THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram-Tamil Nadu

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that Ms. **VARALAKSHMI.V** Reg. No: **20524UCSC068**, a student of III B.Sc. Computer Science, has successfully completed the practice-based practical learning component of **Advanced Database Management System – Practical (UCSR513)** during the academic year 2026–2027.

The practical examination will be held on **16.10.2026**

Course Inchage: Dr. S Manimekalai

Head of the Department: Ms. K Manohari

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** • CREATE • ALTER • DROP • TRUNCATE		
2	**DML Commands** • INSERT • UPDATE • DELETE		
3	**DQL Commands** • SELECT • WHERE • ORDER BY • GROUP BY • HAVING		
4	**Transactions** • COMMIT • ROLLBACK • SAVEPOINT		
5	**Joins** • Inner Join • Left Join • Right Join • Full Join • Self Join		
6	**SQL Functions** • Aggregate Functions • String Functions • Date Functions		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		
10	**Triggers**		
11	**Cursor Implementation**		
12	**PL/SQL Exception Handling**		
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		
	Practice Session (Problems 1 - 19)		
14	**Mini Project**		
	Annexures: • **Khan Academy** • **Hacker Rank** • **Code Chef**		

EX. NO:01 Creation of a DDL operations using Students Table

DATE: 01/07/2026

Aim

To create a Students table, insert student records, display all records, and retrieve students older than 20 years.

Software Used

Oracle / MySQL / SQLite3

Description

DDL (Data Definition Language) is used to define and modify the structure of database objects.

DDL COMMANDS:

- Create
- Alter
- Drop
- Truncate

CREATE

Syntax:

```
CREATE TABLE table_name (
   column1 datatype constraints,
   column2 datatype constraints,
   column3 datatype constraints
);
```

Table Structure:

```
CREATE TABLE students (
   id INTEGER PRIMARY KEY,
   name VARCHAR(100),
   age INTEGER
);
```

Result: Table created.

ALTER

Add Column:

ALTER TABLE students ADD email VARCHAR(100);

Result: Column added.

Modify Column:

ALTER TABLE students MODIFY name VARCHAR(150);

Result: Column modified.

Drop Column:

ALTER TABLE students DROP COLUMN email;

Result: Column dropped.

Rename Table:

ALTER TABLE students RENAME TO learners;

Result: Table renamed.

TRUNCATE

TRUNCATE TABLE students;

Result: All rows deleted, structure intact.

DROP

DROP TABLE students;

Result: Table deleted permanently.

Explanation

Create a table named students. Insert student details into the table. Display all student records. Retrieve students older than 20 years.

Sample Data

INSERT INTO students VALUES (1, 'Arun', 19);

INSERT INTO students VALUES (2, 'Meena', 22);

INSERT INTO students VALUES (3, 'Karthik', 21);

INSERT INTO students VALUES (4, 'Divya', 18);

INSERT INTO students VALUES (5, 'Ravi', 23);

SQL Queries

Query 1: Display all student records

SELECT * FROM students;

Resultant Query Result:

id	name	age
1	arun	19
2	meena	22
3	karthik	21

4	**divya**	**18**
5	**ravi**	**23**

Query 2: Display students older than 20 years

SELECT * FROM students WHERE age > 20 ORDER BY age;

Resultant Query Result:

id	**name**	**age**
3	**karthik**	**21**
2	**meena**	**22**
5	**ravi**	**23**

RESULT

Thus, the Students table was created successfully, records were inserted, all student details were displayed, and the students older than 20 years were retrieved successfull

EX. NO:02 Creation of DML Operations using Books Table

DATE: 08/07/2026

Aim

To create a Books table, insert book records, display all records, update book details, delete a book record, and retrieve books based on their price.

Software Used

Oracle / MySQL / SQLite3

Description

DML (Data Manipulation Language) is used to insert, modify, delete, and retrieve data stored in database tables.

DML COMMANDS:

- INSERT
- UPDATE
- DELETE
- SELECT

CREATE TABLE

sql

```
CREATE TABLE books (
   id INTEGER PRIMARY KEY,
   title VARCHAR(100),
   author VARCHAR(100),
   price INTEGER
);
```

Result: Table created.

INSERT

Syntax:

sql

```
INSERT INTO table_name (column1, column2, column3)
VALUES (value1, value2, value3);
```

Query:

sql

```
INSERT INTO books VALUES (1, 'The Alchemist', 'Paulo Coelho', 350);
INSERT INTO books VALUES (2, 'Wings of Fire', 'A.P.J. Abdul Kalam', 400);
```

INSERT INTO books VALUES (3, 'Harry Potter', 'J.K. Rowling', 550);
INSERT INTO books VALUES (4, 'Ponniyin Selvan', 'Kalki', 600);
INSERT INTO books VALUES (5, 'Atomic Habits', 'James Clear', 450);
Result: Records inserted.

UPDATE

Syntax:

sql
UPDATE table_name
SET column_name = value
WHERE condition;

Query:

sql
UPDATE books
SET price = 500
WHERE id = 1;

Result: Price of *The Alchemist* updated.

DELETE

Syntax:

sql
DELETE FROM table_name
WHERE condition;

Query:

sql
DELETE FROM books
WHERE id = 5;

Result: *Atomic Habits* record deleted.

SELECT

Syntax:

sql
SELECT * FROM table_name;

Query 1: Display all book records

SELECT * FROM books;

Resultant Query Result:

id	title	author	price
1	the alchemist	paulo	500
2	wings of fire	a.p.j abdul kalam	400
3	harry potter	j.k rowling	550
4	ponniyin selvan	kalki	600

Query 2: Display books priced above 450

sql

SELECT * FROM books WHERE price > 450 ORDER BY price;

Resultant Query Result:

id	title	author	price
1	the alchemist	paulo	500
3	harry potter	j.k. rowling	550
4	ponniyin selvan	kalki	600

RESULT

Thus, the Books table was created successfully, records were inserted, book details were displayed, a book price was updated, a book record was deleted, and books priced above 450 were retrieved successfully.

EX. NO:03 Creation of DQL Operations using Products Table

DATE: 13/07/2026

Aim

To create a Products table, insert product records, display all records, filter products based on category, sort products by price, and group products by category.

Software Used

Oracle / MySQL / SQLite3

Description

DQL (Data Query Language) is used to query and retrieve data from database tables.

DQL COMMANDS:

- SELECT
- WHERE
- ORDER BY
- GROUP BY
- HAVING

CREATE TABLE

sql

```
CREATE TABLE products (
   id INTEGER PRIMARY KEY,
   name VARCHAR(100),
   category VARCHAR(50),
   price INTEGER
);
```

Result: Table created.

Sample Data

sql

```
INSERT INTO products VALUES (1, 'Laptop', 'Electronics', 55000);
INSERT INTO products VALUES (2, 'Smartphone', 'Electronics', 30000);
INSERT INTO products VALUES (3, 'Table', 'Furniture', 8000);
INSERT INTO products VALUES (4, 'Chair', 'Furniture', 4000);
INSERT INTO products VALUES (5, 'Shoes', 'Fashion', 2500);
INSERT INTO products VALUES (6, 'Watch', 'Fashion', 6000);
```

Result: Records inserted.

SQL Queries

Query 1: Display all product records

sql

SELECT * FROM products;

Result:

id	name	category	price
1	laptop	electronics	55000
2	smartphone	electronics	30000
3	table	furniture	8000
4	chair	furniture	4000
5	shoes	fashion	2500
6	watch	fashion	6000

Query 2: Display products from Electronics category

sql

SELECT * FROM products WHERE category = 'Electronics';

Result:

id	name	category	price
1	laptop	electronics	55000
2	smartphone	electronics	30000

Query 3: Display products sorted by price (ascending)

sql

SELECT * FROM products ORDER BY price ASC;

Result:

id	name	category	price
5	shoes	fashion	2500
4	chair	furniture	4000

6	**watch**	**fashion**	**6000**
3	**table**	**furniture**	**8000**
2	**smartphone**	**electronics**	**30000**
1	**laptop**	**electronics**	**55000**

Query 4: Group products by category and count them

sql

```
SELECT category, COUNT(*) AS total_products
FROM products
GROUP BY category;
```

Result:

category	total_products
electronics	**2**
furniture	**2**
fashion	**2**

Query 5: Display categories having more than 1 product

sql

```
SELECT category, COUNT(*) AS total_products
FROM products
GROUP BY category
HAVING COUNT(*) > 1;
```

Result:

category	total_products
electronics	**2**
furniture	**2**
fashion	**2**

RESULT

Thus, the Products table was created successfully, records were inserted, product details were displayed, products were filtered by category, sorted by price, grouped by category, and categories with more than one product were retrieved successfully.

EX. NO:04 Creation of Transaction Operations using Accounts Table

DATE: 16/07/2026

Aim

To create an Orders table, insert order records, perform transaction operations using COMMIT, ROLLBACK, and SAVEPOINT, and verify the changes made to the database.

Software Used

Oracle / MySQL / SQLite3

Description

Transaction Control Language (TCL) is used to manage transactions in a database. It ensures data consistency by allowing changes to be saved permanently or undone when required.

TRANSACTION COMMANDS:

- COMMIT
- ROLLBACK
- SAVEPOINT

CREATE TABLE

sql

```
CREATE TABLE orders (
   order_id INTEGER PRIMARY KEY,
   customer_name VARCHAR(100),
   product VARCHAR(100),
   amount INTEGER
);
```

Result: Table created.

Sample Data

sql

```
INSERT INTO orders VALUES (201, 'Arun', 'Laptop', 55000);
INSERT INTO orders VALUES (202, 'Meena', 'Smartphone', 30000);
INSERT INTO orders VALUES (203, 'Karthik', 'Table', 8000);
INSERT INTO orders VALUES (204, 'Divya', 'Shoes', 2500);
INSERT INTO orders VALUES (205, 'Priya', 'Watch', 6000);
```

Result: Records inserted.

COMMIT

Syntax:

sql

COMMIT;

Query:

sql

COMMIT;

Result: All inserted records are permanently saved.

ROLLBACK

Syntax:

sql

ROLLBACK;

Query:

sql

UPDATE orders SET amount = 20000 WHERE order_id = 203;

ROLLBACK;

Result: The update is undone, amount restored to 8000.

SAVEPOINT

Syntax:

sql

SAVEPOINT sp1;

Query:

sql

UPDATE orders SET amount = 35000 WHERE order_id = 202;

SAVEPOINT sp1;

UPDATE orders SET amount = 70000 WHERE order_id = 201;

ROLLBACK TO sp1;

Result: Meena's amount remains updated to 35000, Arun's change is undone.

Explanation

Create a table named orders. Insert order details into the table. Use COMMIT to save records permanently. Update an order and create a SAVEPOINT. Make another change after the savepoint. Use ROLLBACK TO SAVEPOINT to undo changes after the savepoint. Use ROLLBACK to undo uncommitted changes.

RESULT

Thus, the Orders table was created successfully, records were inserted, and transaction operations COMMIT, ROLLBACK, SAVEPOINT, and ROLLBACK TO SAVEPOINT were performed successfully.

EX. NO:05 Creation of a join operations using Suppliers and Products Table

DATE: 17/07/2026

AIM

To create Suppliers and Products tables, insert records, and retrieve related data using different types of joins such as Inner Join, Left Join, Right Join, Full Join, and Self Join.

Software Used

Oracle / MySQL / SQLite3

Description

Joins are used to combine records from two or more tables based on a related column between them.

JOIN COMMANDS:

- Inner Join
- Left Join
- Right Join
- Full Join
- Self Join

Create Tables

sql

```
CREATE TABLE suppliers (
   supplier_id INTEGER PRIMARY KEY,
   supplier_name TEXT,
   city TEXT
);
CREATE TABLE products (
   product_id INTEGER PRIMARY KEY,
   supplier_id INTEGER,
   product_name TEXT,
   price INTEGER
);
```

Result: Tables created.

Sample Data

sql

INSERT INTO suppliers VALUES (1, 'TechWorld', 'Chennai');
INSERT INTO suppliers VALUES (2, 'HomeNeeds', 'Bangalore');
INSERT INTO suppliers VALUES (3, 'FashionHub', 'Delhi');
INSERT INTO suppliers VALUES (4, 'KitchenKing', 'Hyderabad');
INSERT INTO products VALUES (101, 1, 'Laptop', 55000);
INSERT INTO products VALUES (102, 2, 'Table', 8000);
INSERT INTO products VALUES (103, 1, 'Smartphone', 30000);
INSERT INTO products VALUES (104, 3, 'Shoes', 2500);
INSERT INTO products VALUES (105, 5, 'Watch', 6000);

Result: Records inserted.

Query 1: Inner Join

sql

SELECT suppliers.supplier_name, products.product_name, products.price
FROM suppliers
INNER JOIN products
ON suppliers.supplier_id = products.supplier_id;

Resultant Table:

supplier_name	product_name	price
techworld	laptop	55000
homeneeds	table	8000
techworld	smartphone	30000
fashionhub	shoes	2500

Query 2: Left Join

sql

SELECT suppliers.supplier_name, products.product_name, products.price
FROM suppliers
LEFT JOIN products
ON suppliers.supplier_id = products.supplier_id;

Resultant Table:

supplier name	product name	price

techworld	**laptop**	**55000**
techworld	**smartphone**	**30000**
homeneeds	**table**	**8000**
fashionhub	**shoes**	**2500**
kitchenking	**null**	**null**

Query 3: Right Join

sql

```
SELECT suppliers.supplier_name, products.product_name, products.price
FROM suppliers
RIGHT JOIN products
ON suppliers.supplier_id = products.supplier_id;
```

Resultant Table:

supplier name	product name	price
techworld	**laptop**	**55000**
techworld	**smartphone**	**30000**
homeneeds	**table**	**8000**
fashionhub	**shoes**	**2500**
null	**watch**	**6000**

Query 4: Full Join

sql

```
SELECT suppliers.supplier_name, products.product_name, products.price
FROM suppliers
FULL JOIN products
ON suppliers.supplier_id = products.supplier_id;
```

Resultant Table:

supplier name	product name	price

techworld	**laptop**	**55000**
techworld	**smartphone**	**30000**
homeneeds	**table**	**80000**
fashionhub	**shoes**	**2500**
kitchenking	**null**	**null**
null	**watch**	**6000**

Query 5: Self Join (Suppliers Table)

sql

```
SELECT A.supplier_name AS Supplier, B.supplier_name AS Partner
FROM suppliers A
JOIN suppliers B
ON A.city = B.city AND A.supplier_id <> B.supplier_id;
```

Resultant Table:

(If two suppliers share the same city, they appear as pairs. In this dataset, no city is repeated, so result is empty.)

RESULT

Thus, the Suppliers and Products tables were created successfully, and the required records were retrieved using Inner Join, Left Join, Right Join, Full Join, and Self Join operations.

EX. NO:06 Creation of a SQL functions using Library Table

DATE: 18/07/2026

Aim

To create a Library table, insert book records, and retrieve data using different SQL functions such as Aggregate Functions, String Functions, and Date Functions.

Software Used

Oracle / MySQL / SQLite3

Description

SQL Functions are built-in functions used to perform calculations, manipulate text, and work with date values in a database.

SQL FUNCTION TYPES:

- **Aggregate Functions**
- **String Functions**
- **Date Functions**

CREATE TABLE

sql

```
CREATE TABLE library (
   book_id INTEGER PRIMARY KEY,
   title TEXT,
   author TEXT,
   price INTEGER,
   publish_date DATE
);
```

Result: Table created.

Sample Data

sql

```
INSERT INTO library VALUES (1, 'The Alchemist', 'Paulo Coelho', 350, '2025-01-15');
INSERT INTO library VALUES (2, 'Wings of Fire', 'A.P.J. Abdul Kalam', 400, '2025-02-10');
INSERT INTO library VALUES (3, 'Harry Potter', 'J.K. Rowling', 550, '2025-03-05');
INSERT INTO library VALUES (4, 'Ponniyin Selvan', 'Kalki', 600, '2025-04-12');
INSERT INTO library VALUES (5, 'Atomic Habits', 'James Clear', 450, '2025-05-20');
INSERT INTO library VALUES (6, 'The Lion King', 'Disney', 300, '2025-06-18');
```

Result: Records inserted.

SQL Queries

Aggregate Functions

Query 1: Count total books

sql

```
SELECT COUNT(*) AS Total_Books FROM library;
```

Result:

Total_Books
6

Query 2: Calculate total and average price

sql

```
SELECT SUM(price) AS Total_Price, AVG(price) AS Average_Price FROM library;
```

Result:

Total_Price	Average_Price
2650	441.67

Query 3: Find highest and lowest price

sql

```
SELECT MAX(price) AS Highest_Price, MIN(price) AS Lowest_Price FROM library;
```

Result:

Highest_Price	Lowest_Price
600	300

String Functions

Query 4: Display book titles in uppercase

sql

```
SELECT title, UPPER(title) AS Upper_Title FROM library;
```

Result:

title	Upper_Title
the alchemist	THE ALCHEMIST

wings of fire	**WINGS OF FIRE**
harry potter	**HARRY POTTER**
ponniyin selvan	**PONNIYIN SELVAN**
atomic habits	**ATOMIC HABITS**
the lion king	**THE LION KING**

Query 5: Display length of book titles

sql

```
SELECT title, LENGTH(title) AS Title_Length FROM library;
```

Result:

title	**Title_Length**
the alchemist	**13**
wings of fire	**12**
harry potter	**12**
ponniyin selvan	**15**
atomic habits	**13**
the lion king	**13**

Date Functions

Query 6: Display current date

sql

```
SELECT CURRENT_DATE AS Current_Date;
```

Result:

Current_Date
System Date

Query 7: Display books published after March 2025

sql

SELECT title, publish_date FROM library WHERE publish_date > '2025-03-31';

Result:

title	**publish_date**
ponniyin selvan	2025-04-12
atomic habits	2025-05-20
the lion king	2025-06-18

RESULT

Thus, the Library table was created successfully, book records were inserted, and the required data was retrieved successfully using Aggregate Functions, String Functions, and Date Functions.

EX. NO:07 Creation of Subqueries and Nested Queries using Courses

DATE: 20/07/2026

Aim

To create a Courses table, insert course records, and retrieve data using Subqueries and Nested Queries.

Software Used

Oracle / MySQL / SQLite3

Description

A Subquery is a query written inside another SQL query. It is used to retrieve intermediate results and use them in the main query.

A Nested Query is a query placed inside another query to perform operations based on the result of the inner query.

SUBQUERY TYPES:

- **Single Row Subquery**
- **Multiple Row Subquery**
- **Nested Subquery**

CREATE TABLE

sql

```
CREATE TABLE courses (
   course_id INTEGER PRIMARY KEY,
   course_name TEXT,
   instructor TEXT,
   duration INTEGER,   -- duration in hours
   fee INTEGER
);
```

Result: Table created.

Sample Data

sql

```
INSERT INTO courses VALUES (1, 'Java Programming', 'Dr. Kumar', 40, 5000);
INSERT INTO courses VALUES (2, 'Web Development', 'Ms. Priya', 60, 7000);
INSERT INTO courses VALUES (3, 'Data Science', 'Mr. Arun', 80, 12000);
INSERT INTO courses VALUES (4, 'Cyber Security', 'Ms. Divya', 50, 9000);
INSERT INTO courses VALUES (5, 'Cloud Computing', 'Mr. Ravi', 70, 10000);
INSERT INTO courses VALUES (6, 'AI & ML', 'Dr. Meena', 100, 15000);
```

Result: Records inserted.

SQL Queries

Query 1: Display all course records

sql

```
SELECT * FROM courses;
```

Resultant Table:

course_id	course_name	instructor	duration	fee
1	java programming	dr.kumar	40	5000
2	web development	ms.priya	60	7000
3	data science	mr.arun	80	12000
4	cyber security	ms.divya	50	9000
5	cloud computing	mr.ravi	70	10000
6	ai & ml	dr.meena	100	15000

Query 2: Single Row Subquery – Find courses with fee greater than the average fee

sql

```
SELECT course_name, fee
FROM courses
WHERE fee > (SELECT AVG(fee) FROM courses);
```

Result:

course_name	fee
data science	12000
cloud computing	10000
ai & ml	15000

Query 3: Multiple Row Subquery – Find courses taught by instructors in a given set

sql

```
SELECT course_name, instructor
FROM courses
```

WHERE instructor IN (SELECT instructor FROM courses WHERE instructor IN ('Mr. Arun','Ms. Priya'));

Result:

course_name	instructor
web development	ms,priya
data science	mr.arun

Query 4: Nested Subquery – Find the course with the highest fee

sql

SELECT course_name, fee

FROM courses

WHERE fee = (SELECT MAX(fee) FROM courses);

Result:

course_name	fee
ai & ml	15000

Query 5: Subquery – Find courses with duration greater than "Java Programming"

sql

SELECT course_name, duration

FROM courses

WHERE duration > (SELECT duration FROM courses WHERE course_name = 'Java Programming');

Result:

course_name	duration
web development	60
data science	80
cyber security	50
cloud computing	70
ai & ml	100

RESULT

Thus, the Courses table was created successfully, course records were inserted, and the required course details were retrieved successfully using Subqueries and Nested Queries.

EX. NO:08 Creation of views, indexes and sequences

DATE: 24/07/2026

Aim

To create and perform operations on Views, Indexes, and Sequences using SQL commands.

Software Used

Oracle SQL / SQL Command Line

Description

View:
A View is a virtual table created from one or more tables using a SELECT query. It does not store the actual data separately.

Index:
An Index is used to improve the speed of data retrieval from a table.

Sequence:
A Sequence is a database object used to generate unique numeric values automatically.

1. VIEW

Definition

A View is a virtual table based on the result of a SQL query.

Syntax

```
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition;
```

Create Table

```
CREATE TABLE employee_details (
   employee_id NUMBER PRIMARY KEY,
   employee_name VARCHAR2(50),
   department VARCHAR2(30),
   salary NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO employee_details VALUES (101, 'Kavin', 'IT', 45000);
INSERT INTO employee_details VALUES (102, 'Meena', 'HR', 32000);
INSERT INTO employee_details VALUES (103, 'Arun', 'Finance', 38000);
```

INSERT INTO employee_details VALUES (104, 'Divya', 'IT', 50000);

INSERT INTO employee_details VALUES (105, 'Priya', 'HR', 42000);

COMMIT;

Records Inserted

Create View

CREATE VIEW it_employees AS

SELECT employee_id, employee_name, salary

FROM employee_details

WHERE department = 'IT';

View Created

Display View

SELECT * FROM it_employees;

Resultant Query

employee_id	employee_name	salary
101	Kavin	45000
104	Divya	50000

Explanation

The it_employees view displays only the employees who belong to the IT department.

2. INDEX

Definition

An Index is a database object that improves the speed of searching and retrieving records from a table.

Syntax

CREATE INDEX index_name

ON table_name(column_name);

Create Index

CREATE INDEX idx_employee_name

ON employee_details(employee_name);

Index Created

Use Index in Search

SELECT *

FROM employee_details

WHERE employee_name = 'Kavin';

Resultant Query

employee_id	employee_name	department	salary
101	Kavin	IT	45000

Explanation

The index idx_employee_name is created on the employee_name column. It helps improve the performance of searches based on employee names.

3. SEQUENCE

Definition

A Sequence is a database object that automatically generates a series of unique numeric values.

Syntax

CREATE SEQUENCE sequence_name

START WITH initial_value

INCREMENT BY increment_value;

Create Sequence

CREATE SEQUENCE employee_seq

START WITH 106

INCREMENT BY 1;

Sequence Created

Insert Record Using Sequence

INSERT INTO employee_details

VALUES (employee_seq.NEXTVAL, 'Rahul', 'Sales', 28000);

COMMIT;

Record Inserted

Display Sequence Value

SELECT employee_seq.CURRVAL AS current_sequence

FROM dual;

Resultant Query

current_sequence
106

Insert Another Record Using Sequence

```
INSERT INTO employee_details
VALUES (employee_seq.NEXTVAL, 'Anitha', 'Marketing', 35000);
COMMIT;
```

Record Inserted

Display All Employee Records

```
SELECT * FROM employee_details
ORDER BY employee_id;
```

Resultant Query

employee_id	employee_name	department	salary
101	Kavin	IT	45000
102	Meena	HR	32000
103	Arun	Finance	38000
104	Divya	IT	50000
105	Priya	HR	42000
106	Rahul	Sales	28000
107	Anitha	Marketing	35000

Important Commands

Drop View

```
DROP VIEW it_employees;
```

View Dropped

Drop Index

```
DROP INDEX idx_employee_name;
```

Index Dropped

Drop Sequence

```
DROP SEQUENCE employee_seq;
```

Sequence Dropped

Result

Thus, the View, Index, and Sequence were created and successfully operated using SQL commands. The View was used to display selected records, the Index was used to improve data retrieval, and the Sequence was used to generate unique numeric values automatically.

EX. NO:09 Creation of stored procedures and functions

DATE: 27/07/2026

Aim

To create and execute Stored Procedures and Functions using PL/SQL.

Software Used

Oracle SQL / SQL Command Line

Description

Stored Procedure:
A Stored Procedure is a named PL/SQL program stored in the database. It is used to perform a specific task and may or may not return a value.

Function:
A Function is a named PL/SQL program that performs a specific operation and always returns a single value.

1. STORED PROCEDURE

Definition

A Stored Procedure is a stored PL/SQL block that performs a particular operation. It can accept input parameters and can modify or display data.

Syntax

```
CREATE OR REPLACE PROCEDURE procedure_name
(parameter_name IN datatype)
IS
BEGIN
   statements;
END;/
```

Create Table

```
CREATE TABLE student_marks (
   student_id NUMBER PRIMARY KEY,
   student_name VARCHAR2(50),
   mark NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO student_marks VALUES (101, 'Kavin', 85);
INSERT INTO student_marks VALUES (102, 'Meena', 72);
INSERT INTO student_marks VALUES (103, 'Arun', 91);
```

```
INSERT INTO student_marks VALUES (104, 'Divya', 68);
INSERT INTO student_marks VALUES (105, 'Priya', 95);
COMMIT;
```

Records Inserted

Display Student Records

```
SELECT * FROM student_marks;
```

Resultant Query

student_id	student_name	mark
101	Kavin	85
102	Meena	72
103	Arun	91
104	Divya	68
105	Priya	95

Create Stored Procedure

The following procedure displays the details of a student based on the student ID.

```
CREATE OR REPLACE PROCEDURE display_student (
   p_student_id IN NUMBER
)
IS
   v_name student_marks.student_name%TYPE;
   v_mark student_marks.mark%TYPE;
BEGIN
   SELECT student_name, mark
   INTO v_name, v_mark
   FROM student_marks
   WHERE student_id = p_student_id;
   DBMS_OUTPUT.PUT_LINE('Student Name: ' || v_name);
   DBMS_OUTPUT.PUT_LINE('Mark: ' || v_mark);
END;
/
```

Procedure Created

Execute Stored Procedure

```
SET SERVEROUTPUT ON;
EXEC display_student(103);
```

Resultant Query

Student Name	Mark
Arun	91

Output

Student Name: Arun

Mark: 91

Explanation

The procedure display_student accepts the student ID as an input parameter and retrieves the corresponding student name and mark from the student_marks table.

2. FUNCTION

Definition

A Function is a stored PL/SQL program that performs a specific operation and returns a single value.

Syntax

```
CREATE OR REPLACE FUNCTION function_name
(parameter_name IN datatype)
RETURN datatype
IS
BEGIN
   statements;
   RETURN value;
END;
/
```

Create Function

The following function returns the grade based on the student's mark.

```
CREATE OR REPLACE FUNCTION get_grade (
   p_mark IN NUMBER
)
RETURN VARCHAR2
IS
```

```
    v_grade VARCHAR2(5);
BEGIN
    IF p_mark >= 90 THEN
        v_grade := 'A+';
    ELSIF p_mark >= 80 THEN
        v_grade := 'A';
    ELSIF p_mark >= 70 THEN
        v_grade := 'B';
    ELSIF p_mark >= 60 THEN
        v_grade := 'C';
    ELSIF p_mark >= 50 THEN
        v_grade := 'D';
    ELSE
        v_grade := 'F';
    END IF;
    RETURN v_grade;
END;
/
```

Function Created

Execute Function

```
SELECT student_id, student_name, mark,
       get_grade(mark) AS grade
FROM student_marks;
```

Resultant Query

student_id	student_name	mark	grade
101	Kavin	85	A
102	Meena	72	B
103	Arun	91	A+
104	Divya	68	C
105	Priya	95	A+

Explanation

The function get_grade accepts the student's mark as an input and returns the corresponding grade. The function is called inside the SELECT statement for each student.

Important Commands

Display Stored Procedure

SELECT object_name, object_type

FROM user_objects

WHERE object_name = 'DISPLAY_STUDENT';

Resultant Query

object_name	object_type
DISPLAY_STUDENT	PROCEDURE

Display Stored Function

SELECT object_name, object_type

FROM user_objects

WHERE object_name = 'GET_GRADE';

Resultant Query

object_name	object_type
GET_GRADE	FUNCTION

Drop Procedure

DROP PROCEDURE display_student;

Procedure Dropped

Drop Function

DROP FUNCTION get_grade;

Function Dropped

Result

Thus, the Stored Procedure and Function were created and executed successfully using PL/SQL. The Stored Procedure was used to retrieve student details, and the Function was used to calculate the grade based on the student's mark.

EX. NO:10 Creation of Triggers method using Bank Accounts table

DATE: 29/07/2026

Aim

To create and execute Triggers using PL/SQL to automatically perform an action when an INSERT, UPDATE, or DELETE operation occurs on a table.

Software Used

Oracle SQL / SQL Command Line

Description

A Trigger is a stored PL/SQL program that is automatically executed when a specified event occurs on a table or database.

Triggers can be used to automatically validate data, prevent invalid updates, and maintain audit records.

1. CREATE TABLE

sql

```
CREATE TABLE bank_accounts (
   account_id NUMBER PRIMARY KEY,
   account_holder VARCHAR2(50),
   balance NUMBER
);
```

Result: Table created.

Insert Records

sql

```
INSERT INTO bank_accounts VALUES (101, 'Kavin', 25000);
INSERT INTO bank_accounts VALUES (102, 'Meena', 30000);
INSERT INTO bank_accounts VALUES (103, 'Arun', 20000);
INSERT INTO bank_accounts VALUES (104, 'Divya', 35000);
INSERT INTO bank_accounts VALUES (105, 'Priya', 40000);
COMMIT;
```

Result: Records inserted.

Display Records

sql

```
SELECT * FROM bank_accounts;
```

Resultant Table:

account_id	account_holder	balance
101	**kavin**	**25000**
102	**meena**	**30000**
103	**arun**	**20000**
104	**divya**	**35000**
105	**priya**	**40000**

2. BEFORE INSERT TRIGGER

Definition: Prevents inserting an account with a negative balance.

sql

```
CREATE OR REPLACE TRIGGER check_balance_before_insert
BEFORE INSERT ON bank_accounts
FOR EACH ROW
BEGIN
   IF :NEW.balance < 0 THEN
      RAISE_APPLICATION_ERROR(-20001, 'Balance cannot be negative');
   END IF;
END;
/
```

Execution:

sql

```
INSERT INTO bank_accounts VALUES (106, 'Ravi', -5000);
```

Output:

ORA-20001: Balance cannot be negative

3. BEFORE UPDATE TRIGGER

Definition: Prevents updating balance to a negative value.

sql

```
CREATE OR REPLACE TRIGGER check_balance_before_update
BEFORE UPDATE OF balance ON bank_accounts
FOR EACH ROW
BEGIN
```

```
    IF :NEW.balance < 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'Balance cannot be negative');
    END IF;
END;
/
```

Execution:

sql

```
UPDATE bank_accounts SET balance = -1000 WHERE account_id = 103;
```

Output:

ORA-20002: Balance cannot be negative

4. AFTER INSERT TRIGGER (Audit)

Definition: Records every new account creation in an audit table.

Create Audit Table:

sql

```
CREATE TABLE account_audit (
    audit_id NUMBER,
    account_id NUMBER,
    action_type VARCHAR2(20)
);
```

Create Sequence:

sql

```
CREATE SEQUENCE account_audit_seq
START WITH 1
INCREMENT BY 1;
```

Create Trigger:

sql

```
CREATE OR REPLACE TRIGGER account_insert_audit
AFTER INSERT ON bank_accounts
FOR EACH ROW
BEGIN
    INSERT INTO account_audit
    VALUES (account_audit_seq.NEXTVAL, :NEW.account_id, 'INSERT');
END;
/
```

Execution:

sql

INSERT INTO bank_accounts VALUES (106, 'Ravi', 15000);

COMMIT;

Display Audit Records:

sql

SELECT * FROM account_audit;

Resultant Table:

audit_id	account_id	action_type
1	**106**	**INSERT**

5. DISPLAY FINAL ACCOUNT RECORDS

sql

SELECT * FROM bank_accounts ORDER BY account_id;

Resultant Table:

account_id	account_holder	balance
101	**kavin**	**25000**
102	**meena**	**30000**
103	**arun**	**20000**
104	**divya**	**35000**
105	**priya**	**40000**
106	**ravi**	**15000**

Important Commands

Display Triggers:

sql

SELECT trigger_name, status FROM user_triggers;

Result:

trigger_name	status

CHECK_BALANCE_BEFORE_INSERT	**ENABLED**
CHECK_BALANCE_BEFORE_UPDATE	**ENABLED**
ACCOUNT_INSERT_AUDIT	**ENABLED**

Drop Trigger:

sql

DROP TRIGGER check_balance_before_insert;

DROP TRIGGER check_balance_before_update;

DROP TRIGGER account_insert_audit;

RESULT

Thus, the **Bank Accounts table** was created successfully, triggers were executed, invalid balances were prevented, and account insertions were recorded in the audit table.

EX. NO:11 Creation of cursor implementation using Students Marks table

DATE: 30/07/2026

Aim

To implement and execute a Cursor using PL/SQL to retrieve and process records from a database table.

Software Used

Oracle SQL / SQL Command Line

Description

A Cursor is a pointer used to process the rows returned by a SQL query one row at a time.

Cursors are useful when each record needs to be processed individually.

There are two types of cursors:

- Implicit Cursor
- Explicit Cursor

In this practical, an **Explicit Cursor** is implemented.

1. CREATE TABLE

sql

```
CREATE TABLE student_marks (
   student_id NUMBER PRIMARY KEY,
   student_name VARCHAR2(50),
   subject VARCHAR2(50),
   marks NUMBER
);
```

Result: Table created.

Insert Records

sql

```
INSERT INTO student_marks VALUES (401, 'Kavin', 'Maths', 85);
INSERT INTO student_marks VALUES (402, 'Meena', 'Science', 72);
INSERT INTO student_marks VALUES (403, 'Arun', 'English', 91);
INSERT INTO student_marks VALUES (404, 'Divya', 'History', 68);
INSERT INTO student_marks VALUES (405, 'Priya', 'Computer', 95);
COMMIT;
```

Result: Records inserted.

Display Records

sql

SELECT * FROM student_marks;

Resultant Table:

student_id	student_name	subject	marks
401	**kavin**	**maths**	**85**
402	**meena**	**science**	**72**
403	**arun**	**english**	**91**
404	**divya**	**history**	**68**
405	**priya**	**computer**	**95**

2. EXPLICIT CURSOR

Definition: An Explicit Cursor is declared and controlled by the programmer.

Program:

sql

```
SET SERVEROUTPUT ON;
DECLARE
   CURSOR student_cursor IS
      SELECT student_name, marks
      FROM student_marks;
   v_name student_marks.student_name%TYPE;
   v_marks student_marks.marks%TYPE;
BEGIN
   OPEN student_cursor;
   LOOP
      FETCH student_cursor INTO v_name, v_marks;
      EXIT WHEN student_cursor%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE(
         'Student: ' || v_name || ' | Marks: ' || v_marks
      );
   END LOOP;
   CLOSE student_cursor;
END;
```

/

Output:

Code

Student: Kavin | Marks: 85

Student: Meena | Marks: 72

Student: Arun | Marks: 91

Student: Divya | Marks: 68

Student: Priya | Marks: 95

3. CURSOR WITH CONDITION

Program: Display only students who scored above 80.

sql

```
SET SERVEROUTPUT ON;
DECLARE
   CURSOR high_score_cursor IS
      SELECT student_id, student_name, marks
      FROM student_marks
      WHERE marks > 80;
   v_id student_marks.student_id%TYPE;
   v_name student_marks.student_name%TYPE;
   v_marks student_marks.marks%TYPE;
BEGIN
   OPEN high_score_cursor;
   LOOP
      FETCH high_score_cursor INTO v_id, v_name, v_marks;
      EXIT WHEN high_score_cursor%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE(
         'ID: ' || v_id || ' | Student: ' || v_name || ' | Marks: ' || v_marks
      );
   END LOOP;
   CLOSE high_score_cursor;
END;
/
```

Resultant Table:

student_id	student_name	marks
401	kavin	85
403	arun	91
405	priya	95

4. CURSOR ATTRIBUTES

Important attributes:

- %FOUND → TRUE if last fetch returned a row
- %NOTFOUND → TRUE if last fetch did not return a row
- %ROWCOUNT → Number of rows processed
- %ISOPEN → TRUE if cursor is open

5. CURSOR USING %ROWCOUNT

Program: Display total number of students processed.

sql

```
SET SERVEROUTPUT ON;
DECLARE
   CURSOR student_cursor IS
      SELECT student_name, marks
      FROM student_marks;
   v_name student_marks.student_name%TYPE;
   v_marks student_marks.marks%TYPE;
BEGIN
   OPEN student_cursor;
   LOOP
      FETCH student_cursor INTO v_name, v_marks;
      EXIT WHEN student_cursor%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE('Student: ' || v_name || ' | Marks: ' || v_marks);
   END LOOP;
   DBMS_OUTPUT.PUT_LINE('Total Students Processed: ' || student_cursor%ROWCOUNT);
   CLOSE student_cursor;
END;
/
```

Output:

Code

Student: Kavin | Marks: 85

Student: Meena | Marks: 72

Student: Arun | Marks: 91

Student: Divya | Marks: 68

Student: Priya | Marks: 95

Total Students Processed: 5

RESULT

Thus, the Cursor was successfully implemented using PL/SQL. The explicit cursor was used to retrieve and process student records one by one, apply conditions, and count the number of processed records.

EX. NO:12 Creation of PL/SQL Exception Handling using Hospital Patients

DATE: 03/08/2026

Aim

To implement Exception Handling in PL/SQL and handle runtime errors using predefined and user-defined exceptions.

Software Used

Oracle SQL / SQL Command Line

Description

An Exception is an error that occurs during the execution of a PL/SQL program.

Exception Handling is used to handle runtime errors without terminating the program abnormally.

Types of Exceptions:

- Predefined Exception (built-in by Oracle)
- User-Defined Exception (created by programmer)

1. CREATE TABLE

sql

```
CREATE TABLE hospital_patients (
   patient_id NUMBER PRIMARY KEY,
   patient_name VARCHAR2(50),
   disease VARCHAR2(50),
   bill_amount NUMBER
);
```

Result: Table created.

Insert Records

sql

```
INSERT INTO hospital_patients VALUES (701, 'Kavin', 'Fever', 2000);
INSERT INTO hospital_patients VALUES (702, 'Meena', 'Diabetes', 5000);
INSERT INTO hospital_patients VALUES (703, 'Arun', 'Fracture', 8000);
INSERT INTO hospital_patients VALUES (704, 'Divya', 'Asthma', 3000);
INSERT INTO hospital_patients VALUES (705, 'Priya', 'Heart Disease', 15000);
COMMIT;
```

Result: Records inserted.

Display Records

sql

SELECT * FROM hospital_patients;

Resultant Table:

patient_id	patient_name	disease	bill_amount
701	**kavin**	**fever**	**2000**
702	**meena**	**diabetes**	**5000**
703	**arun**	**fracture**	**8000**
704	**divya**	**asthma**	**3000**
705	**priya**	**heart disease**	**15000**

2. NO_DATA_FOUND Exception

sql

```
SET SERVEROUTPUT ON;
DECLARE
   v_name hospital_patients.patient_name%TYPE;
   v_bill hospital_patients.bill_amount%TYPE;
BEGIN
   SELECT patient_name, bill_amount
   INTO v_name, v_bill
   FROM hospital_patients
   WHERE patient_id = 799;
   DBMS_OUTPUT.PUT_LINE('Patient: ' || v_name || ' | Bill: ₹' || v_bill);
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Exception: Patient not found.');
END;
/
```

Output:

Exception: Patient not found.

3. TOO_MANY_ROWS Exception

sql

```
SET SERVEROUTPUT ON;
DECLARE
   v_name hospital_patients.patient_name%TYPE;
BEGIN
   SELECT patient_name
   INTO v_name
   FROM hospital_patients
   WHERE bill_amount > 3000;
   DBMS_OUTPUT.PUT_LINE('Patient: ' || v_name);
EXCEPTION
   WHEN TOO_MANY_ROWS THEN
      DBMS_OUTPUT.PUT_LINE('Exception: More than one patient found.');
END;
/
```

Output:

Exception: More than one patient found.

4. ZERO_DIVIDE Exception

sql

```
SET SERVEROUTPUT ON;
DECLARE
   v_bill NUMBER := 5000;
   v_divisor NUMBER := 0;
   v_result NUMBER;
BEGIN
   v_result := v_bill / v_divisor;
   DBMS_OUTPUT.PUT_LINE('Result: ' || v_result);
EXCEPTION
   WHEN ZERO_DIVIDE THEN
      DBMS_OUTPUT.PUT_LINE('Exception: Cannot divide by zero.');
END;
/
```

Output:

Exception: Cannot divide by zero.

5. USER-DEFINED Exception

```
sql
SET SERVEROUTPUT ON;
DECLARE
   v_bill hospital_patients.bill_amount%TYPE;
   v_payment NUMBER := 20000;
   over_payment EXCEPTION;
BEGIN
   SELECT bill_amount INTO v_bill
   FROM hospital_patients
   WHERE patient_id = 701;
   IF v_payment > v_bill THEN
      RAISE over_payment;
   END IF;
   DBMS_OUTPUT.PUT_LINE('Payment Successful.');
EXCEPTION
   WHEN over_payment THEN
      DBMS_OUTPUT.PUT_LINE('Exception: Payment exceeds bill amount.');
END;
/
```

Output:

Exception: Payment exceeds bill amount.

RESULT

Thus, the **Hospital Patients table** was created successfully, records were inserted, and runtime errors were handled using **NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, and User-Defined Exceptions** in PL/SQL.

EX. NO:13 Creation of Database Normalization using Retail Orders table

DATE: 05/08/2026

Aim

To study and implement Database Normalization up to Boyce-Codd Normal Form (BCNF) to reduce data redundancy and improve data integrity.

Software Used

Oracle SQL / SQL Command Line

Description

Normalization is the process of organizing data in a database to reduce redundancy and avoid insertion, deletion, and update anomalies.

Normal Forms Covered:

- **First Normal Form (1NF)**
- **Second Normal Form (2NF)**
- **Third Normal Form (3NF)**
- **Boyce-Codd Normal Form (BCNF)**

1. UNNORMALIZED TABLE

Consider the following Retail Orders table:

order_id	customer_name	product_id	product_name	supplier_id	supplier_name
901	Kavin	P01	Laptop	S01	TechWorld
902	Meena	P02	Mobile	S02	SmartStore
903	Arun	P01	Laptop	S01	TechWorld
904	Divya	P03	Shoes	S03	FashionHub

Problem: Supplier name is repeated for multiple products.

2. FIRST NORMAL FORM (1NF)

Definition:

- **Each column contains atomic values.**
- **No repeating groups.**
- **Each row is uniquely identifiable.**

1NF Table:

order_id	customer_name	product_id	product_name	supplier_id	supplier_name
901	Kavin	P01	Laptop	S01	TechWorld
902	Meena	P02	Mobile	S02	SmartStore
903	Arun	P01	Laptop	S01	TechWorld
904	Divya	P03	Shoes	S03	FashionHub

Primary Key: order_id

3. SECOND NORMAL FORM (2NF)

Definition:

- **Table is in 1NF.**
- **No partial dependency.**
- **Every non-key attribute depends on the whole primary key.**

Decomposition:

Customer Table

customer_id	customer_name
C01	Kavin
C02	Meena
C03	Arun
C04	Divya

Product Table

product_id	product_name	supplier_id
P01	Laptop	S01
P02	Mobile	S02
P03	Shoes	S03

Order Table

order_id	customer_id	product_id
901	C01	P01
902	C02	P02
903	C03	P01
904	C04	P03

4. THIRD NORMAL FORM (3NF)

Definition:

- **Table is in 2NF.**
- **No transitive dependency.**

Observation: product_id → supplier_id and supplier_id → supplier_name.

Thus, supplier_name has a transitive dependency on product_id.

Final Tables in 3NF:

- **Customer Table (customer_id → customer_name)**
- **Product Table (product_id → product_name, supplier_id)**
- **Supplier Table (supplier_id → supplier_name)**
- **Order Table (order_id → customer_id, product_id)**

5. BOYCE-CODD NORMAL FORM (BCNF)

Definition:

- **For every non-trivial functional dependency, the determinant must be a candidate key.**

Example: If each supplier supplies only one product, then:

supplier_id → product_id

This violates BCNF if supplier_id is not a candidate key in the Product table.

Decomposition:

Supplier_Product Table

supplier_id	product_id
S01	P01
S02	P02
S03	P03

Supplier Table

supplier_id	supplier_name
S01	TechWorld
S02	SmartStore
S03	FashionHub

Product Table

product_id	product_name
P01	Laptop
P02	Mobile
P03	Shoes

RESULT

Thus, the Retail Orders table was normalized step by step into 1NF, 2NF, 3NF, and BCNF. The normalization process reduced redundancy, eliminated anomalies, and improved data consistency and integrity.

PRACTICE SESSION

PRACTICE NO:01 SQL Queries on CABHUB and CUSTOMER Tables

DATE: 06/08/2026

1. Consider the following tables CABHUB and CUSTOMER and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

(i) To display the names of all the white colored vehicles.

(ii) To display name of vehicle, make the capacity of vehicles in ascending order of their sitting Capacity.

(iii) To display the highest charges at which a vehicle can be hired from CABHUB.

(iv) To display the customer and the corresponding name of the vehicle hired by them.

(b) (i) SELECT COUNT (DISTINCT Make) FROM CABHUB;

(ii) SELECT MAX (CHARGES), MIN (Charges) FROM CABHUB;

(iii) SELECT COUNT(*), Make FROM CABHUB;

(iv) SELECT VehicleName FROM CABHUB WHERE Capacity = 4;

Aim

To write and execute SQL queries on the **CABHUB** and **CUSTOMER** tables to retrieve vehicle and customer information using filtering, sorting, aggregate functions, and table joins.

Description

This experiment demonstrates the use of SQL commands to:

- Display white-colored vehicles.
- Display vehicle name, make, and capacity in ascending order.
- Find the highest and lowest vehicle hiring charges.
- Display customers along with the vehicles hired by them.
- Count the number of different vehicle manufacturers.

- Count vehicles based on their make.
- Find vehicles having a specific seating capacity.

Software Used

Oracle Database / Oracle SQL Plus (SQL Command Line)

1. Create CABHUB Table

```
CREATE TABLE CABHUB (
VehicleNo VARCHAR(10),
VehicleName VARCHAR(30),
Make VARCHAR(20),
Color VARCHAR(15),
Capacity INT,
Charges INT
);
```

Insert Records

```
INSERT INTO CABHUB VALUES
('C101', 'Swift Dzire', 'Maruti', 'White', 4, 1500);

INSERT INTO CABHUB VALUES
('C102', 'Innova', 'Toyota', 'Black', 7, 2500);

INSERT INTO CABHUB VALUES
('C103', 'City', 'Honda', 'White', 5, 2000);

INSERT INTO CABHUB VALUES
```

('C104', 'Verna', 'Hyundai', 'Red', 5, 1800);

INSERT INTO CABHUB VALUES

('C105', 'Ertiga', 'Maruti', 'White', 7, 2200);

Results Explain Describe Saved SQL History

VEHICLENO	VEHICLENAME	MAKE	COLOR	CAPACITY	CHARGES
C101	Swift Dzire	Maruti	White	4	1500
C102	Innova	Toyota	Black	7	2500
C103	City	Honda	White	5	2000
C104	Verna	Hyundai	Red	5	1800
C105	Ertiga	Maruti	White	7	2200

5 rows returned in 0.01 seconds CSV Export

2. Create CUSTOMER Table

CREATE TABLE CAB_CUSTOMER (

CustomerID INT,

CustomerName VARCHAR(30),

VehicleNo VARCHAR(10)

);

Insert Record

INSERT INTO CAB_CUSTOMER VALUES

(1, 'Anisha', 'C101');

INSERT INTO CAB_CUSTOMER VALUES

(2, 'Rahul', 'C102');

INSERT INTO CAB_CUSTOMER VALUES

(3, 'Priya', 'C103');

INSERT INTO CAB_CUSTOMER VALUES

(4, 'Kavya', 'C104');
INSERT INTO CAB_CUSTOMER VALUES

(5, 'Arun', 'C105');

SELECT * FROM CAB_CUSTOMER;

Results Explain Describe Saved SQL History

CUSTOMERID	CUSTOMERNAME	VEHICLENO
1	Anisha	C101
2	Rahul	C102
3	Priya	C103
5	Arun	C105

4 rows returned in 0.02 seconds CSV Export

(a) SQL Commands and Output

(i) Display names of all white-colored vehicles

SELECT VehicleName

FROM CABHUB

WHERE Color = 'White';

output

Results | Explain | Describe | Saved SQL | History

VEHICLENAME
Swift Dzire
City
Ertiga

3 rows returned in 0.00 seconds CSV Export

(ii) Display vehicle name, make and capacity in ascending order of capacity

```
SELECT VehicleName, Make, Capacity
FROM CABHUB

ORDER BY Capacity ASC;
```

Results | Explain | Describe | Saved SQL | History

VEHICLENAME	MAKE	CAPACITY
Swift Dzire	Maruti	4
City	Honda	5
Verna	Hyundai	5
Ertiga	Maruti	7
Innova	Toyota	7

5 rows returned in 0.00 seconds CSV Export

(iii) Display the highest charges at which a vehicle can be hired

```
SELECT MAX(Charges) AS Highest_Charges

FROM CABHUB;
```

Output

Results Explain Describe Saved SQL History

HIGHEST_CHARGES
2500

1 rows returned in 0.00 seconds CSV Export

(iv) Display customer and corresponding vehicle name hired by them

```
SELECT CUSTOMER.CustomerName, CABHUB.VehicleName
FROM CUSTOMER
JOIN CABHUB
ON CUSTOMER.VehicleNo = CABHUB.VehicleNo;
```

Results Explain Describe Saved SQL History

CUSTOMERNAME	VEHICLENAME
Anisha	Swift Dzire
Rahul	Innova
Priya	City
Arun	Ertiga

4 rows returned in 0.00 seconds CSV Export

(b) SQL Commands and Output

1.

```
SELECT COUNT(DISTINCT Make)
FROM CABHUB;
```

Results Explain Describe Saved SQL History

COUNT(DISTINCTMAKE)
4

1 rows returned in 0.00 seconds CSV Export

2.

SELECT MAX(Charges), MIN(Charges)

FROM CABHUB;

Results Explain Describe Saved SQL History

MAX(CHARGES)	MIN(CHARGES)
2500	1500

1 rows returned in 0.00 seconds CSV Export

3.

3.SELECT COUNT(*), Make

FROM CABHUB

GROUP BY Make;

Results Explain Describe Saved SQL History

COUNT(*)	MAKE
1	Hyundai
1	Toyota
2	Maruti
1	Honda

4 rows returned in 0.00 seconds CSV Export

4.

```
SELECT VehicleName

FROM CABHUB

WHERE Capacity = 4;
```



PRACTICE NO:02 SQL Queries on CO2_EMISSION Table

DATE: 07/08/2026

2. Given the following CO2_EMISSION relation, write SQL commands for questions (a) to (f).
(a) Display all the details of vehicles that use **Petrol** as the fuel type.

(b) Display the **Company** and **Model** of vehicles whose **EngineSize** is greater than **1.5 litres**.

(c) Display the **Company**, **Model**, and **CO2Emission** in **ascending order of CO2Emission**.

(d) Display the details of vehicles whose **CO2Emission** is greater than **150 g/km**.

(e) Count the number of vehicles that use **Diesel** as the fuel type.

(f) Insert the following record into the **CO2_EMISSION** table.

108, 'MG', 'Hector', 'Petrol', 1.5, 160

(g) Write the output of the following SQL commands.

(i) SELECT MAX(CO2Emission)FROM CO2_EMISSION;
(ii) SELECT MIN(EngineSize)FROM CO2_EMISSION;
(iii) SELECT AVG(CO2Emission)FROM CO2_EMISSIONWHERE FuelType='Petrol'; **(iv)** SELECT COUNT(*)FROM CO2_EMISSIONWHERE EngineSize>=1.5;

Aim

To write and execute SQL queries on the **CO2_EMISSION** table to retrieve and analyze vehicle information based on fuel type, engine size, and CO_2 emission levels using filtering, sorting, counting, and aggregate functions.

Description

This experiment demonstrates the use of SQL commands to:

- Display details of vehicles using **Petrol** as fuel.
- Display the company and model of vehicles with an engine size greater than **1.5 litres**.
- Display vehicle details in ascending order of **CO_2 emission**.
- Display vehicles having CO_2 emissions greater than **150 g/km**.
- Count the number of vehicles using **Diesel**.
- Insert a new vehicle record into the CO2_EMISSION table.

- Find the maximum CO_2 emission.
- Find the minimum engine size.
- Calculate the average CO_2 emission of Petrol vehicles.
- Count vehicles with engine size greater than or equal to **1.5 litres**.

Software Used

Oracle Database / Oracle SQL Plus (SQL Command Line)

1. Create CO2_EMISSION Table

```
CREATE TABLE CO2_EMISSION (
   VehicleID NUMBER(3),
   Company VARCHAR2(20),
   Model VARCHAR2(30),
   FuelType VARCHAR2(10),
   EngineSize NUMBER(3,1),
   CO2Emission NUMBER(5)
);
```

2. Insert Records

```
INSERT INTO CO2_EMISSION VALUES
(101, 'Maruti', 'Swift', 'Petrol', 1.2, 113);
INSERT INTO CO2_EMISSION VALUES
(102, 'Hyundai', 'Creta', 'Diesel', 1.5, 149);
INSERT INTO CO2_EMISSION VALUES
(103, 'Tata', 'Nexon', 'Petrol', 1.2, 135);
INSERT INTO CO2_EMISSION VALUES
(104, 'Toyota', 'Innova', 'Diesel', 2.4, 178);
INSERT INTO CO2_EMISSION VALUES
(105, 'Honda', 'City', 'Petrol', 1.5, 145);
INSERT INTO CO2_EMISSION VALUES
(106, 'Mahindra', 'XUV500', 'Diesel', 2.2, 190);
INSERT INTO CO2_EMISSION VALUES
(107, 'Kia', 'Seltos', 'Petrol', 1.5, 155);
INSERT INTO CO2_EMISSION VALUES
(108, 'MG', 'Hector', 'Petrol', 1.5, 160);
COMMIT;
```

Results Explain Describe Saved SQL History

```
INSERT INTO CO2_EMISSION VALUES
  (101, 'Maruti', 'Swift', 'Petrol', 1.2, 113);
INSERT INTO CO2_EMISSION VALUES
  (102, 'Hyundai', 'Creta', 'Diesel', 1.5, 149);
INSERT INTO CO2_EMISSION VALUES
  (103, 'Tata', 'Nexon', 'Petrol', 1.2, 135);
INSERT INTO CO2_EMISSION VALUES
  (104, 'Toyota', 'Innova', 'Diesel', 2.4, 178);
INSERT INTO CO2_EMISSION VALUES
  (105, 'Honda', 'City', 'Petrol', 1.5, 145);
INSERT INTO CO2_EMISSION VALUES
  (106, 'Mahindra', 'XUV500', 'Diesel', 2.2, 190);
INSERT INTO CO2_EMISSION VALUES
  (107, 'Kia', 'Seltos', 'Petrol', 1.5, 155);
INSERT INTO CO2_EMISSION VALUES
  (108, 'MG', 'Hector', 'Petrol', 1.5, 160);
COMMIT;
```

```
8 rows created.
Commit complete.
```

8 rows inserted. CSV Export

(a) Petrol vehicles

SELECT *

FROM CO2_EMISSION

WHERE FuelType = 'Petrol';

Output:

Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	Maruti	Swift	Petrol	1.2	113
103	Tata	Nexon	Petrol	1.2	135
105	Honda	City	Petrol	1.5	145
107	Kia	Seltos	Petrol	1.5	155
108	MG	Hector	Petrol	1.5	160

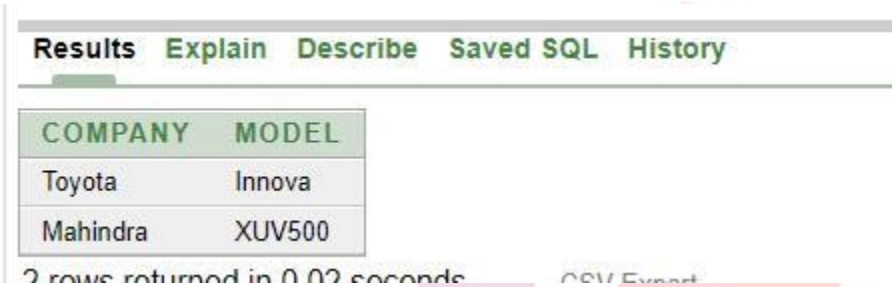
5 rows returned in 0.01 seconds CSV Export

(b) Company and Model where EngineSize > 1.5

```
SELECT Company, Model
FROM CO2_EMISSION

WHERE EngineSize > 1.5;
```

Output:



(c) Company, Model and CO2Emission in ascending order

```
SELECT Company, Model, CO2Emission

FROM CO2_EMISSION

ORDER BY CO2Emission ASC;
```

Output:

Results Explain Describe Saved SQL History

COMPANY	MODEL	CO2EMISSION
Maruti	Swift	113
Tata	Nexon	135
Honda	City	145
Hyundai	Creta	149
Kia	Seltos	155
MG	Hector	160
Toyota	Innova	178
Mahindra	XUV500	190

8 rows returned in 0.00 seconds CSV Export

(d) Vehicles whose CO2Emission > 150 g/km

```
SELECT *

FROM CO2_EMISSION

WHERE CO2Emission > 150;
```

Output:

Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
104	Toyota	Innova	Diesel	2.4	178
106	Mahindra	XUV500	Diesel	2.2	190
107	Kia	Seltos	Petrol	1.5	155
108	MG	Hector	Petrol	1.5	160

4 rows returned in 0.01 seconds CSV Export

(e) Count Diesel vehicles

```
SELECT COUNT(*)
```

FROM CO2_EMISSION

WHERE FuelType = 'Diesel';

Output:



(f) Insert the given record

INSERT INTO CO2_EMISSION VALUES

(108, 'MG', 'Hector', 'Petrol', 1.5, 160);

COMMIT;

(g) Outputs
(i)SELECT MAX(CO2Emission)

FROM CO2_EMISSION;

Output:



(ii)

SELECT MIN(EngineSize)

FROM CO2_EMISSION;

Output:

Results Explain Describe Saved SQL History

MIN(ENGINESIZE)
1.2

rows returned in 0.00 seconds CSV Export

(iii)

SELECT AVG(CO2Emission)

FROM CO2_EMISSION

WHERE FuelType = 'Petrol';

Output:

Results Explain Describe Saved SQL History

AVG(CO2EMISSION)
144.66666666666666666666666666666666666667

1 rows returned in 0.00 seconds CSV Export

(iv)

SELECT COUNT(*)

FROM CO2_EMISSION

WHERE EngineSize >= 1.5;

output



3. APPLICANTS and COURSES

PRACTICE NO:03 SQL Queries on APPLICANTS and COURSES Tables

DATE: 12/08/2026

3. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables APPLICANTS and COURSES.

(a) To display name, fee, gender, joinyear about the applicants, who have joined before 2010.

(b) To display the names of applicants, who are paying fee more than 30000.

(c) To display name of all applicants in ascending order of their joinyear.

(d) To display the year and the total number of applicants joined in each YEAR from the table APPLICANTS.
(e) To display the C_ID (i.e. Course ID) and the number of applicants registered in the course from the APPLICANTS table.

(f) To display the applicant's name with their respective course's name from the tables APPLICANTS and COURSES.

(g) Give the output of following SQL statements:

(g1) SELECT NAME, JOIN YEAR FROM APPLICANTS WHERE GENDER= 'F' AND C_ID= '02';

(g2) SELECT MIN(JOINYEAR) FROM APPLICANTS WHERE Gender= 'M';

(g3) SELE CT AVG(FEE) FROM APPLICANTS WHERE C_ID= 'A01' OR C_ID= 'A05';

(g4) SELECT SUM (FEE), C_ID FROM APPLICATIONS GROUP BY C_ID HAVING COUNT (*)=2;

Aim

To write and execute SQL queries on the **APPLICANTS** and **COURSES** tables to retrieve and analyze applicant and course information using filtering, sorting, grouping, aggregate functions, and table joins.

Description

This experiment demonstrates the use of SQL commands to:

- Display applicant details based on their joining year.
- Display applicants paying a fee greater than **30000**.
- Display applicants in ascending order of their joining year.
- Find the number of applicants who joined in each year.
- Count the number of applicants registered for each course.
- Display applicants along with their respective course names.
- Use aggregate functions such as **MIN, AVG, and SUM**.
- Use **GROUP BY** and **HAVING** clauses to analyze applicant data.

Software Used

Oracle Database / Oracle SQL Plus (SQL Command Line)

1. Create APPLICANTS table

```
CREATE TABLE APPLICANTS (
   A_ID VARCHAR(5),
   NAME VARCHAR(30),
   GENDER CHAR(1),
   JOINYEAR INT,
   FEE INT,
   C_ID VARCHAR(5)
);
```

2. Insert records into APPLICANTS

```
INSERT INTO APPLICANTS VALUES
('A01', 'Anu', 'F', 2009, 20000, '02');
INSERT INTO APPLICANTS VALUES
('A02', 'Bala', 'M', 2008, 35000, 'A01');
INSERT INTO APPLICANTS VALUES
('A03', 'Cathy', 'F', 2011, 32000, '02');
INSERT INTO APPLICANTS VALUES
('A04', 'Deepa', 'F', 2007, 40000, 'A05');
INSERT INTO APPLICANTS VALUES
('A05', 'Ezhil', 'M', 2012, 30000, 'A01');
```

```
INSERT INTO APPLICANTS VALUES
('A06', 'Farah', 'F', 2009, 36000, 'A05');
INSERT INTO APPLICANTS VALUES
('A07', 'Gokul', 'M', 2010, 28000, 'A05');
INSERT INTO APPLICANTS VALUES
('A08', 'Hari', 'M', 2006, 45000, 'A02');
```

Results | Explain | Describe | Saved SQL | History

A_ID	NAME	GENDER	JOINYEAR	FEE	C_ID
A02	Bala	M	2008	35000	A01
A03	Cathy	F	2011	32000	02
A04	Deepa	F	2007	40000	A05
A05	Ezhil	M	2012	30000	A01
A06	Farah	F	2009	36000	A05
A07	Gokul	M	2010	28000	A05
A08	Hari	M	2006	45000	A02

7 rows returned in 0.02 seconds CSV Export

3. Create COURSES table

```
CREATE TABLE COURSES (
   C_ID VARCHAR(5),
   C_NAME VARCHAR(30)
);
```

Insert records

```
INSERT INTO COURSES VALUES
('02', 'Computer Science');
INSERT INTO COURSES VALUES
('A01', 'Python Programming');
INSERT INTO COURSES VALUES
('A02', 'Web Development');
INSERT INTO COURSES VALUES
('A05', 'Data Science');
```

Results Explain Describe Saved SQL History

C_ID	C_NAME
02	Computer Science
A01	Python Programming
A02	Web Development
A05	Data Science

4 rows returned in 0.01 seconds CSV Export

(a) Display name, fee, gender and joinyear of applicants who joined before 2010

```
SELECT NAME, FEE, GENDER, JOINYEAR
FROM APPLICANTS
WHERE JOINYEAR < 2010;
```

Output

Results Explain Describe Saved SQL History

NAME	FEE	GENDER	JOINYEAR
Bala	35000	M	2008
Deepa	40000	F	2007
Farah	36000	F	2009
Hari	45000	M	2006

4 rows returned in 0.00 seconds CSV Export

(b) Display names of applicants paying fee more than 30000

```
SELECT NAME
FROM APPLICANTS
WHERE FEE > 30000;
```

Output

Results | Explain | Describe | Saved SQL | History

NAME
Bala
Cathy
Deepa
Farah
Hari

5 rows returned in 0.00 seconds CSV Export

(c) Display name of all applicants in ascending order of JOINYEAR

```
SELECT NAME, JOINYEAR
FROM APPLICANTS
ORDER BY JOINYEAR ASC;
```

Results | Explain | Describe | Saved SQL | History

NAME	JOINYEAR
Hari	2006
Deepa	2007
Bala	2008
Farah	2009
Gokul	2010
Cathy	2011
Ezhil	2012

7 rows returned in 0.00 seconds CSV Export

(d) Display year and total number of applicants joined in each year

```
SELECT JOINYEAR, COUNT(*)
```

FROM APPLICANTS

GROUP BY JOINYEAR

ORDER BY JOINYEAR;

Output



Results Explain Describe Saved SQL History

JOINYEAR	COUNT(*)
2006	1
2007	1
2008	1
2009	1
2010	1
2011	1
2012	1

7 rows returned in 0.00 seconds CSV Export

(e) Display C_ID and number of applicants registered in each course

SELECT C_ID, COUNT(*)
FROM APPLICANTS
GROUP BY C_ID;

Output



Results Explain Describe Saved SQL History

C_ID	COUNT(*)
02	1
A02	1
A05	3
A01	2

4 rows returned in 0.02 seconds CSV Export

(F)Display applicant's name with respective course name

```
SELECT APPLICANTS.NAME, COURSES.C_NAME
FROM APPLICANTS
JOIN COURSES
ON APPLICANTS.C_ID = COURSES.C_ID;
```

Outputs

Results | Explain | Describe | Saved SQL | Histo

NAME	C_NAME
Bala	Python Programming
Cathy	Computer Science
Deepa	Data Science
Ezhil	Python Programming
Farah	Data Science
Gokul	Data Science
Hari	Web Development

7 rows returned in 0.00 seconds CSV Export

(g1)

```
SELECT NAME, JOINYEAR
FROM APPLICANTS
WHERE GENDER = 'F' AND C_ID = '02';
```

Output

Results Explain Describe Saved SQL History

NAME	JOINYEAR
Cathy	2011

1 rows returned in 0.01 seconds CSV Export

(g2)

SELECT MIN(JOINYEAR)

FROM APPLICANTS

WHERE GENDER = 'M';

Output

Results Explain Describe Saved SQL History

MIN(JOINYEAR)
2006

1 rows returned in 0.01 seconds CSV Export

G (3)

SELECT AVG(FEE)

FROM APPLICANTS

WHERE C_ID = '01' OR C_ID = '05';

Output

```
SELECT AVG(FEE)
FROM APPLICANTS
WHERE C_ID = '01' OR C_ID = '05';
```

Results Explain Describe Saved SQL History

AVG(FEE)
-

1 rows returned in 0.00 seconds CSV Export

(g4)

SELECT SUM(FEE), C_ID

FROM APPLICANTS

GROUP BY C_ID

HAVING COUNT(*) = 2;

Output

Results Explain Describe Saved SQL History

SUM(FEE)	C_ID
65000	A01

1 rows returned in 0.02 seconds CSV Export

4. PRODUCTS AND SUPPLIERS

PRACTICE NO:04 SQL Queries on PRODUCTS and SUPPLIERS Tables

DATE: 18/08/2026

4. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables **PRODUCTS AND SUPPLIERS**

(a) To display the details of all the products in ascending order of product names (i.e. PNAME).

(b) To display product name and price of all those products, whose price is in the range of 10000 and 15000 (both values inclusive).

(c) To display the number of products which are supplied by each supplier i.e. the expected output should be

S01 2

S02 2

S03 1

(d) To display the price, product name (i.e. PName) and quantity (i.e. QTY) of those which have quantity more than 100.

(e) To display the names of those suppliers, who are either from DELHI or from CHENNAI.
(f) To display the name of the companies and the name of the products in descending order of company names.

(g) Obtain the outputs of the following SQL queries based on the data given in tables PRODUCTS and SUPPLIERS.

(g1) SELECT DISTINCT SUPCODE FROM PRODUCTS;

(g2) SELECT MAX(PRICE), MIN (PRICE) FROM PRODUCTS;

(g3) SELECT PRICE * QTY AMOUNT FROM PRODUCTS WHERE PID = 104;

(g4) SELECT PNAME, SNAME FROM PRODUCTS P, SUPPLIERS S WHERE P. SUPCODE = S. SUPCODE AND QTY>100;

1. Create PRODUCTS table

CREATE TABLE PRODUCTS (

```
    PID INT,
    PNAME VARCHAR(30),
    QTY INT,
    PRICE INT,
    COMPANY VARCHAR(30),
    SUPCODE VARCHAR(5)
);
```

2. Insert records into PRODUCTS

```
INSERT INTO PRODUCTS VALUES
(101, 'DIGITAL CAMERA 14X', 120, 12000, 'RENIX', 'S01');
INSERT INTO PRODUCTS VALUES
(102, 'DIGITAL PAD 11i', 100, 22000, 'DIGI POP', 'S02');
INSERT INTO PRODUCTS VALUES
(104, 'PEN DRIVE 16 GB', 500, 1100, 'STOREKING', 'S01');
INSERT INTO PRODUCTS VALUES
(106, 'LED SCREEN', 70, 28000, 'DISEXPERTS', 'S02');
INSERT INTO PRODUCTS VALUES
(105, 'CAR GPS SYSTEM', 60, 12000, 'MOVEON', 'S03');
```

PRODUCTS Table

Results Explain Describe Saved SQL History

PID	PNAME	PRICE	QTY	SUPCODE
101	Laptop	45000	50	S01
102	Mobile	12000	150	S01
103	Keyboard	1500	200	S02
104	Printer	10000	120	S02
105	Monitor	14000	80	S03

5 rows returned in 0.00 seconds CSV Export

3. Create SUPPLIERS table

```
CREATE TABLE SUPPLIERS (
    SUPCODE VARCHAR(5),
```

```
   SNAME VARCHAR(30),
   CITY VARCHAR(20)
);
```

Insert records into SUPPLIERS

```
INSERT INTO SUPPLIERS VALUES
('S01', 'GET ALL INC', 'KOLKATA');
INSERT INTO SUPPLIERS VALUES
('S02', 'DIGI BUSY GROUP', 'CHENNAI');
INSERT INTO SUPPLIERS VALUES
('S03', 'EASY MARKET CORP', 'DELHI');
```

SUPPLIERS Table

Results Explain Describe Saved SQL History

SUPCODE	SNAME	CITY	COMPANY
S01	Raj Traders	DELHI	ABC Electronics
S02	Kumar Suppliers	CHENNAI	XYZ Technologies
S03	Meena Enterprises	MUMBAI	PQR Systems

3 rows returned in 0.00 seconds CSV Export

(a) Display all products in ascending order of PNAME

SQL Query

```
SELECT *

FROM PRODUCTS

ORDER BY PNAME ASC;
```

Output



Results Explain Describe Saved SQL History

PID	PNAME	PRICE	QTY	SUPCODE
103	Keyboard	1500	200	S02
101	Laptop	45000	50	S01
102	Mobile	12000	150	S01
105	Monitor	14000	80	S03
104	Printer	10000	120	S02

5 rows returned in 0.00 seconds CSV Export

(B)Display PNAME and PRICE between 10000 and 15000

SELECT PNAME, PRICE

FROM PRODUCTS

WHERE PRICE BETWEEN 10000 AND 15000;



Results Explain Describe Saved SQL History

PNAME	PRICE
Mobile	12000
Printer	10000
Monitor	14000

3 rows returned in 0.00 seconds CSV Export

(c) Display number of products supplied by each supplier

SELECT SUPCODE, COUNT(*)

FROM PRODUCTS

GROUP BY SUPCODE;

Results Explain Describe Saved SQL History

SUPCODE	COUNT(*)
S01	2
S02	2
S03	1

3 rows returned in 0.00 seconds CSV Export

(d) Display PRICE, PNAME and QTY where QTY > 100

SELECT PRICE, PNAME, QTY

FROM PRODUCTS

WHERE QTY > 100;

Results Explain Describe Saved SQL History

PRICE	PNAME	QTY
12000	Mobile	150
1500	Keyboard	200
10000	Printer	120

3 rows returned in 0.00 seconds CSV Export

(e) Display suppliers from DELHI or CHENNAI

SELECT SNAME

FROM SUPPLIERS

WHERE CITY = 'DELHI' OR CITY = 'CHENNAI';

Results Explain Describe Saved SQL History

SNAME
Raj Traders
Kumar Suppliers

2 rows returned in 0.00 seconds CSV Export

(f) Display company and product names in descending order of company

SQL Query

```
SELECT S.COMPANY, P.PNAME

FROM PRODUCTS P, SUPPLIERS S

WHERE P.SUPCODE = S.SUPCODE
ORDER BY S.COMPANY DESC;
```

Output

Results Explain Describe Saved SQL History

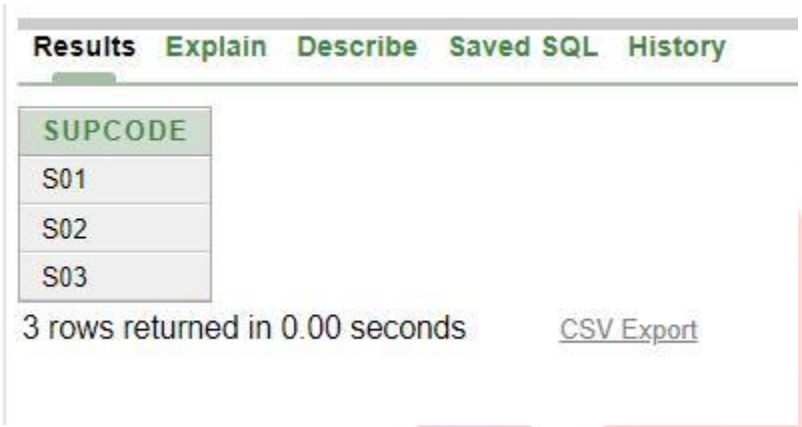
COMPANY	PNAME
XYZ Technologies	Keyboard
XYZ Technologies	Printer
PQR Systems	Monitor
ABC Electronics	Mobile
ABC Electronics	Laptop

5 rows returned in 0.00 seconds CSV Export

(g1)

SELECT DISTINCT SUPCODE

FROM PRODUCTS;



g(2)Query

SELECT MAX(PRICE), MIN(PRICE)
FROM PRODUCTS;

Output



(g3)

Query

SELECT PRICE * QTY AS AMOUNT

FROM PRODUCTS

WHERE PID = 104;

Results Explain Describe Saved SQL History

AMOUNT
1200000

1 rows returned in 0.01 seconds CSV Export

(g4)

Query

SELECT PNAME, SNAME

FROM PRODUCTS P, SUPPLIERS S

WHERE P.SUPCODE = S.SUPCODE

AND QTY > 100;

Results Explain Describe Saved SQL History

PNAME	SNAME
Mobile	Raj Traders
Printer	Kumar Suppliers
Keyboard	Kumar Suppliers

3 rows returned in 0.00 seconds CSV Export

PRACTICE NO:05 SQL Queries on SHOPPE and ACCESSORIES Tables

DATE: 21/08/2026

5. Answer the questions (a) and (b) on the basis of the following tables SHOPPE and ACCESSORIES

(a) Write the SQL queries:

(i) To display Name and Price of all the accessories in ascending order of their Price.

(ii) To display Id and SName of all Shoppe in Nehru Place.

(iii) To display Minimum and Maximum Price of each Name of accessories.

(iv) To display Name, Price of all accessories and their respective SName where they are available.

(b) (i) SELECT DISTINCT Name FROM ACCESSORIES WHERE Price>=500;

(ii) SELECT Area, COUNT (*) FROM GROUP BY Area;

(iii) SELECT COUNT (DISTINCT Area) FROM SHOPPE;

(iv) SELECT Name, Price*0.05 DISCOUNT FROM ACCESSORIES WHERE SNo IN (‘S02, ‘S03’);

5. SHOPPE and ACCESSORIES

Aim

To write and execute SQL queries on the **SHOPPE** and **ACCESSORIES** tables to retrieve and analyze accessory and shop information using sorting, filtering, aggregate functions, and table joins.

Description

This experiment demonstrates the use of SQL commands to:

- Display accessory names and prices in ascending order.
- Display shops located in **Nehru Place**.
- Find the minimum and maximum price for each accessory.
- Display accessories with their respective shop names.

- Find distinct accessories based on price.
- Count shops in each area.
- Find the number of different areas.
- Calculate a 5% discount on selected accessories.

Software Used

Oracle Database / Oracle SQL Plus (SQL Command Line)

1. Create SHOPPE table

```
CREATE TABLE SHOPPE (
   ID VARCHAR(5),
   SNAME VARCHAR(30),
   AREA VARCHAR(20)
);
```

2. Insert records into SHOPPE

```
INSERT INTO SHOPPE VALUES
('S01', 'ABC Computeronics', 'CP');
INSERT INTO SHOPPE VALUES
('S02', 'All Infotech Media', 'GK II');
INSERT INTO SHOPPE VALUES
('S03', 'Tech Shoppe', 'CP');
INSERT INTO SHOPPE VALUES
('S04', 'Geeks Techno Soft', 'Nehru Place');
INSERT INTO SHOPPE VALUES
('S05', 'Hitech Tech Store', 'Nehru Place');
```

SHOPPE Table

3. Create ACCESSORIES table

```
CREATE TABLE ACCESSORIES (
   NO VARCHAR(5),
   NAME VARCHAR(30),
   PRICE INT,
   ID VARCHAR(5)
);
```

4. Insert records into ACCESSORIES

```
INSERT INTO ACCESSORIES VALUES
('A01', 'Mother Board', 12000, 'S01');
INSERT INTO ACCESSORIES VALUES
('A02', 'Hard Disk', 5000, 'S01');
INSERT INTO ACCESSORIES VALUES
('A03', 'Keyboard', 500, 'S02');
INSERT INTO ACCESSORIES VALUES
('A04', 'Mouse', 300, 'S01');
INSERT INTO ACCESSORIES VALUES
('A05', 'Mother Board', 13000, 'S02');
INSERT INTO ACCESSORIES VALUES
('A06', 'Keyboard', 400, 'S03');
INSERT INTO ACCESSORIES VALUES
('A07', 'LCD', 6000, 'S04');
INSERT INTO ACCESSORIES VALUES
('T08', 'LCD', 5500, 'S05');
INSERT INTO ACCESSORIES VALUES
('T09', 'Mouse', 350, 'S05');
INSERT INTO ACCESSORIES VALUES
('T10', 'Hard Disk', 4500, 'S03');
```

ACCESSORIES Table

Results Explain Describe Saved SQL History

NO	NAME	PRICE	ID
101	Keyboard	600	S01
102	Mouse	400	S02
103	Printer	5000	S03
104	Webcam	800	S02
105	Headphone	1200	S03
106	Keyboard	750	S01

6 rows returned in 0.00 seconds CSV Export

(i) Display Name and Price in ascending order of Price

```
SELECT Name, Price
```

FROM ACCESSORIES

ORDER BY Price ASC;

NAME	PRICE
Mouse	400
Keyboard	600
Keyboard	750
Webcam	800
Headphone	1200
Printer	5000

6 rows returned in 0.00 seconds CSV Export

(ii) Display ID and SName of all Shoppe in Nehru Place

SELECT ID, SNAME
FROM SHOPPE
WHERE AREA = 'Nehru Place';

Output

Results Explain Describe Saved SQL History

SNO	SNAME
S01	ABC Electronics
S04	Computer Zone

2 rows returned in 0.00 seconds CSV Export

3.Display Minimum and Maximum Price of each Name

SELECT Name, MIN(Price), MAX(Price)

FROM ACCESSORIES

GROUP BY Name;

Output

```
FROM ACCESSORIES
GROUP BY Name;
```

Results Explain Describe Saved SQL History

NAME	MIN(PRICE)	MAX(PRICE)
Printer	5000	5000
Mouse	400	400
Keyboard	600	750
Headphone	1200	1200
Webcam	800	800

5 rows returned in 0.00 seconds CSV Export

(iv) Display Name, Price and respective SName

SELECT A.NAME, A.PRICE, S.SNAME
FROM ACCESSORIES A, SHOPPE S
WHERE A.ID = S.ID;

Output

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **SHOPPE**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
SHOPPE	SNO	Varchar2	5	-	-	-	✓	-	-
	SNAME	Varchar2	30	-	-	-	✓	-	-
	AREA	Varchar2	30	-	-	-	✓	-	-

1 - 3

(b) Outputs

(i)

SELECT DISTINCT Name

FROM ACCESSORIES

WHERE Price >= 500;

Results Explain Describe Saved SQL History

NAME
Printer
Keyboard
Headphone
Webcam

4 rows returned in 0.00 seconds CSV Export

(ii)

SELECT Area, COUNT(*)

FROM SHOPPE

GROUP BY Area;

```
FROM SHOPPE
GROUP BY Area;
```

Results Explain Describe Saved SQL History

AREA	COUNT(*)
Delhi	1
Nehru Place	2
Chennai	1

3 rows returned in 0.00 seconds CSV Export

(iii)

SELECT COUNT(DISTINCT Area)

FROM SHOPPE;

```
FROM SHOPPE;
```

Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
3

1 rows returned in 0.01 seconds CSV Export

(iv)

DESC ACCESSORIES;

SELECT NAME, PRICE * 0.05 AS DISCOUNT

FROM ACCESSORIES

WHERE SNO IN ('S02', 'S03');

SELECT * FROM ACCESSORIES;

Results Explain Describe Saved SQL History

NO	NAME	PRICE	ID
101	Keyboard	600	S01
102	Mouse	400	S02
103	Printer	5000	S03
104	Webcam	800	S02
105	Headphone	1200	S03
106	Keyboard	750	S01

6 rows returned in 0.00 seconds CSV Export

PRACTICE NO:06 SQL Queries on HOSPITAL Table

DATE: 28/08/2026

6. Write SQL commands for (a) to (f) and Write the outputs for (g) on the basis of table HOSPITAL
(a) To select all the information of patients of cardiology department.

(b) To list the names of female patients who are in ENT department.

(c) To list name of all patients with their date of admission in ascending order.

(d) To display Patient's Name, Charges, Age for only female patients.

(e) To count the number of patients with Age<30.

(f) To inset in a new row in the HOSPITAL table with the following data: 11, "Aftab", 24, "Surgery", {25/02/98}, 300, "M"

(g) Give the output of following SQL statements:

(i) Select COUNT (DISTINCT charges) from HOSPITAL;

(ii) Select MIN (Age) from HOSPITAL where Gender = "F"

(iii) Select SUM (Charges) from HOSPITAL where Department = "ENT"

(iv) Select AVG (Charges) from HOSPITAL where Datofadm< {12/08/98}

6. HOSPITAL Table

Aim

To create and execute SQL queries on the **HOSPITAL** table to retrieve and analyze patient information using filtering, sorting, aggregate functions, and data insertion.

Description

This experiment demonstrates the use of SQL commands to:

- Display patients belonging to the **Cardiology** department.
- Display female patients from the **ENT** department.
- Display patient names with their admission dates in ascending order.
- Display patient name, charges, and age of female patients.

- Count patients whose age is less than **30**.
- Insert a new patient record into the HOSPITAL table.
- Find the number of distinct charges.
- Find the minimum age of female patients.
- Find the total charges for ENT patients.
- Calculate the average charges for patients admitted before a specified date.

Software Used

Oracle Database / Oracle SQL Plus (SQL Command Line)

1. Create the table

```
CREATE TABLE HOSPITAL (
   NO INT,
   NAME VARCHAR(20),
   AGE INT,
   DEPARTMENT VARCHAR(30),
   DATEOFADM DATE,
   CHARGES INT,
   SEX CHAR(1)
);
```

2. Insert the records

```
INSERT INTO PATIENT VALUES

(1, 'Amit', 35, 'Cardiology',

TO_DATE('10/01/1998','DD/MM/YYYY'), 500, 'M');


INSERT INTO PATIENT VALUES

(2, 'Sana', 28, 'ENT',

TO_DATE('15/02/1998','DD/MM/YYYY'), 300, 'F');


INSERT INTO PATIENT VALUES
```

(3, 'Rahul', 42, 'Surgery',

TO_DATE('20/03/1998','DD/MM/YYYY'), 700, 'M');

INSERT INTO PATIENT VALUES

(4, 'Priya', 25, 'Cardiology',
TO_DATE('05/04/1998','DD/MM/YYYY'), 600, 'F');

INSERT INTO PATIENT VALUES

(5, 'Kiran', 31, 'ENT',

TO_DATE('12/05/1998','DD/MM/YYYY'), 400, 'M');

INSERT INTO PATIENT VALUES

(6, 'Meena', 22, 'ENT',

TO_DATE('25/05/1998','DD/MM/YYYY'), 350, 'F');

INSERT INTO PATIENT VALUES

(7, 'Arun', 29, 'Cardiology',

TO_DATE('10/06/1998','DD/MM/YYYY'), 550, 'M');

INSERT INTO PATIENT VALUES

(8, 'Divya', 27, 'Surgery',

TO_DATE('15/07/1998','DD/MM/YYYY'), 450, 'F');

INSERT INTO PATIENT VALUES

(9, 'Vijay', 45, 'ENT',

TO_DATE('05/08/1998','DD/MM/YYYY'), 500, 'M');

INSERT INTO PATIENT VALUES

(10, 'Neha', 26, 'Cardiology',

TO_DATE('10/08/1998','DD/MM/YYYY'), 600, 'F');
SELECT * FROM PATIENT;

Results Explain Describe Saved SQL History

PID	PNAME	AGE	DEPARTMENT	DATEOFADM	CHARGES	GENDER
1	Amit	35	Cardiology	10-JAN-98	500	M
2	Sana	28	ENT	15-FEB-98	300	F
3	Rahul	42	Surgery	20-MAR-98	700	M
4	Priya	25	Cardiology	05-APR-98	600	F
5	Kiran	31	ENT	12-MAY-98	400	M
6	Meena	22	ENT	25-MAY-98	350	F
7	Arun	29	Cardiology	10-JUN-98	550	M
8	Divya	27	Surgery	15-JUL-98	450	F
9	Vijay	45	ENT	05-AUG-98	500	M
10	Neha	26	Cardiology	10-AUG-98	600	F

10 rows returned in 0.00 seconds CSV Export

(a) Display all information of patients in Cardiology

SELECT *

FROM HOSPITAL

WHERE Department = 'Cardiology';

Results Explain Describe Saved SQL History

PID	PNAME	AGE	DEPARTMENT	DATEOFADM	CHARGES	GENDER
1	Amit	35	Cardiology	10-JAN-98	500	M
4	Priya	25	Cardiology	05-APR-98	600	F
7	Arun	29	Cardiology	10-JUN-98	550	M
10	Neha	26	Cardiology	10-AUG-98	600	F

4 rows returned in 0.00 seconds CSV Export

(b) Display names of female patients in ENT

SQL Query

```
SELECT PNAME
FROM PATIENT
WHERE GENDER = 'F'
AND DEPARTMENT = 'ENT';
```

Output

Results Explain Describe Saved SQL History

PNAME
Sana
Meena

2 rows returned in 0.00 seconds CSV Export

(c) Display names with date of admission in ascending order

```
SELECT PNAME, DATEOFADM

FROM PATIENT

ORDER BY DATEOFADM ASC;
```

Results Explain Describe Saved SQL History

PNAME	DATEOFADM
Amit	10-JAN-98
Sana	15-FEB-98
Rahul	20-MAR-98
Priya	05-APR-98
Kiran	12-MAY-98
Meena	25-MAY-98
Arun	10-JUN-98
Divya	15-JUL-98
Vijay	05-AUG-98
Neha	10-AUG-98

10 rows returned in 0.00 seconds CSV Export

(e) Display Name, Charges and Age of female patients

```
SELECT PNAME, CHARGES, AGE

FROM PATIENT

WHERE GENDER = 'F';
```

Output

Results Explain Describe Saved SQL History

PNAME	CHARGES	AGE
Sana	300	28
Priya	600	25
Meena	350	22
Divya	450	27
Neha	600	26

5 rows returned in 0.01 seconds CSV Export

(f) Count patients whose Age < 30

```
SELECT COUNT(*)
```

FROM PATIENT

WHERE AGE < 30;

Output

Results Explain Describe Saved SQL History

PID	PNAME	AGE	DEPARTMENT	DATEOFADM	CHARGES	GENDER
1	Amit	35	Cardiology	10-JAN-98	500	M
2	Sana	28	ENT	15-FEB-98	300	F
3	Rahul	42	Surgery	20-MAR-98	700	M
4	Priya	25	Cardiology	05-APR-98	600	F
5	Kiran	31	ENT	12-MAY-98	400	M
6	Meena	22	ENT	25-MAY-98	350	F
7	Arun	29	Cardiology	10-JUN-98	550	M
8	Divya	27	Surgery	15-JUL-98	450	F
9	Vijay	45	ENT	05-AUG-98	500	M
10	Neha	26	Cardiology	10-AUG-98	600	F

More than 10 rows available. Increase rows selector to view more rows.

10 rows returned in 0.00 seconds CSV Export

(g) Outputs

(i) COUNT(DISTINCT CHARGES)

SELECT COUNT(DISTINCT CHARGES)

FROM PATIENT;

Output

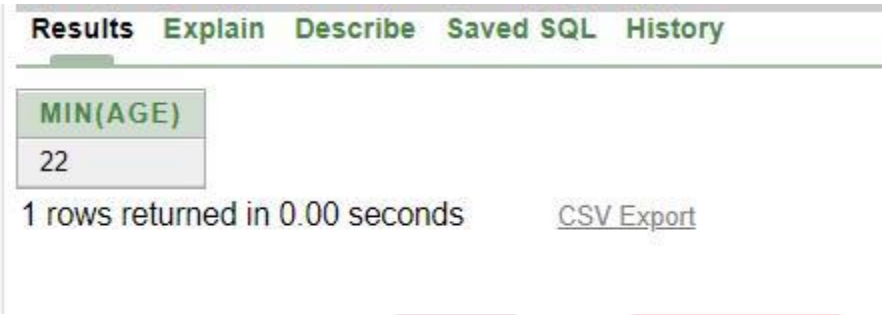
Results Explain Describe Saved SQL History

COUNT(DISTINCTCHARGES)
8

1 rows returned in 0.00 seconds CSV Export

2. age of female patients

```
SELECT MIN(AGE)
FROM PATIENT
WHERE GENDER = 'F';
```



(iii) Sum of charges of ENT patients

```
SELECT SUM(CHARGES)
FROM PATIENT
WHERE DEPARTMENT = 'ENT';
```



(iv) Average charges where DATEOFADM < 12/02/98

Results Explain Describe Saved SQL History

AVG(CHARGES)
477.27272727272727272727272727272727

1 rows returned in 0.02 seconds CSV Export

PRACTICE NO:07 SQL Queries on STORE and ITEM Tables

DATE: 01/09/2026

7. Answer the questions (a) and (b) on the basis of the following tables **STORE** and **ITEM**

(a) Write the SQL queries (i) to (iv):

(i) To display IName and Price of all the items in ascending order of their Price.

(ii) To display SNo and SName of all store location in CP.

(iii) To display Minimum and maximum Price of each IName from the table ITEM.

(iv) To display IName, Price of all items and their respective SName where they are available.

(b) Write the output of the following SQL commands (i) to (iv):

(i) SELECT DISTINCT IName FROM ITEM WHERE Price >=5000;

(ii) SELECT Area, COUNT (*) FROM STORE GROUP BY Area;
(iii) SELECT COUNT (DISTINCT Area) FROM STORE:

(iv) SELECT IName, Price * 0.05 DISCOUNT FROM ITEM WHERE SNo IN ('S02', 'S03');

7. STORE and ITEM

Aim

To create and execute SQL queries on the **STORE** and **ITEM** tables to retrieve item and store information using sorting, grouping, aggregate functions, filtering, and table joins.

Description

This experiment demonstrates the use of SQL commands to:

- Display item names and prices in ascending order.
- Display store details based on a particular area.
- Find the minimum and maximum price of items.
- Display items along with their respective store names.
- Find distinct items based on price.
- Count stores in each area.

- Count the number of different store areas.
- Calculate a 5% discount on selected items.

Software Used

Oracle Database / Oracle SQL Plus (SQL Command Line)

1. Create STORE table

```
CREATE TABLE STORE (
   SNO VARCHAR(5),
   SNAME VARCHAR(30),
   AREA VARCHAR(20)
);
```

2. Insert records into STORE

```
INSERT INTO STORE VALUES
('S01', 'ABC STORES', 'CP');
INSERT INTO STORE VALUES
('S02', 'ALL MART', 'GK II');
INSERT INTO STORE VALUES
('S03', 'CITY STORE', 'CP');
INSERT INTO STORE VALUES
('S04', 'TECH STORE', 'NEHRU PLACE');
INSERT INTO STORE VALUES
('S05', 'SUPER MART', 'NEHRU PLACE');
```

STORE Table

Results | Explain | Describe | Saved SQL | History

SNO	SNAME	AREA
S01	City Store	CP
S02	Mega Store	Delhi
S03	Super Store	CP
S04	Family Store	Chennai

4 rows returned in 0.02 seconds CSV Export

3. Create ITEM table

```
CREATE TABLE ITEM (
   INO VARCHAR(5),
   INAME VARCHAR(30),
   PRICE INT,
   SNO VARCHAR(5)
);
```

4. Insert records into ITEM

```
INSERT INTO ITEM VALUES
('I01', 'Laptop', 50000, 'S01');
INSERT INTO ITEM VALUES
('I02', 'Hard Disk', 5000, 'S01');
INSERT INTO ITEM VALUES
('I03', 'Keyboard', 500, 'S02');
INSERT INTO ITEM VALUES
('I04', 'Mouse', 300, 'S01');
INSERT INTO ITEM VALUES
('I05', 'Laptop', 55000, 'S02');
INSERT INTO ITEM VALUES
('I06', 'Keyboard', 600, 'S03');
INSERT INTO ITEM VALUES
```

('I07', 'Monitor', 6000, 'S04');
INSERT INTO ITEM VALUES
('I08', 'Monitor', 5500, 'S05');
INSERT INTO ITEM VALUES
('I09', 'Mouse', 400, 'S05');
INSERT INTO ITEM VALUES
('I10', 'Hard Disk', 4500, 'S03');

ITEM Table

Results Explain Describe Saved SQL History

INO	INAME	PRICE	SNO
101	Laptop	55000	S01
102	Mobile	25000	S02
103	Keyboard	1500	S03
104	Monitor	8000	S01
105	Printer	12000	S02
106	Tablet	18000	S03

6 rows returned in 0.02 seconds CSV Export

(a) SQL Queries

(i) Display IName and Price in ascending order of Price

SELECT IName, Price

FROM ITEM

ORDER BY Price ASC;

Results Explain Describe Saved SQL History

INAME	PRICE
Keyboard	1500
Monitor	8000
Printer	12000
Tablet	18000
Mobile	25000
Laptop	55000

6 rows returned in 0.00 seconds CSV Export

(ii) Display SNo and SName of stores located in CP

```
SELECT SNo, SName
FROM STORE
WHERE Area = 'CP';
```

Results Explain Describe Saved SQL History

SNO	SNAME
S01	City Store
S03	Super Store

2 rows returned in 0.00 seconds CSV Export

3. Display minimum and maximum price of each IName

```
SELECT IName, MIN(Price), MAX(Price)
FROM ITEM
GROUP BY IName;
```

Results Explain Describe Saved SQL History

INAME	MIN(PRICE)	MAX(PRICE)
Mobile	25000	25000
Printer	12000	12000
Tablet	18000	18000
Laptop	55000	55000
Keyboard	1500	1500
Monitor	8000	8000

6 rows returned in 0.02 seconds CSV Export

(iv) Display IName, Price and respective SName

SELECT I.IName, I.Price, S.SName

FROM ITEM I, STORE S

WHERE I.SNo = S.SNo;

Output

Results Explain Describe Saved SQL History

INAME	PRICE	SNAME
Laptop	55000	City Store
Mobile	25000	Mega Store
Keyboard	1500	Super Store
Monitor	8000	City Store
Printer	12000	Mega Store
Tablet	18000	Super Store

6 rows returned in 0.00 seconds CSV Export

(b) Outputs

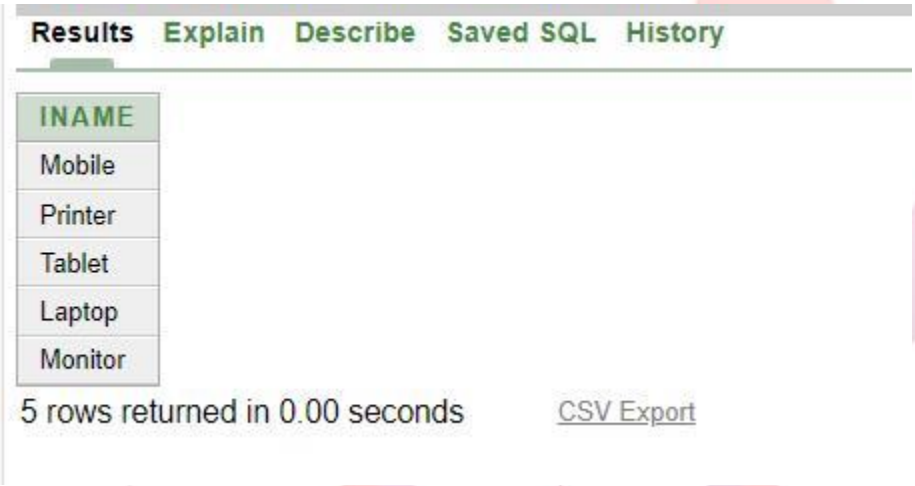
(i) DISTINCT item names where Price >= 5000

Query

```
SELECT DISTINCT IName
FROM ITEM

WHERE Price >= 5000;
```

Output



(ii) Area and number of stores in each area

Query

```
SELECT COUNT(DISTINCT Area)

FROM STORE;
```

Output

Results Explain Describe Saved SQL His

AREA	COUNT(*)
Delhi	1
CP	2
Chennai	1

3 rows returned in 0.01 seconds CSV Exp

(iii) Number of different areas

Query

SELECT COUNT(DISTINCT Area)

FROM STORE;

Output

Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
3

1 rows returned in 0.00 seconds CSV Export

(iv) 5% discount for items in S02 and S03

Query

SELECT IName, Price * 0.05 AS DISCOUNT

FROM ITEM

WHERE SNo IN ('S02', 'S03');

Output

Results | Explain | Describe | Saved SQL | History

INAME	DISCOUNT
Mobile	1250
Keyboard	75
Printer	600
Tablet	900

4 rows returned in 0.00 seconds CSV Export

PRACTICE NO:08 SQL Queries on TEACHER Relation

DATE: 02/09/2026

8. Write SQL commands for (a) to (f) and write output for (g) on the basis of Teacher relation given below:

(a) To show all information about the teacher of history department

(b) To list the names of female teacher who are in Hindi department

(c) To list names of all teachers with their date of joining in ascending order.

(d) To display teacher's Name, Salary, Age for male teacher only

(e) To count the number of teachers with Age>23.

(f) To inset a new row in the TEACHER table with the following data: 9, "Raja", 26, "Computer", {13/05/95}, 2300, "M"

(g) Give the output of following SQL statements:

(i) Select COUNT (distinct department) from TEACHER;

(II) Select MAX (Age) from TEACHER where Gender= "F"

(iii) Select AVG (Salary) from TEACHER where Date of join < {12/07/96};

(iv) Select SUM (Salary) from TEACHER where Date of join < {12/07/96};

8. TEACHER Relation

Aim

To create and execute SQL queries on the **TEACHER** table to retrieve teacher information using filtering, sorting, counting, aggregate functions, and date conditions.

Description

This experiment demonstrates the use of SQL commands to:

- Display teachers belonging to the History department.
- Display female teachers in the Hindi department.
- Display teachers according to their date of joining in ascending order.

- Display the name, salary, and age of male teachers.
- Count teachers whose age is greater than 23.
- Insert a new teacher record into the table.
- Use aggregate functions such as COUNT(), MAX(), AVG(), and SUM().

Software Used

Oracle Database / Oracle SQL Plus (SQL Command Line)

1. Create TEACHER table

```
CREATE TABLE TEACHER (
   TNO INT,
   NAME VARCHAR(20),
   AGE INT,
   DEPARTMENT VARCHAR(20),
   DATEOFJOIN DATE,
   SALARY INT,
   GENDER CHAR(1)
);
```

2. Insert records

```
INSERT INTO TEACHER VALUES
(1, 'Anita', 28, 'History', '1995-05-10', 2500, 'F');
INSERT INTO TEACHER VALUES
(2, 'Bharat', 35, 'Hindi', '1996-01-15', 3000, 'M');
INSERT INTO TEACHER VALUES
(3, 'Geeta', 25, 'Hindi', '1996-03-20', 2800, 'F');
INSERT INTO TEACHER VALUES
(4, 'Kamal', 30, 'History', '1995-08-12', 3200, 'M');
INSERT INTO TEACHER VALUES
(5, 'Meena', 22, 'Computer', '1997-02-10', 2600, 'F');
INSERT INTO TEACHER VALUES
(6, 'Ravi', 24, 'Mathematics', '1996-06-18', 2700, 'M');
INSERT INTO TEACHER VALUES
(7, 'Sita', 26, 'History', '1995-11-25', 2900, 'F');
INSERT INTO TEACHER VALUES
(8, 'Vijay', 21, 'Computer', '1997-04-15', 2400, 'M');
```

Results Explain Describe Saved SQL History

TID	TNAME	AGE	DEPARTMENT	DOJ	SALARY	GENDER
1	Anita	25	History	10-JAN-95	2500	F
2	Bala	30	Hindi	15-MAR-96	3000	M
3	Deepa	28	Hindi	20-APR-95	2800	F
4	Kumar	35	History	12-JUN-94	3500	M
5	Priya	22	Computer	05-AUG-96	2200	F
6	Ravi	27	Mathematics	10-FEB-95	2700	M
7	Meena	24	History	18-MAY-96	2400	F
8	Suresh	32	Computer	25-SEP-94	3200	M

8 rows returned in 0.01 seconds CSV Export

(a) Show all information about teachers of History department

```
SELECT *
FROM TEACHER
WHERE DEPARTMENT = 'History';
```

Output

Results Explain Describe Saved SQL History

TID	TNAME	AGE	DEPARTMENT	DOJ	SALARY	GENDE
1	Anita	25	History	10-JAN-95	2500	F
4	Kumar	35	History	12-JUN-94	3500	M
7	Meena	24	History	18-MAY-96	2400	F

3 rows returned in 0.00 seconds CSV Export

(b) List names of female teachers in Hindi department

```
SELECT TNAME
FROM TEACHER
```

```
WHERE GENDER = 'F'
AND DEPARTMENT = 'Hindi';
```

Output



(c) List names of all teachers with date of joining in ascending order

```
SELECT TNAME, DOJ

FROM TEACHER

ORDER BY DOJ ASC;
```

Output



Results Explain Describe Saved SQL History

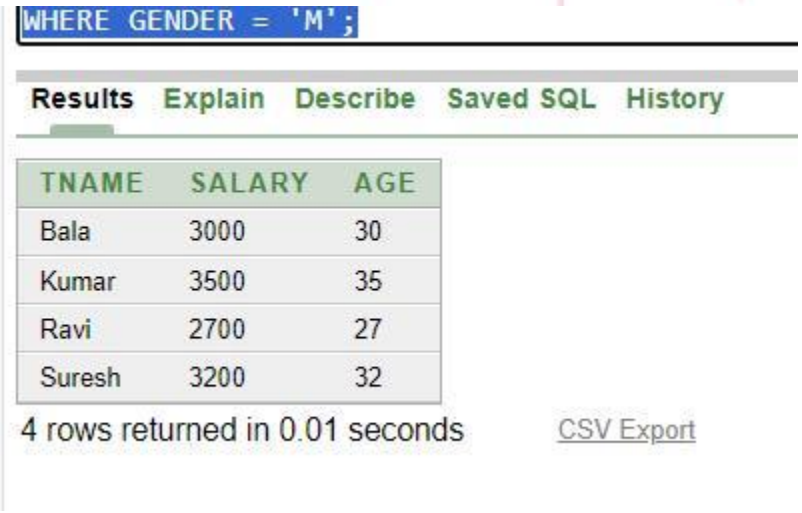
TNAME	DOJ
Kumar	12-JUN-94
Suresh	25-SEP-94
Anita	10-JAN-95
Ravi	10-FEB-95
Deepa	20-APR-95
Bala	15-MAR-96
Meena	18-MAY-96
Priya	05-AUG-96

8 rows returned in 0.00 seconds CSV Export

(d) Display Name, Salary and Age of male teachers

```
SELECT TNAME, SALARY, AGE
FROM TEACHER
WHERE GENDER = 'M';
```

Output



```
WHERE GENDER = 'M';
```

Results Explain Describe Saved SQL History

TNAME	SALARY	AGE
Bala	3000	30
Kumar	3500	35
Ravi	2700	27
Suresh	3200	32

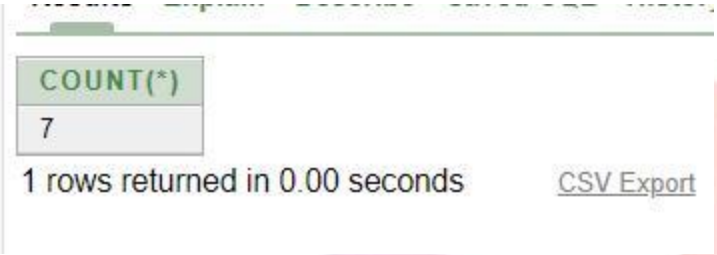
4 rows returned in 0.01 seconds CSV Export

(e) Count teachers whose Age > 23

SELECT COUNT(*)

FROM TEACHER

WHERE AGE > 23;



COUNT(*)
7

1 rows returned in 0.00 seconds CSV Export

(f)Insert new teacher

INSERT INTO TEACHER VALUES

(9, 'Raja', 26, 'Computer',
TO_DATE('13/05/1995','DD/MM/YYYY'), 2300, 'M');

Results Explain Describe Saved SQL History

TID	TNAME	AGE	DEPARTMENT	DOJ	SALARY	GENDER
1	Anita	25	History	10-JAN-95	2500	F
2	Bala	30	Hindi	15-MAR-96	3000	M
3	Deepa	28	Hindi	20-APR-95	2800	F
4	Kumar	35	History	12-JUN-94	3500	M
5	Priya	22	Computer	05-AUG-96	2200	F
6	Ravi	27	Mathematics	10-FEB-95	2700	M
7	Meena	24	History	18-MAY-96	2400	F
8	Suresh	32	Computer	25-SEP-94	3200	M
9	Raja	26	Computer	13-MAY-95	2300	M

9 rows returned in 0.00 seconds CSV Export

(g) Outputs

(i) Count distinct departments

SELECT COUNT(DISTINCT DEPARTMENT)FROM TEACHER;

Departments are:

- History
- Hindi
- Computer
- Mathematics

Output



(ii) Maximum age of female teachers

```
SELECT MAX(AGE)

FROM TEACHER

WHERE GENDER = 'F';
```



(iii) Average salary where Date of Join < 12/07/96

```
SELECT AVG(SALARY)

FROM TEACHER
```

WHERE DOJ < TO_DATE('12/07/1996','DD/MM/YYYY');



(iv) Sum of salary where Date of Join < 12/07/9

SELECT SUM(SALARY)

FROM TEACHER

WHERE DOJ < TO_DATE('12/07/1996','DD/MM/YYYY');

PRACTICE NO:09 SQL Queries on LIBRARY Database

DATE: 05/09/2026

9. Given the following tables for a database LIBRARY:

Write SQL queries for (a) to (f):
(a) To show Book name, Author name and Price of books of EPB publishers.

(b) To list the names of the books of Fiction Type.

(c) To display the names and price of the books in descending order of their price.

(d) To increase the price of all books of first publisher by 50.

(e) To display the Book_Id, Book_Name and Quantity issued for all books which have been issued. (The query will require contents from both the tables).

(f) To insert a new row in the table Issued following the data: “F0002”,4

(g) Give the output of the following queries based on the above tables:

(i) SELECT COUNT(DISTINCT Publishers) FROM Books;

(ii) SELECT SUM(Price) FROM Books WHERE Quantity > 5;

(iii) SELECT BOOK_NAME,AUTHOR_NAME FROM Books WHERE Price < 500;

(iv) SELECT COUNT (*) FROM Books;

9. LIBRARY Database

Aim

To create and execute SQL queries on the **BOOKS** and **ISSUED** tables to manage library book details, including authors, publishers, prices, quantities, and issued books.

Description

This experiment demonstrates the use of SQL commands to:

- Display books based on their publisher.
- Display books belonging to the Fiction type.

- Sort books according to price.
- Update the price of books.
- Display issued book details using a table join.
- Insert a new record into the ISSUED table.
- Use aggregate functions such as COUNT() and SUM().

Software Used

Oracle Database / Oracle SQL Plus (SQL Command Line)
create table

```
CREATE TABLE LIBRARY_BOOKS (
   BOOK_ID VARCHAR2(10),
   BOOK_NAME VARCHAR2(50),
   AUTHOR_NAME VARCHAR2(40),
   PUBLISHERS VARCHAR2(30),
   PRICE NUMBER(8,2),
   BOOK_TYPE VARCHAR2(20),
   QUANTITY NUMBER(5)
);
```

2. Insert records

```
INSERT INTO LIBRARY_BOOKS VALUES
('B001', 'The Guide', 'R.K. Narayan', 'EPB', 450, 'Fiction', 8);

INSERT INTO LIBRARY_BOOKS VALUES
('B002', 'Malgudi Days', 'R.K. Narayan', 'EPB', 550, 'Fiction', 6);
```

INSERT INTO LIBRARY_BOOKS VALUES

('B003', 'Wings of Fire', 'A.P.J. Abdul Kalam', 'Penguin', 600, 'Biography', 10);

INSERT INTO LIBRARY_BOOKS VALUES

('B004', 'The Alchemist', 'Paulo Coelho', 'Harper', 400, 'Fiction', 4);
INSERT INTO LIBRARY_BOOKS VALUES

('B005', 'India 2020', 'A.P.J. Abdul Kalam', 'EPB', 500, 'Non-Fiction', 7);

INSERT INTO LIBRARY_BOOKS VALUES

('B006', 'Train to Pakistan', 'Khushwant Singh', 'Penguin', 350, 'Fiction', 3);

BOOKS Table

```
SELECT * FROM LIBRARY_BOOKS;
```

Results Explain Describe Saved SQL History

BOOK_ID	BOOK_NAME	AUTHOR_NAME	PUBLISHERS	PRICE	BOOK_TYPE	QUANTITY
B001	The Guide	R.K. Narayan	EPB	450	Fiction	8
B002	Malgudi Days	R.K. Narayan	EPB	550	Fiction	6
B003	Wings of Fire	A.P.J. Abdul Kalam	Penguin	600	Biography	10
B004	The Alchemist	Paulo Coelho	Harper	400	Fiction	4
B005	India 2020	A.P.J. Abdul Kalam	EPB	500	Non-Fiction	7
B006	Train to Pakistan	Khushwant Singh	Penguin	350	Fiction	3

6 rows returned in 0.00 seconds CSV Export

3. Create ISSUED table

```
CREATE TABLE BOOK_ISSUED (
   BOOK_ID VARCHAR2(10),
   QUANTITY_ISSUED NUMBER(5)
);
```

Insert issued-book records

INSERT INTO BOOK_ISSUED VALUES

('B001', 2);

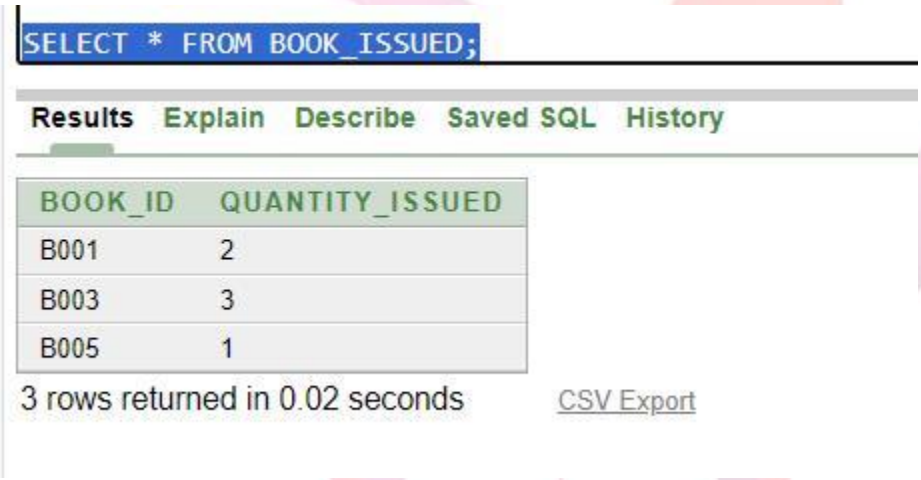
```
INSERT INTO BOOK_ISSUED VALUES

('B003', 3);


INSERT INTO BOOK_ISSUED VALUES
('B005', 1);


COMMIT;
```

ISSUED Table



```
SELECT * FROM BOOK_ISSUED;
```

Results | Explain | Describe | Saved SQL | History

BOOK_ID	QUANTITY_ISSUED
B001	2
B003	3
B005	1

3 rows returned in 0.02 seconds CSV Export

(a) Book name, Author name and Price of books published by EPB

```
SELECT BOOK_NAME, AUTHOR_NAME, PRICE

FROM LIBRARY_BOOKS

WHERE PUBLISHERS = 'EPB';
```

(b) Output

Results Explain Describe Saved SQL History

BOOK_NAME	AUTHOR_NAME	PRICE
The Guide	R.K. Narayan	450
Malgudi Days	R.K. Narayan	550
India 2020	A.P.J. Abdul Kalam	500

3 rows returned in 0.00 seconds CSV Export

Language: en-us

(b) Names of books of Fiction type

```
SELECT BOOK_NAME
FROM LIBRARY_BOOKS
WHERE BOOK_TYPE = 'Fiction';
```

Output



Results Explain Describe Saved SQL History

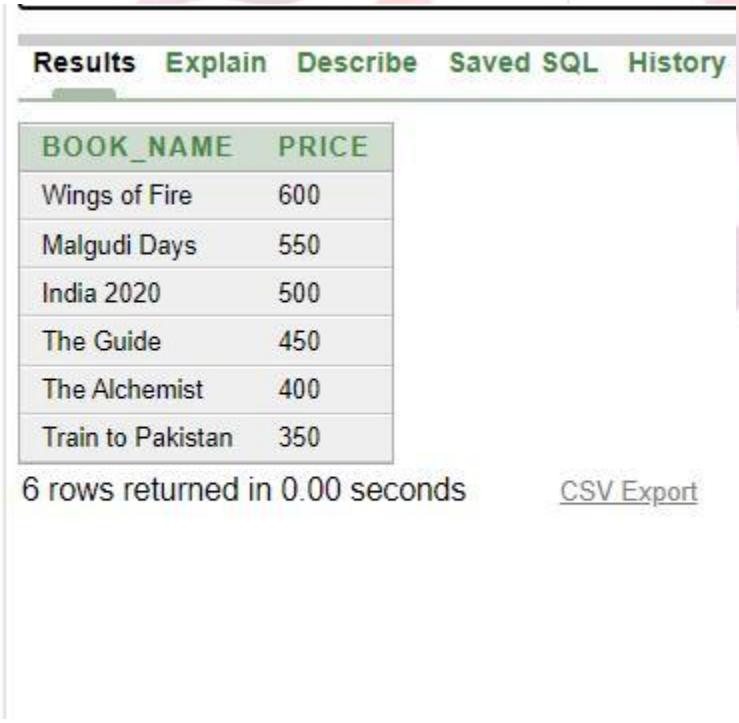
BOOK_NAME
The Guide
Malgudi Days
The Alchemist
Train to Pakistan

4 rows returned in 0.00 seconds CSV Export

(c) Names and prices in descending order of price

```
SELECT BOOK_NAME, PRICE
FROM LIBRARY_BOOKS
ORDER BY PRICE DESC;
```

Output



Results Explain Describe Saved SQL History

BOOK_NAME	PRICE
Wings of Fire	600
Malgudi Days	550
India 2020	500
The Guide	450
The Alchemist	400
Train to Pakistan	350

6 rows returned in 0.00 seconds CSV Export

(d) Increase price of all books of first publisher by 50

```
UPDATE LIBRARY_BOOKS

SET PRICE = PRICE + 50

WHERE PUBLISHERS = 'EPB';
```

Results Explain Describe Saved SQL History

BOOK_ID	BOOK_NAME	AUTHOR_NAME	PUBLISHERS	PRICE	BOOK_TYPE	QUANTITY
B001	The Guide	R.K. Narayan	EPB	500	Fiction	8
B002	Malgudi Days	R.K. Narayan	EPB	600	Fiction	6
B003	Wings of Fire	A.P.J. Abdul Kalam	Penguin	600	Biography	10
B004	The Alchemist	Paulo Coelho	Harper	400	Fiction	4
B005	India 2020	A.P.J. Abdul Kalam	EPB	550	Non-Fiction	7
B006	Train to Pakistan	Khushwant Singh	Penguin	350	Fiction	3

6 rows returned in 0.01 seconds CSV Export

(e) Display Book_ID, Book_Name and Quantity Issued for issued books

```
SELECT B.BOOK_ID, B.BOOK_NAME, I.QUANTITY_ISSUED

FROM LIBRARY_BOOKS B, BOOK_ISSUED I

WHERE B.BOOK_ID = I.BOOK_ID;
```

Output

```
WHERE B.BOOK_ID = I.BOOK_ID;
```

Results Explain Describe Saved SQL History

BOOK_ID	BOOK_NAME	QUANTITY_ISSUED
B001	The Guide	2
B003	Wings of Fire	3
B005	India 2020	1

3 rows returned in 0.00 seconds CSV Export

(e) Insert a new row into ISSUED

INSERT INTO BOOK_ISSUED VALUES

('F0002', 4);

Updated ISSUED table

Results Explain Describe Saved SQL History

BOOK_ID	QUANTITY_ISSUED
B001	2
B003	3
B005	1
F0002	4

4 rows returned in 0.00 seconds CSV Export

(g) Outputs

(i) Number of distinct publishers

```
SELECT COUNT(DISTINCT PUBLISHERS)

FROM LIBRARY_BOOKS;
```

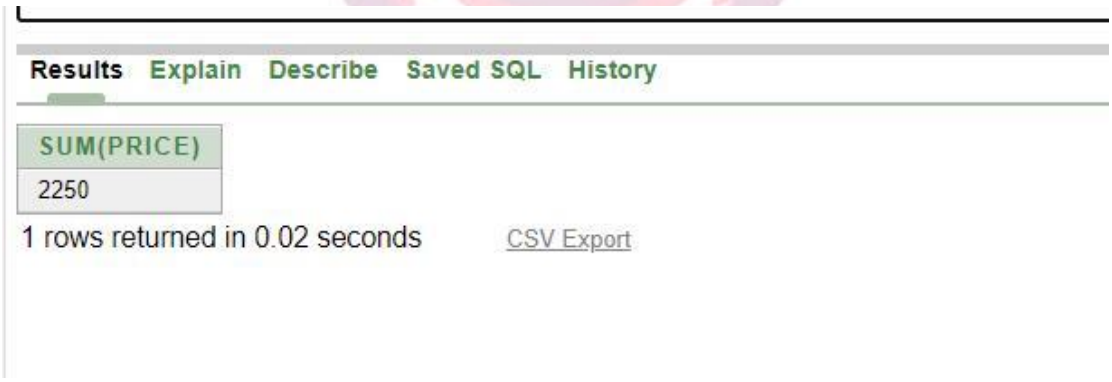


(ii) Sum of price where Quantity > 5

```
SELECT SUM(PRICE)

FROM LIBRARY_BOOKS

WHERE QUANTITY > 5;
```

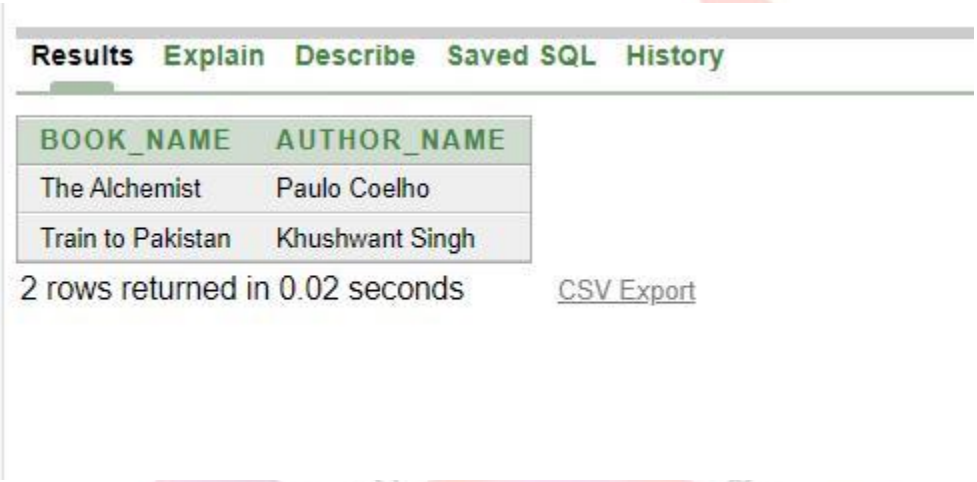


(iii) Book name and author where Price < 500

```
SELECT BOOK_NAME, AUTHOR_NAME
FROM LIBRARY_BOOKS

WHERE PRICE < 500;
```

Output



Results Explain Describe Saved SQL History

BOOK_NAME	AUTHOR_NAME
The Alchemist	Paulo Coelho
Train to Pakistan	Khushwant Singh

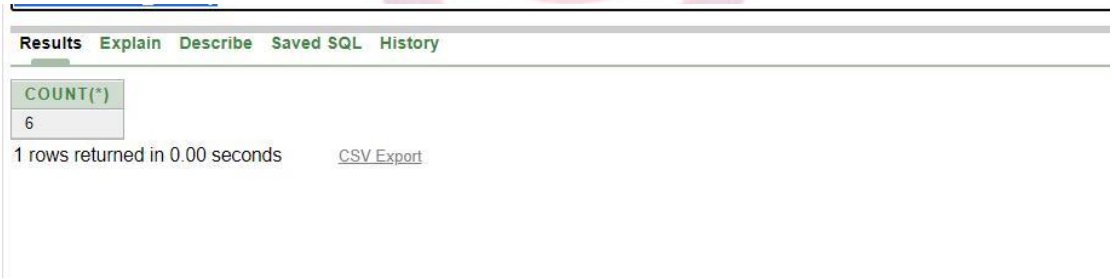
2 rows returned in 0.02 seconds CSV Export

(iv) Count all books

```
SELECT COUNT(*)

FROM LIBRARY_BOOKS;
```

Output



Results Explain Describe Saved SQL History

COUNT(*)
6

1 rows returned in 0.00 seconds CSV Export

PRACTICE NO:10 SQL Queries on CARDEN and CUSTOMER Tables

DATE: 07/09/2026

10. Consider the following tables CARDEN and CUSTOMER and answer (a) and (b) parts of this question:
(a) Write SQL commands for the following statements:

(i) To display the name of all the SILVER colored cars.

(ii) To display name of car, make and capacity of cars in descending order of their sitting capacity.

(iii) To display the highest Charges at which a vehicle can be hired from CARDEN.

(iv) To display the customer name and the corresponding name of the cards hired by them.

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (DISTINCT Make) FROM CARDEN;

(ii) SELECT MAX (Charges), MIN (Charges) FROM CARDEN;

(iii) SELECT COUNT (*), Make FROM CARDEN;

(iv) SELECT CarName FROM CARDEN WHERE Capacity = 4;

10. CARDEN and CUSTOMER

Aim

To create and execute SQL queries on the **CARDEN** and **CUSTOMER** tables to retrieve car and customer information using filtering, sorting, aggregate functions, and table joins.

Description

This experiment demonstrates the use of SQL commands to:

- Display silver-colored cars.
- Display car details in descending order of seating capacity.
- Find the highest and lowest hiring charges.
- Display customers with the corresponding cars hired by them.

- Count different car manufacturers.
- Count cars based on their make.
- Display cars having a specific seating capacity.

Software Used

Oracle Database / Oracle SQL Plus (SQL Command Line)

1. Create CARDEN table

```
CREATE TABLE CARDEN (
   Ccode INT,
   CarName VARCHAR(20),
   Make VARCHAR(20),
   Color VARCHAR(15),
   Capacity INT,
   Charges INT
);
```

3. Insert records into CARDEN

```
INSERT INTO CARDEN VALUES

('C101', 'Swift Dzire', 'Maruti', 'Silver', 4, 1500);


INSERT INTO CARDEN VALUES

('C102', 'Innova', 'Toyota', 'Black', 7, 2500);


INSERT INTO CARDEN VALUES

('C103', 'City', 'Honda', 'Silver', 5, 2000);


INSERT INTO CARDEN VALUES
```

('C104', 'Verna', 'Hyundai', 'Red', 5, 1800);

INSERT INTO CARDEN VALUES

('C105', 'Ertiga', 'Maruti', 'Silver', 7, 2200);

SELECT * FROM CARDEN;

COMMIT;

Results Explain Describe Saved SQL History

CARNO	CARNAME	MAKE	COLOR	CAPACITY	CHARGES
C101	Swift Dzire	Maruti	Silver	4	1500
C102	Innova	Toyota	Black	7	2500
C103	City	Honda	Silver	5	2000
C104	Verna	Hyundai	Red	5	1800
C105	Ertiga	Maruti	Silver	7	2200

5 rows returned in 0.00 seconds CSV Export

4. Create CUSTOMER table

CREATE TABLE CUSTOMER (

CustomerID NUMBER(3),

CustomerName VARCHAR2(30),

CarNo VARCHAR2(10)

);

Insert records

INSERT INTO CAR_CUSTOMER VALUES

(1, 'Anisha', 'C101');

INSERT INTO CAR_CUSTOMER VALUES

\(2, 'Rahul', 'C102');

INSERT INTO CAR_CUSTOMER VALUES
(3, 'Priya', 'C103');

INSERT INTO CAR_CUSTOMER VALUES

(4, 'Kiran', 'C105');

COMMIT;

CUSTOMER Table

s

Results Explain Describe Saved SQL History

CUSTOMERID	CUSTOMERNAME	CARNO
1	Anisha	C101
2	Rahul	C102
3	Priya	C103
4	Kiran	C105

4 rows returned in 0.00 seconds CSV Export

(a) SQL Commands

(i) Display names of all SILVER coloured cars

SELECT CarName

FROM CARDEN

WHERE Color = 'Silver';

Output



(ii) Display CarName, Make and Capacity in descending order of Capacity

SELECT CarName, Make, Capacity

FROM CARDEN

ORDER BY Capacity DESC;

Output



Results Explain Describe Saved SQL History

CARNAME	MAKE	CAPACITY
Ertiga	Maruti	7
Innova	Toyota	7
City	Honda	5
Verna	Hyundai	5
Swift Dzire	Maruti	4

5 rows returned in 0.00 seconds CSV Export



Language: en-us



34°

(iii) Display the highest Charges

SELECT CarName, Make, Capacity

FROM CARDEN

ORDER BY Capacity DESC;

Output

Results Explain Describe Saved SQL History

CARNAME	MAKE	CAPACITY
Ertiga	Maruti	7
Innova	Toyota	7
City	Honda	5
Verna	Hyundai	5
Swift Dzire	Maruti	4

5 rows returned in 0.00 seconds CSV Export

(iv) Display customer name and corresponding car name

```
SELECT C.CustomerName, D.CarName
FROM CAR_CUSTOMER C, CARDEN D
WHERE C.CarNo = D.CarNo;
```

Output

```
WHERE C.CarNo = D.CarNo;
```

Results Explain Describe Saved SQL History

CUSTOMERNAME	CARNAME
Anisha	Swift Dzire
Rahul	Innova
Priya	City
Kiran	Ertiga

4 rows returned in 0.02 seconds CSV Export

(b) Outputs

(i)

SELECT COUNT(DISTINCT Make)

FROM CARDEN;



Results Explain Describe Saved SQL History

COUNT(DISTINCTMAKE)
4

1 rows returned in 0.00 seconds CSV Export

(ii)

SELECT MAX(Charges), MIN(Charges)

FROM CARDEN;



Results Explain Describe Saved SQL History

MAX(CHARGES)	MIN(CHARGES)
2500	1500

1 rows returned in 0.00 seconds CSV Export

(iii)

SELECT COUNT(*), MakeFROM CARDEN;

Expected output

Results Explain Describe Saved SQL History

COUNT(*)	MAKE
1	Hyundai
1	Toyota
2	Maruti
1	Honda

4 rows returned in 0.00 seconds CSV Export

(iv) Display cars with Capacity = 4

```
SELECT CarName

FROM CARDEN

WHERE Capacity = 4;
```

Output

Results Explain Describe Saved SQL Hi

CARNAME
Swift Dzire

1 rows returned in 0.00 seconds CSV Exp

PRACTICE NO:11 SQL Queries on EMPLOYEE and SALGRADE Tables

DATE: 12/09/2026

11. Consider the following tables EMPLOYEE and SALGRADE and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

(i) To display the detail of all the EMPLOYEE in descending order of DOJ.
(ii) To display name and design of those EMPLOYEE, whose sgrade is either S02 or S03.

(iii) To display the content of all the EMPLOYEE table, whose DOJ is in between '09-FEB- 2006' and '08-AUG-2009'.

(iv) TO add a new row in the EMPLOYEE table with the following data:

109, 'Harish Roy', 'HEAD-IT', 'S02', '09-SEP-2007', '21-APR-1983'.

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (SGRADE), SGRADE FROM EMPLOYEE GROUP BY SGRADE;

(ii) SELECT MIN (DOB), MAX (DOJ) FROM EMPLOYEE;

(iii) SELECT NAME, SALARY FROM EMPLOYEE E, SALGRADE S

WHERE E. SGRADE = S. SGRADE AND E. ECODE<103;

(v) SELECT SGRADE, SALARY+HRA FROM SALGRADE WHERE SGRADE = 'S02';

Aim

To perform SQL queries on the **EMPLOYEE** and **SALGRADE** tables using SELECT, WHERE, ORDER BY, GROUP BY, aggregate functions, INSERT, and table joining.

Description

This practical demonstrates how to:

- Display employee details in descending order of DOJ.
- Filter employees based on their salary grade.
- Display employees whose DOJ falls within a specified date range.

- Insert a new employee record.
- Use aggregate functions such as COUNT, MIN, and MAX.
- Join the EMPLOYEE and SALGRADE tables to retrieve salary details.

Software Used

- **Oracle Database 21c / Oracle XE**
- **SQL*Plus** or **Oracle SQL Developer**

1. Create EMPLOYEE table

```
CREATE TABLE EMPLOYEE_NEW
(
   ECODE NUMBER(3),
   NAME VARCHAR2(30),
   DESIGN VARCHAR2(20),
   SGRADE VARCHAR2(3),
   DOJ DATE,
   DOB DATE
);
```

Insert records

```
INSERT INTO EMPLOYEE_NEW
VALUES (101, 'Amit Kumar', 'MANAGER', 'S01',
      '12-JAN-2005', '15-MAR-1980');

INSERT INTO EMPLOYEE_NEW
VALUES (102, 'Ravi Kumar', 'CLERK', 'S02',
      '09-FEB-2006', '20-JUL-1985');

INSERT INTO EMPLOYEE_NEW
```

```
VALUES (103, 'Priya Sharma', 'ANALYST', 'S03',
      '15-MAY-2007', '10-DEC-1984');

INSERT INTO EMPLOYEE_NEW
VALUES (104, 'Sneha Rao', 'PROGRAMMER', 'S02',
      '08-AUG-2009', '25-JAN-1986');

INSERT INTO EMPLOYEE_NEW
VALUES (105, 'Kiran Das', 'TESTER', 'S03',
      '20-JUN-2008', '18-NOV-1983');
```

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	SGRADE	DOJ	DOB
101	Amit Kumar	MANAGER	S01	12-JAN-05	15-MAR-80
102	Ravi Kumar	CLERK	S02	09-FEB-06	20-JUL-85
103	Priya Sharma	ANALYST	S03	15-MAY-07	10-DEC-84
104	Sneha Rao	PROGRAMMER	S02	08-AUG-09	25-JAN-86
105	Kiran Das	TESTER	S03	20-JUN-08	18-NOV-83

5 rows returned in 0.00 seconds CSV Export

2. Create SALGRADE table

Because the queries in part (b) require SALARY and HRA:

```
CREATE TABLE SALGRADE (
   SGRADE VARCHAR(5),
   SALARY INT,
   HRA INT
);
```

Insert records

```
INSERT INTO SALGRADE VALUES
('S01', 56000, 18000);
INSERT INTO SALGRADE VALUES
('S02', 32000, 12000);
INSERT INTO SALGRADE VALUES
('S03', 24000, 8000);
```

Results Explain Describe Saved SQL History

SGRADE	SALARY	HRA
S01	50000	5000
S02	40000	4000
S03	30000	3000

3 rows returned in 0.00 seconds CSV Export

(a) SQL Commands

(i) Display all EMPLOYEE details in descending order of DOJ

```
SELECT *
FROM EMPLOYEE
ORDER BY DOJ DESC;
```

Output

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	SGRADE	DOJ	DOB
104	Sneha Rao	PROGRAMMER	S02	08-AUG-09	25-JAN-86
105	Kiran Das	TESTER	S03	20-JUN-08	18-NOV-83
103	Priya Sharma	ANALYST	S03	15-MAY-07	10-DEC-84
102	Ravi Kumar	CLERK	S02	09-FEB-06	20-JUL-85
101	Amit Kumar	MANAGER	S01	12-JAN-05	15-MAR-80

5 rows returned in 0.02 seconds CSV Export

(ii) Display NAME and DESIGN of employees whose SGRADE is S02 or S03

```
SELECT NAME, DESIGN

FROM EMPLOYEE_NEW

WHERE SGRADE IN ('S02', 'S03');
```

Output



Results Explain Describe Saved SQL History

NAME	DESIGN
Ravi Kumar	CLERK
Priya Sharma	ANALYST
Sneha Rao	PROGRAMMER
Kiran Das	TESTER

4 rows returned in 0.02 seconds CSV Export

(iii) Display employees whose DOJ is between 09-Feb-2006 and 08-Aug-2009

```
SELECT *
FROM EMPLOYEE_NEW
WHERE DOJ BETWEEN '09-FEB-2006' AND '08-AUG-2009';
```

Output



Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	SGRADE	DOJ	DOB
102	Ravi Kumar	CLERK	S02	09-FEB-06	20-JUL-85
103	Priya Sharma	ANALYST	S03	15-MAY-07	10-DEC-84
104	Sneha Rao	PROGRAMMER	S02	08-AUG-09	25-JAN-86
105	Kiran Das	TESTER	S03	20-JUN-08	18-NOV-83

4 rows returned in 0.00 seconds CSV Export

(iv) Insert the new employee

```
INSERT INTO EMPLOYEE_NEW
VALUES (109, 'Harish Roy', 'HEAD-IT', 'S02',
    '09-SEP-2007', '21-APR-1983');
```

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	SGRADE	DOJ	DOB
101	Amit Kumar	MANAGER	S01	12-JAN-05	15-MAR-80
102	Ravi Kumar	CLERK	S02	09-FEB-06	20-JUL-85
103	Priya Sharma	ANALYST	S03	15-MAY-07	10-DEC-84
104	Sneha Rao	PROGRAMMER	S02	08-AUG-09	25-JAN-86
105	Kiran Das	TESTER	S03	20-JUN-08	18-NOV-83
109	Harish Roy	HEAD-IT	S02	09-SEP-07	21-APR-83

6 rows returned in 0.00 seconds CSV Export

(b) Outputs

(i) Count employees in each SGRADE

```
SELECT COUNT(SGRADE), SGRADE

FROM EMPLOYEE_NEW

GROUP BY SGRADE;
```

Results Explain Describe Saved SQL History

COUNT(SGRADE)	SGRADE
1	S01
3	S02
2	S03

3 rows returned in 0.00 seconds CSV Export

(ii) Minimum DOB and maximum DOJ

```
SELECT MIN(DOB), MAX(DOJ)

FROM EMPLOYEE_NEW;
```

Output

Results Explain Describe Saved SQL History

MIN(DOB)	MAX(DOJ)
15-MAR-80	08-AUG-09

1 rows returned in 0.01 seconds CSV Export

(iii) Name and salary for employees with ECODE < 103

```
SELECT NAME, SALARY

FROM EMPLOYEE_NEW E, SALGRADE_NEW S

WHERE E.SGRADE = S.SGRADE
AND E.ECODE < 103;
```

Output

Results Explain Describe Saved SQL History

NAME	SALARY
Amit Kumar	50000
Ravi Kumar	40000

2 rows returned in 0.00 seconds CSV Export

(iv) SGRADE and Salary + HRA for S02

```
SELECT SGRADE, SALARY + HRA

FROM SALGRADE_NEW

WHERE SGRADE = 'S02';
```

```
WHERE SGRADE = 'S02';
```

Results Explain Describe Saved SQL History

SGRADE	SALARY+HRA
S02	44000

1 rows returned in 0.00 seconds CSV Export

PRACTICE NO:12 SQL Queries on WORKER and PAYLEVEL Tables

DATE: 15/09/2026

12. Consider the following tables WORKER and PAYLAVEL and answer (a) and (b) parts of this question

(a) Write SQL commands for the following statements:

(i) To display the detail of all WORKER in descending order of DOB.

(ii) To display name and design of those WORKER, whoseplevel is either P001 to P002.
(iii) To display the content of all the WORKER table, whose DOB is in between '19-JAN-1984' and '18-JAN-1987'.

(iv) To add a new row with the following:

19, 'Daya Kishore', 'Operator', 'P003', '19-JUN-2008', '11-JUL-1984'.

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (PLEVEL), PLEVEL FROM WORKER GROUP BY PLEVEL;

(ii) SELECT MAX (DOB), MIN (DOJ) FROM WORKER;

(iii) SELECT NAME, PAY FROM WORKER W, PAYLEVEL P WHERE W. PLEVEL= P.LEVEL AND W. ECODE<13;

(iv) SELECT PLEVEL, PAY+ALLOWANCE FROM PLEVEL WHERE PLEVEL = 'P003';

12. WORKER and PAYLEVEL

Aim

To create the **WORKER** and **PAYLEVEL** tables and perform SQL operations such as sorting, filtering, grouping, aggregate functions, insertion, and joining tables.

Description

This practical demonstrates how to manage worker and pay-level details using SQL. The queries are used to display worker records in descending order, filter workers based on pay level and date, insert a new worker record, calculate aggregate values, and combine data from the WORKER and PAYLEVEL tables.

Software Used

- **Oracle SQL Developer**
- **Oracle Database 21c / Oracle XE**

1. Create the WORKER table

```
CREATE TABLE WORKER (
   ECODE INT,
   NAME VARCHAR(30),
   DESIG VARCHAR(20),
   PLEVEL VARCHAR(5),
   DOJ DATE,
   DOB DATE
);
```

2. Insert records into WORKER

```
INSERT INTO WORKER VALUES
(11, 'Radhe Shyam', 'Supervisor', 'P001', '2004-09-13', '1981-08-23');
INSERT INTO WORKER VALUES
(12, 'Chander Nath', 'Operator', 'P003', '2010-02-22', '1987-07-12');
INSERT INTO WORKER VALUES
(13, 'Fizza', 'Operator', 'P003', '2009-06-14', '1983-10-14');
INSERT INTO WORKER VALUES
(15, 'Ameen Ahmed', 'Mechanic', 'P002', '2006-08-21', '1984-03-13');
INSERT INTO WORKER VALUES
(18, 'Sanya', 'Clerk', 'P002', '2005-12-19', '1983-06-09');
```

WORKER Table

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOB	DOJ
11	Arun Kumar	Manager	P001	19-JAN-83	10-JUN-05
12	Ravi Kumar	Clerk	P002	20-JAN-85	15-MAR-06
13	Priya Devi	Supervisor	P003	18-JAN-87	20-JUL-07
14	Kiran Raj	Operator	P002	25-MAY-86	12-DEC-08

4 rows returned in 0.00 seconds CSV Export

3. Create PAYLEVEL table

```
CREATE TABLE PAYLEVEL (
   PLEVEL VARCHAR(5),
   PAY INT,
   ALLOWANCE INT
);
```

Insert records

```
INSERT INTO PAYLEVEL VALUES
('P001', 26000, 12000);
INSERT INTO PAYLEVEL VALUES
('P002', 22000, 10000);
INSERT INTO PAYLEVEL VALUES
('P003', 12000, 6000);
```

PAYLEVEL Table

Results Explain Describe Saved SQL History

PLEVEL	PAY	ALLOWANCE
P001	50000	5000
P002	40000	4000
P003	30000	3000

3 rows returned in 0.00 seconds CSV Export

(a) SQL Commands

(i) Display details of all WORKERs in descending order of DOB

SELECT *FROM WORKERORDER BY DOB DESC;

Output

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOB	DOJ
13	Priya Devi	Supervisor	P003	18-JAN-87	20-JUL-07
14	Kiran Raj	Operator	P002	25-MAY-86	12-DEC-08
12	Ravi Kumar	Clerk	P002	20-JAN-85	15-MAR-06
11	Arun Kumar	Manager	P001	19-JAN-83	10-JUN-05

4 rows returned in 0.02 seconds CSV Export

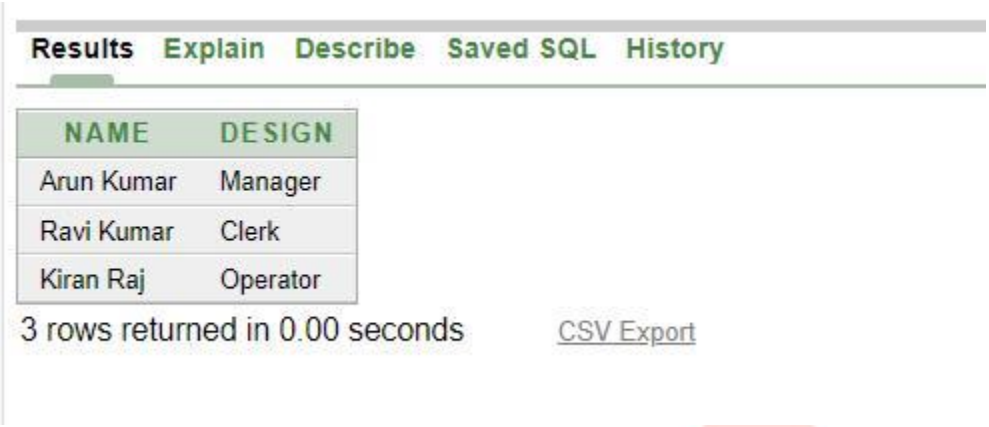
(ii) Display NAME and DESIG of workers whose PLEVEL is P001 or P002

SELECT NAME, DESIGN

FROM WORKER_NEW

WHERE PLEVEL IN ('P001', 'P002');

Output



Results Explain Describe Saved SQL History

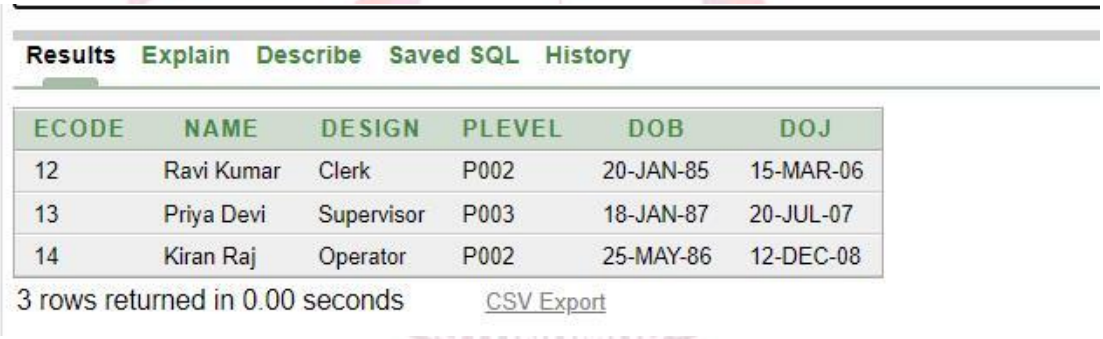
NAME	DESIGN
Arun Kumar	Manager
Ravi Kumar	Clerk
Kiran Raj	Operator

3 rows returned in 0.00 seconds CSV Export

(iii) Display workers whose DOB is between 19-Jan-1984 and 18-Jan-1987

```
SELECT *
FROM WORKER_NEW
WHERE DOB BETWEEN '19-JAN-1984' AND '18-JAN-1987';
```

Output



Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOB	DOJ
12	Ravi Kumar	Clerk	P002	20-JAN-85	15-MAR-06
13	Priya Devi	Supervisor	P003	18-JAN-87	20-JUL-07
14	Kiran Raj	Operator	P002	25-MAY-86	12-DEC-08

3 rows returned in 0.00 seconds CSV Export

(iv) Add the new worker

```
INSERT INTO WORKER_NEW
VALUES
(19, 'Daya Kishore', 'Operator', 'P003',
'19-JUN-2008', '11-JUL-1984');
```

Inserted row

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOB	DOJ
11	Arun Kumar	Manager	P001	19-JAN-83	10-JUN-05
12	Ravi Kumar	Clerk	P002	20-JAN-85	15-MAR-06
13	Priya Devi	Supervisor	P003	18-JAN-87	20-JUL-07
14	Kiran Raj	Operator	P002	25-MAY-86	12-DEC-08
19	Daya Kishore	Operator	P003	19-JUN-08	11-JUL-84

5 rows returned in 0.00 seconds CSV Export

(b) Outputs

(i) Count workers in each PLEVEL

```
SELECT COUNT(PLEVEL), PLEVEL
FROM WORKER_NEW
GROUP BY PLEVEL;
```

Output

COUNT(PLEVEL)	PLEVEL
1	P001
2	P003
2	P002

3 rows returned in 0.00 seconds CSV Export

(ii) Maximum DOB and minimum DOJ

```
SELECT MAX(DOB),MIN(DOJ)
FROM WORKER;
```

Output



Results Explain Describe Saved SQL History

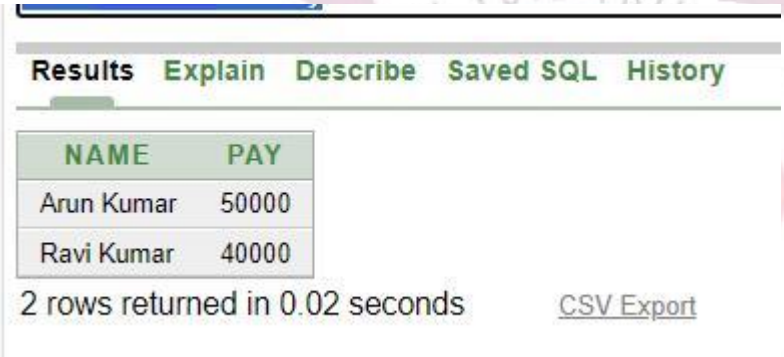
MAX(DOB)	MIN(DOJ)
19-JUN-08	11-JUL-84

1 rows returned in 0.00 seconds CSV Export

(iii) Display NAME and PAY where ECODE < 13

```
SELECT NAME, PAY
FROM WORKER_NEW W, PAYLEVEL_NEW P
WHERE W.PLEVEL = P.PLEVEL
AND W.ECODE < 13;
```

Output



Results Explain Describe Saved SQL History

NAME	PAY
Arun Kumar	50000
Ravi Kumar	40000

2 rows returned in 0.02 seconds CSV Export

(iv) PLEVEL and PAY + ALLOWANCE for P003

```
SELECT PLEVEL, PAY + ALLOWANCE
FROM PAYLEVEL_NEW
WHERE PLEVEL = 'P003';
```

Output

Results Explain Describe Saved SQL Histor

PLEVEL	PAY+ALLOWANCE
P003	33000

rows returned in 0.00 seconds CSV Export

PRACTICE NO:13 SQL Queries on STORE and SUPPLIERS Tables

DATE: 16/09/2026

13. Consider the following tables STORE and SUPPLIERS and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

(i) To display details of all the items in the Store table in ascending order of LastBuy.

(ii) To display ItemNo and Item name of those items from STORE table whose Rate is more than15 Rupees.

(iii)To display the details of those items whose Supplier code (Scode) is 22 or Quantity in Store (Qty) is more than 110 from the table STORE.

(iv)To display minimum Rate of items for each supplier individually as per Scode from the table STORE.

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (DISTINCT Scode) FROM STORE;

(ii)SELECT Rate * Qty FROM STORE WHERE ItemNo=2004;

(iii)SELECT Item,Sname FROM Store S, SUPPLIERS P WHERE S.Scode=P.ScodeAND ItemNo=2006;

(iv) SELECT MAX(LastBuy) FROM

13. STORE and SUPPLIERS

Aim

To create the **STORE** and **SUPPLIERS** tables and perform SQL queries using sorting, filtering, grouping, aggregate functions, and table joins.

Description

This practical demonstrates how to manage store item and supplier information using SQL. The queries retrieve item details based on purchase date, rate, supplier code, and quantity. Aggregate

functions such as COUNT(), MIN(), and MAX() are also used, along with a join between the STORE and SUPPLIERS tables.

Software Used

- **Oracle Database 21c / Oracle XE**
- **Oracle SQL Developer**

1. Create the STORE table

```
CREATE TABLE STORE (
   ItemNo INT,
   Item VARCHAR(30),
   Scode INT,
   Qty INT,
   Rate INT,
   LastBuy DATE
);
```

2. Insert records

```
INSERT INTO STORE VALUES
(2005, 'Sharpener Classic', 23, 60, 8, '2009-06-30');
INSERT INTO STORE VALUES
(2003, 'Ball Pen 0.25', 22, 50, 25, '2010-02-01');
INSERT INTO STORE VALUES
(2002, 'Gel Pen Premium', 21, 150, 12, '2010-02-24');
INSERT INTO STORE VALUES
(2006, 'Gel Pen Classic', 21, 250, 20, '2009-03-11');
INSERT INTO STORE VALUES
(2001, 'Eraser Small', 22, 220, 6, '2009-01-19');
INSERT INTO STORE VALUES
(2004, 'Eraser Big', 22, 110, 8, '2009-12-02');
INSERT INTO STORE VALUES
(2009, 'Ball Pen 0.5', 21, 180, 18, '2009-11-03');
```

STORE Table

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2001	Pen	21	100	10	10-JAN-26
2002	Notebook	22	120	25	15-FEB-26
2003	Pencil	21	150	8	20-MAR-26
2004	Bag	23	50	500	10-APR-26
2006	Marker	23	80	20	15-JUN-26
2005	Eraser	22	130	6	25-MAY-26

6 rows returned in 0.00 seconds CSV Export

3. Create SUPPLIERS table

```
CREATE TABLE SUPPLIERS (
   Scode INT,
   Sname VARCHAR(30)
);
```

Insert records

```
INSERT INTO SUPPLIERS VALUES
(21, 'Premium Stationers');
INSERT INTO SUPPLIERS VALUES
(23, 'Soft Plastics');
INSERT INTO SUPPLIERS VALUES
(22, 'Tetra Supply');
```

SUPPLIERS Table

Results Explain Describe Saved SC

SCODE	SNAME
21	ABC Suppliers
22	XYZ Suppliers
23	PQR Suppliers

3 rows returned in 0.00 seconds

(a) SQL Commands

(i) Display all items in ascending order of LastBuy

```
SELECT *

FROM STORE_NEW

ORDER BY LastBuy ASC;
```

Output

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2001	Pen	21	100	10	10-JAN-26
2002	Notebook	22	120	25	15-FEB-26
2003	Pencil	21	150	8	20-MAR-26
2004	Bag	23	50	500	10-APR-26
2005	Eraser	22	130	6	25-MAY-26
2006	Marker	23	80	20	15-JUN-26

6 rows returned in 0.00 seconds CSV Export

(ii) Display ItemNo and Item where Rate > 15

```
SELECT ItemNo, Item

FROM STORE_NEW
```

WHERE Rate > 15;

Output

```
WHERE Rate > 15;
```

Results | Explain | Describe | Saved SQL | History

ITEMNO	ITEM
2002	Notebook
2004	Bag
2006	Marker

3 rows returned in 0.00 seconds CSV Export

(iii) Display details where Scode = 22 OR Qty > 110

SELECT *

FROM STORE_NEW

WHERE Scode = 22 OR Qty > 110;

Output

Results | Explain | Describe | Saved SQL | History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2002	Notebook	22	120	25	15-FEB-26
2003	Pencil	21	150	8	20-MAR-26
2005	Eraser	22	130	6	25-MAY-26

(iv) Display minimum Rate for each supplier

SELECT Scode, MIN(Rate)

FROM STORE_NEW

GROUP BY Scode;

Output



Results Explain Describe Saved SQL History

SCODE	MIN(RATE)
22	6
21	8
23	20

3 rows returned in 0.00 seconds CSV Export

(b) Outputs

(i) Count distinct supplier codes
Query

```
SELECT COUNT(DISTINCT Scode)

FROM STORE_NEW;
```

Output



Results Explain Describe Saved SQL History

COUNT(DISTINCTSCODE)
3

1 rows returned in 0.00 seconds CSV Export

(ii) Rate × Qty for ItemNo = 2004

Query

```
SELECT Rate * Qty

FROM STORE_NEW

WHERE ItemNo = 2004;
```



(iii) Item and supplier name for ItemNo = 2006
Query

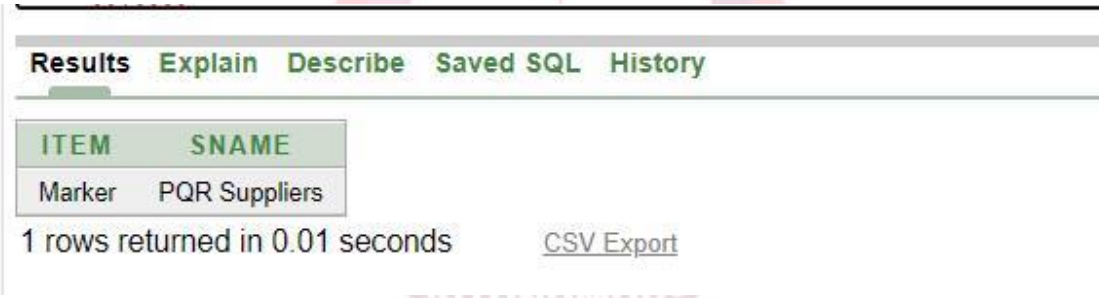
```
SELECT Item, Sname

FROM STORE_NEW S, SUPPLIERS_NEW P

WHERE S.Scode = P.Scode

AND ItemNo = 2006;
```

Output



(iv) Maximum LastBuy date

Query

```
SELECT MAX(LastBuy)
FROM STORE;
```

Output


Results
Explain
Describe
Saved SQL
History
MAX(LASTBUY)
15-JUN-26
1 rows returned in 0.00 seconds
CSV Export

PRACTICE NO:14 SQL Queries on GARMENT and FABRIC Tables

DATE: 17/09/2026

14. Consider the following table GARMENT and FABRIC,

Write SQL commands for the statements (i) to (iv) and give outputs for the SQL queries (v) to (viii).

(i) To display GCODE and DESCRIPTION of each GARMENT in descending order of GCODE.

(ii) To display the details of all the GARMENT, which have READYDATE in between 08- DEC-07 and 16-JUN-08 (inclusive if both the dates).

(iii) To display the average PRICE of all the GARMENT, which are made up of fabric with FCODE as F03.

(iv) To display fabric wise highest and lowest price of GARMENT from GARMENT table. (Display FCODE of each GARMENT along with highest and lowest Price).

(v) SELECT SUM (PRICE) FROM GARMENT WHERE FCODE = 'F01';

(vi) SELECT DESCRIPTION, TYPE FROM GARMENT, FABRIC WHERE GARMENT, FCODE = FABRIC.FCODE AND GARMENT.PRICE >=1260;

(vii) SELECT MAX (FCODE) FROM FABRIC;

(viii) SELECT COUNT (DISTINCT PRICE) FROM GARMENT;

Aim

To create the **GARMENT** and **FABRIC** tables and execute SQL queries to retrieve, filter, sort, and summarize garment and fabric information using SQL.

Description

This practical demonstrates the use of SQL commands such as SELECT, WHERE, ORDER BY, BETWEEN, AVG(), SUM(), MAX(), MIN(), COUNT(DISTINCT), GROUP BY, and table joins. The queries are used to display garment details, calculate average and total prices, find highest and lowest prices, and retrieve information by joining the GARMENT and FABRIC tables.

Software Used

- **Oracle Database 21c / Oracle XE**
- **Oracle SQL Developer**
- **SQL*Plus**

1. Create FABRIC table

```
CREATE TABLE FABRIC (
   FCODE VARCHAR(5),
   DESCRIPTION VARCHAR(30),
   TYPE VARCHAR(20)
);
```

Insert records

```
INSERT INTO FABRIC VALUES
('F01', 'Cotton', 'Natural');
INSERT INTO FABRIC VALUES
('F02', 'Silk', 'Natural');
INSERT INTO FABRIC VALUES
('F03', 'Polyester', 'Synthetic');
INSERT INTO FABRIC VALUES
('F04', 'Wool', 'Natural');
```

FABRIC Table

Results Explain Describe Saved SQL History

GCODE	DESCRIPTION	TYPE	FCODE	PRICE	READYDATE
101	Cotton Shirt	Shirt	F01	1200	10-DEC-07
103	Denim Jeans	Jeans	F03	1500	20-MAY-08
104	Cotton Skirt	Skirt	F01	1000	16-JUN-08
105	Wool Coat	Coat	F03	1800	25-JUL-08

4 rows returned in 0.01 seconds CSV Export

2. Create GARMENT table

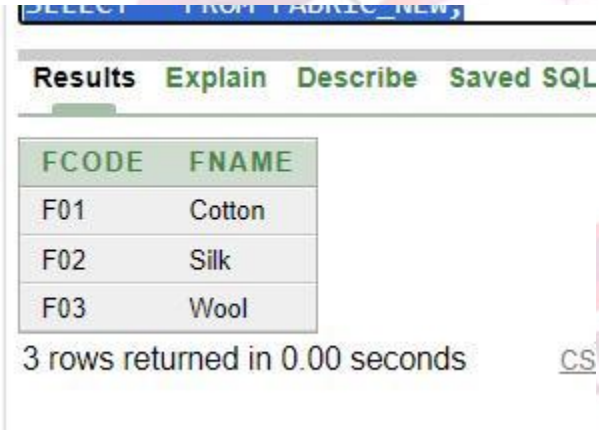
```
CREATE TABLE GARMENT (
   GCODE VARCHAR(5),
   DESCRIPTION VARCHAR(30),
   PRICE INT,
```

```
    FCODE VARCHAR(5),
    READYDATE DATE
);
```

Insert records

```
INSERT INTO GARMENT VALUES
('G001', 'Formal Shirt', 1200, 'F01', '2007-12-10');
INSERT INTO GARMENT VALUES
('G002', 'Silk Saree', 1500, 'F02', '2008-01-15');
INSERT INTO GARMENT VALUES
('G003', 'T-Shirt', 900, 'F03', '2008-06-10');
INSERT INTO GARMENT VALUES
('G004', 'Party Dress', 1800, 'F03', '2008-06-16');
INSERT INTO GARMENT VALUES
('G005', 'Winter Coat', 2200, 'F04', '2008-07-20');
INSERT INTO GARMENT VALUES
('G006', 'Cotton Pants', 1300, 'F01', '2008-05-12');
INSERT INTO GARMENT VALUES
('G007', 'Polyester Skirt', 1260, 'F03', '2007-11-20');
```

GARMENT Table



(i) Display GCODE and DESCRIPTION in descending order of GCODE

```
SELECT GCODE, DESCRIPTION

FROM GARMENT_NEW

ORDER BY GCODE DESC;
```

Output



Results Explain Describe Saved SQL History

GCODE	DESCRIPTION
105	Wool Coat
104	Cotton Skirt
103	Denim Jeans
101	Cotton Shirt

4 rows returned in 0.00 seconds CSV Export

(ii) Display details of garments with READYDATE between 08-Dec-07 and 16-Jun-08

```
SELECT *

FROM GARMENT_NEW

WHERE READYDATE BETWEEN '08-DEC-2007' AND '16-JUN-2008';
```

Output



Results Explain Describe Saved SQL History

GCODE	DESCRIPTION	TYPE	FCODE	PRICE	READYDATE
101	Cotton Shirt	Shirt	F01	1200	10-DEC-07
103	Denim Jeans	Jeans	F03	1500	20-MAY-08
104	Cotton Skirt	Skirt	F01	1000	16-JUN-08

3 rows returned in 0.00 seconds CSV Export

(iii) Average PRICE of garments made from F03

```
SELECT AVG(PRICE)

FROM GARMENT_NEW

WHERE FCODE = 'F03';
```

Results Explain Describe Saved SQL History

AVG(PRICE)
1650

rows returned in 0.00 seconds CSV Export

(iv) Fabric-wise highest and lowest price

```
SELECT FCODE, MAX(PRICE), MIN(PRICE)
FROM GARMENT_NEW
GROUP BY FCODE;
```

Output

Results Explain Describe Saved SQL History

FCODE	MAX(PRICE)	MIN(PRICE)
F03	1800	1500
F01	1200	1000

2 rows returned in 0.00 seconds CSV Export

(v) SUM of PRICE for F01

Query

```
SELECT SUM(PRICE)
FROM GARMENT_NEW
WHERE FCODE = 'F01';
```

Results Explain Describe Saved SQL History

SUM(PRICE)
2200

1 rows returned in 0.00 seconds CSV Export

(vi) Display DESCRIPTION and TYPE where PRICE >= 1260

Query

```
SELECT DESCRIPTION, TYPE
FROM GARMENT_NEW, FABRIC_NEW
WHERE GARMENT_NEW.FCODE = FABRIC_NEW.FCODE
AND GARMENT_NEW.PRICE >= 1260;
```

Results Explain Describe Saved SQL History

DESCRIPTION	TYPE
Wool Coat	Coat
Denim Jeans	Jeans

2 rows returned in 0.00 seconds CSV Export

(vii) Find maximum FCODE from FABRIC

```
SELECT MAX(FCODE)
FROM FABRIC_NEW;
```

MAX(FCODE)
F03

1 rows returned in 0.00 seconds

(viii) Count distinct prices in GARMENT

Query

SELECT COUNT(DISTINCT PRICE)

FROM GARMENT_NEW;

Output



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

PRACTICE NO:15 SQL Queries on FURNITURE and ARRIVALS Tables

DATE: 19/09/2026

15. Given the following tables for a database FURNITURE : Write SQL command for (a) to (f) and write the outputs for (g) on the bases of tables **FURNITURE** AND **ARRIVALS.**

(a) To show all information about the baby cots from the FURNITURE table.

(b) To list the ITEMNAME which are priced at more than 15000 from the FURNITURE table.

(c) To list ITEMNAME AND TYPE of those items, in which DATEOFSTOCK is before 22/01/02 from the FURNITURE table in descending order of ITEMNAME.

(d) To display ITEMNAME and DATEOFSTOCK of those items, in which the DISCOUNT percentage is more than 25 from FURNITURE table.

(e) To count the number of items, whose TYPE is "Sofa" from FURNITURE table.

(f) To insert a new row in the ARRIVALS table with the following data: 14, "Velvet touch", Double bed", {25/03/03}, 25000, 30

(g) Give the output of following SQL statement:

Note: outputs of the below mentioned queries should be based on original data given in both the tables i.e., without considering the insertion done in (g) part of this question.

(i) Select COUNT(distinct TYPE) from FURNITURE;

(ii) Select MAX(DISCOUNT) from FURNITURE, ARRIVALS;

(iii) Select AVG(DISCOUNT) from FURNITURE where TYPE = "Baby cot";

(iv) Select SUM(PRICE) from FURNIT

Aim

To create the **FURNITURE** and **ARRIVALS** tables and execute SQL commands to retrieve, filter, sort, count, insert, and calculate aggregate values from the tables.

Description

This practical demonstrates SQL operations on furniture and arrival records. The queries are used to display baby cot details, find items based on price and discount, sort items by name, count sofa

items, insert a new arrival record, and use aggregate functions such as COUNT(), MAX(), AVG(), and SUM().

Software Used

- **Oracle Database 21c / Oracle XE**
- **Oracle SQL Developer**
- **SQL*Plus**

1. Create the FURNITURE table

```
CREATE TABLE FURNITURE (
   NO INT,
   ITEMNAME VARCHAR(30),
   TYPE VARCHAR(20),
   DATEOFSTOCK DATE,
   PRICE INT,
   DISCOUNT INT
);
```

Insert records

```
INSERT INTO FURNITURE VALUES
(1, 'White lotus', 'Double Bed', '2002-02-23', 30000, 25);
INSERT INTO FURNITURE VALUES
(2, 'Pink feather', 'Baby cot', '2002-01-20', 7000, 20);
INSERT INTO FURNITURE VALUES
(3, 'Dolphin', 'Baby cot', '2002-02-19', 9500, 20);
INSERT INTO FURNITURE VALUES
(4, 'Decent', 'Office Table', '2002-01-01', 25000, 30);
INSERT INTO FURNITURE VALUES
(5, 'Comfort zone', 'Double Bed', '2002-01-12', 25000, 25);
INSERT INTO FURNITURE VALUES
(6, 'Donald', 'Baby cot', '2002-02-24', 6500, 15);
INSERT INTO FURNITURE VALUES
(7, 'Royal Finish', 'Office Table', '2002-02-20', 18000, 30);
INSERT INTO FURNITURE VALUES
(8, 'Royal tiger', 'Sofa', '2002-02-22', 31000, 30);
INSERT INTO FURNITURE VALUES
```

(9, 'Econo sitting', 'Sofa', '2001-12-13', 9500, 25);
INSERT INTO FURNITURE VALUES
(10, 'Eating Paradise', 'Dining Table', '2002-02-19', 11500, 25);

FURNITURE Table

Results Explain Describe Saved SQL History

ITEMNO	ITEMNAME	TYPE	DATEOFSTOCK	PRICE	DISCOUNT
1	Baby Cot Classic	Baby cot	15-JAN-02	12000	20
2	Modern Sofa	Sofa	20-DEC-01	18000	30
3	Wooden Table	Table	10-JAN-02	16000	15
4	Baby Cot Deluxe	Baby cot	18-JAN-02	14000	28
5	Luxury Sofa	Sofa	25-JAN-02	22000	35

5 rows returned in 0.00 seconds CSV Export

2. Create ARRIVALS table

```
CREATE TABLE ARRIVALS (
   NO INT,
   ITEMNAME VARCHAR(30),
   TYPE VARCHAR(20),
   DATEOFSTOCK DATE,
   PRICE INT,
   DISCOUNT INT
);
```

Insert original records

INSERT INTO ARRIVALS VALUES
(11, 'Wood Comfort', 'Double Bed', '2003-03-23', 25000, 25);
INSERT INTO ARRIVALS VALUES
(12, 'Old Fox', 'Sofa', '2003-02-20', 17000, 20);
INSERT INTO ARRIVALS VALUES
(13, 'Micky', 'Baby cot', '2003-02-21', 7500, 15);

ARRIVALS Table

(a) SQL Commands

Results Explain Describe Saved SQL History

ITEMNO	ITEMNAME	TYPE	DATEOFSTOCK	PRICE	DISCOUNT
11	Royal Chair	Chair	10-FEB-02	8000	10
12	Comfort Bed	Bed	18-FEB-02	20000	25
13	Office Table	Table	20-MAR-02	15000	20

(i) Display all information about Baby cots

SELECT *FROM FURNITUREWHERE TYPE = 'Baby cot';

Output

Results Explain Describe Saved SQL History

ITEMNO	ITEMNAME	TYPE	DATEOFSTOCK	PRICE	DISCOUNT
1	Baby Cot Classic	Baby cot	15-JAN-02	12000	20
4	Baby Cot Deluxe	Baby cot	18-JAN-02	14000	28

2 rows returned in 0.00 seconds CSV Export

(ii) Display ITEMNAME priced more than 15000

SELECT ITEMNAME

FROM FURNITURE_NEW

WHERE PRICE > 15000;

Output

Results Explain Describe Saved SQL History

ITEMNAME
Modern Sofa
Wooden Table
Luxury Sofa

3 rows returned in 0.02 seconds CSV Export

(iii) Display ITEMNAME and TYPE where DATEOFSTOCK is before 22-Jan-02, in descending order of ITEMNAME

```
SELECT ITEMNAME, TYPE

FROM FURNITURE_NEW

WHERE DATEOFSTOCK < '22-JAN-2002'

ORDER BY ITEMNAME DESC;
```

Output

Results Explain Describe Saved SQL History

ITEMNAME	TYPE
Wooden Table	Table
Modern Sofa	Sofa
Baby Cot Deluxe	Baby cot
Baby Cot Classic	Baby cot

4 rows returned in 0.00 seconds CSV Export

(iv) Display ITEMNAME and DATEOFSTOCK where DISCOUNT > 25

```
SELECT ITEMNAME, DATEOFSTOCK

FROM FURNITURE_NEW

WHERE DISCOUNT > 25;
```

Output

Results Explain Describe Saved SQL History

ITEMNAME	DATEOFSTOCK
Modern Sofa	20-DEC-01
Baby Cot Deluxe	18-JAN-02
Luxury Sofa	25-JAN-02

3 rows returned in 0.02 seconds CSV Export

(v) Count the number of Sofa items

```
SELECT COUNT(*)
FROM FURNITURE_NEW
WHERE TYPE = 'Sofa';
```

Output

Results Explain Describe Saved SQL History

COUNT(*)
2

1 rows returned in 0.00 seconds CSV Export

(vi) Insert the new row into ARRIVALS

```
INSERT INTO ARRIVALS_NEW
VALUES
(14, 'Velvet touch', 'Double bed',
'25-MAR-2003', 25000, 30);
```

Results Explain Describe Saved SQL History

ITEMNO	ITEMNAME	TYPE	DATEOFSTOCK	PRICE	DISCOUNT
11	Royal Chair	Chair	10-FEB-02	8000	10
12	Comfort Bed	Bed	18-FEB-02	20000	25
13	Office Table	Table	20-MAR-02	15000	20
14	Velvet touch	Double bed	25-MAR-03	25000	30

4 rows returned in 0.00 seconds CSV Export

(i) Count distinct TYPE

SELECT COUNT(DISTINCT TYPE)

FROM FURNITURE_NEW;

Output

Results Explain Describe Saved SQL History

COUNT(DISTINCTTYPE)
3

1 rows returned in 0.00 seconds CSV Export

(ii) Maximum DISCOUNT from FURNITURE and ARRIVALS

SELECT MAX(F.DISCOUNT)

FROM FURNITURE_NEW F;

Output



(iii) Average DISCOUNT for Baby cot

SELECT AVG(DISCOUNT)

FROM FURNITURE_NEW

WHERE TYPE = 'Baby cot';

Output



(iv) Sum of PRICE where DATEOFSTOCK is before 12-Feb-02

SELECT SUM(PRICE)

FROM FURNITURE_NEW;

Results Explain Describe Saved SQL History

SUM(PRICE)
82000

1 rows returned in 0.00 seconds CSV Export

PRACTICE NO:16 SQL Queries on COVID19 Relation

DATE: 21/09/2026

16. **Given the following COVID19 relation, write SQL commands for questions (a) to (f).**

a) Display all details of patients from **Tamil Nadu**.

b) Display the names of patients whose **Status** is **Recovered**.

c) Display **PatientName** and **Age** in ascending order of **Age**.

d) Display the names of patients who are **Vaccinated = 'Yes'**.

e) Count the number of patients whose status is **Active**.

f) Insert the following record into the **COVID19** table:

108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes'

(g) Write the output of the following SQL commands.

a. SELECT COUNT(*) FROM COVID19;

b. SELECT MAX(Age) FROM COVID19;

c. SELECT COUNT(*) FROM COVID19 WHERE Vaccinated='Yes';

d. SELECT MIN(Age)FROM COVID19 WHERE Status='Recovered';

Aim

To create the **COVID19** table and perform SQL operations to retrieve, filter, sort, count, and insert patient records based on their location, status, age, and vaccination details.

Description

This practical demonstrates the use of SQL commands such as SELECT, WHERE, ORDER BY, COUNT(), MAX(), MIN(), and INSERT. The queries are used to display patients from Tamil Nadu, recovered and vaccinated patients, arrange patients by age, count active patients, and add a new patient record.

Software Used

- **Oracle Database 21c / Oracle XE**
- **Oracle SQL Developer**
- **SQL*Plus**

1. Create the COVID19 table

```
CREATE TABLE COVID19 (
   PatientID INT,
   PatientName VARCHAR(30),
   State VARCHAR(30),
   Age INT,
   Status VARCHAR(20),
   Vaccinated VARCHAR(5)
);
```

2. Insert records

```
INSERT INTO COVID19 VALUES
(101, 'Rahul', 'Tamil Nadu', 25, 'Recovered', 'Yes');
INSERT INTO COVID19 VALUES
(102, 'Priya', 'Kerala', 32, 'Active', 'Yes');
INSERT INTO COVID19 VALUES
(103, 'Arun', 'Tamil Nadu', 45, 'Active', 'No');
INSERT INTO COVID19 VALUES
(104, 'Meena', 'Karnataka', 28, 'Recovered', 'Yes');
INSERT INTO COVID19 VALUES
(105, 'Karthik', 'Tamil Nadu', 52, 'Recovered', 'No');
INSERT INTO COVID19 VALUES
(106, 'Divya', 'Andhra Pradesh', 35, 'Active', 'Yes');
INSERT INTO COVID19 VALUES
(107, 'Suresh', 'Tamil Nadu', 19, 'Recovered', 'Yes');
```

COVID19 Table

101	Arun	Tamil Nadu	25	Recovered	Yes
102	Priya	Kerala	32	Active	Yes
103	Ravi	Tamil Nadu	45	Active	No
104	Meena	Karnataka	28	Recovered	Yes
105	Kumar	Tamil Nadu	52	Recovered	No
106	Divya	Andhra Pradesh	36	Active	Yes
107	Suresh	Tamil Nadu	41	Active	Yes

rows returned in 0.00 seconds CSV Export

(a) Display all details of patients from Tamil Nadu

SELECT *FROM COVID19WHERE State = 'Tamil Nadu';

Output

Results Explain Describe Saved SQL History

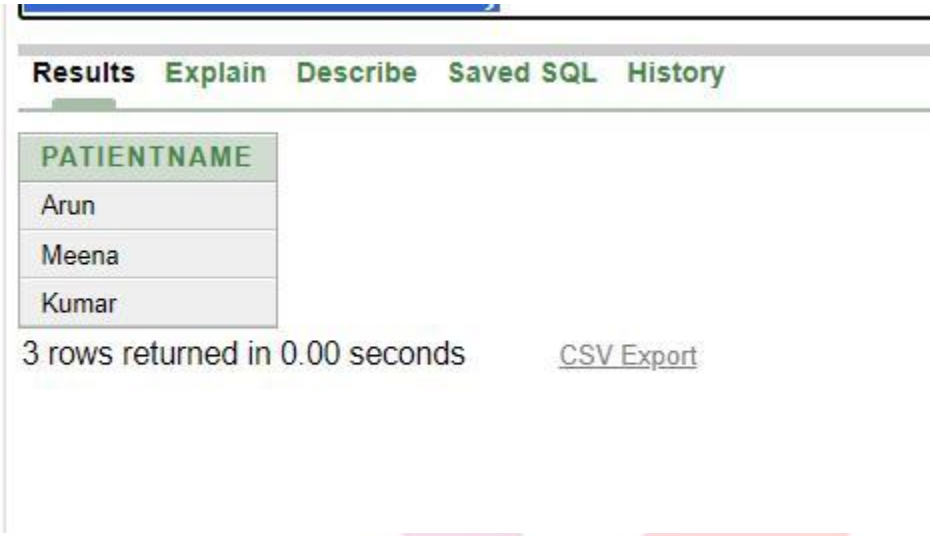
PATIENTID	PATIENTNAME	STATE	AGE	STATUS	VACCINATED
101	Arun	Tamil Nadu	25	Recovered	Yes
103	Ravi	Tamil Nadu	45	Active	No
105	Kumar	Tamil Nadu	52	Recovered	No
107	Suresh	Tamil Nadu	41	Active	Yes

4 rows returned in 0.02 seconds CSV Export

(b) Display names of patients whose Status is Recovered

```
SELECT PatientName
FROM COVID19_NEW
WHERE Status = 'Recovered';
```

Output



(c) Display PatientName and Age in ascending order of Age

```
SELECT PatientName, Age

FROM COVID19_NEW

ORDER BY Age ASC;
```

Output

```
SELECT PatientName, Age
FROM COVID19_NEW
ORDER BY Age ASC;
```

Results | Explain | Describe | Saved SQL | History

PATIENTNAME	AGE
Arun	25
Meena	28
Priya	32
Divya	36
Suresh	41
Ravi	45
Kumar	52

7 rows returned in 0.00 seconds CSV Export

(d) Display names of vaccinated patients

SELECT PatientName

FROM COVID19_NEW

WHERE Vaccinated = 'Yes';

Output

PATIENTNAME
Arun
Priya
Meena
Divya
Suresh

5 rows returned in 0.02

(e) Count patients whose Status is Active

```
SELECT COUNT(*)

FROM COVID19_NEW

WHERE Status = 'Active';
```

Output



(f) Insert the given record

Given:

108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes'

```
INSERT INTO COVID19VALUES
(108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes');
```

Inserted record

Results Explain Describe Saved SQL History

PATIENTID	PATIENTNAME	STATE	AGE	STATUS	VACCINATED
101	Arun	Tamil Nadu	25	Recovered	Yes
102	Priya	Kerala	32	Active	Yes
103	Ravi	Tamil Nadu	45	Active	No
104	Meena	Karnataka	28	Recovered	Yes
105	Kumar	Tamil Nadu	52	Recovered	No
106	Divya	Andhra Pradesh	36	Active	Yes
107	Suresh	Tamil Nadu	41	Active	Yes
108	Anitha	Puducherry	39	Recovered	Yes

8 rows returned in 0.00 seconds CSV Export

(g) Outputs

(a) Count all patients

SELECT COUNT(*)FROM COVID19;

Output

Results Explain Describe Saved SQL History

COUNT(*)
8

1 rows returned in 0.00 seconds CSV Export

(b) Maximum Age

SELECT MAX(Age)
FROM COVID19;

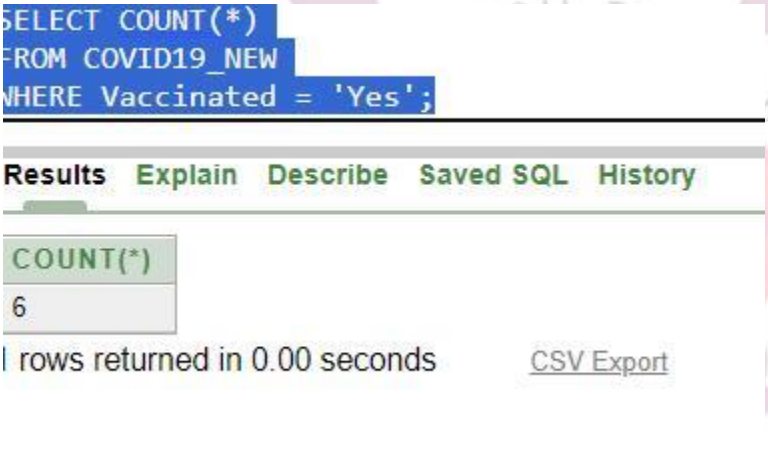
Output



(c) Count vaccinated patients

```
SELECT COUNT(*)
FROM COVID19_NEW
WHERE Vaccinated = 'Yes';
```

Output



(d) Minimum Age of Recovered patients

```
SELECT MIN(Age)
FROM COVID19_NEW
WHERE Status = 'Recovered';
```

Results Explain Describe Saved SQL History

MIN(AGE)
25

1 rows returned in 0.00 seconds CSV Export

PRACTICE NO:17 SQL Queries on HOUSEPRICE Relation (Set 1)

DATE: 23/09/2026

17. **Given the following HOUSEPRICE relation, write SQL commands for questions (a) to (f).**

a) Display all houses located in **Chennai**.

b) Display the **Location** and **Price** of houses having **3 Bedrooms**.

c) Display **Location**, **AreaSqft**, and **Price** in descending order of **Price**.

d) Display the details of houses costing more than **₹70,00,000**.

e) Count the number of **Apartments**.

f) Insert the following record:

208, 'Vellore', 3, 1700, 7300000, 'Villa'

(g) Write the output of the following SQL commands.

(i) SELECT MAX(Price)FROM HOUSEPRICE;

(i) SELECT MIN(AreaSqft)FROM HOUSEPRICE;

(ii) SELECT AVG(Price)FROM HOUSEPRICE WHERE HouseType='Apartment';

(iii) SELECT COUNT(*)FROM HOUSEPRICE WHERE Bedrooms=3;

Aim

To create the **HOUSEPRICE** table and perform SQL operations to display, filter, sort, count, insert, and calculate aggregate values of houses based on location, bedrooms, area, price, and house type.

Description

This practical demonstrates the use of SQL commands such as SELECT, WHERE, ORDER BY, COUNT(), MAX(), MIN(), AVG(), and INSERT. These commands are used to retrieve house details, filter houses based on price and bedrooms, arrange records by price, count apartments, and calculate maximum, minimum, and average values.

Software Used

- Oracle Database 21c / Oracle XE
- Oracle SQL Developer
- SQL*Plus

1. Create HOUSEPRICE table

```
CREATE TABLE HOUSEPRICE (
   HouseID INT,
   Location VARCHAR(30),
   Bedrooms INT,
   AreaSqft INT,
   Price INT,
   HouseType VARCHAR(20)
);
```

2. Insert records

```
INSERT INTO HOUSEPRICE VALUES
(201, 'Chennai', 2, 1200, 5500000, 'Apartment');
INSERT INTO HOUSEPRICE VALUES
(202, 'Chennai', 3, 1500, 6800000, 'Villa');
INSERT INTO HOUSEPRICE VALUES
(203, 'Bangalore', 3, 1600, 7200000, 'Apartment');
INSERT INTO HOUSEPRICE VALUES
(204, 'Coimbatore', 2, 1100, 4500000, 'Apartment');
INSERT INTO HOUSEPRICE VALUES
(205, 'Chennai', 4, 2000, 8500000, 'Villa');
INSERT INTO HOUSEPRICE VALUES
(206, 'Madurai', 3, 1400, 6000000, 'Apartment');
INSERT INTO HOUSEPRICE VALUES
(207, 'Vellore', 2, 1000, 4000000, 'Villa');
```

HOUSEPRICE Table

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
201	Chennai	2	1200	5500000	Apartment
202	Chennai	3	1500	7500000	Villa
201	Chennai	2	1200	5500000	Apartment
201	Chennai	2	1200	5500000	Apartment
202	Chennai	3	1500	7500000	Villa
203	Bangalore	2	1000	4800000	Apartment
204	Coimbatore	3	1400	6500000	Apartment
205	Chennai	4	2000	9000000	Villa
206	Madurai	2	1100	5000000	Apartment
207	Salem	3	1300	6000000	Villa

10 rows returned in 0.00 seconds CSV Export

(a) Display all houses located in Chennai

SELECT *

FROM HOUSEPRICE_NEW

WHERE Location = 'Chennai';

Output

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
201	Chennai	2	1200	5500000	Apartment
202	Chennai	3	1500	7500000	Villa
201	Chennai	2	1200	5500000	Apartment
201	Chennai	2	1200	5500000	Apartment
202	Chennai	3	1500	7500000	Villa
205	Chennai	4	2000	9000000	Villa

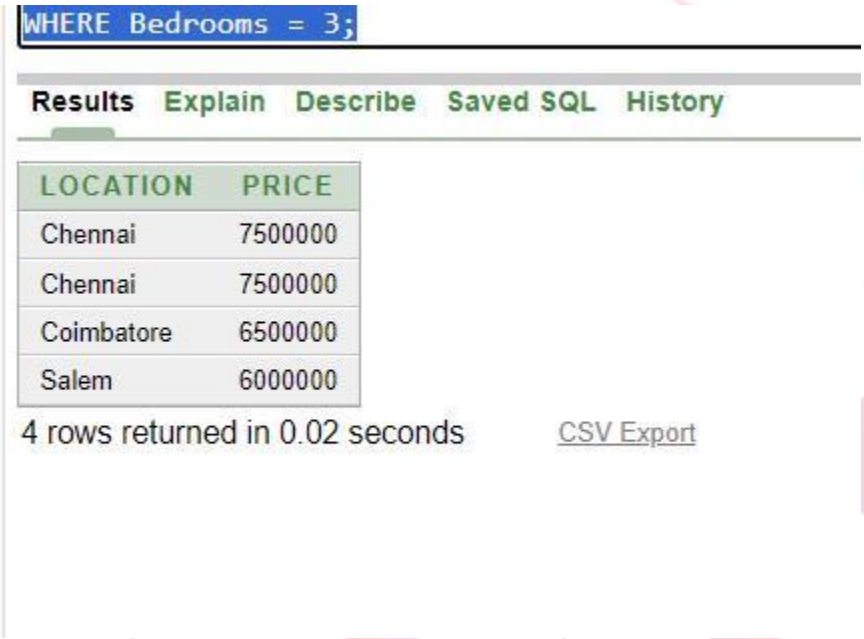
6 rows returned in 0.01 seconds CSV Export

(b) Display Location and Price of houses having 3 Bedrooms

SELECT Location, Price

FROM HOUSEPRICE_NEW
WHERE Bedrooms = 3;

Output



(c) Display Location, AreaSqft and Price in descending order of Price

SELECT Location, AreaSqft, Price

FROM HOUSEPRICE_NEW

ORDER BY Price DESC;

Output

LOCATION	AREASQFT	PRICE
Chennai	2000	9000000
Chennai	1500	7500000
Chennai	1500	7500000
Coimbatore	1400	6500000
Salem	1300	6000000
Chennai	1200	5500000
Chennai	1200	5500000
Chennai	1200	5500000
Madurai	1100	5000000
Bangalore	1000	4800000

10 rows returned in 0.00 seconds CSV Export

(d) Display details of houses costing more than ₹70,00,000

SELECT *

FROM HOUSEPRICE_NEW

WHERE Price > 7000000;

Output

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
202	Chennai	3	1500	7500000	Villa
202	Chennai	3	1500	7500000	Villa
205	Chennai	4	2000	9000000	Villa

3 rows returned in 0.00 seconds CSV Export

(e) Count the number of Apartments

SELECT COUNT(*)
FROM HOUSEPRICE
WHERE HouseType = 'Apartment';

Output

COUNT(*)
6

1 rows returned in 0.00 seconds CSV Export

(f) Insert the given record

INSERT INTO HOUSEPRICE_NEW

VALUES

(208, 'Vellore', 3, 1700, 7300000, 'Villa');

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
201	Chennai	2	1200	5500000	Apartment
202	Chennai	3	1500	7500000	Villa
201	Chennai	2	1200	5500000	Apartment
201	Chennai	2	1200	5500000	Apartment
202	Chennai	3	1500	7500000	Villa
203	Bangalore	2	1000	4800000	Apartment
204	Coimbatore	3	1400	6500000	Apartment
205	Chennai	4	2000	9000000	Villa
206	Madurai	2	1100	5000000	Apartment
207	Salem	3	1300	6000000	Villa
More than 10 rows available. Increase rows selector to view more rows.					

10 rows returned in 0.01 seconds CSV Export

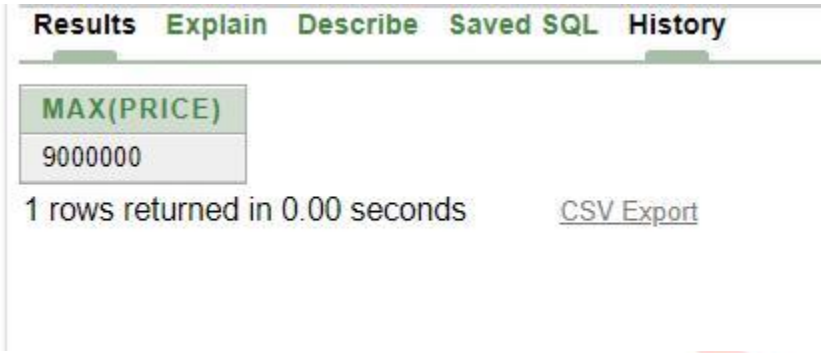
(g) Outputs

(i) Maximum Price

SELECT MAX(Price)

FROM HOUSEPRICE_NEW;

Output



(ii) Minimum AreaSqft

```
SELECT MIN(AreaSqft)

FROM HOUSEPRICE_NEW;
```

Output



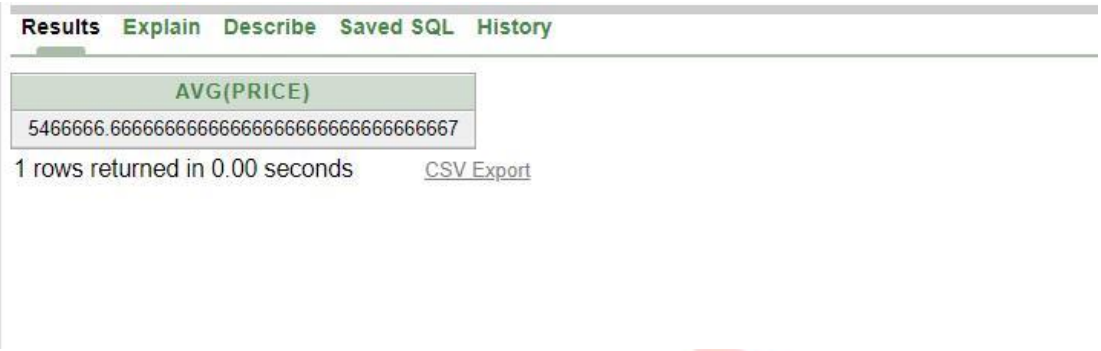
(iii) Average Price of Apartments

```
SELECT AVG(Price)

FROM HOUSEPRICE_NEW

WHERE HouseType = 'Apartment';
```

Output



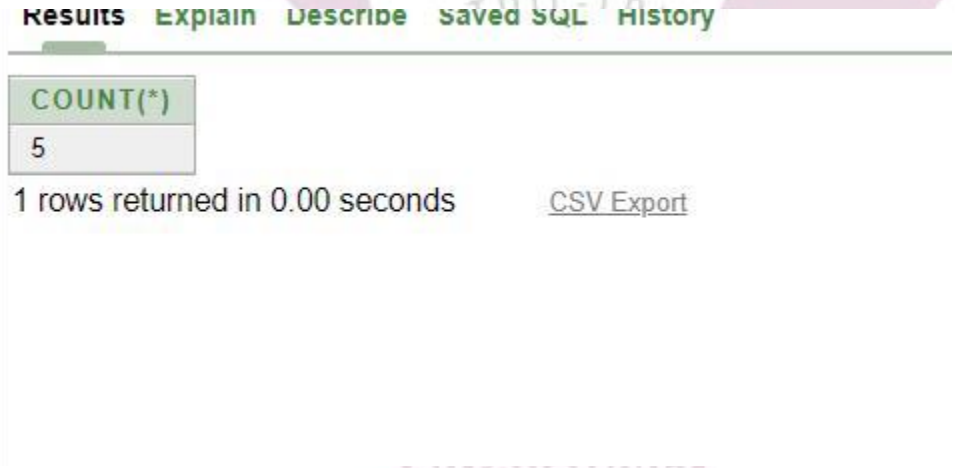
(iv) Count houses having 3 Bedrooms

```
SELECT COUNT(*)

FROM HOUSEPRICE_NEW

WHERE Bedrooms = 3;
```

Output



PRACTICE NO:18 SQL Queries on CANCERDATA Relation

DATE: 24/09/2026

18. **Given the following CANCERDATA relation, write SQL commands for questions (a) to (f).**

a) Display all patients diagnosed with **Breast Cancer**.

b) Display the names of patients whose **Stage** is **III**.

c) Display **PatientName** and **Age** in ascending order of **Age**.

d) Display the names of patients having **SurvivalYears greater than 5**.

e) Count the number of patients diagnosed with **Lung Cancer**.
f) Insert the following record:

308, 'Anand', 'Leukemia', 50, 'II', 7

(g) Write the output of the following SQL commands.

(i) SELECT COUNT(DISTINCT CancerType) FROM CANCERDATA;

(ii) SELECT MAX(SurvivalYears) FROM CANCERDATA;

(iii) SELECT AVG(Age) FROM CANCERDATA WHERE CancerType='Breast';

(iv) SELECT MIN(Age) FROM CANCERDATA WHERE Stage='II'

Aim

To create the **CANCERDATA** table and perform SQL operations to retrieve, filter, sort, count, insert, and calculate aggregate values from cancer patient records.

Description

This practical demonstrates the use of SQL commands such as SELECT, WHERE, ORDER BY, COUNT(), MAX(), AVG(), MIN(), and INSERT. The queries are used to display patients based on cancer type and stage, arrange patients by age, identify patients with higher survival years, count patients with Lung Cancer, and calculate aggregate values.

Software Used

- **Oracle Database 21c / Oracle XE**
- **Oracle SQL Developer**
- **SQL*Plus**

1. Create CANCERDATA table

```
CREATE TABLE CANCERDATA (
   PatientID INT,
   PatientName VARCHAR(30),
   CancerType VARCHAR(20),
   Age INT,
   Stage VARCHAR(5),
   SurvivalYears INT
);
```

2. Insert records

```
INSERT INTO CANCERDATA VALUES
(301, 'Rahul', 'Breast', 45, 'II', 6);
INSERT INTO CANCERDATA VALUES
(302, 'Priya', 'Lung', 52, 'III', 4);
INSERT INTO CANCERDATA VALUES
(303, 'Meena', 'Breast', 38, 'I', 8);
INSERT INTO CANCERDATA VALUES
(304, 'Arun', 'Lung', 60, 'II', 3);
INSERT INTO CANCERDATA VALUES
(305, 'Kavitha', 'Liver', 55, 'III', 7);
INSERT INTO CANCERDATA VALUES
(306, 'Suresh', 'Breast', 42, 'II', 5);
INSERT INTO CANCERDATA VALUES
(307, 'Divya', 'Liver', 35, 'I', 9);
```

CANCERDATA Table

Results Explain Describe Saved SQL History

PATIENTID	PATIENTNAME	CANCERTYPE	AGE	STAGE	SURVIVALYEARS
301	Priya	Breast	45	II	8
302	Ravi	Lung	60	III	4
303	Meena	Breast	38	I	6
305	Divya	Liver	50	III	3
306	Suresh	Colon	42	II	7
307	Anjali	Breast	35	I	9

6 rows returned in 0.00 seconds CSV Export

(a)

Display all patients diagnosed with Breast Cancer

```
SELECT *
FROM CANCERDATA
WHERE CancerType = 'Breast';
```

Output

Results Explain Describe Saved SQL History

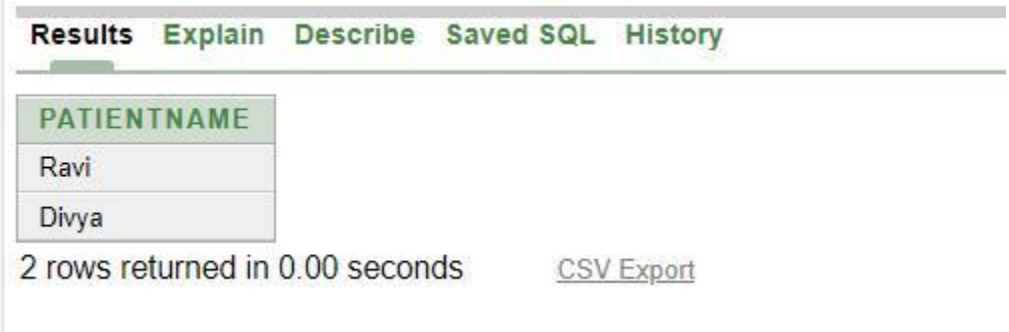
PATIENTID	PATIENTNAME	CANCERTYPE	AGE	STAGE	SURVIVALYEARS
301	Priya	Breast	45	II	8
303	Meena	Breast	38	I	6
307	Anjali	Breast	35	I	9

3 rows returned in 0.00 seconds CSV Export

(b) Display names of patients whose Stage is III

```
SELECT PatientName
FROM CANCERDATA
WHERE Stage = 'III';
```

Output


Results Explain Describe Saved SQL History

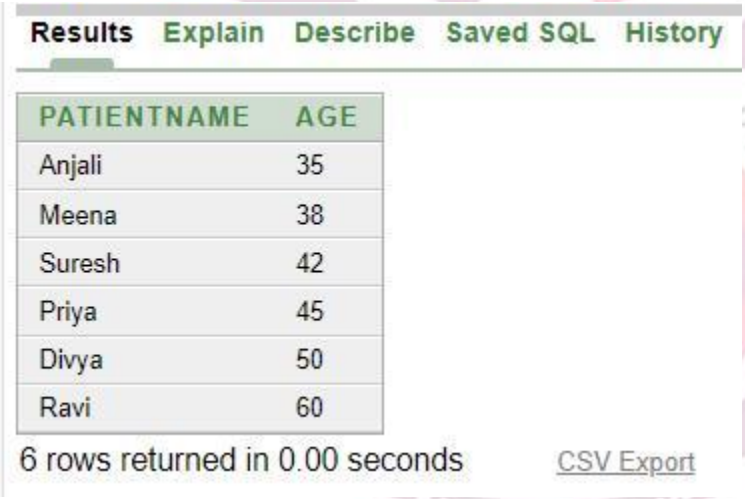
PATIENTNAME
Ravi
Divya

2 rows returned in 0.00 seconds CSV Export

(c) Display PatientName and Age in ascending order of Age

```
SELECT PatientName,
AgeFROM CANCERDATA
ORDER BY Age ASC;
```

Output


Results Explain Describe Saved SQL History

PATIENTNAME	AGE
Anjali	35
Meena	38
Suresh	42
Priya	45
Divya	50
Ravi	60

6 rows returned in 0.00 seconds CSV Export

(d) Display names of patients having SurvivalYears greater than 5

```
SELECT PatientName
FROM CANCERDATA
WHERE SurvivalYears > 5;
```

Output

Results Explain Describe Saved SQL History

PATIENTNAME
Priya
Meena
Suresh
Anjali

4 rows returned in 0.00 seconds CSV Export

(e) Count patients diagnosed with Lung Cancer

```
SELECT COUNT(*)
FROM CANCERDATA
WHERE CancerType = 'Lung';
```

There are **2 Lung Cancer patients**.

Output

COUNT(*)
1

1 rows returned in 0.00 seconds CSV Export

(f)

F. Inserted Record

```
INSERT INTO CANCERDATA_NEW

VALUES

(308, 'Anand', 'Leukemia', 50, 'II', 7);
```

PATIENTID	PATIENTNAME	CANCERTYPE	AGE	STAGE	SURVIVALYEARS
301	Priya	Breast	45	II	8
302	Ravi	Lung	60	III	4
303	Meena	Breast	38	I	6
305	Divya	Liver	50	III	3
306	Suresh	Colon	42	II	7
307	Anjali	Breast	35	I	9
308	Anand	Leukemia	50	II	7

7 rows returned in 0.00 seconds CSV Export

(g) Outputs

(i) Count distinct CancerType

```
SELECT COUNT
(DISTINCT CancerType)
FROM CANCERDATA;
```

Cancer types are:

- Breast
- Lung
- Liver

Output

COUNT(DISTINCTCANCERTYPE)
5

rows returned in 0.00 seconds CSV Export

(ii) Maximum SurvivalYears

```
SELECT MAX(SurvivalYears)
FROM CANCERDATA;
```

The maximum survival is **9 years**.

Output



(iii) Average Age of Breast Cancer patients

```
SELECT AVG(Age)
FROM CANCERDATA
WHERE CancerType = 'Breast';
```

Breast Cancer ages:

- 45
- 38
- 42

Calculation:

(45 + 38 + 42) / 3 = 125 / 3 = 41.67

Output

```
SELECT COUNT(DISTINCT CancerType)
FROM CANCERDATA_NEW;


SELECT MAX(SurvivalYears)
FROM CANCERDATA_NEW;


SELECT AVG(Age)
FROM CANCERDATA_NEW
WHERE CancerType = 'Breast';
```

Results | Explain | Describe | Saved SQL | History

AVG(AGE)
39.333333333333333333333333333333333333

1 rows returned in 0.00 seconds CSV Export

(iv) Minimum Age of Stage II patients

SELECT MIN(Age)
FROM CANCERDATA
WHERE Stage = 'II';

Stage II ages:

- Rahul → 45
- Arun → 60
- Suresh → 42

Minimum = **42**

Output

MIN(AGE)
42

rows returned in 0.00 seconds CSV Export

PRACTICE NO:19 SQL Queries on HOUSEPRICE Relation (Set 2)

DATE: 25/09/2026

19. **Given the following HOUSEPRICE relation, write SQL commands for questions (a) to (f).**

a) Display all houses located in **Chennai**.

b) Display the **Location** and **Price** of houses having **3 Bedrooms**.

c) Display **Location**, **AreaSqft**, and **Price** in descending order of **Price**.

d) Display the details of houses costing more than **₹70,00,000**.

e) Count the number of **Apartments**.

f) Insert the following record:

208, 'Vellore', 3, 1700, 7300000, 'Villa'

(g) Write the output of the following SQL commands.

(i) SELECT MAX(Price)FROM HOUSEPRICE;

(i) SELECT MIN(AreaSqft)FROM HOUSEPRICE;

(ii) SELECT AVG(Price)FROM HOUSEPRICE WHERE HouseType='Apartment';

(iii) SELECT COUNT(*)FROM HOUSEPRICE WHERE Bedrooms=3;

Aim

To create the **HOUSEPRICE_NEW** table and perform SQL operations to display, filter, sort, count, insert, and calculate aggregate values of houses based on location, bedrooms, area, price, and house type.

Description

This practical demonstrates the use of SQL commands such as SELECT, WHERE, ORDER BY, COUNT(), MAX(), MIN(), AVG(), and INSERT. The queries are used to display houses in

Chennai, find houses with 3 bedrooms, arrange houses according to price, identify houses costing more than ₹70,00,000, count apartments, and calculate aggregate values.

Software Used

- **Oracle Database 21c / Oracle XE**
- **Oracle SQL Developer**
- **SQL*Plus**

1. Create the table

```
CREATE TABLE HOUSEPRICE (
   HouseID INT,
   Location VARCHAR(30),
   Bedrooms INT,
   AreaSqft INT,
   Price INT,
   HouseType VARCHAR(20)
);
```

2. Insert sample data

```
INSERT INTO HOUSEPRICE VALUES
(201, 'Chennai', 2, 1200, 5500000, 'Apartment');
INSERT INTO HOUSEPRICE VALUES
(202, 'Chennai', 3, 1500, 6800000, 'Villa');
INSERT INTO HOUSEPRICE VALUES
(203, 'Bangalore', 3, 1600, 7200000, 'Apartment');
INSERT INTO HOUSEPRICE VALUES
(204, 'Coimbatore', 2, 1100, 4500000, 'Apartment');
INSERT INTO HOUSEPRICE VALUES
(205, 'Chennai', 4, 2000, 8500000, 'Villa');
INSERT INTO HOUSEPRICE VALUES
(206, 'Madurai', 3, 1400, 6000000, 'Apartment');
INSERT INTO HOUSEPRICE VALUES
(207, 'Vellore', 2, 1000, 4000000, 'Villa');
```

HOUSEPRICE Table

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
201	Chennai	2	1200	5500000	Apartment
202	Chennai	3	1500	7500000	Villa
203	Bangalore	2	1000	4800000	Apartment
204	Coimbatore	3	1400	6500000	Apartment
205	Chennai	4	2000	9000000	Villa
206	Madurai	2	1100	5000000	Apartment
207	Salem	3	1300	6000000	Villa

7 rows returned in 0.01 seconds CSV Export

(a) Display all houses located in Chennai

```
SELECT *
FROM HOUSEPRICE
WHERE Location = 'Chennai';
```

Output

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
201	Chennai	2	1200	5500000	Apartment
202	Chennai	3	1500	7500000	Villa
205	Chennai	4	2000	9000000	Villa

rows returned in 0.00 seconds CSV Export

(b) Display Location and Price of houses having 3 Bedrooms

```
SELECT Location, Price
FROM HOUSEPRICE
WHERE Bedrooms = 3;
```

Output

Results Explain Describe Saved SQL History

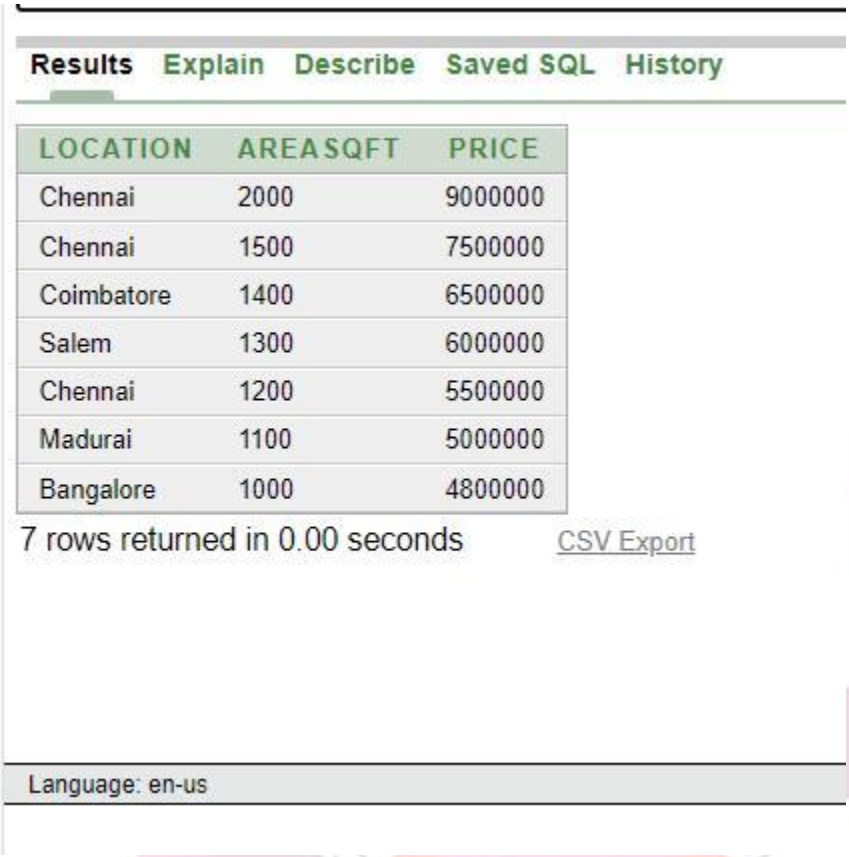
LOCATION	PRICE
Chennai	7500000
Coimbatore	6500000
Salem	6000000

3 rows returned in 0.01 seconds CSV Export

(c) Display Location, AreaSqft and Price in descending order of Price

```
SELECT Location, AreaSqft, Price
FROM HOUSEPRICE
ORDER BY Price DESC;
```

Output



Results Explain Describe Saved SQL History

LOCATION	AREASQFT	PRICE
Chennai	2000	9000000
Chennai	1500	7500000
Coimbatore	1400	6500000
Salem	1300	6000000
Chennai	1200	5500000
Madurai	1100	5000000
Bangalore	1000	4800000

7 rows returned in 0.00 seconds CSV Export

Language: en-us

(d) Display details of houses costing more than ₹70,00,000

SELECT *FROM HOUSEPRICEWHERE Price > 7000000;

Output

(e) Count the number of Apartments

SELECT COUNT(*)
FROM HOUSEPRICE
WHERE HouseType = 'Apartment';

Output

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
202	Chennai	3	1500	7500000	Villa
205	Chennai	4	2000	9000000	Villa

2 rows returned in 0.00 seconds CSV Export

(f) Insert the given record

INSERT INTO HOUSEPRICEVALUES
(208, 'Vellore', 3, 1700, 7300000, 'Villa');

Inserted Record

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
201	Chennai	2	1200	5500000	Apartment
202	Chennai	3	1500	7500000	Villa
203	Bangalore	2	1000	4800000	Apartment
204	Coimbatore	3	1400	6500000	Apartment
205	Chennai	4	2000	9000000	Villa
206	Madurai	2	1100	5000000	Apartment
207	Salem	3	1300	6000000	Villa
208	Vellore	3	1700	7300000	Villa

(g) Outputs
(i) Maximum Price

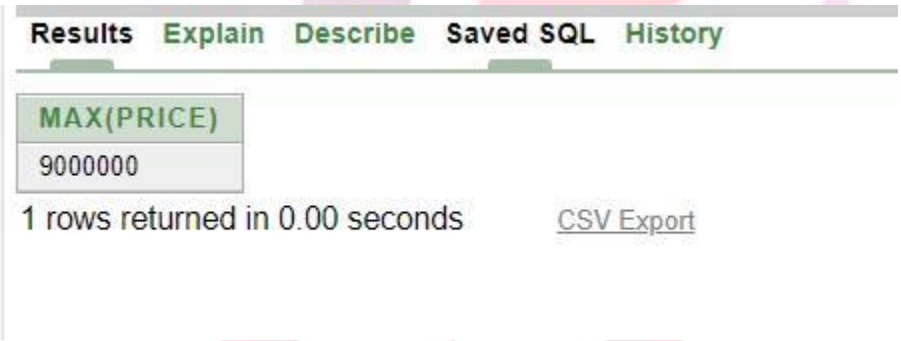
SELECT MAX(Price)
FROM HOUSEPRICE;

Output:



(ii) Minimum AreaSqft

SELECT MIN(AreaSqft)
FROM HOUSEPRICE;



Output:

(iii) Average Price of Apartments

SELECT AVG(Price)
FROM HOUSEPRICE
WHERE HouseType = 'Apartment';

Apartment prices:

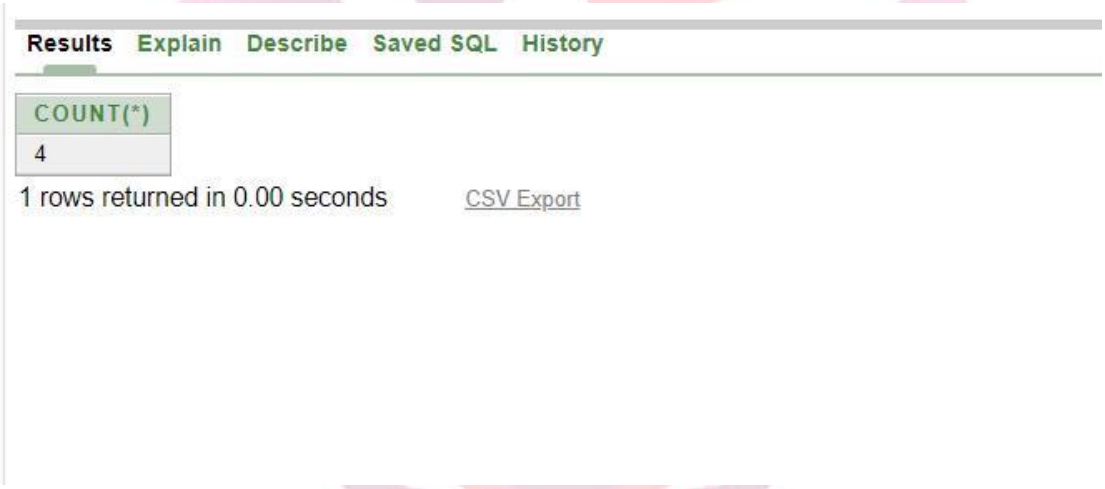
5500000 + 7200000 + 4500000 + 6000000
Average = 5800000



Output:

(iv) Count houses having 3 Bedrooms

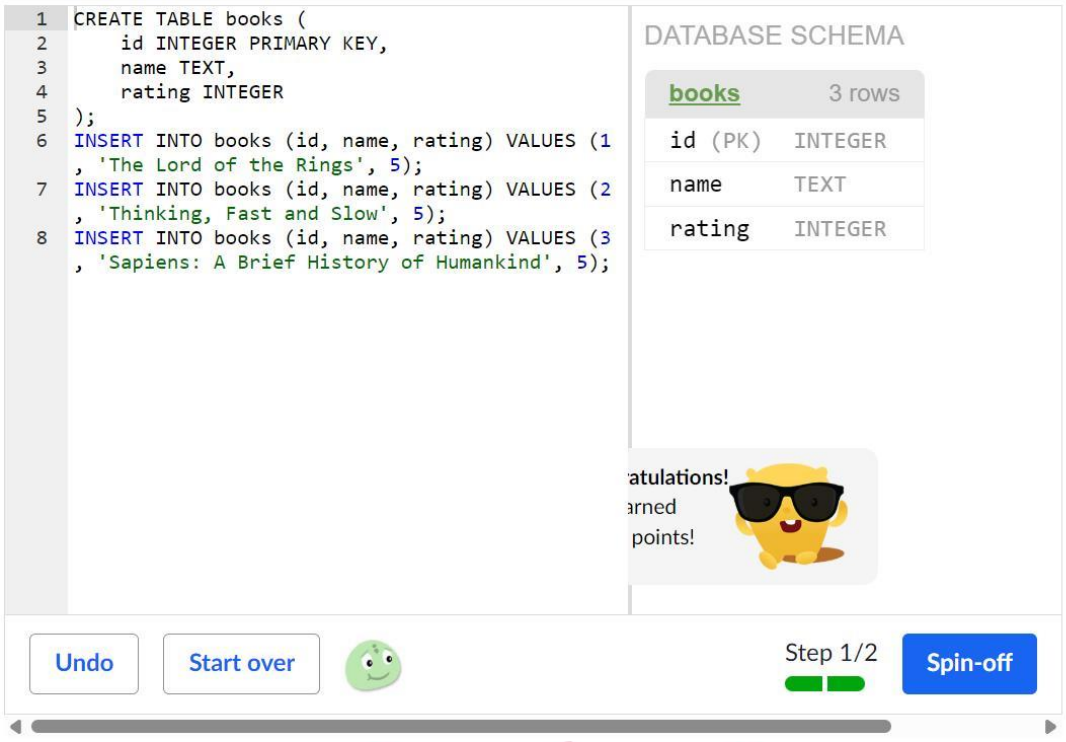
```
SELECT COUNT(*)
FROM HOUSEPRICE
WHERE Bedrooms = 3;
```



ANNEXURE - KHAN ACADEMY

Step 1

What are your favorite books? You can make a database table to store them in! In this first step, create a table to store your list of books. It should have columns for id, name, and rating.



Step 2

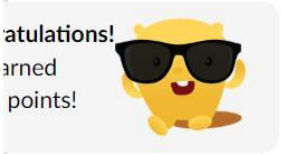
Now, add three of your favorite books into the table.

Hint
INSERT INTO _ VALUES (…);

```sql
CREATE TABLE books (
    id INTEGER PRIMARY KEY,
    name TEXT,
    rating INTEGER
);
INSERT INTO books (id, name, rating) VALUES (1, 'The Lord of the Rings', 5);
INSERT INTO books (id, name, rating) VALUES (2, 'Thinking, Fast and Slow', 5);
INSERT INTO books (id, name, rating) VALUES (3, 'Sapiens: A Brief History of Humankind', 5);
```

DATABASE SCHEMA

books	3 rows
id (PK)	INTEGER
name	TEXT
rating	INTEGER



2. This database contains an incomplete list of box office hits and their release year. In this challenge, you're going to get the results back out of the database in different ways! In this first step, just select all the movies.

```sql
CREATE TABLE movies (id INTEGER PRIMARY KEY, name TEXT, release_year INTEGER);

INSERT INTO movies VALUES (1, "Avatar", 2009);
INSERT INTO movies VALUES (2, "Titanic", 1997);
INSERT INTO movies VALUES (3, "Star Wars: Episode IV - A New Hope", 1977);
INSERT INTO movies VALUES (4, "Shrek 2", 2004);
INSERT INTO movies VALUES (5, "The Lion King", 1994);
INSERT INTO movies VALUES (6, "Disney's Up", 2009);

SELECT * FROM movies;

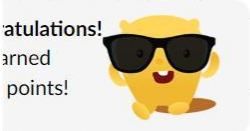
SELECT * FROM movies
WHERE release_year >= 2000
ORDER BY release_year;
```

DATABASE SCHEMA

movies	6 rows
id (PK)	INTEGER
name	TEXT
release_year	INTEGER

QUERY RESULTS

id	name	rel
1	Avatar	20
2	Titanic	19
3	Episode Hope	19
4		20
5	The Lion King	19



Step 1

Here's a table containing a TODO list with the number of minutes it will take to complete each item. Insert another item to your todo list with the estimated minutes it will take.

```
CREATE TABLE todo_list (id INTEGER PRIMARY KEY
, item TEXT, minutes INTEGER);

INSERT INTO todo_list VALUES (1, "Wash the
dishes", 15);
INSERT INTO todo_list VALUES (2, "vacuuming",
20);
INSERT INTO todo_list VALUES (3, "Learn some
stuff on KA", 30);

INSERT INTO todo_list VALUES (4, "Do laundry",
45);
SELECT SUM(minutes) FROM todo_list;
```

DATABASE SCHEMA

todo_list	4 rows
id (PK)	INTEGER
item	TEXT
minutes	INTEGER

QUERY RESULTS

SUM(minutes)
110



Step 2

Select the SUM of minutes it will take to do all of the items on your TODO list. You should only have one SELECT statement.

Hint

SELECT _(…) …;

```
CREATE TABLE todo_list (id INTEGER PRIMARY KEY
, item TEXT, minutes INTEGER);

INSERT INTO todo_list VALUES (1, "Wash the
dishes", 15);
INSERT INTO todo_list VALUES (2, "vacuuming",
20);
INSERT INTO todo_list VALUES (3, "Learn some
stuff on KA", 30);

INSERT INTO todo_list VALUES (4, "Do laundry",
45);
SELECT SUM(minutes) FROM todo_list;
```

DATABASE SCHEMA

todo_list	4 rows
id (PK)	INTEGER
item	TEXT
minutes	INTEGER

QUERY RESULTS

SUM(minutes)
110

3.Create your own store! Your store should sell one type of things, like clothing or bikes,

whatever you want your store to specialize in. You should have a table for all the items in your store, and at least 5 columns for the kind of data you think you'd need to store. You should sell at least 15 items, and use select statements to order your items by price and show at least one statistic about the items.

Your project is now marked as complete. You can continue working on it, of course, if you think of other ways to improve it. Keep practicing!

```
CREATE TABLE bike_store (
    id INTEGER PRIMARY KEY,
    name TEXT,
    brand TEXT,
    price INTEGER,
    color TEXT,
    stock INTEGER,
    wheel_size INTEGER
);

INSERT INTO bike_store VALUES (1, "Mountain Bike X1", "Trek", 500, "Red", 10, 29);
INSERT INTO bike_store VALUES (2, "Road Bike R2", "Giant", 700, "Blue", 5, 28);
INSERT INTO bike_store VALUES (3, "City Bike C3", "Cannondale", 400, "Black", 8, 27);
INSERT INTO bike_store VALUES (4, "Mountain Bike X2", "Trek", 650, "Green", 6, 29);
INSERT INTO bike_store VALUES (5, "Road Bike R3", "Giant", 850, "White", 4, 28);
INSERT INTO bike_store VALUES (6, "Hybrid Bike H1", "Specialized", 550, "Gray", 7, 28);
INSERT INTO bike_store VALUES (7, "Electric
```

DATABASE SCHEMA

bike_store	15 rows
id (PK)	INTEGER
name	TEXT
brand	TEXT
price	INTEGER
color	TEXT
stock	INTEGER
wheel_size	INTEGER

QUERY RESULTS

id	name	brand
8	Kids	Schwinn

```
INSERT INTO bike_store VALUES (8, "Kids Bike K1", "Schwinn", 200, "Yellow", 12, 20);
INSERT INTO bike_store VALUES (9, "BMX Bike B1", "Mongoose", 300, "Orange", 9, 20);
INSERT INTO bike_store VALUES (10, "Cruiser Bike CR1", "Huffy", 350, "Pink", 5, 26);
INSERT INTO bike_store VALUES (11, "Mountain Bike X3", "Trek", 800, "Blue", 4, 29);
INSERT INTO bike_store VALUES (12, "Road Bike R4", "Giant", 950, "Red", 2, 28);
INSERT INTO bike_store VALUES (13, "Hybrid Bike H2", "Specialized", 600, "White", 6, 28);
INSERT INTO bike_store VALUES (14, "Electric Bike E2", "Rad", 1500, "Silver", 2, 27);
INSERT INTO bike_store VALUES (15, "Kids Bike K2", "Schwinn", 250, "Purple", 10, 20);

-- Order items by price (lowest to highest)
SELECT * FROM bike_store
ORDER BY price;
```

DATABASE SCHEMA

bike_store	15 rows
id (PK)	INTEGER
name	TEXT
brand	TEXT
price	INTEGER
color	TEXT
stock	INTEGER
wheel_size	INTEGER

QUERY RESULTS

id	name	brand
8	Kids	Schwinn

```sql
-- Order items by price (lowest to highest)
SELECT * FROM bike_store
ORDER BY price;

-- Statistics
SELECT COUNT(*) AS total_items FROM
bike_store;

SELECT AVG(price) AS average_price FROM
bike_store;

SELECT MAX(price) AS highest_price FROM
bike_store;

SELECT MIN(price) AS lowest_price FROM
bike_store;

SELECT SUM(stock) AS total_stock FROM
bike store;
```

name	TEXT
brand	TEXT
price	INTEGER
color	TEXT
stock	INTEGER
wheel_size	INTEGER

QUERY RESULTS

id	name	brand
	Kids	

Step 1

Ever sung karaoke? It's a place where you sing songs with your friends, and it's a lot of fun. We've created a table with songs, and in this challenge, you'll use queries to decide what songs to sing. For the first step, select all the song titles.

```sql
CREATE TABLE songs (
    id INTEGER PRIMARY KEY,
    title TEXT,
    artist TEXT,
    mood TEXT,
    duration INTEGER,
    released INTEGER
);

INSERT INTO songs (title, artist, mood,
duration, released)
    VALUES ("Bohemian Rhapsody", "Queen",
"epic", 60, 1975);
INSERT INTO songs (title, artist, mood,
duration, released)
    VALUES ("Let it go", "Idina Menzel",
"epic", 227, 2013);
INSERT INTO songs (title, artist, mood,
duration, released)
    VALUES ("I will survive", "Gloria
Gaynor", "epic", 198, 1978);
INSERT INTO songs (title, artist, mood,
duration, released)
    VALUES ("Twist and Shout", "The
Beatles", "happy", 152, 1963);
```

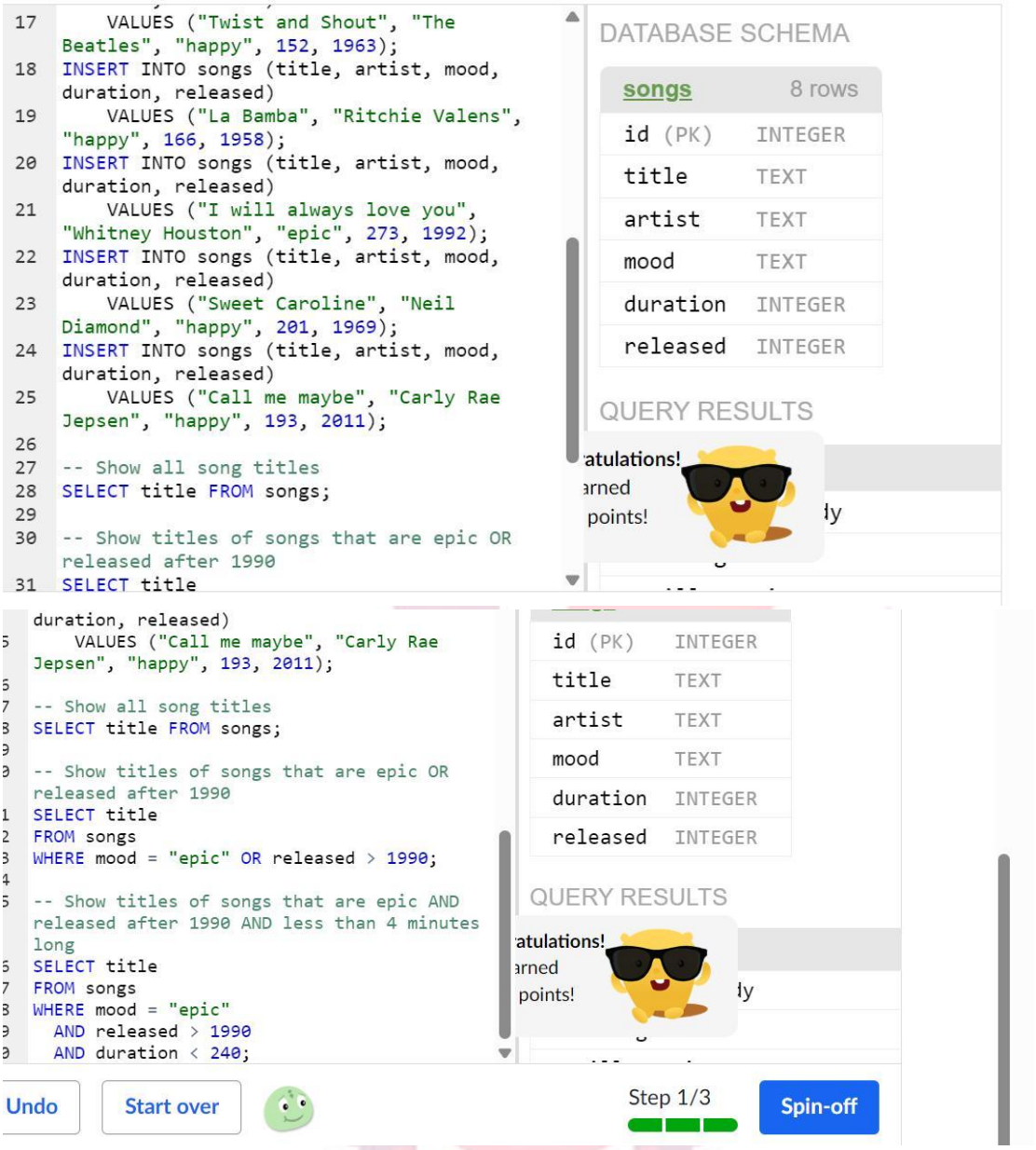
DATABASE SCHEMA

songs	8 rows
id (PK)	INTEGER
title	TEXT
artist	TEXT
mood	TEXT
duration	INTEGER
released	INTEGER

QUERY RESULTS

atulations!
arned
points!





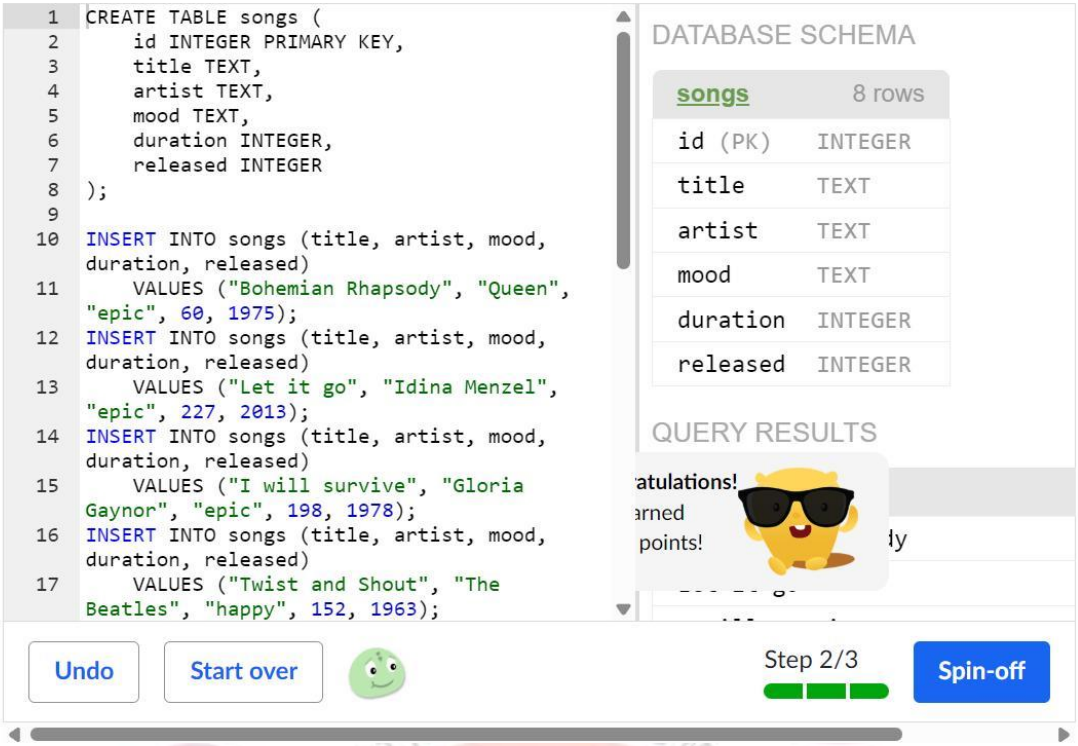
Step 2

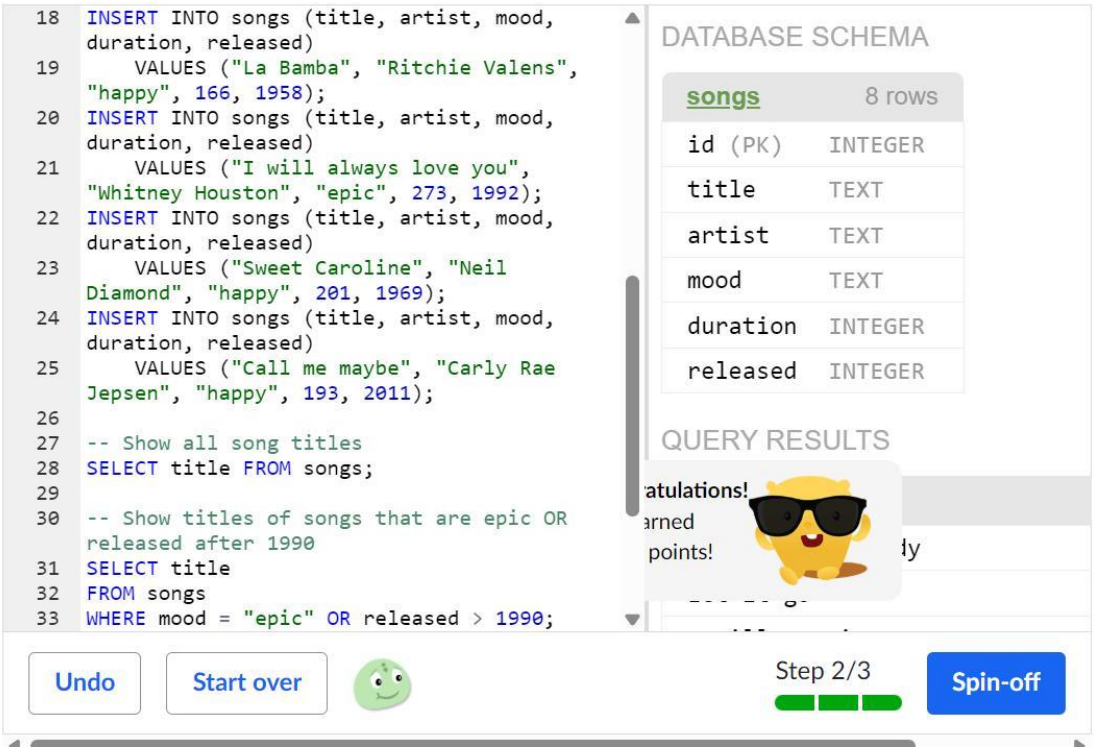
Maybe your friends only like singing either recent songs or truly epic songs. Add another SELECT that uses OR to show the titles of the songs that have an 'epic' mood or a release date after 1990.

(Tip: If you're not sure how to select rows where a column equals a value, remember that you can check the documentation below.)

Hint

SELECT _ WHERE … OR …;





```
duration, released)
    VALUES ("Sweet Caroline", "Neil Diamond", "happy", 201, 1969);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Call me maybe", "Carly Rae Jepsen", "happy", 193, 2011);

-- Show all song titles
SELECT title FROM songs;

-- Show titles of songs that are epic OR released after 1990
SELECT title
FROM songs
WHERE mood = "epic" OR released > 1990;

-- Show titles of songs that are epic AND released after 1990 AND less than 4 minutes long
SELECT title
FROM songs
WHERE mood = "epic"
  AND released > 1990
  AND duration < 240;
```

DATABASE SCHEMA

songs	8 rows
id (PK)	INTEGER
title	TEXT
artist	TEXT
mood	TEXT
duration	INTEGER
released	INTEGER

QUERY RESULTS

atulations!
arned
points!

Undo | Start over | Step 2/3 | Spin-off

Step 3

People get picky at the end of the night. Add another SELECT that uses AND to show the titles of songs that are 'epic', **and** released after 1990, **and** less than 4 minutes long.
Note that the duration column is measured in seconds.

```
CREATE TABLE songs (
    id INTEGER PRIMARY KEY,
    title TEXT,
    artist TEXT,
    mood TEXT,
    duration INTEGER,
    released INTEGER
);

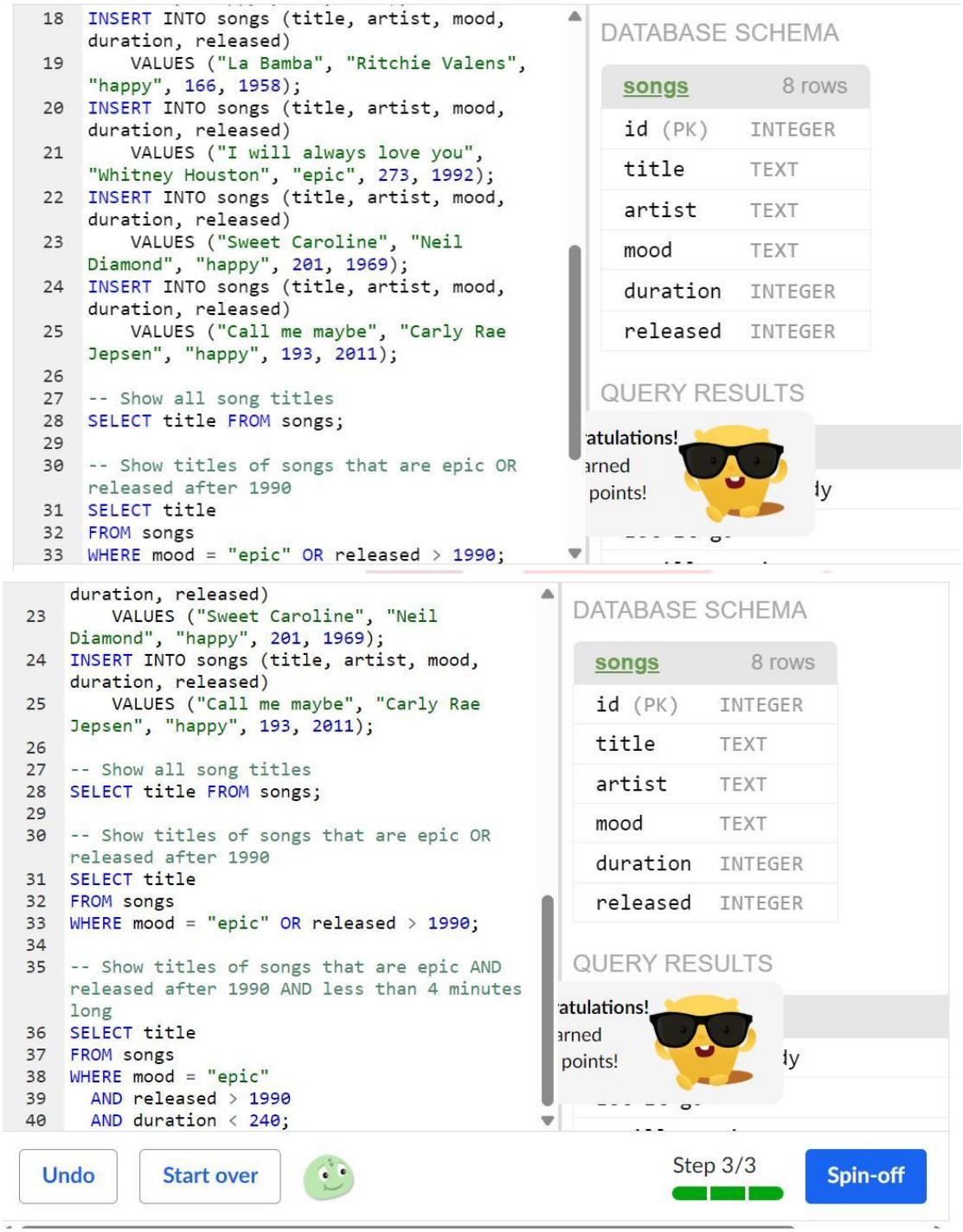
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Bohemian Rhapsody", "Queen", "epic", 60, 1975);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Let it go", "Idina Menzel", "epic", 227, 2013);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will survive", "Gloria Gaynor", "epic", 198, 1978);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Twist and Shout", "The Beatles", "happy", 152, 1963);
```

DATABASE SCHEMA

songs	8 rows
id (PK)	INTEGER
title	TEXT
artist	TEXT
mood	TEXT
duration	INTEGER
released	INTEGER

QUERY RESULTS

atulations!
arned
points!



Step 1

We've created a database of songs and artists, and you'll make playlists from them in this challenge. In this first step, select the title of all the songs by the artist named 'Queen'.

Hint

SELECT title FROM …;

```
CREATE TABLE artists (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    country TEXT,
    genre TEXT
);

INSERT INTO artists (name, country, genre)
    VALUES ("Taylor Swift", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Led Zeppelin", "US", "Hard
rock");
INSERT INTO artists (name, country, genre)
    VALUES ("ABBA", "Sweden", "Disco");
INSERT INTO artists (name, country, genre)
    VALUES ("Queen", "UK", "Rock");
INSERT INTO artists (name, country, genre)
    VALUES ("Celine Dion", "Canada", "Pop"
);
INSERT INTO artists (name, country, genre)
    VALUES ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("Garth Brooks", "US", "Country"
);
INSERT INTO artists (name, country, genre)
    VALUES ("Shania Twain", "Canada",
"Country");
INSERT INTO artists (name, country, genre)
    VALUES ("Rihanna", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Guns N' Roses", "US", "Hard
rock");
INSERT INTO artists (name, country, genre)
    VALUES ("Gloria Estefan", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Bob Marley", "Jamaica",
"Reggae");

CREATE TABLE songs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    artist TEXT,
    title TEXT
);

INSERT INTO songs (artist, title)
    VALUES ("Taylor Swift", "Shake it off"
);
INSERT INTO songs (artist, title)
    VALUES ("Rihanna", "Stay");
INSERT INTO songs (artist, title)
```

DATABASE SCHEMA

artists	12 rows
id (PK)	INTEGER
name	TEXT
country	TEXT
genre	TEXT

songs	11 rows
id (PK)	INTEGER
artist	TEXT

atulations!
arned
points!



title

DATABASE SCHEMA

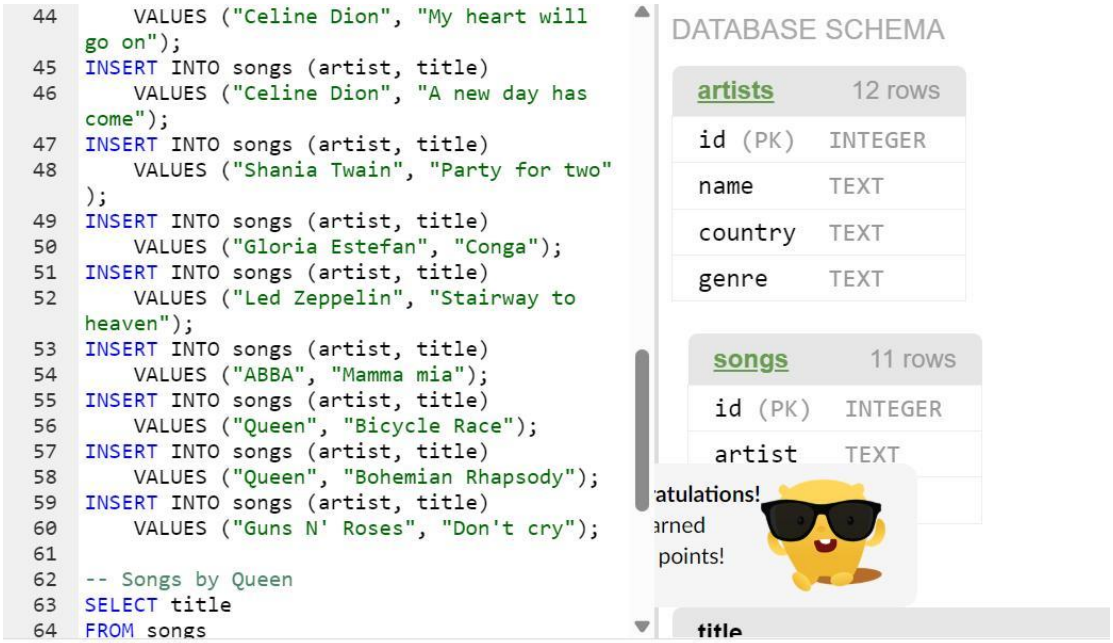
artists	12 rows
id (PK)	INTEGER
name	TEXT
country	TEXT
genre	TEXT

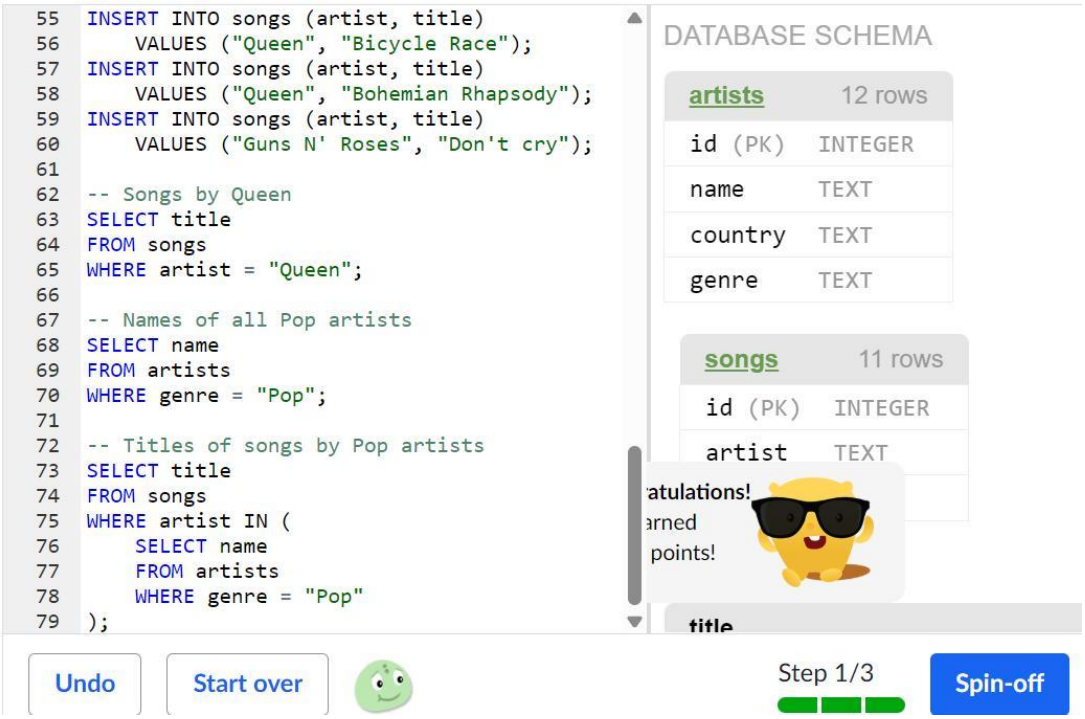
songs	11 rows
id (PK)	INTEGER
artist	TEXT

atulations!
arned
points!



title





Step 2

Now you'll make a 'Pop' playlist. In preparation, select the name of all of the artists from the 'Pop' genre.

(Tip: Make sure you type it 'Pop', SQL considers that different from 'pop'.)



```
    VALUES ("Celine Dion", "My heart will
go on");
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "A new day has
come");
INSERT INTO songs (artist, title)
    VALUES ("Shania Twain", "Party for two"
);
INSERT INTO songs (artist, title)
    VALUES ("Gloria Estefan", "Conga");
INSERT INTO songs (artist, title)
    VALUES ("Led Zeppelin", "Stairway to
heaven");
INSERT INTO songs (artist, title)
    VALUES ("ABBA", "Mamma mia");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bicycle Race");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bohemian Rhapsody");
INSERT INTO songs (artist, title)
    VALUES ("Guns N' Roses", "Don't cry");

-- Songs by Queen
SELECT title
FROM songs
```

DATABASE SCHEMA

artists 12 rows

id (PK)	INTEGER
name	TEXT
country	TEXT
genre	TEXT

songs 11 rows

id (PK)	INTEGER
artist	TEXT

atulations!
arned
points!

title

Undo | Start over | Step 2/3 | Spin-off

```
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bicycle Race");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bohemian Rhapsody");
INSERT INTO songs (artist, title)
    VALUES ("Guns N' Roses", "Don't cry");

-- Songs by Queen
SELECT title
FROM songs
WHERE artist = "Queen";

-- Names of all Pop artists
SELECT name
FROM artists
WHERE genre = "Pop";

-- Titles of songs by Pop artists
SELECT title
FROM songs
WHERE artist IN (
    SELECT name
    FROM artists
    WHERE genre = "Pop"
);
```

DATABASE SCHEMA

artists 12 rows

id (PK)	INTEGER
name	TEXT
country	TEXT
genre	TEXT

songs 11 rows

id (PK)	INTEGER
artist	TEXT

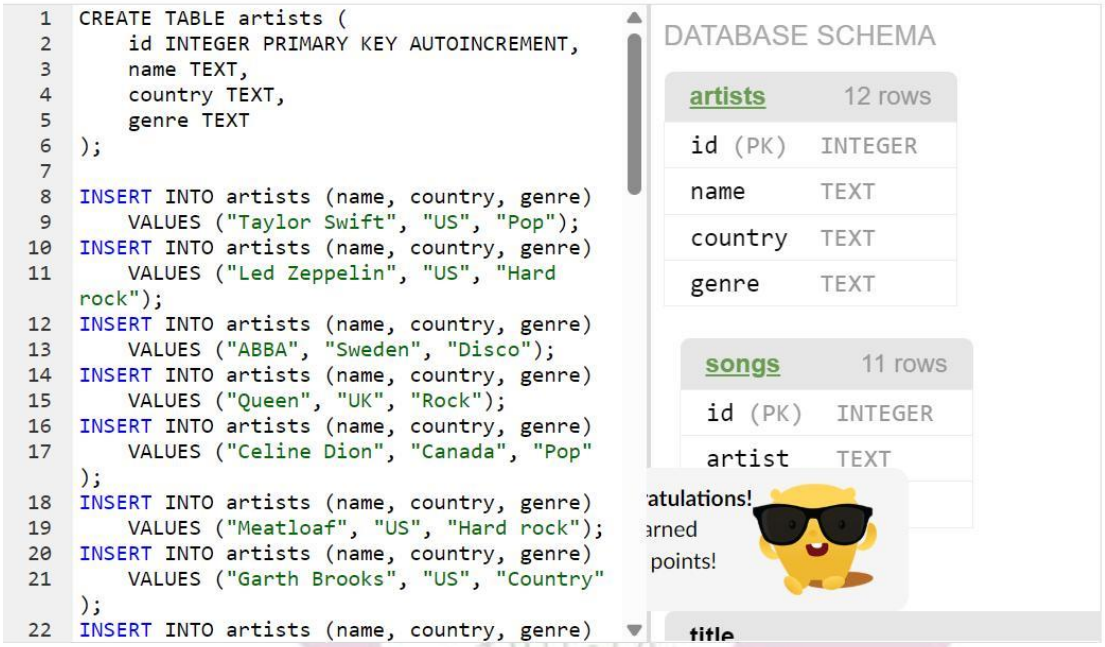
atulations!
arned
points!

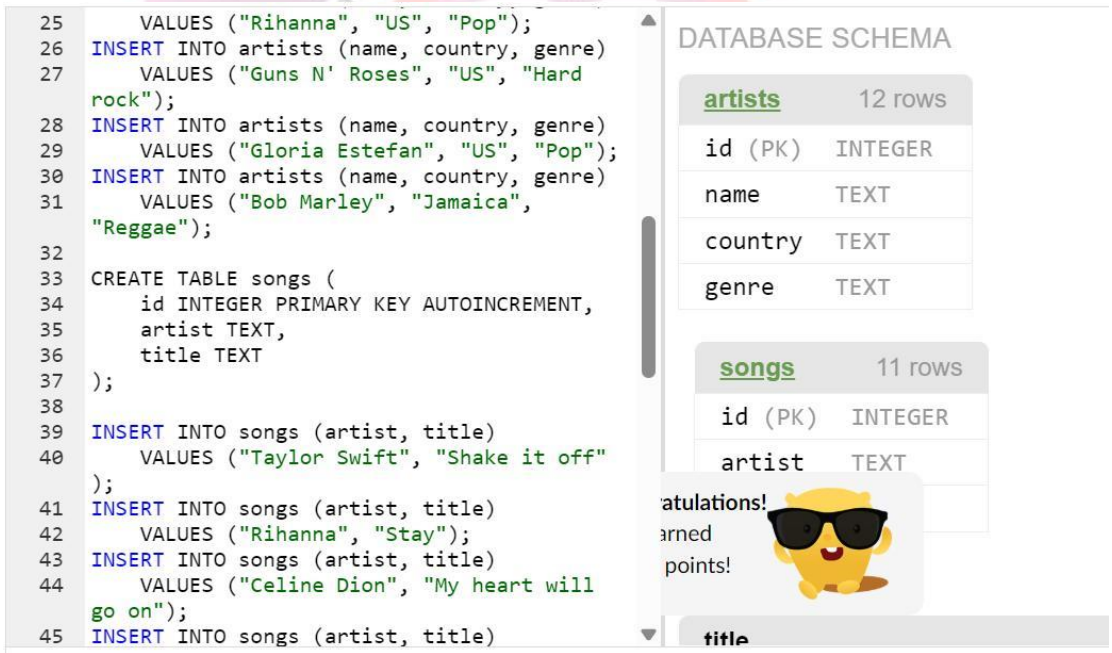
title

Undo | Start over | Step 2/3 | Spin-off

Step 3

To finish creating the 'Pop' playlist, add another query that will select the title of all the songs from the 'Pop' artists. It should use IN on a nested subquery that's based on your previous query.





```
    VALUES ("Celine Dion", "A new day has come");
INSERT INTO songs (artist, title)
    VALUES ("Shania Twain", "Party for two");
INSERT INTO songs (artist, title)
    VALUES ("Gloria Estefan", "Conga");
INSERT INTO songs (artist, title)
    VALUES ("Led Zeppelin", "Stairway to heaven");
INSERT INTO songs (artist, title)
    VALUES ("ABBA", "Mamma mia");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bicycle Race");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bohemian Rhapsody");
INSERT INTO songs (artist, title)
    VALUES ("Guns N' Roses", "Don't cry");

-- Songs by Queen
SELECT title
FROM songs
WHERE artist = "Queen";

-- Names of all Pop artists
```

DATABASE SCHEMA

artists 12 rows

id (PK)	INTEGER
name	TEXT
country	TEXT
genre	TEXT

songs 11 rows

id (PK)	INTEGER
artist	TEXT

atulations!
arned
points!



title

Step 1

We've created a database of a few popular authors and their books, with word counts for each book. In this first step, select all the authors who have written more than 1 million words, using GROUP BY and HAVING. Your results table should include the 'author' and their total word count as a 'total_words' column.

```
CREATE TABLE books (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    author TEXT,
    title TEXT,
    words INTEGER);

INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
INSERT INTO books (author, title, words)
```

DATABASE SCHEMA

books 15 rows

id (PK)	INTEGER
author	TEXT
title	TEXT
words	INTEGER

QUERY RESULTS

author	total_wo
J.K. Rowling	1086594

atulations!
arned
points!



avg_words
155227.7142

```
    VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);

INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Twilight", 118501);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "New Moon", 132807);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Eclipse", 147930);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Breaking Dawn", 192196);

INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "The Hobbit", 95022);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Two Towers", 143436);
```

DATABASE SCHEMA

books	15 rows
id (PK)	INTEGER
author	TEXT
title	TEXT
words	INTEGER

QUERY RESULTS

author	total_wo
J.K. Rowling	1086594

atulations!
arned
points!



avg_words
155227.71428

```
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "The Hobbit", 95022);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Two Towers", 143436);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Return of the King", 134462);
    SELECT
    author,
    SUM(words) AS total_words
FROM books
GROUP BY author
HAVING SUM(words) > 1000000;
 SELECT
    author,
    AVG(words) AS avg_words
FROM books
GROUP BY author
HAVING AVG(words) > 150000;
```

DATABASE SCHEMA

books	15 rows
id (PK)	INTEGER
author	TEXT
title	TEXT
words	INTEGER

QUERY RESULTS

author	total_wo
J.K. Rowling	1086594

atulations!
arned
points!



avg_words
155227.7142

AUTONOMOUS

```sql
CREATE TABLE books (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    author TEXT,
    title TEXT,
    words INTEGER);

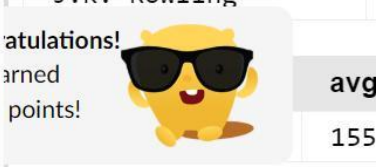
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
INSERT INTO books (author, title, words)
```

DATABASE SCHEMA

books	15 rows
id (PK)	INTEGER
author	TEXT
title	TEXT
words	INTEGER

QUERY RESULTS

author
J.K. Rowling



avg
155

```sql
    VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);

INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Twilight", 118501);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "New Moon", 132807);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Eclipse", 147930);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Breaking Dawn", 192196);

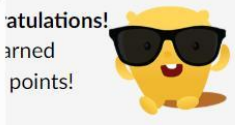
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "The Hobbit", 95022);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Two Towers", 143436);
```

DATABASE SCHEMA

books	15 rows
id (PK)	INTEGER
author	TEXT
title	TEXT
words	INTEGER

QUERY RESULTS

author	total_w
J.K. Rowling	1086594



avg_words
155227.7142

```
30
31  INSERT INTO books (author, title, words)
32      VALUES ("J.R.R. Tolkien", "The Hobbit",
    95022);
33  INSERT INTO books (author, title, words)
34      VALUES ("J.R.R. Tolkien", "Fellowship
    of the Ring", 177227);
35  INSERT INTO books (author, title, words)
36      VALUES ("J.R.R. Tolkien", "Two Towers",
    143436);
37  INSERT INTO books (author, title, words)
38      VALUES ("J.R.R. Tolkien", "Return of
    the King", 134462);
39      SELECT
40      author,
41      SUM(words) AS total_words
42  FROM books
43  GROUP BY author
44  HAVING SUM(words) > 1000000;
45   SELECT
46      author,
47      AVG(words) AS avg_words
48  FROM books
49  GROUP BY author
50  HAVING AVG(words) > 150000;
```

DATABASE SCHEMA

books	15 rows
id (PK)	INTEGER
author	TEXT
title	TEXT
words	INTEGER

QUERY RESULTS

author	total_wo
J.K. Rowling	1086594

avg_words
155227.71428



Step 1

We've created a database to track student grades, with their name, number grade, and what percent of activities they've completed. In this first step, select all of the rows, and display the name, number_grade, and percent_completed, which you can compute by multiplying and rounding the fraction_completed column.

```
1   CREATE TABLE student_grades (
2       id INTEGER PRIMARY KEY AUTOINCREMENT,
3       name TEXT,
4       number_grade INTEGER,
5       fraction_completed REAL
6   );
7
8   INSERT INTO student_grades (name, number_gr
    ade, fraction_completed) VALUES ("Winston",
    90, 0.805);
9   INSERT INTO student_grades (name, number_gr
    ade, fraction_completed) VALUES ("Winnefer"
    , 95, 0.901);
10  INSERT INTO student_grades (name, number_gr
    ade, fraction_completed) VALUES ("Winsteen"
    , 85, 0.906);
11  INSERT INTO student_grades (name, number_gr
    ade, fraction_completed) VALUES ("Wincifer"
    , 66, 0.7054);
12  INSERT INTO student_grades (name, number_gr
    ade, fraction_completed) VALUES ("Winster",
    76, 0.5013);
13  INSERT INTO student_grades (name, number_gr
    ade, fraction_completed) VALUES ("Winstonia
    ", 82, 0.9045);
```

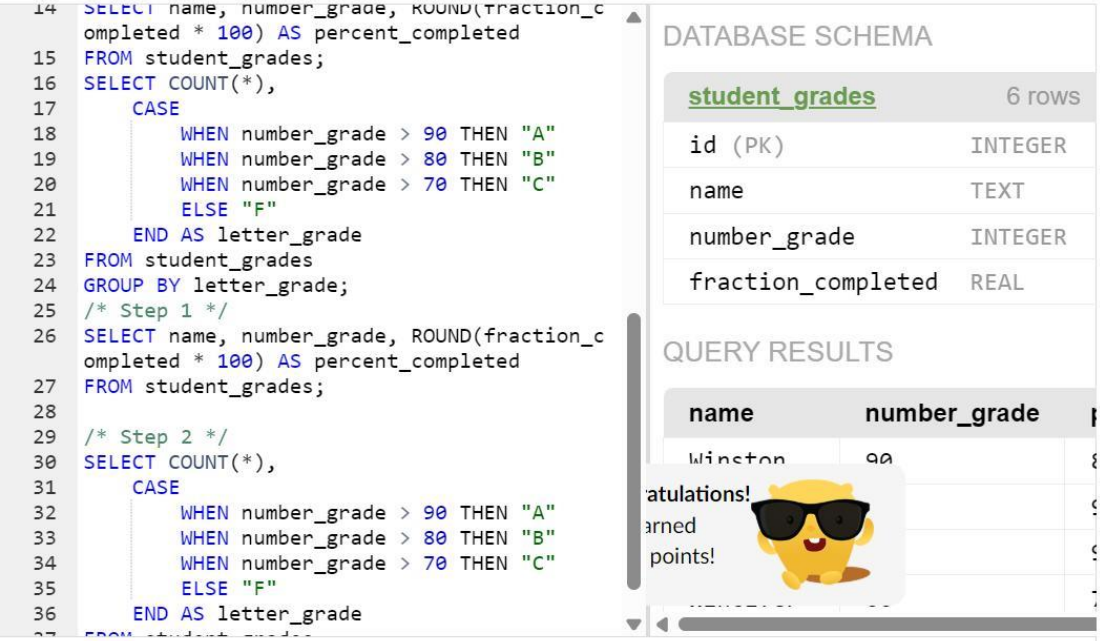
DATABASE SCHEMA

student_grades	6 rows
id (PK)	INTEGER
name	TEXT
number_grade	INTEGER
fraction_completed	REAL

QUERY RESULTS

name	number_grade
Winston	90





We've curated a set of interesting data sets for you: NASA astronauts, Superbowl results, Pokemon stats, NBA players, Top movies, Top countries by population, Solar system objects by size, Marvel characters, Furniture store sales, Earned KA badges, Winston's donut logs, Card game results, and NFL draft picks.

Pick one of those data sets or create a data set like that, and use advanced SELECT queries to discover things about the data. What sort of questions might one have about that data, like if they were using it for an app or a business idea? Here are some ideas:

- What are average, max, and min values in the data?
- What about those numbers per category in the data (using HAVING)?
- What ways are there to group the data values that don't exist yet (using CASE)?
- What interesting ways are there to filter the data (using AND/OR)?

Your project is now marked as complete. You can continue working on it, of course, if you think of other ways to improve it. Keep practicing!

```sql
CREATE TABLE pokemon (
    id NUMBER,
    name VARCHAR2(50),
    type VARCHAR2(20),
    hp NUMBER,
    attack NUMBER,
    defense NUMBER,
    speed NUMBER
);
SELECT
    AVG(hp) AS avg_hp,
    MAX(hp) AS max_hp,
    MIN(hp) AS min_hp,
    AVG(attack) AS avg_attack,
    MAX(attack) AS max_attack,
    MIN(attack) AS min_attack
FROM pokemon;

SELECT
    type,
    COUNT(*) AS total_pokemon,
    AVG(hp) AS avg_hp,
    MAX(attack) AS max_attack,
    MIN(defense) AS min_defense
FROM pokemon
```

```sql
SELECT
    type,
    COUNT(*) AS total_pokemon,
    AVG(hp) AS avg_hp,
    MAX(attack) AS max_attack,
    MIN(defense) AS min_defense
FROM pokemon
GROUP BY type;
SELECT
    type,
    COUNT(*) AS total_pokemon,
    AVG(attack) AS avg_attack
FROM pokemon
GROUP BY type
HAVING AVG(attack) > 80;
SELECT
    name,
    attack,
    CASE
        WHEN attack >= 100 THEN 'Strong'
        WHEN attack >= 60 THEN 'Medium'
        ELSE 'Weak'
    END AS attack_category
FROM pokemon;
```

AUTONOMOUS

```sql
SELECT name, type, hp, attack, speed
FROM pokemon
WHERE hp > 80
AND attack > 90
AND speed > 70;

SELECT name, type, attack, defense
FROM pokemon
WHERE attack > 120
OR defense > 120;
SELECT name, type, attack
FROM pokemon
WHERE attack = (SELECT MAX(attack) FROM
pokemon);

SELECT type, COUNT(*) AS total_pokemon
FROM pokemon
GROUP BY type
HAVING COUNT(*) >= 5
ORDER BY total_pokemon DESC;

SELECT
    name,
    type,
    (hp + attack + defense + speed) AS
```



```
63
64  SELECT
65      name,
66      type,
67      (hp + attack + defense + speed) AS
    total_power
68  FROM pokemon
69  ORDER BY total_power DESC;
70
71  SELECT
72      name,
73      hp,
74      attack,
75      defense,
76      CASE
77          WHEN attack >= 100 AND defense >=
    100 THEN 'Powerful'
78          WHEN attack >= 100 THEN 'Attack
    Specialist'
79          WHEN defense >= 100 THEN 'Defense
    Specialist'
80          ELSE 'Normal'
81      END AS pokemon_class
82  FROM pokemon;
83
```


Step 1

We've created a database of people and hobbies, and each row in hobbies is related to a row in persons via the person_id column. In this first step, insert one more row in persons and then one more row in hobbies that is related to the newly inserted person.

```sql
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    age INTEGER);

INSERT INTO persons (name, age) VALUES
("Bobby McBobbyFace", 12);
INSERT INTO persons (name, age) VALUES
("Lucy BoBucie", 25);
INSERT INTO persons (name, age) VALUES
("Banana FoFanna", 14);
INSERT INTO persons (name, age) VALUES
("Shish Kabob", 20);
INSERT INTO persons (name, age) VALUES
("Fluffy Sparkles", 8);

CREATE table hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT);

INSERT INTO hobbies (person_id, name)
VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name)
VALUES (1, "coding");
```

DATABASE SCHEMA

persons	6 rows
id (PK)	INTEGER
name	TEXT
age	INTEGER

hobbies	11 rows
id (PK)	INTEGER
person_id	INTEGER
name	TEXT

atulations!
arned
points!



name
Bobby McBobbyFace	drawing

```sql
INSERT INTO hobbies (person_id, name)
VALUES (1, "coding");
INSERT INTO hobbies (person_id, name)
VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name)
VALUES (2, "coding");
INSERT INTO hobbies (person_id, name)
VALUES (3, "skating");
INSERT INTO hobbies (person_id, name)
VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name)
VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name)
VALUES (4, "coding");
INSERT INTO hobbies (person_id, name)
VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name)
VALUES (4, "meowing");
INSERT INTO persons (name, age)
VALUES ("Alex Wonder", 18);

INSERT INTO hobbies (person_id, name)
VALUES (6, "gaming");
SELECT persons.name, hobbies.name
```

DATABASE SCHEMA

persons	6 rows
id (PK)	INTEGER
name	TEXT
age	INTEGER

hobbies	11 rows
id (PK)	INTEGER
person_id	INTEGER
name	TEXT

atulations!
arned
points!



name
Bobby McBobbyFace	drawing

```
    VALUES (3, "skating");
22  INSERT INTO hobbies (person_id, name)
    VALUES (3, "rowing");
23  INSERT INTO hobbies (person_id, name)
    VALUES (3, "drawing");
24  INSERT INTO hobbies (person_id, name)
    VALUES (4, "coding");
25  INSERT INTO hobbies (person_id, name)
    VALUES (4, "dilly-dallying");
26  INSERT INTO hobbies (person_id, name)
    VALUES (4, "meowing");
27  INSERT INTO persons (name, age)
28  VALUES ("Alex Wonder", 18);
29
30  INSERT INTO hobbies (person_id, name)
31  VALUES (6, "gaming");
32  SELECT persons.name, hobbies.name
33  FROM persons
34  JOIN hobbies
35  ON persons.id = hobbies.person_id;
36  SELECT persons.name, hobbies.name
37  FROM persons
38  JOIN hobbies
39  ON persons.id = hobbies.person_id
40  WHERE persons.name = 'Bobby McBobbyFace';
```

DATABASE SCHEMA

persons	6 rows
id (PK)	INTEGER
name	TEXT
age	INTEGER

hobbies	11 rows
id (PK)	INTEGER
person_id	INTEGER
name	TEXT

atulations!
arned
points!



name
Bobby McBobbyFace

Step 2

Now, select the 2 tables with a join so that you can see each person's name next to their hobby.

Hint

SELECT … JOIN …;

```
 1  CREATE TABLE persons (
 2      id INTEGER PRIMARY KEY AUTOINCREMENT,
 3      name TEXT,
 4      age INTEGER);
 5
 6  INSERT INTO persons (name, age) VALUES
    ("Bobby McBobbyFace", 12);
 7  INSERT INTO persons (name, age) VALUES
    ("Lucy BoBucie", 25);
 8  INSERT INTO persons (name, age) VALUES
    ("Banana FoFanna", 14);
 9  INSERT INTO persons (name, age) VALUES
    ("Shish Kabob", 20);
10  INSERT INTO persons (name, age) VALUES
    ("Fluffy Sparkles", 8);
11
12  CREATE table hobbies (
13      id INTEGER PRIMARY KEY AUTOINCREMENT,
14      person_id INTEGER,
15      name TEXT);
16
17  INSERT INTO hobbies (person_id, name)
    VALUES (1, "drawing");
18  INSERT INTO hobbies (person_id, name)
    VALUES (1, "coding");
```

DATABASE SCHEMA

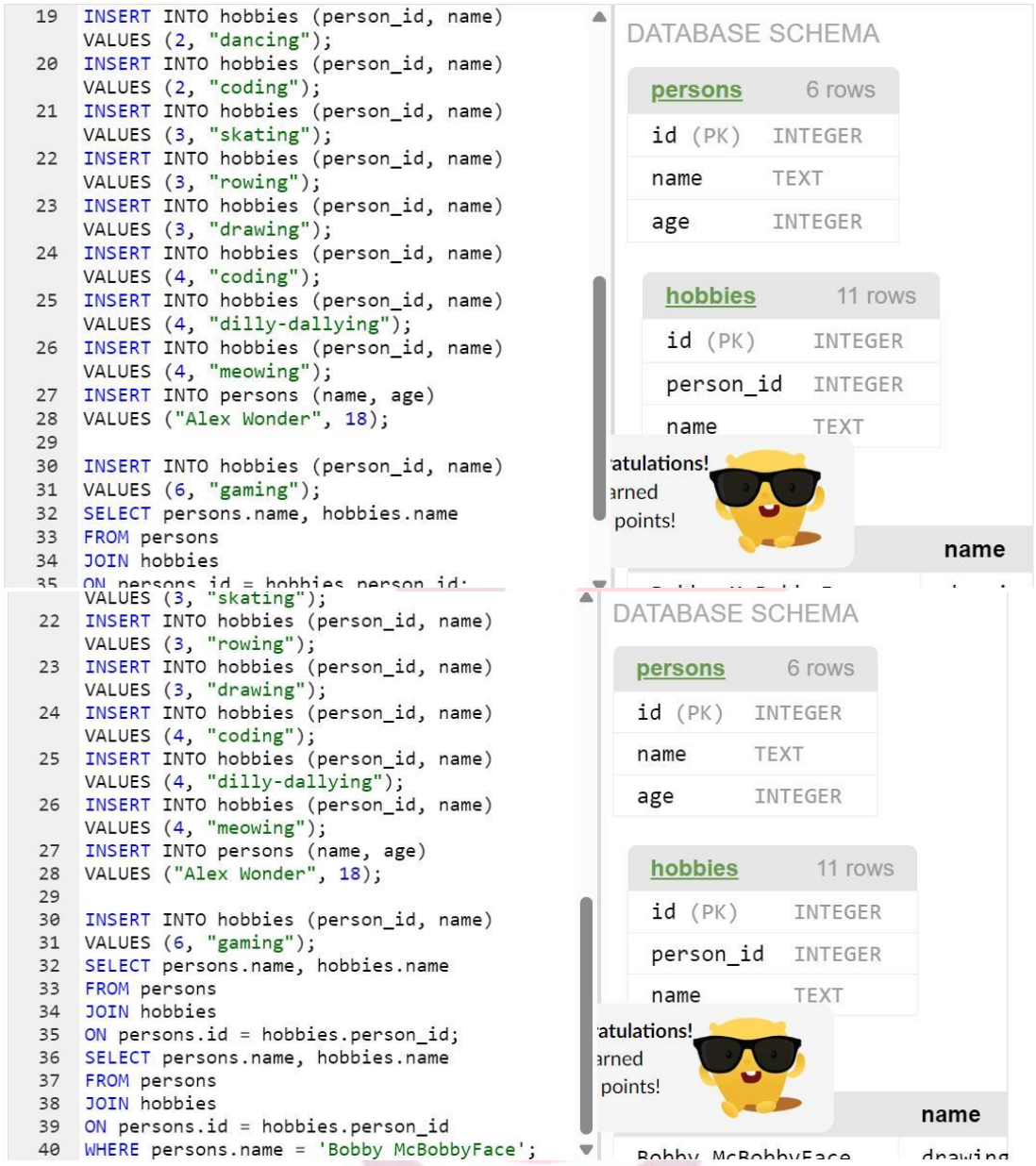
persons	6 rows
id (PK)	INTEGER
name	TEXT
age	INTEGER

hobbies	11 rows
id (PK)	INTEGER
person_id	INTEGER
name	TEXT

atulations!
arned
points!



name
Bobby McBobbyFace

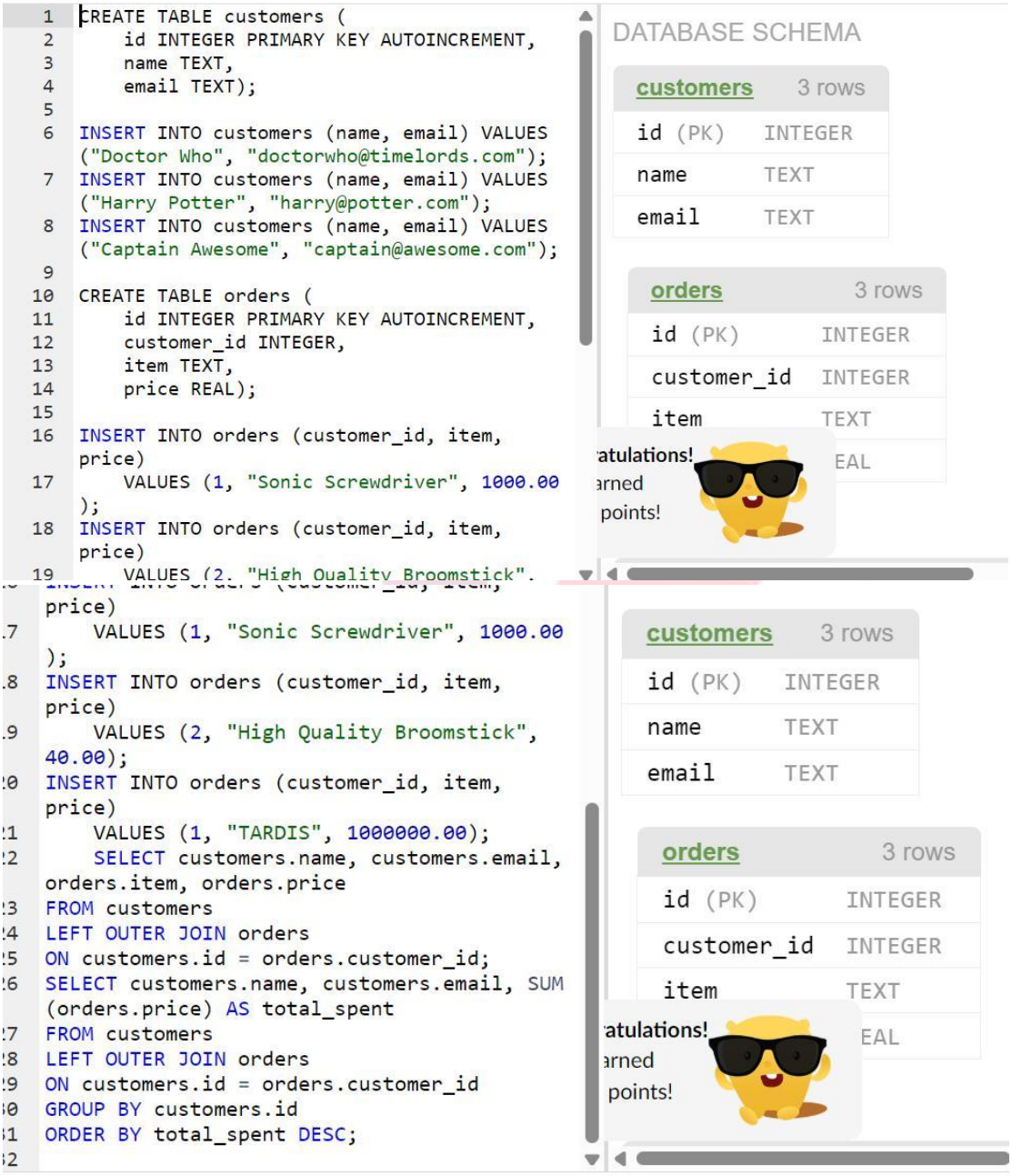


Step 1

We've created a database for customers and their orders. Not all of the customers have made orders, however. Come up with a query that lists the name and email of every customer followed by the item and price of orders they've made. Use a LEFT OUTER JOIN so that a customer is listed even if they've made no orders, and don't add any ORDER BY.

Hint

SELECT … LEFT OUTER JOIN …;



Step 2

Now, create another query that will result in one row per each customer, with their name, email, and total amount of money they've spent on orders. Sort the rows

according to the total money spent, from the most spent to the least spent.

(Tip: You should always GROUP BY on the column that is most likely to be unique in a row.)

```
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    email TEXT);

INSERT INTO customers (name, email) VALUES
("Doctor Who", "doctorwho@timelords.com");
INSERT INTO customers (name, email) VALUES
("Harry Potter", "harry@potter.com");
INSERT INTO customers (name, email) VALUES
("Captain Awesome", "captain@awesome.com");

CREATE TABLE orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER,
    item TEXT,
    price REAL);

INSERT INTO orders (customer_id, item,
price)
    VALUES (1, "Sonic Screwdriver", 1000.00
);
INSERT INTO orders (customer_id, item,
price)
    VALUES (2, "High Quality Broomstick",
```

DATABASE SCHEMA

customers	3 rows
id (PK)	INTEGER
name	TEXT
email	TEXT

orders	3 rows
id (PK)	INTEGER
customer_id	INTEGER
item	TEXT
price	REAL

atulations!
arned
points!



```
    customer_id INTEGER,
    item TEXT,
    price REAL);

INSERT INTO orders (customer_id, item,
price)
    VALUES (1, "Sonic Screwdriver", 1000.00
);
INSERT INTO orders (customer_id, item,
price)
    VALUES (2, "High Quality Broomstick",
40.00);
INSERT INTO orders (customer_id, item,
price)
    VALUES (1, "TARDIS", 1000000.00);
    SELECT customers.name, customers.email,
orders.item, orders.price
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id;
SELECT customers.name, customers.email, SUM
(orders.price) AS total_spent
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id
```

DATABASE SCHEMA

customers	3 rows
id (PK)	INTEGER
name	TEXT
email	TEXT

orders	3 row
id (PK)	INTEGER
customer_id	INTEGER
item	TEXT
EAL	

atulations!
arned
points!



Undo Start over



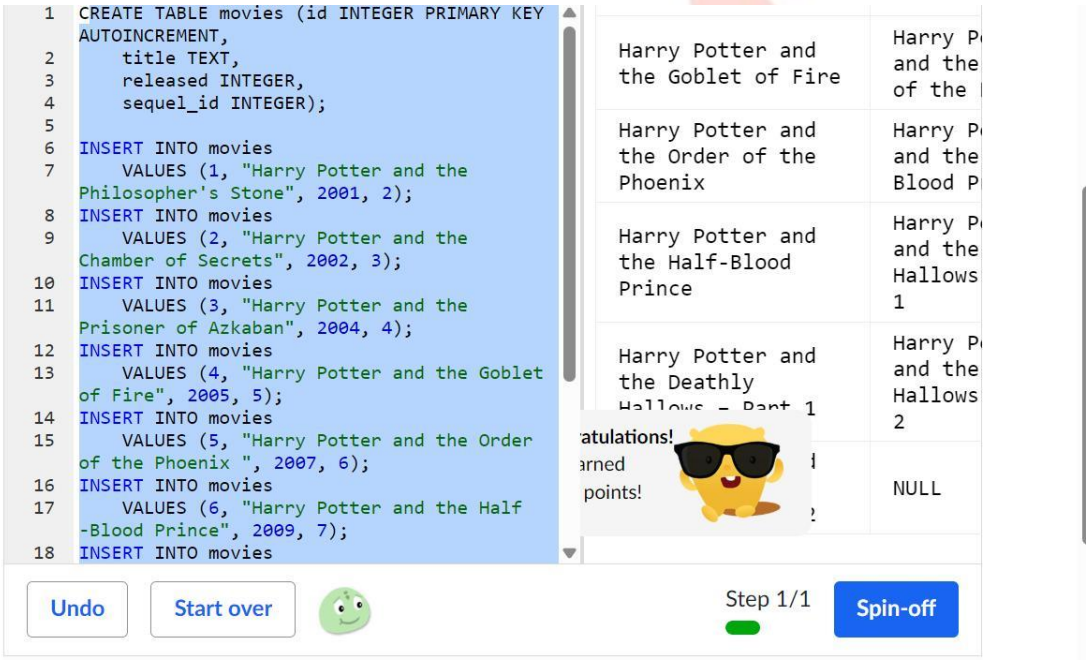
Step 2/2 Sp

Step 1

We've created a table with all the 'Harry Potter' movies, with a sequel_id column that matches the id of the sequel for each movie. Issue a SELECT that will show the title of each movie next to its sequel's title (or NULL if it doesn't have a sequel).

Hint

SELECT … FROM …
LEFT OUTER JOIN …
ON …;



```
Chamber of Secrets", 2002, 3);
INSERT INTO movies
    VALUES (3, "Harry Potter and the
Prisoner of Azkaban", 2004, 4);
INSERT INTO movies
    VALUES (4, "Harry Potter and the Goblet
of Fire", 2005, 5);
INSERT INTO movies
    VALUES (5, "Harry Potter and the Order
of the Phoenix ", 2007, 6);
INSERT INTO movies
    VALUES (6, "Harry Potter and the Half
-Blood Prince", 2009, 7);
INSERT INTO movies
    VALUES (7, "Harry Potter and the
Deathly Hallows - Part 1", 2010, 8);
INSERT INTO movies
    VALUES (8, "Harry Potter and the
Deathly Hallows - Part 2", 2011, NULL);
    SELECT movies.title AS movie_title,
sequel.title AS sequel_title
FROM movies
LEFT OUTER JOIN movies sequel
ON movies.sequel_id = sequel.id;
```

Harry Potter and the Goblet of Fire	Harry and th of the
Harry Potter and the Order of the Phoenix	Harry and th Blood
Harry Potter and the Half-Blood Prince	Harry and th Hallow 1
Harry Potter and the Deathly Hallows - Part 1	Harry and th Hallow 2
	NULL

atulations!
arned
points!



Step 1

We've created a database for a friend networking site, with a table storing data on each person, a table on each person's hobbies, and a table of friend connections between the people. In this first step, use a JOIN to display a table showing people's names with their hobbies.

Hint

fullname	name
Bobby McBobbyFace	drawing
Bobby McBobbyFace	coding
Lucy BoBucie	dancing
. . .	. . .

SELECT … JOIN …;

```sql
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    fullname TEXT,
    age INTEGER);

INSERT INTO persons (fullname, age) VALUES ("Bobby McBobbyFace", "12");
INSERT INTO persons (fullname, age) VALUES ("Lucy BoBucie", "25");
INSERT INTO persons (fullname, age) VALUES ("Banana FoFanna", "14");
INSERT INTO persons (fullname, age) VALUES ("Shish Kabob", "20");
INSERT INTO persons (fullname, age) VALUES ("Fluffy Sparkles", "8");

CREATE table hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT);

INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
```

DATABASE SCHEMA

persons	5 rows
id (PK)	INTEGER
fullname	TEXT
age	INTEGER

hobbies	10 rows
id (PK)	INTEGER
person_id	INTEGER
name	TEXT

atulations!
arned
points!

2 rows

TEGER

person1_id INTEGER

```sql
VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");

CREATE table friends (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person1_id INTEGER,
    person2_id INTEGER);

INSERT INTO friends (person1_id, person2_id )
    VALUES (1, 4);
INSERT INTO friends (person1_id, person2_id
```

DATABASE SCHEMA

persons	5 rows
id (PK)	INTEGER
fullname	TEXT
age	INTEGER

hobbies	10 rows
id (PK)	INTEGER
person_id	INTEGER
name	TEXT

atulations!
arned
points!

2 rows

TEGER

person1_id INTEGER

```
27
28  CREATE table friends (
29      id INTEGER PRIMARY KEY AUTOINCREMENT,
30      person1_id INTEGER,
31      person2_id INTEGER);
32
33  INSERT INTO friends (person1_id, person2_id
    )
34      VALUES (1, 4);
35  INSERT INTO friends (person1_id, person2_id
    )
36      VALUES (2, 3);
37      SELECT persons.fullname, hobbies.name
38  FROM persons
39  JOIN hobbies
40  ON persons.id = hobbies.person_id;
41      SELECT p1.fullname AS person1, p2
    .fullname AS person2
42  FROM friends
43  JOIN persons p1 ON friends.person1_id = p1
    .id
44  JOIN persons p2 ON friends.person2_id = p2
    .id;
45
46
```

DATABASE SCHEMA

persons	5 rows
id (PK)	INTEGER
fullname	TEXT
age	INTEGER

hobbies	10 rows
id (PK)	INTEGER
person_id	INTEGER
name	TEXT

atulations!
arned
points!

2 rows

TEGER

person1_id INTEGER

Step 2

Now, use another SELECT with a JOIN to show the names of each pair of friends, based on the data in the friends table.

```
1   CREATE TABLE persons (
2       id INTEGER PRIMARY KEY AUTOINCREMENT,
3       fullname TEXT,
4       age INTEGER);
5
6   INSERT INTO persons (fullname, age) VALUES
    ("Bobby McBobbyFace", "12");
7   INSERT INTO persons (fullname, age) VALUES
    ("Lucy BoBucie", "25");
8   INSERT INTO persons (fullname, age) VALUES
    ("Banana FoFanna", "14");
9   INSERT INTO persons (fullname, age) VALUES
    ("Shish Kabob", "20");
10  INSERT INTO persons (fullname, age) VALUES
    ("Fluffy Sparkles", "8");
11
12  CREATE table hobbies (
13      id INTEGER PRIMARY KEY AUTOINCREMENT,
14      person_id INTEGER,
15      name TEXT);
16
17  INSERT INTO hobbies (person_id, name)
    VALUES (1, "drawing");
18  INSERT INTO hobbies (person_id, name)
    VALUES (1, "coding");
```

DATABASE SCHEMA

persons	5 rows
id (PK)	INTEGER
fullname	TEXT
age	INTEGER

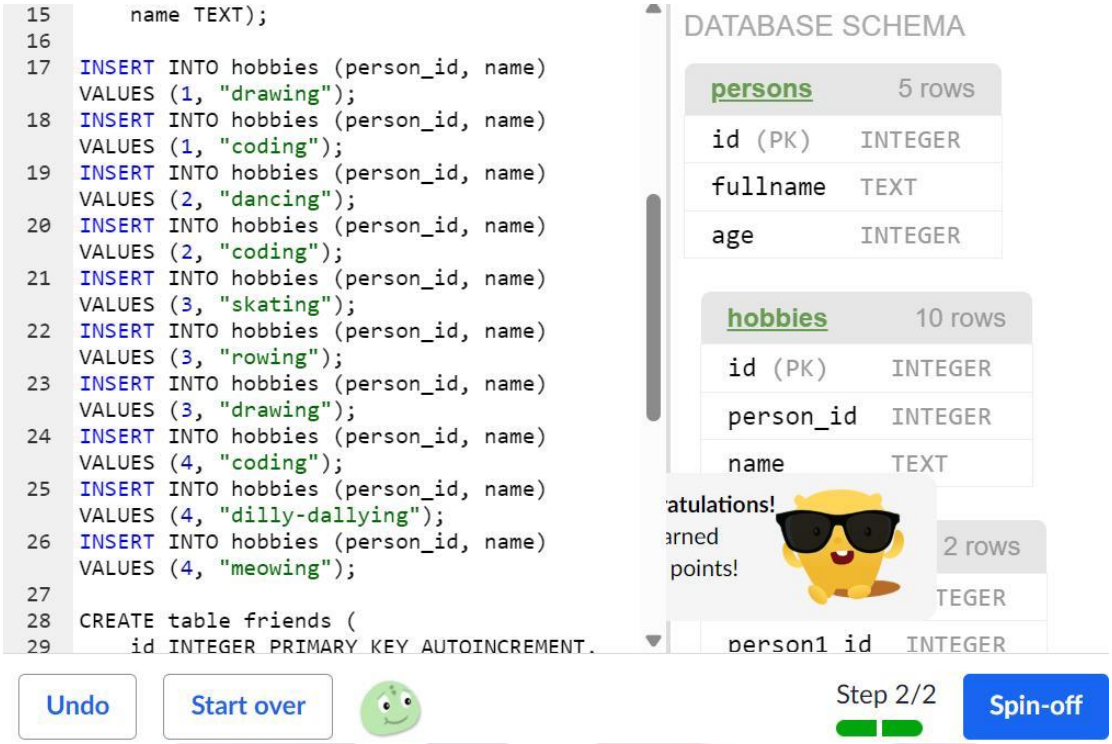
hobbies	10 rows
id (PK)	INTEGER
person_id	INTEGER
name	TEXT

atulations!
arned
points!

2 rows

TEGER

person1_id INTEGER



n this project, you're going to make your own table with some small set of "famous people", then make more tables about things they do and join those to create nice human readable lists.

For example, here are types of famous people and the questions your data could answer:

- Movie stars: What movies are they in? Are they married to each other?
- Singers: What songs did they write? Where are they from?
- Authors: What books did they write?
- Fictional characters: How are they related to other characters? What books do they show up in?

Your project is now marked as complete. You can continue working on it, of course, if you think of other ways to improve it. Keep practicing

```sql
CREATE TABLE authors (
    id INTEGER PRIMARY KEY,
    name TEXT,
    country TEXT
);

INSERT INTO authors VALUES
(1, 'J.K. Rowling', 'United Kingdom'),
(2, 'George Orwell', 'United Kingdom'),
(3, 'Mark Twain', 'United States');

CREATE TABLE books (
    id INTEGER PRIMARY KEY,
    title TEXT,
    author_id INTEGER
);

INSERT INTO books VALUES
(1, 'Harry Potter and the Sorcerer''s Stone', 1),
(2, '1984', 2),
(3, 'Animal Farm', 2),
(4, 'The Adventures of Tom Sawyer', 3);

CREATE TABLE genres (
```

book_id	INTEGER
genre_id	INTEGER

QUERY RESULTS

author	book
J.K. Rowling	Harry Potter and the Sorcerer's Stone
George Orwell	1984
George Orwell	Animal Farm
Mark Twain	The Adventures of Tom Sawyer

```sql
CREATE TABLE books (
    id INTEGER PRIMARY KEY,
    title TEXT,
    author_id INTEGER
);

INSERT INTO books VALUES
(1, 'Harry Potter and the Sorcerer''s Stone', 1),
(2, '1984', 2),
(3, 'Animal Farm', 2),
(4, 'The Adventures of Tom Sawyer', 3);

CREATE TABLE genres (
    id INTEGER PRIMARY KEY,
    genre_name TEXT
);

INSERT INTO genres VALUES
(1, 'Fantasy'),
(2, 'Dystopian'),
(3, 'Adventure');

CREATE TABLE book_genres (
    book_id INTEGER,
```

book_id	INTEGER
genre_id	INTEGER

QUERY RESULTS

author	book
J.K. Rowling	Harry Potter and the Sorcerer's Stone
George Orwell	1984
George Orwell	Animal Farm
Mark Twain	The Adventures of Tom Sawyer

```
(2, 'Dystopian'),
(3, 'Adventure');

CREATE TABLE book_genres (
    book_id INTEGER,
    genre_id INTEGER
);

INSERT INTO book_genres VALUES
(1, 1),
(2, 2),
(3, 2),
(4, 3);

SELECT
    authors.name AS author,
    books.title AS book,
    genres.genre_name AS genre
FROM books
JOIN authors
ON books.author_id = authors.id
JOIN book_genres
ON books.id = book_genres.book_id
JOIN genres
ON book_genres.genre_id = genres.id;
```

book_id	INTEGER
genre_id	INTEGER

QUERY RESULTS

author	book
J.K. Rowling	Harry Potter and the Sorcerer's Stone
George Orwell	1984
George Orwell	Animal Farm
Mark Twain	The Adventures of Tom Sawyer

Step 1

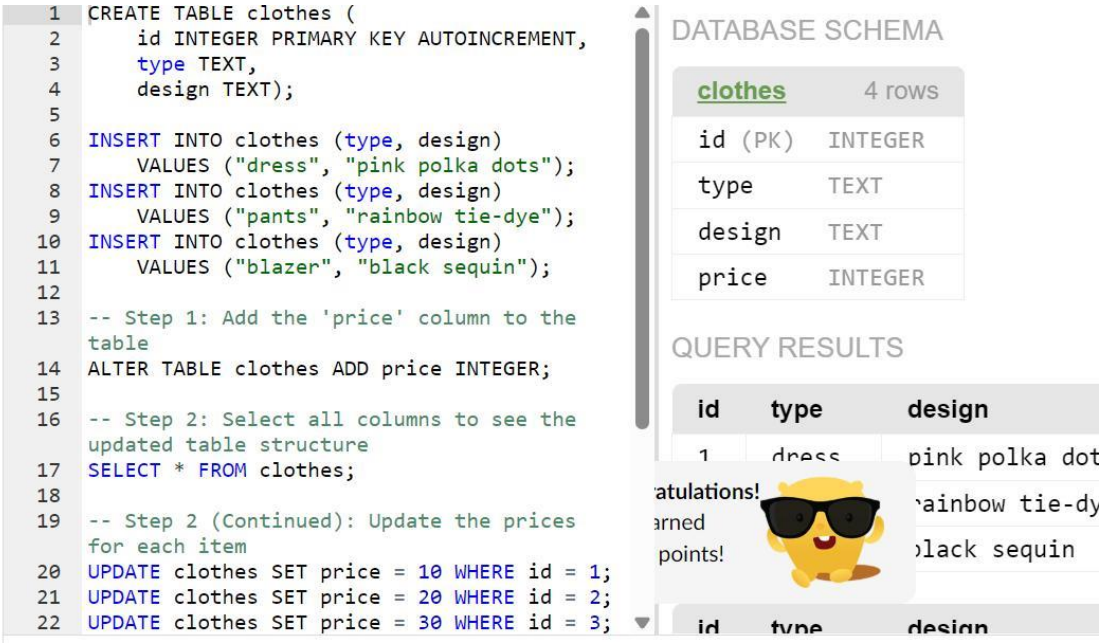
We've created a database for a documents app, with rows for each document with it's title, content, and author. In this first step, use UPDATE to change the author to 'Jackie Draper' for all rows where it's currently 'Jackie Paper'. Then re-select all the rows to make sure the table changed like you expected.

Hint
UPDATE …;
SELECT …;



Step 1

We've created a database of clothes, and decided we need a price column. Use ALTER to add a 'price' column to the table. Then select all the columns in each row to see what your table looks like now.



```
CREATE TABLE clothes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    type TEXT,
    design TEXT);

INSERT INTO clothes (type, design)
    VALUES ("dress", "pink polka dots");
INSERT INTO clothes (type, design)
    VALUES ("pants", "rainbow tie-dye");
INSERT INTO clothes (type, design)
    VALUES ("blazer", "black sequin");

-- Step 1: Add the 'price' column to the table
ALTER TABLE clothes ADD price INTEGER;

-- Step 2: Select all columns to see the updated table structure
SELECT * FROM clothes;

-- Step 2 (Continued): Update the prices for each item
UPDATE clothes SET price = 10 WHERE id = 1;
UPDATE clothes SET price = 20 WHERE id = 2;
UPDATE clothes SET price = 30 WHERE id = 3;
```

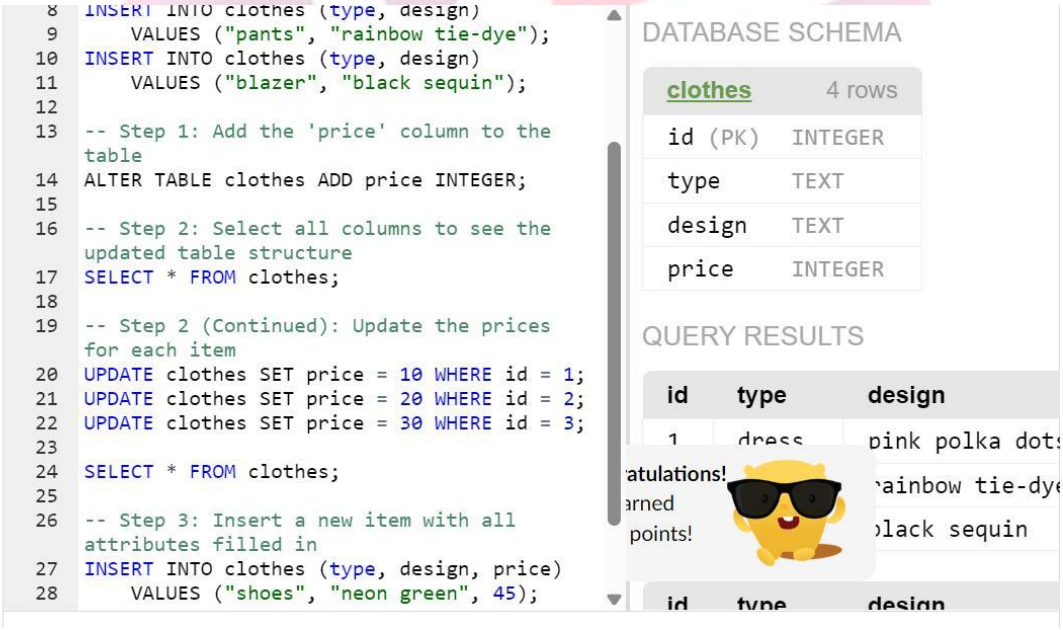
DATABASE SCHEMA

clothes	4 rows
id (PK)	INTEGER
type	TEXT
design	TEXT
price	INTEGER

QUERY RESULTS

id	type	design
1	dress	pink polka dot
		rainbow tie-dy
		black sequin

atulations!
arned
points!



```
INSERT INTO clothes (type, design)
    VALUES ("pants", "rainbow tie-dye");
INSERT INTO clothes (type, design)
    VALUES ("blazer", "black sequin");

-- Step 1: Add the 'price' column to the table
ALTER TABLE clothes ADD price INTEGER;

-- Step 2: Select all columns to see the updated table structure
SELECT * FROM clothes;

-- Step 2 (Continued): Update the prices for each item
UPDATE clothes SET price = 10 WHERE id = 1;
UPDATE clothes SET price = 20 WHERE id = 2;
UPDATE clothes SET price = 30 WHERE id = 3;

SELECT * FROM clothes;

-- Step 3: Insert a new item with all attributes filled in
INSERT INTO clothes (type, design, price)
    VALUES ("shoes", "neon green", 45);
```

DATABASE SCHEMA

clothes	4 rows
id (PK)	INTEGER
type	TEXT
design	TEXT
price	INTEGER

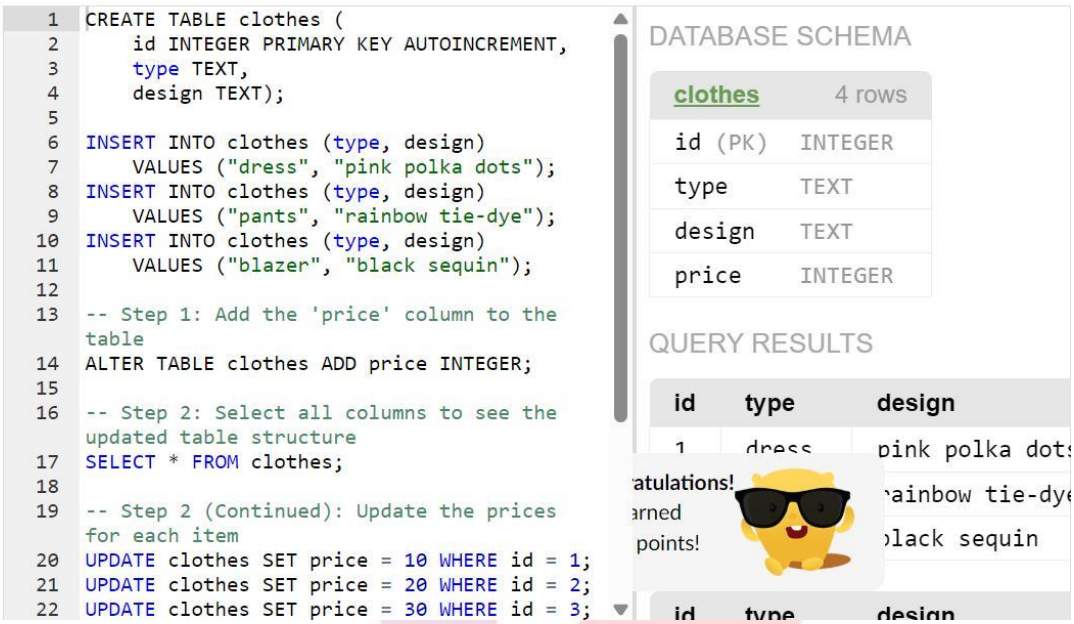
QUERY RESULTS

id	type	design
1	dress	pink polka dots
		rainbow tie-dye
		black sequin

atulations!
arned
points!

Step 2

Now assign each item a price, using UPDATE - item 1 should be 10 dollars, item 2 should be 20 dollars, item 3 should be 30 dollars. When you're done, do another SELECT of all the rows to check that it worked as expected.



```
CREATE TABLE clothes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    type TEXT,
    design TEXT);

INSERT INTO clothes (type, design)
    VALUES ("dress", "pink polka dots");
INSERT INTO clothes (type, design)
    VALUES ("pants", "rainbow tie-dye");
INSERT INTO clothes (type, design)
    VALUES ("blazer", "black sequin");

-- Step 1: Add the 'price' column to the
table
ALTER TABLE clothes ADD price INTEGER;

-- Step 2: Select all columns to see the
updated table structure
SELECT * FROM clothes;

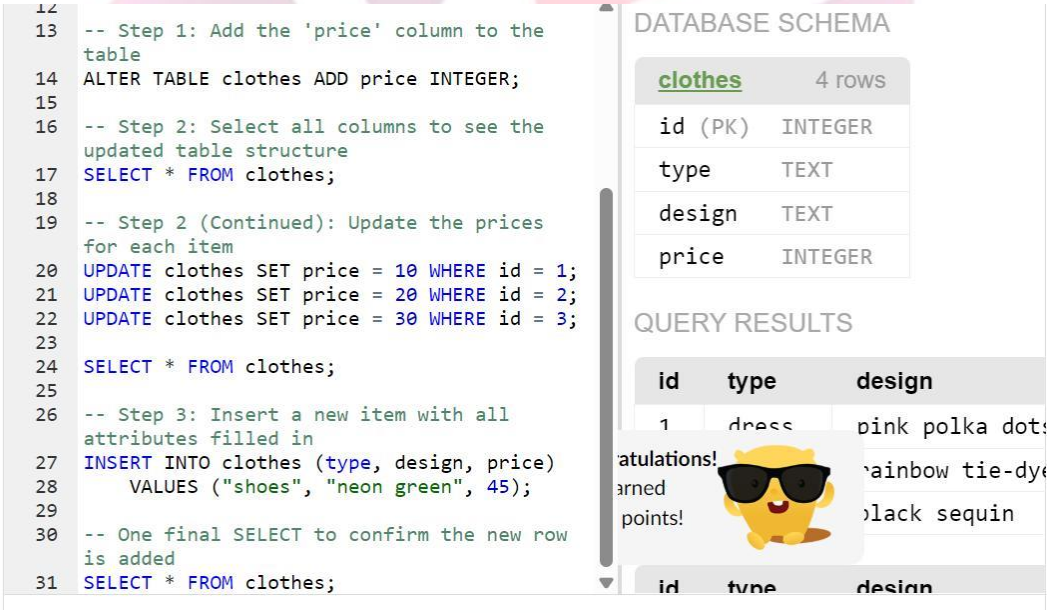
-- Step 2 (Continued): Update the prices
for each item
UPDATE clothes SET price = 10 WHERE id = 1;
UPDATE clothes SET price = 20 WHERE id = 2;
UPDATE clothes SET price = 30 WHERE id = 3;
```

DATABASE SCHEMA

clothes	4 rows
id (PK)	INTEGER
type	TEXT
design	TEXT
price	INTEGER

QUERY RESULTS

id	type	design
1	dress	pink polka dots
		rainbow tie-dye
		black sequin



```
-- Step 1: Add the 'price' column to the
table
ALTER TABLE clothes ADD price INTEGER;

-- Step 2: Select all columns to see the
updated table structure
SELECT * FROM clothes;

-- Step 2 (Continued): Update the prices
for each item
UPDATE clothes SET price = 10 WHERE id = 1;
UPDATE clothes SET price = 20 WHERE id = 2;
UPDATE clothes SET price = 30 WHERE id = 3;

SELECT * FROM clothes;

-- Step 3: Insert a new item with all
attributes filled in
INSERT INTO clothes (type, design, price)
    VALUES ("shoes", "neon green", 45);

-- One final SELECT to confirm the new row
is added
SELECT * FROM clothes;
```

DATABASE SCHEMA

clothes	4 rows
id (PK)	INTEGER
type	TEXT
design	TEXT
price	INTEGER

QUERY RESULTS

id	type	design
1	dress	pink polka dots
		rainbow tie-dye
		black sequin

Step 3

Now insert a new item into the table that has all three attributes filled in, including 'price'. Do one final SELECT of all the rows to check it worked.

Hint

INSERT … ;

SELECT * FROM …;

```sql
CREATE TABLE clothes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    type TEXT,
    design TEXT);

INSERT INTO clothes (type, design)
    VALUES ("dress", "pink polka dots");
INSERT INTO clothes (type, design)
    VALUES ("pants", "rainbow tie-dye");
INSERT INTO clothes (type, design)
    VALUES ("blazer", "black sequin");

-- Step 1: Add the 'price' column to the table
ALTER TABLE clothes ADD price INTEGER;

-- Step 2: Select all columns to see the updated table structure
SELECT * FROM clothes;

-- Step 2 (Continued): Update the prices for each item
UPDATE clothes SET price = 10 WHERE id = 1;
UPDATE clothes SET price = 20 WHERE id = 2;
UPDATE clothes SET price = 30 WHERE id = 3;
```

type	TEXT
design	TEXT
price	INTEGER

QUERY RESULTS

id	type	design
1	dress	pink polka dots
2	pants	rainbow tie-dye
3	blazer	black sequin

atulations!
arned
points!



id	type	design
1	dress	pink polka dots
2	pants	rainbow tie-dye
3	blazer	black sequin

```sql
-- Step 1: Add the 'price' column to the table
ALTER TABLE clothes ADD price INTEGER;

-- Step 2: Select all columns to see the updated table structure
SELECT * FROM clothes;

-- Step 2 (Continued): Update the prices for each item
UPDATE clothes SET price = 10 WHERE id = 1;
UPDATE clothes SET price = 20 WHERE id = 2;
UPDATE clothes SET price = 30 WHERE id = 3;

SELECT * FROM clothes;

-- Step 3: Insert a new item with all attributes filled in
INSERT INTO clothes (type, design, price)
    VALUES ("shoes", "neon green", 45);

-- One final SELECT to confirm the new row is added
SELECT * FROM clothes;
```

type	TEXT
design	TEXT
price	INTEGER

QUERY RESULTS

id	type	design
1	dress	pink polka dots
2	pants	rainbow tie-dye
3	blazer	black sequin

atulations!
arned
points!



id	type	design
1	dress	pink polka dots
2	pants	rainbow tie-dye
3	blazer	black sequin

```sql
/* What does the app's SQL look like? */
-- 1. CREATE the table to store workout data
CREATE TABLE workouts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    activity_type TEXT,
    duration_minutes INTEGER,
    calories_burned INTEGER,
    date_logged TEXT
);

-- 2. INSERT a few example rows (initial app data)
INSERT INTO workouts (activity_type, duration_minutes, calories_burned, date_logged)
    VALUES ("Running", 30, 350, "2026-06-20");

INSERT INTO workouts (activity_type, duration_minutes, calories_burned, date_logged)
    VALUES ("Weightlifting", 45, 200, "2026-06-21");

INSERT INTO workouts (activity_type, duration_minutes, calories_burned, date_logged)
    VALUES ("Cycling", 60, 500, "2026-06-22");

INSERT INTO workouts (activity_type, duration_minutes, calories_burned, date_logged)
    VALUES ("Yoga", 15, 50, "2026-06-23");

-- Let's see the initial log
SELECT * FROM workouts;

-- 3. UPDATE: Emulate a user editing a workout
-- Scenario: The user realized they actually ran for 40 minutes instead of 30, burning 450 calories.
UPDATE workouts
SET duration_minutes = 40, calories_burned = 450
WHERE id = 1;
```

DATABASE SCHEMA

workouts	3 rows
id (PK)	INTEGER
activity_type	TEXT
duration_minutes	INTEGER
calories_burned	INTEGER
date_logged	TEXT

QUERY RESULTS

id	activity_type	durati
1	Running	30
2	Weightlifting	45
3	Cycling	60

DATABASE SCHEMA

workouts	3 rows
id (PK)	INTEGER
activity_type	TEXT
duration_minutes	INTEGER
calories_burned	INTEGER
date_logged	TEXT

QUERY RESULTS

id	activity_type	durati
1	Running	30
2	Weightlifting	45
3	Cycling	60

ANNEXURE - HACKER RANK

1.Query all columns for all American cities in the **CITY** table with populations larger than `100000`. The **CountryCode** for America is `USA`.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Change Theme Language MySQL

```
1  SELECT *
2  FROM CITY
3  WHERE CountryCode = 'USA'
4    AND Population > 100000;
5
```

Query the **NAME** field for all American cities in the **CITY** table with populations larger than `120000`. The *CountryCode* for America is `USA`.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Change Theme Language MySQL

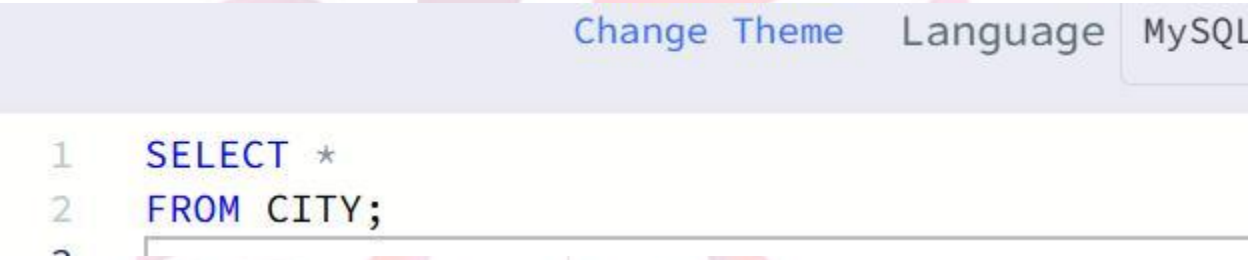
```
1  SELECT NAME
2  FROM CITY
3  WHERE CountryCode = 'USA'
4    AND Population > 120000;
5
```

Query all columns (attributes) for every row in the **CITY** table.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER



Query all columns for a city in **CITY** with the *ID* `1661`.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Change Theme Language MySQL

```
SELECT *
FROM CITY
WHERE ID = 1661;
```

Query all attributes of every Japanese city in the **CITY** table. The **COUNTRYCODE** for Japan is `JPN`.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Change Theme Language MySQL

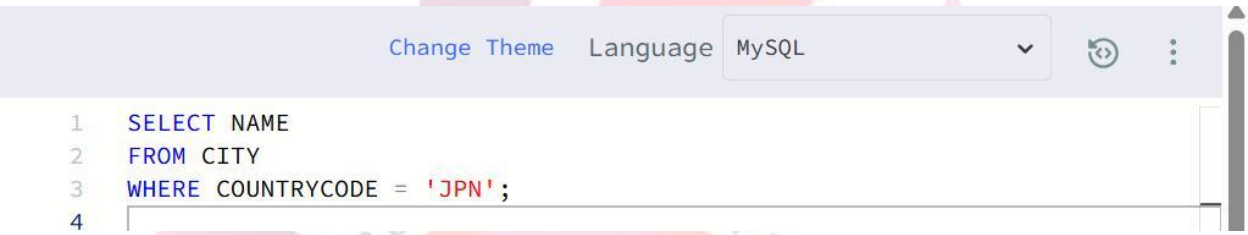
```
SELECT *
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

Query the names of all the Japanese cities in the **CITY** table. The **COUNTRYCODE** for Japan is `JPN`.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER



Query the names of all the Japanese cities in the **CITY** table. The **COUNTRYCODE** for Japan is `JPN`.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Change Theme Language MySQL

```
1  SELECT NAME
2  FROM CITY
3  WHERE COUNTRYCODE = 'JPN';
```

Query a list of **CITY** and **STATE** from the **STATION** table.

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where **LAT_N** is the northern latitude and **LONG_W** is the western longitude.

Change Theme Language MySQL

```
1  SELECT CITY, STATE
2  FROM STATION;
3
```

Query a list of **CITY** names from **STATION** for cities that have an even **ID** number. Print the results in any order, but exclude duplicates from the answer.

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where **LAT_N** is the northern latitude and **LONG_W** is the western longitude.

Change Theme Language MySQL

```
1  SELECT DISTINCT CITY
2  FROM STATION
3  WHERE ID % 2 = 0;
4
```

Find the difference between the total number of **CITY** entries in the table and the number of distinct **CITY** entries in the table.

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where **LAT_N** is the northern latitude and **LONG_W** is the western longitude.

For example, if there are three records in the table with **CITY** values 'New York', 'New York', 'Bengalaru', there are 2 different city names: 'New York' and 'Bengalaru'. The query returns , because .

Change Theme Language MySQL

```
1  SELECT COUNT(CITY) - COUNT(DISTINCT CITY)
2  FROM STATION;
3
```

Query the two cities in **STATION** with the shortest and longest *CITY* names, as well as their respective lengths (i.e.: number of characters in the name). If there is more than one smallest or largest city, choose the one that comes first when ordered alphabetically.

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where **LAT_N** is the northern latitude and **LONG_W** is the western longitude.

Sample Input

For example, **CITY** has four entries: **DEF, ABC, PQRS** and **WXY**.

Sample Output

```
ABC 3
PQRS 4
```

Explanation

When ordered alphabetically, the **CITY** names are listed as **ABC, DEF, PQRS,** and **WXY**, with lengths and . The

longest name is **PQRS**, but there are options for shortest named city. Choose **ABC**, because it comes first alphabetically.

Change Theme Language MySQL

```
SELECT CITY, LENGTH(CITY)
FROM STATION
ORDER BY LENGTH(CITY), CITY
LIMIT 1;

SELECT CITY, LENGTH(CITY)
FROM STATION
ORDER BY LENGTH(CITY) DESC, CITY
LIMIT 1;
```

Query the list of *CITY* names starting with vowels (i.e., `a`, `e`, `i`, `o`, or `u`) from **STATION**. Your result *cannot* contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Change Theme Language MySQL

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY REGEXP '^[aeiou]';
```

Query the list of *CITY* names ending with vowels (a, e, i, o, u) from **STATION**. Your result *cannot* contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Change Theme Language MySQL

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY REGEXP '[aeiou]$';
```

Query the list of *CITY* names from **STATION** which have vowels (i.e., *a*, *e*, *i*, *o*, and *u*) as both their first *and* last characters. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Change Theme Language MySQL

```
1  SELECT DISTINCT CITY
2  FROM STATION
3  WHERE CITY REGEXP '^[aeiouAEIOU].*[aeiouAEIOU]$';
4
```

Query the list of *CITY* names from **STATION** that *do not start* with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Change Theme Language MySQL

```
1  SELECT DISTINCT CITY
2  FROM STATION
3  WHERE CITY NOT REGEXP '^[aeiouAEIOU]';
```

Query the list of *CITY* names from **STATION** that *do not end* with vowels. Your result cannot contain duplicates.

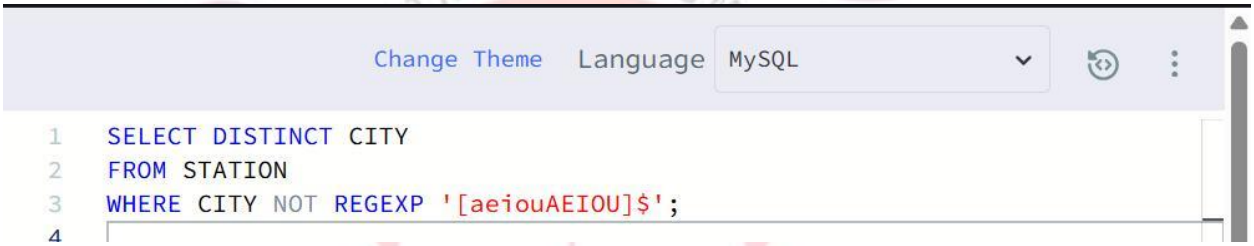
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.



Query the list of *CITY* names from **STATION** that either do not start with vowels or do not end with vowels.

Your result cannot contain duplicates.

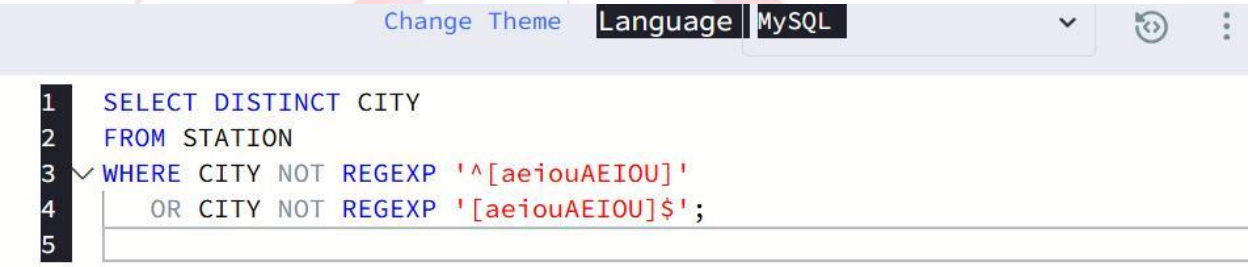
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.



uery the list of *CITY* names from **STATION** that either do not start with vowels or do not end with vowels.

Your result cannot contain duplicates.

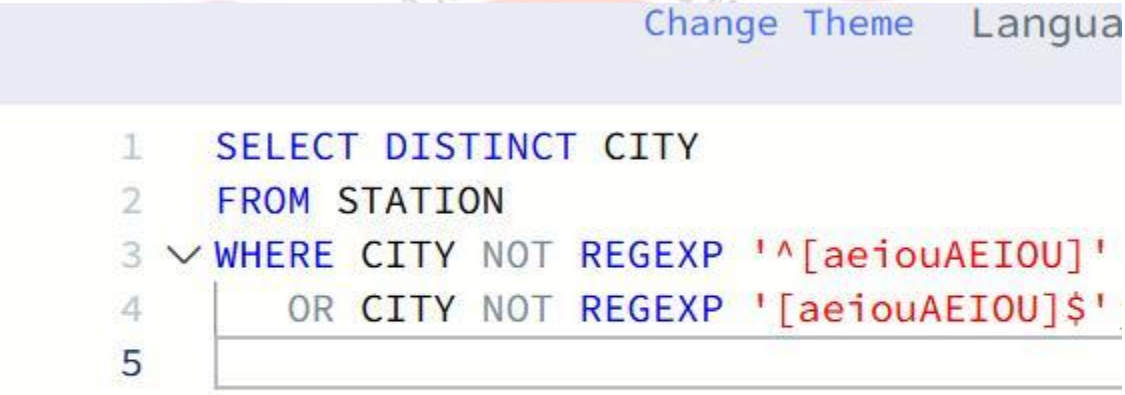
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.



Query the list of *CITY* names from **STATION** that *do not start* with vowels and *do not end* with vowels. Your result cannot contain duplicates.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Change Theme Language MySQL

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY NOT REGEXP '^[aeiouAEIOU]'
  AND CITY NOT REGEXP '[aeiouAEIOU]$';
```

Write a query that prints a list of employee names (i.e.: the *name* attribute) from the **Employee** table in alphabetical order.

Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is their monthly salary.

Sample Input

employee_id	name	months	salary
12228	Rose	15	1968
33645	Angela	1	3443
45692	Frank	17	1608
56118	Patrick	7	1345
59725	Lisa	11	2330
74197	Kimberly	16	4372
78454	Bonnie	8	1771
83565	Michael	6	2017
98607	Todd	5	3396
99989	Joe	9	3573

Sample Output

```
Angela
Bonnie


Frank


Joe


Kimberly


Lisa


Michael


Patrick


Rose


Todd
```

Change Theme Language MySQL

```sql
SELECT name
FROM Employee
ORDER BY name ASC;
```

Write a query that prints a list of employee names (i.e.: the *name* attribute) for employees in **Employee** having a salary greater than per month who have been employees for less than months. Sort your result by

ascending *employee_id*.

Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is the their monthly salary.

Sample Input

employee_id	name	months	salary
12228	Rose	15	1968
33645	Angela	1	3443
45692	Frank	17	1608
56118	Patrick	7	1345
59725	Lisa	11	2330
74197	Kimberly	16	4372
78454	Bonnie	8	1771
83565	Michael	6	2017
98607	Todd	5	3396
99989	Joe	9	3573

Sample Output

```
Angela


Michael


Todd


Joe
```

Explanation

Angela has been an employee for month and earns per month.

Michael has been an employee for months and earns per month.

Todd has been an employee for months and earns per month.

Joe has been an employee for months and earns per month.

We order our output by ascending *employee_id*.



Write a query identifying the *type* of each record in the **TRIANGLES** table using its three side lengths. Output

one of the following statements for each record in the table:

- **Equilateral**: It's a triangle with sides of equal length.
- **Isosceles**: It's a triangle with sides of equal length.
- **Scalene**: It's a triangle with sides of differing lengths.
- **Not A Triangle**: The given values of *A*, *B*, and *C* don't form a triangle.

Input Format

The **TRIANGLES** table is described as follows:

Column	Type
A	Integer
B	Integer
C	Integer

Each row in the table denotes the lengths of each of a triangle's three sides.

Sample Input

A	B	C
20	20	23
20	20	20
20	21	22
13	14	30

Sample Output

```
Isosceles

Equilateral

Scalene

Not A Triangle
```

Explanation

Values in the tuple form an Isosceles triangle, because .

Values in the tuple form an Equilateral triangle, because . Values in the tuple form a Scalene triangle, because .

Values in the tuple cannot form a triangle because the combined value of sides and is not larger than that of side .

Change Theme Language MySQL

```mysql
SELECT
    CASE
        WHEN A + B <= C OR A + C <= B OR B + C <= A THEN 'Not A Triangle'
        WHEN A = B AND B = C THEN 'Equilateral'
        WHEN A = B OR B = C OR A = C THEN 'Isosceles'
        ELSE 'Scalene'
    END
FROM TRIANGLES;
```

generate the following two result sets:

1. Query an *alphabetically ordered* list of all names in **OCCUPATIONS**, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For example: `AnActorName(A)`, `ADoctorName(D)`, `AProfessorName(P)`, and `ASingerName(S)`.

Query the number of ocurrences of each occupation in **OCCUPATIONS**. Sort the occurrences in *ascending order*, and output them in the following format:

```
There are a total of [occupation_count] [occupation]s.
```

2. where `[occupation_count]` is the number of occurrences of an occupation in **OCCUPATIONS** and `[occupation]` is the *lowercase* occupation name. If more than one *Occupation* has the same `[occupation_count]`, they should be ordered alphabetically.

Note: There will be at least two entries in the table for each type of occupation.

Input Format

Column	Type
Name	String
Occupation	String

The **OCCUPATIONS** table is described as follows:

Occupation will only contain one of the following values: **Doctor**, **Professor**, **Singer** or **Actor**.

Sample Input

An **OCCUPATIONS** table that contains the following records:

Name	Occupation
Samantha	Doctor
Julia	Actor
Maria	Actor
Meera	Singer
Ashely	Professor
Ketty	Professor
Christeen	Professor
Jane	Actor
Jenny	Doctor
Priya	Singer

Sample Output

```
Ashely(P)

Christeen(P)

Jane(A)

Jenny(D)

Julia(A)
Ketty(P)

Maria(A)

Meera(S)

Priya(S)

Samantha(D)

There are a total of 2 doctors.

There are a total of 2 singers.

There are a total of 3 actors.

There are a total of 3 professors.
```

Explanation

The results of the first query are formatted to the problem description's specifications.

The results of the second query are ascendingly ordered first by number of names corresponding to each profession (), and then alphabetically by profession (, and).

Change Theme Language MySQL

```sql
-- Query 1: Alphabetical list of names with parenthetical occu
SELECT CONCAT(Name, '(', SUBSTRING(Occupation, 1, 1), ')')
FROM OCCUPATIONS
ORDER BY Name;

-- Query 2: Summary counts for each occupation formatted as a
SELECT CONCAT('There are a total of ', COUNT(Occupation), ' '
(Occupation), 's.')
FROM OCCUPATIONS
GROUP BY Occupation
ORDER BY COUNT(Occupation) ASC, Occupation ASC;
```

Pivot the *Occupation* column in **OCCUPATIONS** so that each *Name* is sorted alphabetically and displayed underneath its corresponding *Occupation*. The output should consist of four columns (*Doctor*, *Professor*, *Singer*, and *Actor*) in that specific order, with their respective names listed alphabetically under each column.

Note: Print **NULL** when there are no more names corresponding to an occupation.

Input Format

The **OCCUPATIONS** table is described as follows:

Column	Type
Name	String
Occupation	String

Occupation will only contain one of the following values: **Doctor**, **Professor**, **Singer** or **Actor**.

Sample Input

Name	Occupation
Samantha	Doctor
Julia	Actor
Maria	Actor
Meera	Singer
Ashely	Professor
Ketty	Professor
Christeen	Professor
Jane	Actor
Jenny	Doctor
Priya	Singer

Sample Output

```
Jenny    Ashley     Meera Jane

Samantha Christeen Priya Julia

NULL     Ketty      NULL Maria
```

Explanation

The first column is an alphabetically ordered list of Doctor names.

The second column is an alphabetically ordered list of Professor names.
The third column is an alphabetically ordered list of Singer names.

The fourth column is an alphabetically ordered list of Actor names.

The empty cell data for columns with less than the maximum number of names per occupation (in this case, the Professor and Actor columns) are filled with **NULL** values.

Change Theme Language MySQL

```sql
SELECT
    MAX(CASE WHEN Occupation = 'Doctor' THEN Name END) AS Doctor,
    MAX(CASE WHEN Occupation = 'Professor' THEN Name END) AS Professor,
    MAX(CASE WHEN Occupation = 'Singer' THEN Name END) AS Singer,
    MAX(CASE WHEN Occupation = 'Actor' THEN Name END) AS Actor
FROM (
    SELECT Name, Occupation,
           ROW_NUMBER() OVER (PARTITION BY Occupation ORDER BY Name) AS
           row_num
    FROM OCCUPATIONS
) AS subquery
GROUP BY row_num
ORDER BY row_num;
```

You are given a table, *BST*, containing two columns: *N* and *P,* where *N* represents the value of a node in

Binary Tree, and *P* is the parent of *N*.

Column	Type
N	Integer
P	Integer

Write a query to find the node type of *Binary Tree* ordered by the value of the node. Output one of the following for each node:

- *Root*: If node is root node.
- *Leaf*: If node is leaf node.
- *Inner*: If node is neither root nor leaf node.

Sample Input

N	P
1	2
3	2
6	8
9	8
2	5
8	5
5	null

Sample Output

```
1 Leaf


2 Inner


3 Leaf

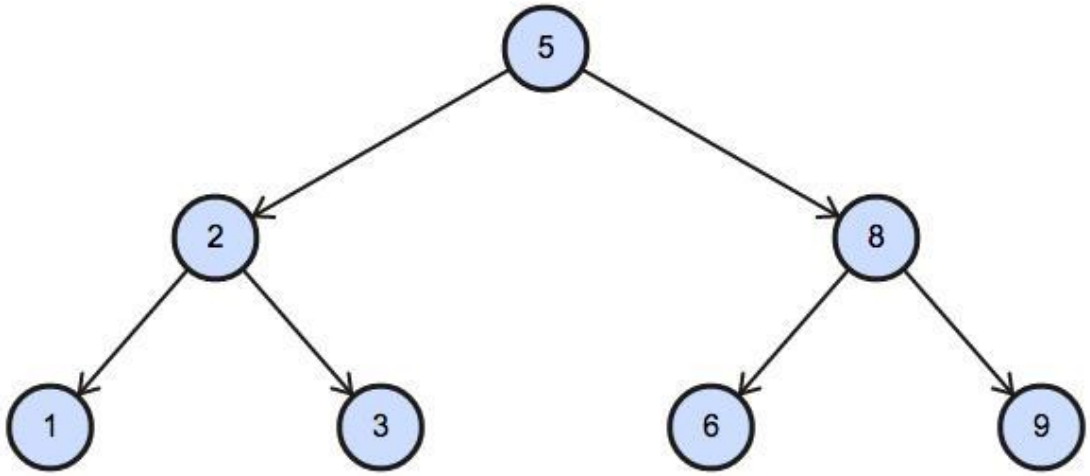

5 Root
6 Leaf


8 Inner
```

```
9 Leaf
```

Explanation

The *Binary Tree* below illustrates the sample:




```
Language

MySQL

More
```

Change Theme Language MySQL

```
SELECT N,
    CASE
        WHEN P IS NULL THEN 'Root'
        WHEN N IN (SELECT DISTINCT P FROM BST WHERE P IS NOT NULL) THE
        'Inner'
        ELSE 'Leaf'
    END AS NodeType
FROM BST
ORDER BY N ASC;
```

Amber's conglomerate corporation just acquired some new companies. Each of the companies follows this



hierarchy:

Given the table schemas below, write a query to print the *company_code*, *founder* name, total number of *lead* managers, total number of *senior* managers, total number of *managers*, and total number of *employees*. Order your output by ascending *company_code*.

Note:

- The tables may contain duplicate records.
- The *company_code* is string, so the sorting should not be **numeric**. For example, if the *company_codes* are *C_1*, *C_2*, and *C_10*, then the ascending *company_codes* will be *C_1*, *C_10*, and *C_2*.

Input Format

The following tables contain company data:

- *Company:* The *company_code* is the code of the company and *founder* is the founder of the

Column	Type
company_code	String
founder	String

company.

- *Lead_Manager:* The *lead_manager_code* is the code of the lead manager, and the *company_code*

Column	Type
lead_manager_code	String
company_code	String

is the code of the working company.

- *Senior_Manager:* The *senior_manager_code* is the code of the senior manager, the *lead_manager_code* is the code of its lead manager, and the *company_code* is the code of the

Column	Type
senior_manager_code	String
lead_manager_code	String
company_code	String

working company.

- *Manager:* The *manager_code* is the code of the manager, the *senior_manager_code* is the code of its senior manager, the *lead_manager_code* is the code of its lead manager, and the

Column	Type
manager_code	String
senior_manager_code	String
lead_manager_code	String
company_code	String

company_code is the code of the working company.

- *Employee:* The *employee_code* is the code of the employee, the *manager_code* is the code of its manager, the *senior_manager_code* is the code of its senior manager, the *lead_manager_code* is the code of its lead manager, and the *company_code* is the code of the working company.

Column	Type
employee_code	String
manager_code	String
senior_manager_code	String
lead_manager_code	String
company_code	String

Sample Input

company_code	founder
C1	Monika
C2	Samantha

Company Table:

Lead_Manager Table:

lead_manager_code	company_code
LM1	C1
LM2	C2

Senior_Manager Table:

senior_manager_code	lead_manager_code	company_code
SM1	LM1	C1
SM2	LM1	C1
SM3	LM2	C2

Manager Table:

manager_code	senior_manager_code	lead_manager_code	company_code
M1	SM1	LM1	C1
M2	SM3	LM2	C2
M3	SM3	LM2	C2

Employee Table:

employee_code	manager_code	senior_manager_code	lead_manager_code	company_code
E1	M1	SM1	LM1	C1
E2	M1	SM1	LM1	C1
E3	M2	SM3	LM2	C2
E4	M3	SM3	LM2	C2

Sample Output

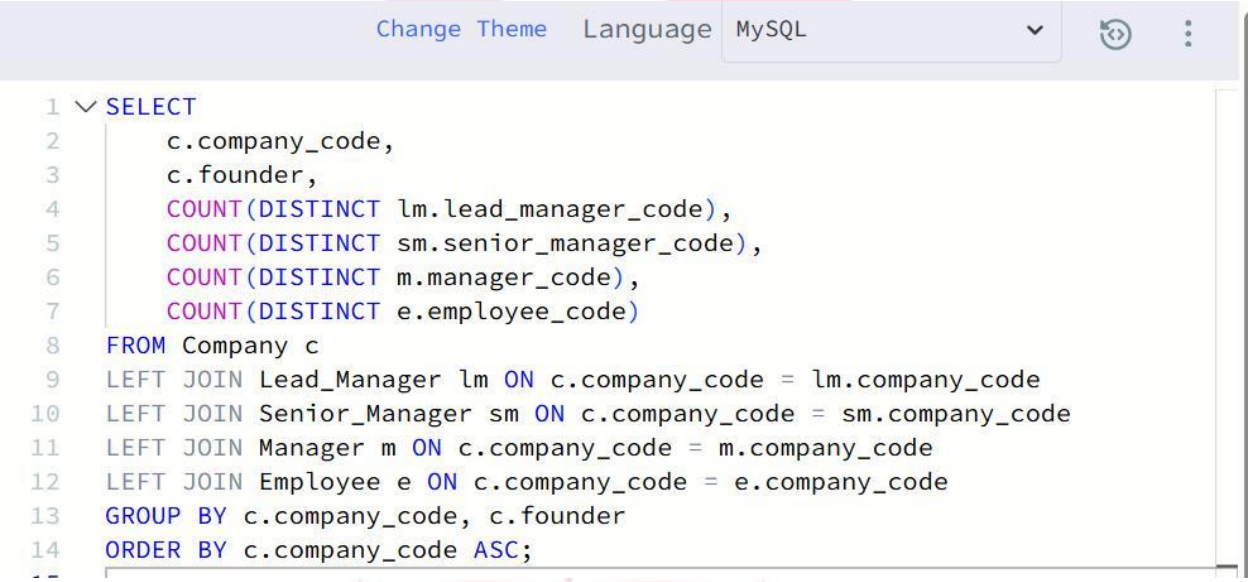
```
C1 Monika 1 2 1 2


C2 Samantha 1 1 2 2
```

Explanation

In company *C1*, the only lead manager is *LM1*. There are two senior managers, *SM1* and *SM2*, under *LM1*. There is one manager, *M1*, under senior manager *SM1*. There are two employees, *E1* and *E2*, under manager *M1*.

In company *C2*, the only lead manager is *LM2*. There is one senior manager, *SM3*, under *LM2*. There are two managers, *M2* and *M3*, under senior manager *SM3*. There is one employee, *E3*, under manager *M2*, and another employee, *E4*, under manager, *M3*.




```sql
SELECT
    c.company_code,
    c.founder,
    COUNT(DISTINCT lm.lead_manager_code),
    COUNT(DISTINCT sm.senior_manager_code),
    COUNT(DISTINCT m.manager_code),
    COUNT(DISTINCT e.employee_code)
FROM Company c
LEFT JOIN Lead_Manager lm ON c.company_code = lm.company_code
LEFT JOIN Senior_Manager sm ON c.company_code = sm.company_code
LEFT JOIN Manager m ON c.company_code = m.company_code
LEFT JOIN Employee e ON c.company_code = e.company_code
GROUP BY c.company_code, c.founder
ORDER BY c.company_code ASC;
```

Query the total population of all cities in **CITY** where *District* is **California**.

Input Format

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

The **CITY** table is described as follows:

Change Theme Language MySQL

```
1  SELECT SUM(POPULATION)
2  FROM CITY
3  WHERE DISTRICT = 'California';
4
```

Query the average population of all cities in **CITY** where *District* is **California**.

Input Format

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

The **CITY** table is described as follows:

Change Theme Language MySQL

```
SELECT AVG(POPULATION)
FROM CITY
WHERE DISTRICT = 'California';
```

Query the average population for all cities in **CITY**, rounded *down* to the nearest integer.

Input Format

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

The **CITY** table is described as follows:

Change Theme Language MySQL

```
1  SELECT FLOOR(AVG(POPULATION))
2  FROM CITY;
3
```

Query the sum of the populations for all Japanese cities in **CITY**. The *COUNTRYCODE* for Japan is **JPN**.

Input Format

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

The **CITY** table is described as follows:

Change Theme Language MySQL

```
SELECT SUM(POPULATION)
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

Query the difference between the maximum and minimum populations in **CITY**.

Input Format

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

The **CITY** table is described as follows:

Change Theme Language MySQL

```
1  SELECT MAX(POPULATION) - MIN(POPULATION)
2  FROM CITY;
3
```

Samantha was tasked with calculating the average monthly salaries for all employees in the **EMPLOYEES** table, but did not realize her keyboard's key was broken until after completing the calculation. She wants your help finding the difference between her miscalculation (using salaries with any zeros removed), and the actual average salary.

Write a query calculating the amount of error (i.e.: average monthly salaries), and round it up to the next integer.

Input Format

The **EMPLOYEES** table is described as follows:

Column	Type
ID	Integer
Name	String
Salary	Integer

Note: *Salary* is per month.

Constraints

.

Sample Input

ID	Name	Salary
1	Kristeen	1420
2	Ashley	2006
3	Julia	2210
4	Maria	3000

Sample Output

```
2061
```

Explanation

The table below shows the salaries *without zeros* as they were entered by Samantha:

ID	Name	Salary
1	Kristeen	142
2	Ashley	26
3	Julia	221
4	Maria	3

Samantha computes an average salary of . The *actual* average salary is .

The resulting error between the two calculations is . Since it is equal to the integer , it does not get rounded up.

Change Theme Language MySQL

```
1  SELECT CEIL(AVG(Salary) - AVG(REPLACE(Salary, '0', ''))
2  FROM EMPLOYEES;
```

We define an employee's *total earnings* to be their monthly worked, and the *maximum total earnings* to be the maximum total earnings for any employee in the **Employee** table. Write a query to find the *maximum total earnings* for all employees as well as the total number of employees who have maximum total earnings. Then print these values as space-separated integers.

Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is the their monthly salary.

Sample Input

employee_id	name	months	salary
12228	Rose	15	1968
33645	Angela	1	3443
45692	Frank	17	1608
56118	Patrick	7	1345
59725	Lisa	11	2330
74197	Kimberly	16	4372
78454	Bonnie	8	1771
83565	Michael	6	2017
98607	Todd	5	3396
99989	Joe	9	3573

Sample Output

```
69952 1
```

Explanation

The table and earnings data is depicted in the following diagram:

employee_id	name	months	salary	earnings
12228	Rose	15	1968	29520
33645	Angela	1	3443	3443
45692	Frank	17	1608	27336
56118	Patrick	7	1345	9415
59725	Lisa	11	2330	25630
74197	Kimberly	16	4372	69952
78454	Bonnie	8	1771	14168
83565	Michael	6	2017	12102
98607	Todd	5	3396	16980
99989	Joe	9	3573	32157

The maximum *earnings* value is . The only employee with *earnings* is *Kimberly*, so we print the maximum *earnings* value () and a count of the number of employees who have earned (which is) as two space-separated values.

Change Theme Language MySQL

```
SELECT (months * salary) AS earnings, COUNT(*)
FROM Employee
GROUP BY earnings
ORDER BY earnings DESC
LIMIT 1;
```

Query the following two values from the **STATION** table:

1. The sum of all values in *LAT_N* rounded to a scale of decimal places.

2. The sum of all values in *LONG_W* rounded to a scale of decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Output Format

Your results must be in the form:

```
lat lon
```

where is the sum of all values in *LAT_N* and is the sum of all values in

Change Theme Language MySQL

```
1  SELECT ROUND(SUM(LAT_N), 2), ROUND(SUM(LONG_W), 2)
2  FROM STATION;
```

Query the sum of *Northern Latitudes* (*LAT_N*) from **STATION** having values greater than and less than .

Truncate your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

```
SELECT ROUND(SUM(LAT_N), 4)
FROM STATION
WHERE LAT_N > 38.7880 AND LAT_N < 137.2345;
```

Query the greatest value of the *Northern Latitudes* (*LAT_N*) from **STATION** that is less than . Truncate your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Change Theme Language MySQL

```
SELECT ROUND(MAX(LAT_N), 4)
FROM STATION
WHERE LAT_N < 137.2345;
```

Query the *Western Longitude* (*LONG_W*) for the largest *Northern Latitude* (*LAT_N*) in **STATION** that is less than . Round your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Change Theme Language MySQL

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
    SELECT MAX(LAT_N)
    FROM STATION
    WHERE LAT_N < 137.2345
);
```

Query the smallest *Northern Latitude* (*LAT_N*) from **STATION** that is greater than . Round your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Change Theme Language MySQL

```
1  SELECT ROUND(MIN(LAT_N), 4)
2  FROM STATION
3  WHERE LAT_N > 38.7780;
```

Query the *Western Longitude* (*LONG_W*)where the smallest *Northern Latitude* (*LAT_N*) in **STATION** is greater than . Round your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude

Change Theme Language MySQL

```mysql
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
    SELECT MIN(LAT_N)
    FROM STATION
    WHERE LAT_N > 38.7780
);
```

Consider and to be two points on a *2D* plane.

- happens to equal the minimum value in *Northern Latitude* (*LAT_N* in **STATION**).
- happens to equal the minimum value in *Western Longitude* (*LONG_W* in **STATION**).
- happens to equal the maximum value in *Northern Latitude* (*LAT_N* in **STATION**).
- happens to equal the maximum value in *Western Longitude* (*LONG_W* in **STATION**).

Query the Manhattan Distance between points and and round it to a scale of decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Change Theme Language MySQL

```mysql
SELECT ROUND(
    (MAX(LAT_N) - MIN(LAT_N)) + (MAX(LONG_W) - MIN(LONG_
    4
)
FROM STATION;
```

Consider and to be two points on a 2D plane where are the respective minimum and maximum values of *Northern Latitude* (*LAT_N*) and are the respective minimum and maximum values of *Western Longitude* (*LONG_W*) in **STATION**.

Query the Euclidean Distance between points and and *format your answer* to display decimal digits.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Change Theme Language MySQL

```
SELECT ROUND(
    SQRT(
        POWER(MIN(LAT_N) - MAX(LAT_N), 2) +
        POWER(MIN(LONG_W) - MAX(LONG_W), 2)
    ),
    4
)
FROM STATION;
```

A *median* is defined as a number separating the higher half of a data set from the lower half. Query the *median* of the *Northern Latitudes* (*LAT_N*) from **STATION** and round your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Change Theme Language MySQL

```mysql
WITH RankedStations AS (
    SELECT LAT_N,
           ROW_NUMBER() OVER (ORDER BY LAT_N) as row_num,
           COUNT(*) OVER () as total_count
    FROM STATION
)
SELECT ROUND(AVG(LAT_N), 4)
FROM RankedStations
WHERE row_num IN (FLOOR((total_count + 1) / 2), CEIL((total_count + 1) / 2
```

Given the **CITY** and **COUNTRY** tables, query the sum of the populations of all cities where the *CONTINENT* is *'Asia'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Change Theme Language MySQL

```sql
SELECT SUM(CITY.POPULATION)
FROM CITY
JOIN COUNTRY ON CITY.COUNTRYCODE = COUNTRY.CODE
WHERE COUNTRY.CONTINENT = 'Asia';
```

Given the **CITY** and **COUNTRY** tables, query the names of all cities where the *CONTINENT* is *'Africa'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

AUTONOMOUS

COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

Change Theme Language MySQL

```
1  SELECT CITY.NAME
2  FROM CITY
3  JOIN COUNTRY ON CITY.CountryCode = COUNTRY.Code
4  WHERE COUNTRY.CONTINENT = 'Africa';
5
```

Given the **CITY** and **COUNTRY** tables, query the names of all the continents (*COUNTRY.Continent*) and their

respective average city populations (*CITY.Population*) rounded *down* to the nearest integer.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

Change Theme Language MySQL

```mysql
SELECT COUNTRY.Continent, FLOOR(AVG(CITY.Population))
FROM CITY
JOIN COUNTRY ON CITY.CountryCode = COUNTRY.Code
GROUP BY COUNTRY.Continent;
```

You are given two tables: *Students* and *Grades*. *Students* contains three columns *ID*, *Name* and *Marks*.

Column	Type
ID	Integer
Name	String
Marks	Integer

Grades contains the following data:

Grade	Min_Mark	Max_Mark
1	0	9
2	10	19
3	20	29
4	30	39
5	40	49
6	50	59
7	60	69
8	70	79
9	80	89
10	90	100

Ketty gives *Eve* a task to generate a report containing three columns: *Name*, *Grade* and *Mark*. *Ketty* doesn't want the NAMES of those students who received a grade lower than *8*. The report must be in descending order by grade -- i.e. higher grades are entered first. If there is more than one student with the same grade (8-10) assigned to them, order those particular students by their name alphabetically. Finally, if the grade is

lower than 8, use "NULL" as their name and list them by their grades in descending order. If there is more than one student with the same grade (1-7) assigned to them, order those particular students by their marks in ascending order.

Write a query to help Eve.

Sample Input

ID	Name	Marks
1	Julia	88
2	Samantha	68
3	Maria	99
4	Scarlet	78
5	Ashley	63
6	Jane	81

Sample Output

```
Maria 10 99

Jane 9 81

Julia 9 88

Scarlet 8 78

NULL 7 63

NULL 7 68
```

Note

Print "NULL" as the name if the grade is less than 8.

Explanation

Consider the following table with the grades assigned to the students:

ID	Name	Marks	Grade
1	Julia	88	9
2	Samantha	68	7
3	Maria	99	10
4	Scarlet	78	8
5	Ashley	63	7
6	Jane	81	9

So, the following students got *8*, *9* or *10* grades:

- *Maria (grade 10)*
- *Jane (grade 9)*
- *Julia (grade 9)*
- *Scarlet (grade 8)*

Change Theme Language MySQL

```mysql
SELECT
    CASE
        WHEN g.Grade >= 8 THEN s.Name
        ELSE NULL
    END AS Name,
    g.Grade,
    s.Marks
FROM Students s
JOIN Grades g ON s.Marks BETWEEN g.Min_Mark AND g.Max_Mark
ORDER BY
    g.Grade DESC,
    Name ASC,
    s.Marks ASC;
```

Julia just finished conducting a coding contest, and she needs your help assembling the leaderboard! Write a query to print the respective *hacker_id* and *name* of hackers who achieved full scores for *more than one* challenge. Order your output in descending order by the total number of challenges in which the hacker earned a full score. If more than one hacker received full scores in same number of challenges, then sort them by ascending *hacker_id*.

Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Difficulty:* The *difficult_level* is the level of difficulty of the challenge, and *score* is the maximum

score that can be achieved for a challenge at that difficulty level.

Column	Type
difficulty_level	Integer
score	Integer

- *Challenges:* The *challenge_id* is the id of the challenge, the *hacker_id* is the id of the hacker who created the challenge, and *difficulty_level* is the level of difficulty of the challenge.

Column	Type
challenge_id	Integer
hacker_id	Integer
difficulty_level	Integer

- *Submissions:* The *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, *challenge_id* is the id of the challenge that the submission belongs to, and *score* is the score of the submission.

Column	Type
submission_id	Integer
hacker_id	Integer
challenge_id	Integer
score	Integer

Sample Input

hacker_id	name
5580	Rose
8439	Angela
27205	Frank
52243	Patrick
52348	Lisa
57645	Kimberly
77726	Bonnie
83082	Michael
86870	Todd
90411	Joe

Hackers Table:

difficulty_level	score
1	20
2	30
3	40
4	60
5	80
6	100
7	120

Difficulty Table:

challenge_id	hacker_id	difficulty_level
4810	77726	4
21089	27205	1
36566	5580	7
66730	52243	6
71055	52243	2

Challenges Table:

Submissions Table:

submission_id	hacker_id	challenge_id	score
68628	77726	36566	30
65300	77726	21089	10
40326	52243	36566	77
8941	27205	4810	4
83554	77726	66730	30
43353	52243	66730	0
55385	52348	71055	20
39784	27205	71055	23
94613	86870	71055	30
45788	52348	36566	0
93058	86870	36566	30
7344	8439	66730	92
2721	8439	4810	36
523	5580	71055	4
49105	52348	66730	0
55877	57645	66730	80
38355	27205	66730	35
3924	8439	36566	80
97397	90411	66730	100
84162	83082	4810	40
97431	90411	71055	30

Sample Output

```
90411 Joe
```

Explanation

Hacker *86870* got a score of *30* for challenge *71055* with a difficulty level of *2*, so *86870* earned a full score for this challenge.

Hacker *90411* got a score of *30* for challenge *71055* with a difficulty level of *2*, so *90411* earned a full score for this challenge.

Hacker *90411* got a score of *100* for challenge *66730* with a difficulty level of *6*, so *90411* earned a full score for this challenge.

Only hacker *90411* managed to earn a full score for more than one challenge, so we print the their *hacker_id* and *name* as space-separated values.

Change Theme Language MySQL

```mysql
SELECT
    h.hacker_id,
    h.name
FROM Submissions s
JOIN Challenges c ON s.challenge_id = c.challenge_id
JOIN Difficulty d ON c.difficulty_level = d.difficulty_level
JOIN Hackers h ON s.hacker_id = h.hacker_id
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(s.challenge_id) > 1
ORDER BY COUNT(s.challenge_id) DESC, h.hacker_id ASC;
```

Harry Potter and his friends are at Ollivander's with Ron, finally replacing Charlie's old broken wand.

Hermione decides the best way to choose is by determining the minimum number of gold galleons needed to buy each *non-evil* wand of high power and age. Write a query to print the *id*, *age*, *coins_needed*, and *power* of the wands that Ron's interested in, sorted in order of descending *power*. If more than one wand has same

power, sort the result in order of descending *age*.

Input Format

The following tables contain data on the wands in Ollivander's inventory:

- *Wands:* The *id* is the id of the wand, *code* is the code of the wand, *coins_needed* is the total number of gold galleons needed to buy the wand, and *power* denotes the quality of the wand

Column	Type
id	Integer
code	Integer
coins_needed	Integer
power	Integer

(the higher the power, the better the wand is).

- *Wands_Property:* The *code* is the code of the wand, *age* is the age of the wand, and *is_evil* denotes whether the wand is good for the dark arts. If the value of *is_evil* is *0*, it means that the wand is not evil. The mapping between *code* and *age* is one-one, meaning that if there are two

Column	Type
code	Integer
age	Integer
is_evil	Integer

pairs, and , then and .

Sample Input

id	code	coins_needed	power
1	4	3688	8
2	3	9365	3
3	3	7187	10
4	3	734	8
5	1	6020	2
6	2	6773	7
7	3	9873	9
8	3	7721	7
9	1	1647	10
10	4	504	5
11	2	7587	5
12	5	9897	10
13	3	4651	8
14	2	5408	1
15	2	6018	7
16	4	7710	5
17	2	8798	7
18	2	3312	3
19	4	7651	6
20	5	5689	3

Wands Table:

Wands_Property Table:

code	age	is_evil
1	45	0
2	40	0
3	4	1
4	20	0
5	17	0

Sample Output

```
9 45 1647 10

12 17 9897 10

1 20 3688 8

15 40 6018 7

19 20 7651 6

11 40 7587 5

10 20 504 5

18 40 3312 3

20 17 5689 3

5 45 6020 2

14 40 5408 1
```

Explanation

id	age	coins_needed	power
5	45	6020	2
9	45	1647	10

The data for wands of *age 45* (code 1):

- The minimum number of galleons needed for
- The minimum number of galleons needed for

id	age	coins_needed	power
14	40	5408	1
18	40	3312	3
11	40	7587	5
15	40	6018	7
17	40	8798	7
6	40	6773	7

The data for wands of *age 40* (code 2):

- The minimum number of galleons needed for
- The minimum number of galleons needed for
- The minimum number of galleons needed for
- The minimum number of galleons needed for

id	age	coins_needed	power
10	20	504	5
16	20	7710	5
19	20	7651	6
1	20	3688	8

The data for wands of *age 20* (code 4):

- The minimum number of galleons needed for
- The minimum number of galleons needed for
- The minimum number of galleons needed for

id	age	coins_needed	power
20	17	5689	3
12	17	9897	10

The data for wands of *age 17* (code 5):

- The minimum number of galleons needed for
- The minimum number of galleons needed for

Change Theme Language MySQL

```mysql
SELECT w.id, wp.age, w.coins_needed, w.power
FROM Wands w
JOIN Wands_Property wp ON w.code = wp.code
WHERE wp.is_evil = 0
  AND w.coins_needed = (
      SELECT MIN(w2.coins_needed)
      FROM Wands w2
      JOIN Wands_Property wp2 ON w2.code = wp2.code
      WHERE w2.power = w.power
        AND wp2.age = wp.age
  )
ORDER BY w.power DESC, wp.age DESC;
```

Julia asked her students to create some coding challenges. Write a query to print the *hacker_id*, *name*, and the total number of challenges created by each student. Sort your results by the total number of challenges in descending order. If more than one student created the same number of challenges, then sort the result by *hacker_id*. If more than one student created the same number of challenges and the count is less than the maximum number of challenges created, then exclude those students from the result.

Input Format

The following tables contain challenge data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Challenges:* The *challenge_id* is the id of the challenge, and *hacker_id* is the id of the student who

Column	Type
challenge_id	Integer
hacker_id	Integer

created the challenge.

Sample Input 0

hacker_id	name
5077	Rose
21283	Angela
62743	Frank
88255	Patrick
96196	Lisa

Hackers Table:

challenge_id	hacker_id
61654	5077
58302	21283
40587	88255
29477	5077
1220	21283
69514	21283
46561	62743
58077	62743
18483	88255
76766	21283
52382	5077
74467	21283
33625	96196
26053	88255
42665	62743
12859	62743
70094	21283
34599	88255
54680	88255
61881	5077

Challenges Table:

Sample Output 0

```
21283 Angela 6
88255 Patrick 5

96196 Lisa 1
```

Sample Input 1

challenge_id	hacker_id
63963	81041
63117	79345
28225	34856
21989	12299
4653	12299
70070	79345
36905	34856
61136	80491
17234	12299
80308	79345
40510	34856
79820	80491
22720	12299
21394	12299
36261	34856
15334	12299
71435	79345
23157	34856
54102	34856
69065	80491

hacker_id	name
12299	Rose
34856	Angela
79345	Frank
80491	Patrick
81041	Lisa

Hackers Table:

Challenges Table:

Sample Output 1

```
12299 Rose 6
34856 Angela 6

79345 Frank 4

80491 Patrick 3

81041 Lisa 1
```

Explanation

For *Sample Case 0*, we can get the following details:

hacker_id	name	challenges_created
21283	Angela	6
88255	Patrick	5
5077	Rose	4
62743	Frank	4
96196	Lisa	1

Students and both created challenges, but the maximum number of challenges created is so these students are excluded from the result.

For *Sample Case 1*, we can get the following details:

hacker_id	name	challenges_created
12299	Rose	6
34856	Angela	6
79345	Frank	4
80491	Patrick	3
81041	Lisa	1

Students and both created challenges. Because is the maximum number of challenges created, these students are included in the result.

```sql
WITH ChallengeCounts AS (
    SELECT
        h.hacker_id,
        h.name,
        COUNT(c.challenge_id) AS total_challenges
    FROM Hackers h
    JOIN Challenges c ON h.hacker_id = c.hacker_id
    GROUP BY h.hacker_id, h.name
),
MaxAndDupCheck AS (
    SELECT
        total_challenges,
        COUNT(*) AS count_of_hackers_with_this_total,
        MAX(total_challenges) OVER() AS max_challenges_ove
    FROM ChallengeCounts
    GROUP BY total_challenges
)
SELECT
    cc.hacker_id,
    cc.name,
    cc.total_challenges
FROM ChallengeCounts cc
JOIN MaxAndDupCheck mdc ON cc.total_challenges = mdc.total
WHERE
    cc.total_challenges = mdc.max_challenges_overall
```

You did such a great job helping Julia with her last coding contest challenge that she wants you to work on this one, too!

The total score of a hacker is the sum of their maximum scores for all of the challenges. Write a query to print the *hacker_id*, *name*, and total score of the hackers ordered by the descending score. If more than one hacker achieved the same total score, then sort the result by ascending *hacker_id*. Exclude all hackers with a total score of from your result.

Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Submissions:* The *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, *challenge_id* is the id of the challenge for which the submission belongs to,

Column	Type
submission_id	Integer
hacker_id	Integer
challenge_id	Integer
score	Integer

and *score* is the score of the submission.

Sample Input

hacker_id	name
4071	Rose
4806	Angela
26071	Frank
49438	Patrick
74842	Lisa
80305	Kimberly
84072	Bonnie
87868	Michael
92118	Todd
95895	Joe

Hackers Table:

submission_id	hacker_id	challenge_id	score
67194	74842	63132	76
64479	74842	19797	98
40742	26071	49593	20
17513	4806	49593	32
69846	80305	19797	19
41002	26071	89343	36
52826	49438	49593	9
31093	26071	19797	2
81614	84072	49593	100
44829	26071	89343	17
75147	80305	49593	48
14115	4806	49593	76
6943	4071	19797	95
12855	4806	25917	13
73343	80305	49593	42
84264	84072	63132	0
9951	4071	49593	43
45104	49438	25917	34
53795	74842	19797	5
26363	26071	19797	29
10063	4071	49593	96

Submissions Table:

Sample Output

```
4071 Rose 191


74842 Lisa 174


84072 Bonnie 100


4806 Angela 89


26071 Frank 85


80305 Kimberly 67


49438 Patrick 43
```

Explanation

Hacker *4071* submitted solutions for challenges *19797* and *49593*, so the total score .

Hacker *74842* submitted solutions for challenges *19797* and *63132*, so the total score

Hacker *84072* submitted solutions for challenges *49593* and *63132*, so the total score .

The total scores for hackers *4806*, *26071*, *80305*, and *49438* can be similarly calculated.

Change Theme Language MySQL

```mysql
SELECT
    h.hacker_id,
    h.name,
    SUM(max_scores.max_score) AS total_score
FROM Hackers h
JOIN (
    SELECT
        hacker_id,
        challenge_id,
        MAX(score) AS max_score
    FROM Submissions
    GROUP BY hacker_id, challenge_id
) max_scores ON h.hacker_id = max_scores.hacker_id
GROUP BY h.hacker_id, h.name
HAVING SUM(max_scores.max_score) > 0
ORDER BY total_score DESC, h.hacker_id ASC;
```

You are given a table, *Projects*, containing three columns: *Task_ID*, *Start_Date* and *End_Date*. It is guaranteed that the difference between the *End_Date* and the *Start_Date* is equal to *1* day for each row in the table.

Column	*Type*
Task_ID	*Integer*
Start_Date	*Date*
End_Date	*Date*

If the *End_Date* of the tasks are consecutive, then they are part of the same project. Samantha is interested in finding the total number of different projects completed.
Write a query to output the start and end dates of projects listed by the number of days it took to complete

the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project.

Sample Input

Task_ID	Start_Date	End_Date
1	2015-10-01	2015-10-02
2	2015-10-02	2015-10-03
3	2015-10-03	2015-10-04
4	2015-10-13	2015-10-14
5	2015-10-14	2015-10-15
6	2015-10-28	2015-10-29
7	2015-10-30	2015-10-31

Sample Output

```
2015-10-28 2015-10-29

2015-10-30 2015-10-31

2015-10-13 2015-10-15

2015-10-01 2015-10-04
```

Explanation

The example describes following *four* projects:

- *Project 1*: Tasks *1*, *2* and *3* are completed on consecutive days, so these are part of the project. Thus start date of project is *2015-10-01* and end date is *2015-10-04*, so it took *3 days* to complete the project.
- *Project 2*: Tasks *4* and *5* are completed on consecutive days, so these are part of the project. Thus, the start date of project is *2015-10-13* and end date is *2015-10-15*, so it took *2 days* to complete the project.
- *Project 3*: Only task *6* is part of the project. Thus, the start date of project is *2015-10-28* and end date is *2015-10-29*, so it took *1 day* to complete the project.
- *Project 4*: Only task *7* is part of the project. Thus, the start date of project is *2015-10-30* and end date is *2015-10-31*, so it took *1 day* to complete the project.

Change Theme Language MySQL

```mysql
SELECT s.Start_Date, MIN(e.End_Date)
FROM
    (SELECT Start_Date FROM Projects WHERE Start_Date NOT IN (SELECT
    End_Date FROM Projects)) s
JOIN
    (SELECT End_Date FROM Projects WHERE End_Date NOT IN (SELECT Start_Da
    FROM Projects)) e
ON s.Start_Date < e.End_Date
GROUP BY s.Start_Date
ORDER BY DATEDIFF(MIN(e.End_Date), s.Start_Date) ASC, s.Start_Date ASC;
```

You are given a table, *Functions*, containing two columns: *X* and *Y*.

Column	Type
X	Integer
Y	Integer

Two pairs (X_1, Y_1) and (X_2, Y_2) are said to be *symmetric pairs* if $X_1 = Y_2$ and $X_2 = Y_1$.

Write a query to output all such *symmetric pairs* in ascending order by the value of *X*. List the rows such that $X_1 \leq Y_1$.

Sample Input

X	Y
20	20
20	20
20	21
23	22
22	23
21	20

Sample Output

```
20 20
20 21


22 23
```

Change Theme Language MySQL

```mysql
SELECT X, Y
FROM Functions f1
WHERE X < Y
  AND EXISTS (
      SELECT 1
      FROM Functions f2
      WHERE f2.X = f1.Y AND f2.Y = f1.X
  )

UNION ALL

SELECT X, Y
FROM Functions
WHERE X = Y
GROUP BY X, Y
HAVING COUNT(*) > 1

ORDER BY X;
```

Samantha interviews many candidates from different colleges using coding challenges and contests. Write a query to print the *contest_id*, *hacker_id*, *name*, and the sums of *total_submissions*, *total_accepted_submissions*, *total_views*, and *total_unique_views* for each contest sorted by *contest_id*. Exclude the contest from the result if all four sums are .

Note: A specific contest can be used to screen candidates at more than one college, but each college only holds screening contest.

Input Format

The following tables hold interview data:

- *Contests:* The *contest_id* is the id of the contest, *hacker_id* is the id of the hacker who created the

Column	Type
contest_id	Integer
hacker_id	Integer
name	String

  contest, and *name* is the name of the hacker.

- *Colleges:* The *college_id* is the id of the college, and *contest_id* is the id of the contest that

Column	Type
college_id	Integer
contest_id	Integer

  Samantha used to screen the candidates.

- *Challenges:* The *challenge_id* is the id of the challenge that belongs to one of the contests whose contest_id Samantha forgot, and *college_id* is the id of the college where the challenge was given

Column	Type
challenge_id	Integer
college_id	Integer

  to candidates.

- *View_Stats:* The *challenge_id* is the id of the challenge, *total_views* is the number of times the challenge was viewed by candidates, and *total_unique_views* is the number of times the

Column	Type
challenge_id	Integer
total_views	Integer
total_unique_views	Integer

challenge was viewed by unique candidates.

- *Submission_Stats:* The *challenge_id* is the id of the challenge, *total_submissions* is the number of submissions for the challenge, and *total_accepted_submission* is the number of submissions that

Column	Type
challenge_id	Integer
total_submissions	Integer
total_accepted_submissions	Integer

achieved full scores.

Sample Input

contest_id	hacker_id	name
66406	17973	Rose
66556	79153	Angela
94828	80275	Frank

Contests Table:

Colleges Table:

college_id	contest_id
11219	66406
32473	66556
56685	94828

Challenges Table:

challenge_id	college_id
18765	11219
47127	11219
60292	32473
72974	56685

View_Stats

challenge_id	total_views	total_unique_views
47127	26	19
47127	15	14
18765	43	10
18765	72	13
75516	35	17
60292	11	10
72974	41	15
75516	75	11

Table:

Submission_Stats Table:

challenge_id	total_submissions	total_accepted_submissions
75516	34	12
47127	27	10
47127	56	18
75516	74	12
75516	83	8
72974	68	24
72974	82	14
47127	28	11

Sample Output

```
66406 17973 Rose 111 39 156 56


66556 79153 Angela 0 0 11 10
94828 80275 Frank 150 38 41 15
```

Explanation

The contest is used in the college . In this college , challenges and are asked, so from the *view* and *submission* stats:

- Sum of total submissions
- Sum of total accepted submissions
- Sum of total views
- Sum of total unique views

Similarly, we can find the sums for contests and .

Change Theme Language MySQL

```
SELECT
    con.contest_id,
    con.hacker_id,
    con.name,
    SUM(COALESCE(sub.sum_submissions, 0)) AS total_submissions,
    SUM(COALESCE(sub.sum_accepted_submissions, 0)) AS
    total_accepted_submissions,
    SUM(COALESCE(vw.sum_views, 0)) AS total_views,
    SUM(COALESCE(vw.sum_unique_views, 0)) AS total_unique_views
FROM Contests con
JOIN Colleges col ON con.contest_id = col.contest_id
JOIN Challenges cha ON col.college_id = cha.college_id
LEFT JOIN (
    SELECT
        challenge_id,
        SUM(total_submissions) AS sum_submissions,
        SUM(total_accepted_submissions) AS sum_accepted_submissi
    FROM Submission_Stats
    GROUP BY challenge_id
) sub ON cha.challenge_id = sub.challenge_id
LEFT JOIN (
    SELECT
        challenge_id,
        SUM(total_views) AS sum_views,
        SUM(total_unique_views) AS sum_unique_views
```

Julia conducted a days of learning SQL contest. The start date of the contest was *March 01, 2016* and the end date was *March 15, 2016*.

Write a query to print total number of unique hackers who made at least submission each day (starting on the first day of the contest), and find the *hacker_id* and *name* of the hacker who made maximum number of submissions each day. If more than one such hacker has a maximum number of submissions, print the lowest *hacker_id*. The query should print this information for each day of the contest, sorted by the date.

Input Format

The following tables hold contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the

Column	Type
hacker_id	Integer
name	String

hacker.

- *Submissions:* The *submission_date* is the date of the submission, *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, and *score* is the score of

Column	Type
submission_date	Date
submission_id	Integer
hacker_id	Integer
score	Integer

the submission.

Sample Input

For the following sample input, assume that the end date of the contest was *March 06, 2016*.

hacker_id	name
15758	Rose
20703	Angela
36396	Frank
38289	Patrick
44065	Lisa
53473	Kimberly
62529	Bonnie
79722	Michael

Hackers Table:

Submissions Table:

submission_date	submission_id	hacker_id	score
2016-03-01	8494	20703	0
2016-03-01	22403	53473	15
2016-03-01	23965	79722	60
2016-03-01	30173	36396	70
2016-03-02	34928	20703	0
2016-03-02	38740	15758	60
2016-03-02	42769	79722	25
2016-03-02	44364	79722	60
2016-03-03	45440	20703	0
2016-03-03	49050	36396	70
2016-03-03	50273	79722	5
2016-03-04	50344	20703	0
2016-03-04	51360	44065	90
2016-03-04	54404	53473	65
2016-03-04	61533	79722	45
2016-03-05	72852	20703	0
2016-03-05	74546	38289	0
2016-03-05	76487	62529	0
2016-03-05	82439	36396	10
2016-03-05	90006	36396	40
2016-03-06	90404	20703	0

Sample Output

```
2016-03-01 4 20703 Angela
```

```
2016-03-02 2 79722 Michael

2016-03-03 2 20703 Angela

2016-03-04 2 20703 Angela

2016-03-05 1 36396 Frank

2016-03-06 1 20703 Angela
```

Explanation

On *March 01, 2016* hackers , , , and made submissions. There are unique hackers who made at least one submission each day. As each hacker made one submission, is considered to be the hacker who made maximum number of submissions on this day. The name of the hacker is *Angela*.

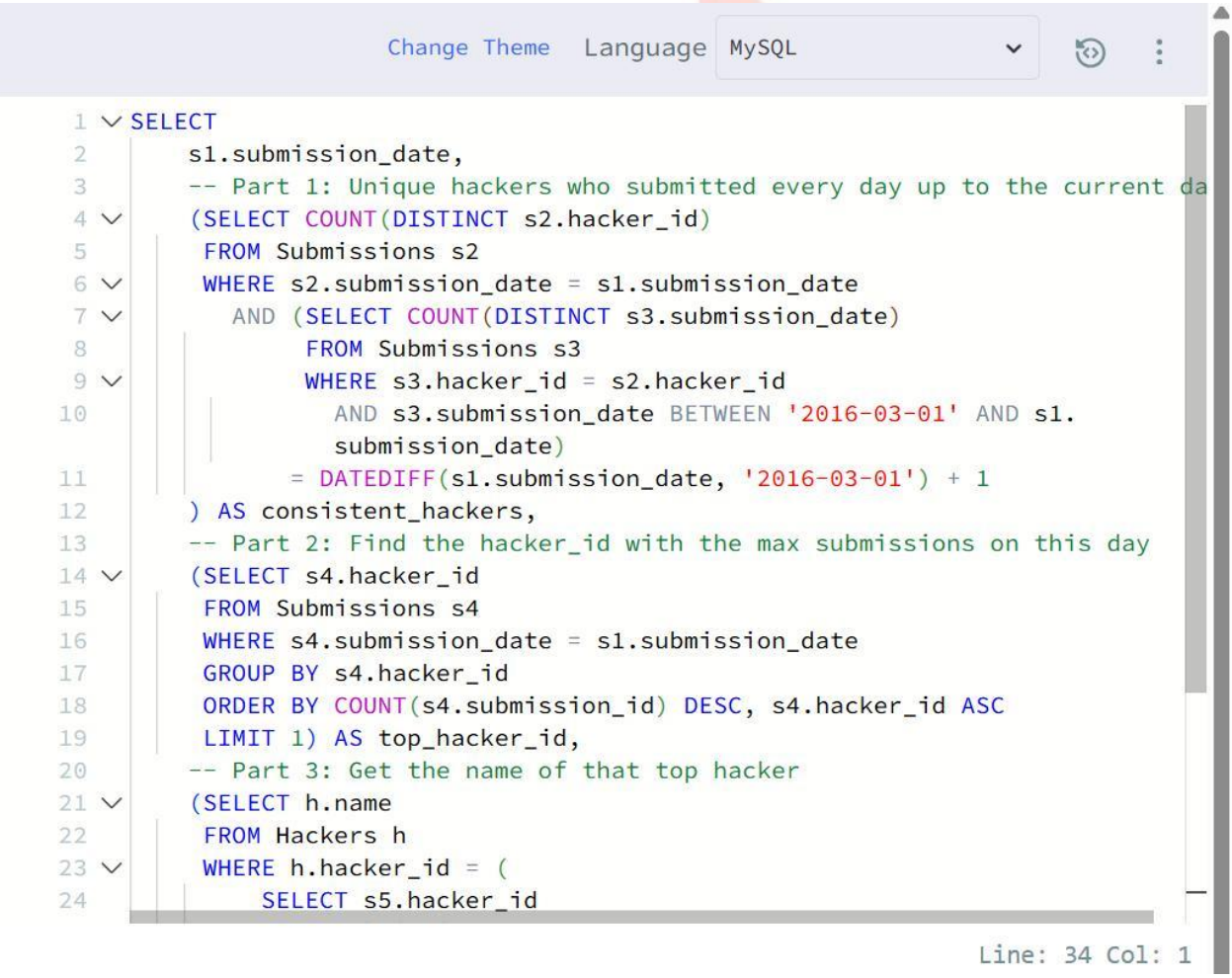
On *March 02, 2016* hackers , , and made submissions. Now and were the only ones to submit every day, so there are unique hackers who made at least one submission each day. made submissions, and name of the hacker is *Michael*.

On *March 03, 2016* hackers , , and made submissions. Now and were the only ones, so there are unique hackers who made at least one submission each day. As each hacker made one submission so is considered to be the hacker who made maximum number of submissions on this day. The name of the hacker is *Angela*.

On *March 04, 2016* hackers , , , and made submissions. Now and only submitted each day, so there are unique hackers who made at least one submission each day. As each hacker made one submission so is considered to be the hacker who made maximum number of submissions on this day. The name of the hacker is *Angela*.

On *March 05, 2016* hackers , , and made submissions. Now only submitted each day, so there is only unique hacker who made at least one submission each day. made submissions and name of the hacker is *Frank*.

On *March 06, 2016* only made submission, so there is only unique hacker who made at least one submission each day. made submission and name of the hacker is *Angela*.



```sql
SELECT
    (SELECT COUNT(DISTINCT s2.hacker_id)
    ) AS consistent_hackers,
    -- Part 2: Find the hacker_id with the max submissions on this
    (SELECT s4.hacker_id
     FROM Submissions s4
     WHERE s4.submission_date = s1.submission_date
     GROUP BY s4.hacker_id
     ORDER BY COUNT(s4.submission_id) DESC, s4.hacker_id ASC
     LIMIT 1) AS top_hacker_id,
    -- Part 3: Get the name of that top hacker
    (SELECT h.name
     FROM Hackers h
     WHERE h.hacker_id = (
         SELECT s5.hacker_id
         FROM Submissions s5
         WHERE s5.submission_date = s1.submission_date
         GROUP BY s5.hacker_id
         ORDER BY COUNT(s5.submission_id) DESC, s5.hacker_id ASC
         LIMIT 1
     )
    ) AS top_hacker_name
FROM (SELECT DISTINCT submission_date FROM Submissions) s1
ORDER BY s1.submission_date ASC;
```

P(R) represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:

```
* * * * *

* * * *

* * *

* *

*
```

Write a query to print the pattern *P(20)*.

```sql
WITH RECURSIVE NumberSequence AS (
    SELECT 20 AS num
    UNION ALL
    SELECT num - 1 FROM NumberSequence WHERE num > 1
)
SELECT REPEAT('* ', num)
FROM NumberSequence;
```

P(R) represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:

```
*

* *

* * *

* * * *

* * * * *
```

Write a query to print the pattern *P(20)*.

Change Theme Language MySQL

```sql
WITH RECURSIVE Pattern AS (
    SELECT 1 AS num
    UNION ALL
    SELECT num + 1 FROM Pattern WHERE num < 20
)
SELECT REPEAT('* ', num)
FROM Pattern;
```

Write a query to print all *prime numbers* less than or equal to . Print your result on a single line, and use the ampersand () character as your separator (instead of a space).

For example, the output for all prime numbers would be:

2&3&5&7

Change Theme Language MySQL

```mysql
WITH RECURSIVE Numbers AS (
    SELECT 2 AS num
    UNION ALL
    SELECT num + 1 FROM Numbers WHERE num < 1000
)
SELECT GROUP_CONCAT(n1.num SEPARATOR '&')
FROM Numbers n1
WHERE NOT EXISTS (
    SELECT 1
    FROM Numbers n2
    WHERE n2.num < n1.num
      AND n1.num % n2.num = 0
);
```

ANNEXURE - CODE CHEF

All Products

1.Task

Write a query to output Products table.

Table name: Products

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

ANSWER
SELECT * FROM Products;

Your Output

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

2. High Price of Products

Task

Write a query to find all product_name and category that have a price greater than 100.00 from the **Products** table.

Table name: Products

Answer:

```
SELECT product_name, category
FROM Products
WHERE price > 100.00;
```

Your Output

product_name	category
Laptop	Electronics
Desk Chair	Furniture
Smartphone	Electronics
Standing Desk	Furniture
Tablet	Electronics

3.Average Salary

Task

Write a query to calculate the average salary across all companies combined. Rename the column as avg_salary.

Table name: Works

employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10

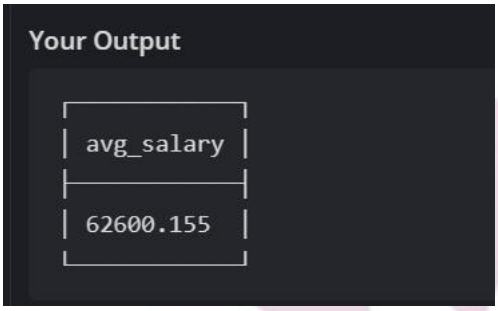
9	EduWorks	51000.00
10	GreenTech	72000.00

Expected Column: avg_salary

Answer :

```
SELECT AVG(salary) AS avg_salary

FROM Works;
```



4.Write a query to retrieve the department_name and location of departments located in a city that starts with 'S'.

Table Name: departments

department_id	department_name	location
1	Human Resources	New York
2	Research and Development	San Francisco
3	Sales	Los Angeles
4	Marketing	New York
5	Customer Support	Boston
6	Finance	Austin
7	IT Support	Seattle

8 Product Management San Francisco

department_id department_name location

9 Quality Assurance Los Angeles

10 Legal Boston

Expected Columns

- department_name
- Location

Answer:

SELECT department_name, location

FROM departments

WHERE location LIKE 'S%';

Your Output

department_name	location
Research and Development	San Francisco
IT Support	Seattle
Product Management	San Francisco

5.Write a query to select all the distinct companies (company_name) from the Works table.

Table Name: Works

employee_id company_name salary

1 TechCorp 75000.00

2 InnovateLtd 62000.50

employee_id company_name salary

3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

Expected Column

- company_name

Answer

```
SELECT DISTINCT company_name
FROM Works;
```

Your Output

company_name
TechCorp
InnovateLtd
HealthPlus
EduWorks
GreenTech

7.Fiction Collection Size

Write a query to find the total count of books whose genre is Fiction

Note: The output column name should be fiction_count.

Table Name: Books

id	title	author	genre	price	published_year
1	The Great Gatsby	F. Scott Fitzgerald	Fiction	10.99	1925
2	1984	George Orwell	Dystopian	15.99	1949
3	To Kill a Mockingbird	Harper Lee	Fiction	12.99	1960
4	The Catcher in the Rye	J.D. Salinger	Fiction	14.99	1951
5	Brave New World	Aldous Huxley	Dystopian	13.99	1932
6	The Hobbit	J.R.R. Tolkien	Fantasy	9.99	1937
7	Moby Dick	Herman Melville	Fiction	18.50	1851
8	War and Peace	Leo Tolstoy	Historical	20.00	1869
9	The Picture of Dorian Gray	Oscar Wilde	Fiction	11.50	1890
10	The Alchemist	Paulo Coelho	Fiction	16.00	1988
11	Fahrenheit 451	Ray Bradbury	Dystopian	12.50	1953
12	The Chronicles of Narnia	C.S. Lewis	Fantasy	14.00	1950
13	The Handmaid's Tale	Margaret Atwood	Dystopian	15.50	1985
14	A Tale of Two Cities	Charles Dickens	Historical	13.00	1859
15	Little Women	Louisa May Alcott	Fiction	9.50	1868

Expected Column

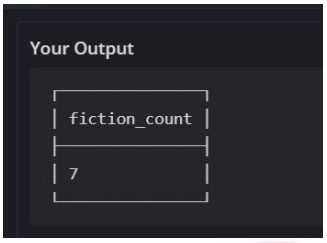
- fiction_count

Answer:

```
SELECT COUNT(*) AS fiction_count

FROM Books

WHERE genre = 'Fiction';
```



7.List of Movies with Ratings

Task

Write a query to select only the **movie names** where the **ratings are greater than 7 but less than 9**.

Table Name: Cinema

Movie_id	Movie_name	Description	Rating
1	War	great 3D	8.9
2	Science	fiction	8.5
3	Irish	boring	6.2
4	Ice Song	Fantasy	8.6
5	House Card	Interesting	9.1
6	The Escape	Thriller	7.8

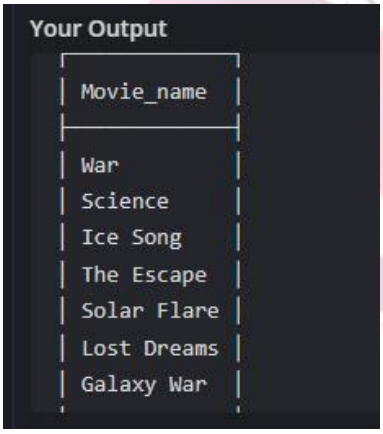
Movie_id	Movie_name	Description	Rating
7	Solar Flare	Sci-Fi	8.3
8	The Joker	Drama	9.0
9	Lost Dreams	Mystery	7.5
10	Galaxy War	Action	8.7

Expected Column

- Movie_name

Answer:

```
SELECT Movie_name
FROM Cinema
WHERE Rating > 7 AND Rating < 9;
```



8.Handling NULL Values
Task

Write a query to retrieve **book_id, title, author, and published_year** of the books which have **NULL rating**.

Table Name: Library

book_id	title	author	published_year	rating
1	The Great Gatsby	F. Scott Fitzgerald	1925	4.2
2	To Kill a Mockingbird	Harper Lee	1960	NULL
3	1984	George Orwell	1949	4.8
4	The Catcher in the Rye	J.D. Salinger	1951	NULL
5	Brave New World	Aldous Huxley	1932	4.3

Expected Columns

- book_id
- title
- author
- published_year

Answer:

```
SELECT book_id, title, author, published_year

FROM Library

WHERE rating IS NULL;
```

Your Output

book_id	title	author	published_year
2	To Kill a Mockingbird	Harper Lee	1960
4	The Catcher in the Rye	J.D. Salinger	1951

9.Salary of Employees

Task

Create a query to retrieve the **employee_name, company, and salary** for employees in the **Full-Time** category, ordered by salary in **descending** order.

Table Name: Employees

employee_id	employee_name	company	category	department	salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	IT	95000
employee_id	employee_name	company	category	department	salary
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

Expected Columns

- employee_name
- company
- Salary

Answer:

SELECT employee_name, company, salary

FROM Employees

WHERE category = 'Full-Time'

ORDER BY salary DESC;

Your Output

employee_name	company	salary
Bob Brown	FinServ	90000
David Green	TechCorp	85000
John Smith	TechCorp	80000
Grace Grey	TechCorp	70000
Frank Black	HealthPlus	60000

10.Department of Each Employee

Task

Write a query to group the employees by their **department** and display the total number of employees (as total_employees) in each department.

Table Name: Employees

employee_id employee_name company category department salary

1 John Smith TechCorp Full-Time Engineering 80000

employee_id employee_name company category department salary

2 Alice Johnson TechCorp Part-Time HR 30000

3 Bob Brown FinServ Full-Time Finance 90000

4 Carol White HealthPlus Contract IT 75000

5 David Green TechCorp Full-Time Engineering 85000

6	Emma Blue	FinServ Part-Time Finance	32000
7	Frank Black	HealthPlus Full-Time HR	60000
8	Grace Grey	TechCorp Full-Time Marketing	70000
9	Henry Red	FinServ Contract Sales	95000
10	Ivy Yellow	HealthPlus Part-Time Marketing	28000

Expected Columns

- department
- total_employees_

Answer:

Your Output

department	total_employees
Engineering	2
Finance	2
HR	2
IT	1
Marketing	2
Sales	1

11.Article Views – SQL Query

Write a query to retrieve the **author_id, author_name, and publication_name** for authors whose articles received **zero views**. The result should be sorted by **author_id in ascending order**.

Table: Views

article_id	author_id	author_name	viewer_id	view_count	publication_name
101	1	John Doe	1	3	The Daily Times
102	2	Jane Smith	2	1	Global News
103	3	Emily Clark	4	0	Tech Monthly

104	4	Mark Lewis 4	2	Health Digest
105	5	Sara White 3	0	The Business Journal
106	1	John Doe 2	0	The Daily Times
107	2	Jane Smith 2	2	Global News

Answer:

```
SELECT author_id, author_name, publication_name
FROM Views
WHERE view_count = 0
ORDER BY author_id ASC;
```

Your Output

author_id	author_name	publication_name
1	John Doe	The Daily Times
3	Emily Clark	Tech Monthly
5	Sara White	The Business Journal

12.Player Performance Insights

Write a query to find the names of the **top 3 distinct players by highest score** who have **won at least one match**, including their scores.

Table 1: Players

player_id player_name score rank

1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4

player_id player_name score rank

4 David 1600 1

5 Eve 1100 6

Table 2: Matches

match_id player1 player2 winner match_date

101 Alice Bob Bob 2024-01-15

102 Charlie David David 2024-01-16

103 Eve Bob Bob 2024-01-17

104 Alice David David 2024-01-18

105 Charlie Eve Charlie 2024-01-19

Answer:

```
SELECT DISTINCT p.player_name, p.score
FROM Players p
JOIN Matches m
ON p.player_name = m.winner
ORDER BY p.score DESC
LIMIT 3;
```

Your Output

player_name	score
David	1600
Bob	1500
Charlie	1300

13.Player Details

Write a query to retrieve the details of the **last five matches played**, including:

- Match ID
- Names of both players
- Winning player
- Match date
- Final score of the winner

Table 1: Players

player_id player_name score rank

1 Alice 1200 5

2 Bob 1500 2

3 Charlie 1300 4

4 David 1600 1

5 Eve 1100 6

6 Frank 1450 3

7 Hank 1250 7

8 Jack 1400 8

Table 2: Matches

match_id player1 player2 winner match_date

106 Frank Hank Frank 2024-01-29

101 Alice Bob Bob 2024-01-25

110 David Eve David 2024-01-24

108 Jack Alice Jack 2024-01-19

103 Eve Bob Bob 2024-01-17

Answer:

```
SELECT
   m.match_id,
   m.player_1,
   m.player_2,
   m.winner,
   m.match_date,
   p.score
FROM Matches AS m

JOIN Players AS p
ON m.winner = p.player_name
ORDER BY m.match_date DESC
LIMIT 5;
```

Your Output

match_id	player_1	player_2	winner	match_date	score
106	Frank	Hank	Frank	2024-01-29	1450
101	Alice	Bob	Bob	2024-01-25	1500
110	David	Eve	David	2024-01-24	1600
108	Jack	Alice	Jack	2024-01-19	1400
103	Eve	Bob	Bob	2024-01-17	1500

14.Find the Error in Query

Question:
Spot the error in the following query:

SELECT COUNT(), user_name FROM Users;

Options

A. No Error

B. Group By Clause is not specified

C. Column name not specified in count function

D. No such column user_name

✓ **Correct Answer: B. Group By Clause is not specified**

15.Find the Output of SQL Query

Question:
Determine the output of the following query:

SELECT MAX(rating) FROM Users;

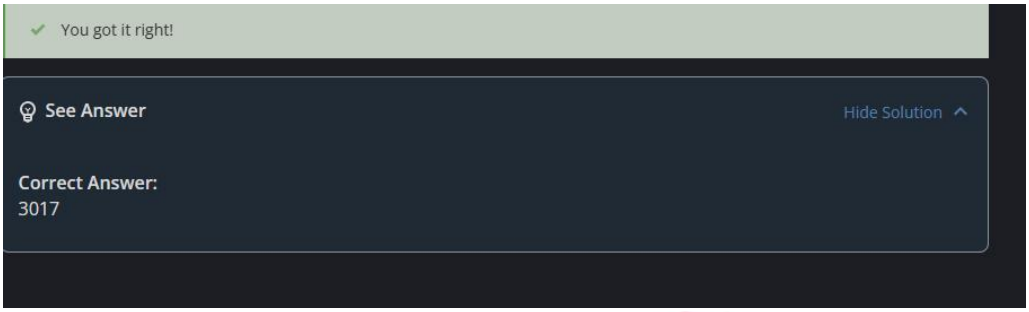
Options

A. 3017

B. 3012

C. 4120

D. 1450



16.Return Type of COUNT Function

Question:
What is the return type of the COUNT function?

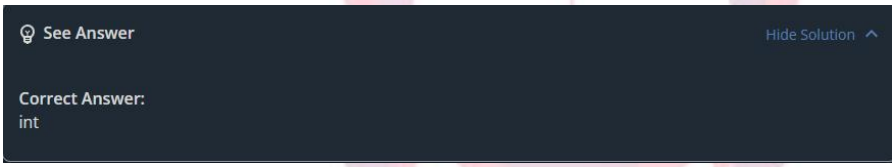
Options

A. int

B. boolean

C. Date

D. Float



17.Find Incorrect Queries

Question:
Which among the given queries is/are incorrect?

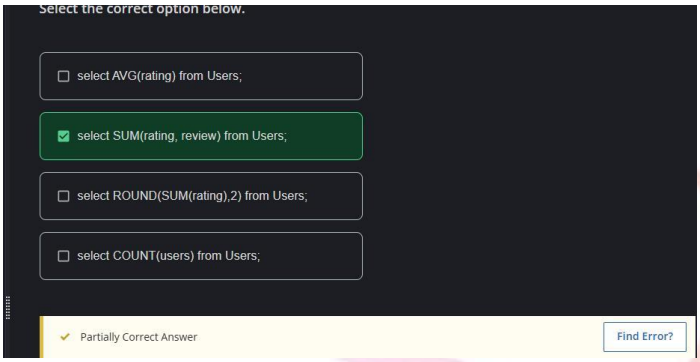
Options

A. SELECT AVG(rating) FROM Users;

B. SELECT SUM(rating, review) FROM Users;

C. SELECT ROUND(SUM(rating),2) FROM Users;

D. SELECT COUNT(users) FROM Users;



18.Nesting in Aggregate Functions

Question:

Till how many levels (maximum) can we nest Aggregate functions?

Options

A. 1

B. Nesting Not Possible

C. 2

D. Any number of levels



19.Output of a Query

Question:

Choose the output of the following query:

SELECT MIN(college) FROM Users;

Options

A. Error

B. Caltech

C. ITMO University

D. IIT Lucknow



20.Choose the Correct Statements

Question:
Which among the given options is/are correct about aggregate functions?

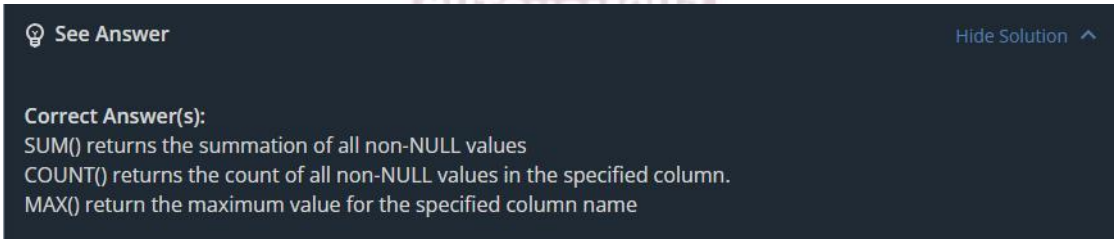
Options

A. SUM() returns the summation of all non-NULL values.

B. COUNT() returns the count of all non-NULL values in the specified column.

C. AVG() returns the average of all distinct non-NULL values.

D. MAX() returns the maximum value for the specified column name.



21.Count Star in NULL Columns

Question:
Mark the output of the following query for the given table:

SELECT COUNT(*) FROM Given_table;

Given Table

column1 column2

NULL NULL

NULL NULL

Options

A. 1
B. NULL
C. 2
D. Error



22.Group Users
Question:
Rearrange the following code to print user_name and the **category of users depending on ratings**.

Arrange the given blocks in this order:

1. select user_name, case

2. when rating between 1800 and 1999 then "4 Star"

3. when rating between 2000 and 2199 then "5 Star"

4. when rating between 2200 and 2499 then "6 Star"

5. else "None" end as category

6. from Users group by user_name;

See Answer

```
select user_name , case
when rating between 1800 and 1999 then "4 Star"
when rating between 2000 and 2199 then "5 Star"
when rating between 2200 and 2499 then "6 Star"
else "None" end as category
from Users group by user_name;
```

23.Question

Which JOIN type returns only the rows that have matching values in both tables?

Options

A. INNER JOIN
B. LEFT JOIN
C. RIGHT JOIN
D. FULL JOIN

See Answer

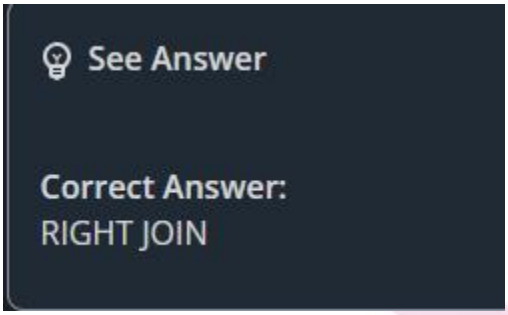
Correct Answer:
INNER JOIN

24.Question

Which JOIN type returns all the rows from the right table and the matching rows from the left table?

Options

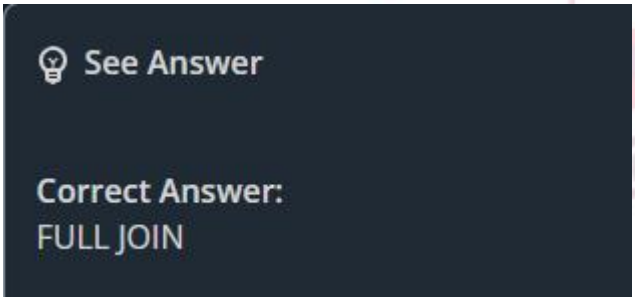
A. INNER JOIN
B. LEFT JOIN
C. RIGHT JOIN
D. FULL JOIN



25.Question
Which JOIN type returns all the rows from both tables, regardless of whether they have matching rows or not?

Options

A. INNER JOIN
B. LEFT JOIN
C. RIGHT JOIN
D. FULL JOIN

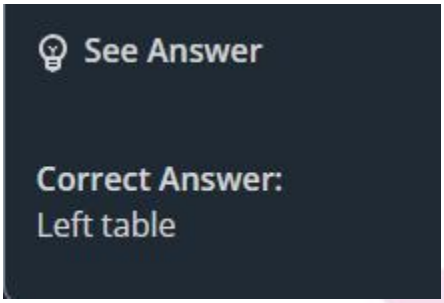


26.Question

In a LEFT JOIN operation, which table's rows are included in the result, even if there is no matching row in the other table?

Options

A. Left table
B. Right table
C. Both tables
D. None of the above

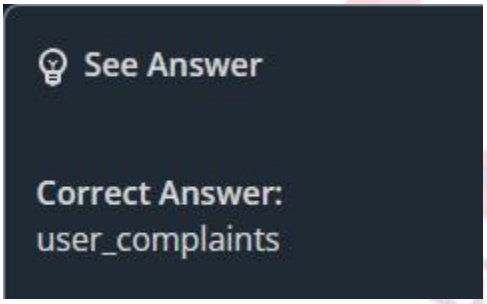


27.Question

Which table in the case study schema stores information about the users' complaints?

Options

A. user_information
B. complaint_status
C. user_complaints
D. complaint_category



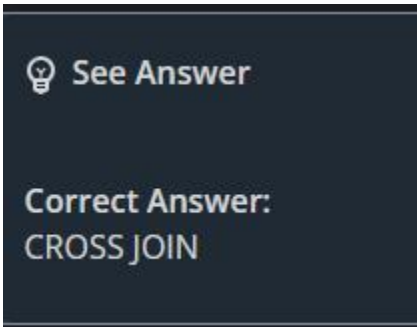
28.Question

Which JOIN type combines rows from two or more tables based on a condition that is not related to a specific column?

Options

A. INNER JOIN
B. LEFT JOIN

C. RIGHT JOIN
D. CROSS JOIN



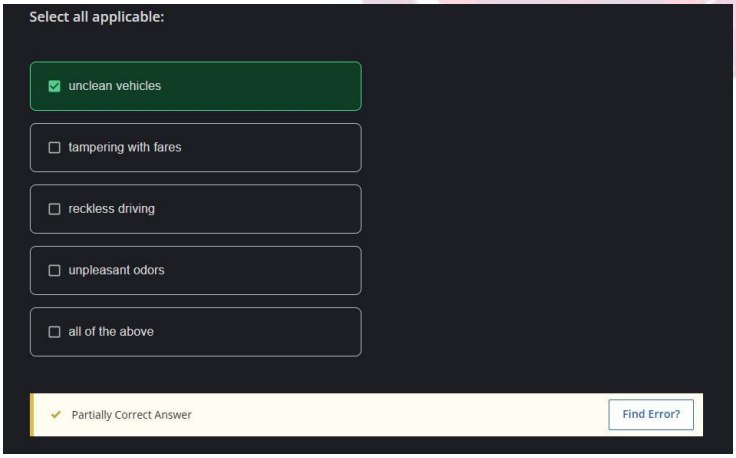
29.Find the Output for Query

Question:
What is the correct output for the following query?

SELECT reasonFROM category_descriptionWHERE category_id = 4;

Options

A. unclean vehicles
B. tampering with fares
C. reckless driving
D. unpleasant odors
E. all of the above

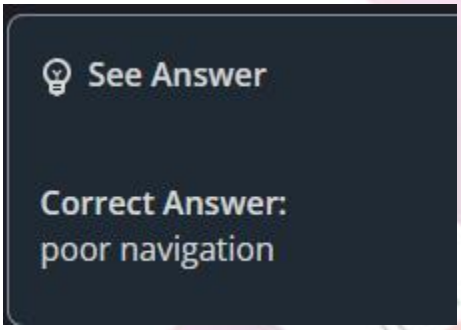


30.Find Complaint Category

Question:
"Taking incorrect routes" is associated with which of the following complaint categories?

Options

A. Overcharging
B. Driver behavior
C. Cleanliness
D. Poor navigation
E. Payment issues
F. None of the above



31.Rearrange the Query – 1

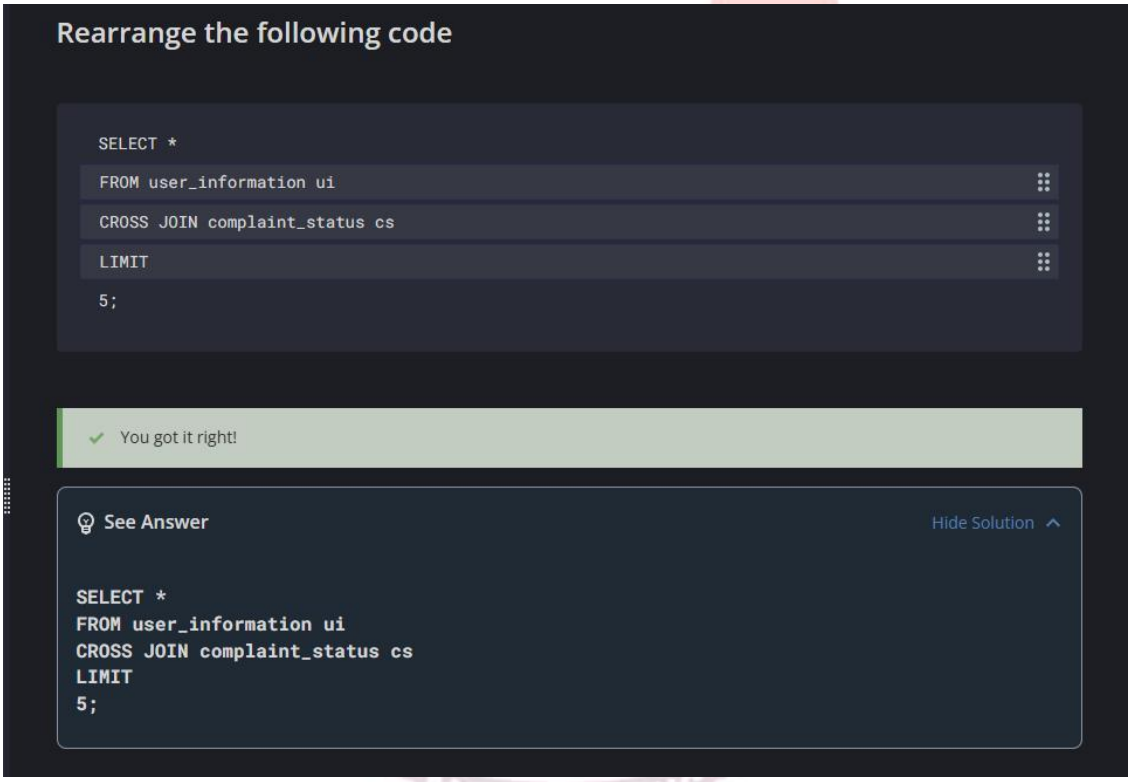
Question:
Apply JOINs and fetch the **category ID, category name, subcategory ID, and reason** for the category with **ID 1**.

```
SELECT
cc.category_id, cc.category_name, cd.subcategory_id, cd.reason
FROM
complaint_category cc
INNER JOIN
category_description cd
ON cc.category_id = cd.category_id
WHERE
cc.category_id = 1
;
```

```sql
SELECT cc.category_id, cc.category_name, cd.subcategory_id, cd.reason
FROM complaint_category cc
INNER JOIN category_description cd
    ON cc.category_id = cd.category_id
WHERE cc.category_id = 1;
```

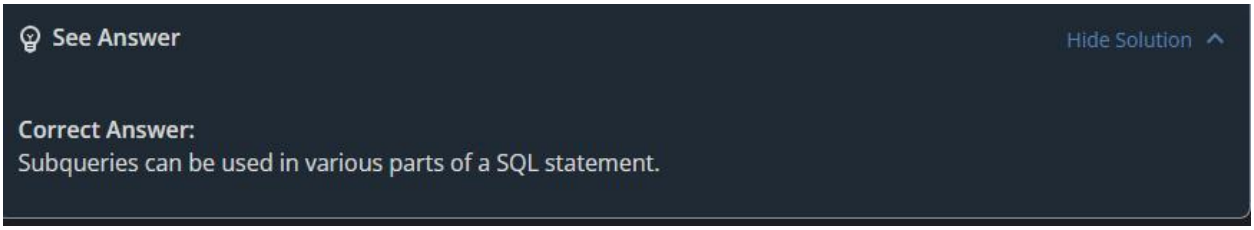
32.The question is:

"Combine the user_information **and** complaint_status **tables using a CROSS JOIN and fetch the first 5 rows. What do you see?"**



33.Question:
Which of the following statements about subqueries is true?

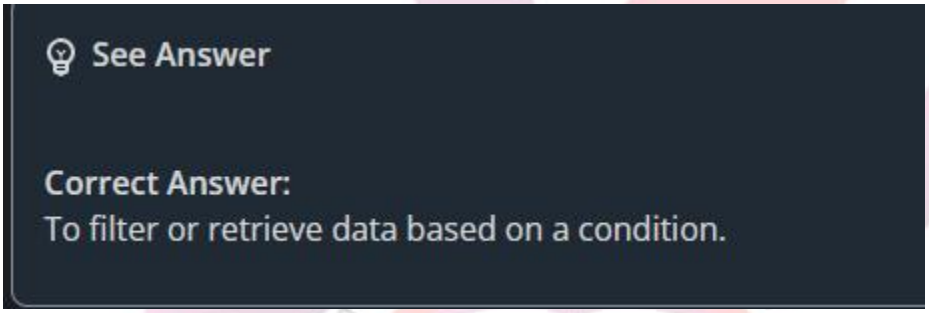
A. Subqueries can only be used in SELECT statements.
B. Subqueries always return a single row of data.
C. Subqueries must be enclosed in square brackets.
D. Subqueries can be used in various parts of a SQL statement.



34.Question:

What is the purpose of a subquery?

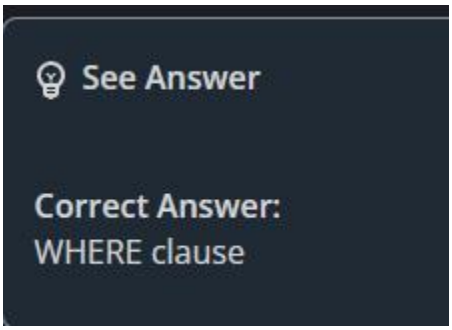
A. To improve query performance.
B. To filter or retrieve data based on a condition.
C. To combine data from multiple tables.
D. To simplify complex SQL statements.



35.Question:

In which clause is a subquery typically used?

A. FROM clause
B. SELECT clause
C. WHERE clause
D. ORDER BY clause



36.Question:

Which type of subquery is executed once for the entire outer query?

A. Scalar subquery
B. Correlated subquery
C. Nested subquery
D. Inline subquery

See Answer

Correct Answer:
Scalar subquery

37.Question:

Which keyword is used to compare a subquery result with a column value in the outer query?

A. BETWEEN
B. EXISTS
C. IN
D. LIKE

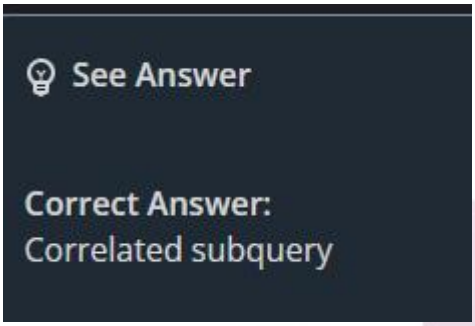
See Answer

Correct Answer:
IN

38.Question:

Which type of subquery can reference columns from the outer query?

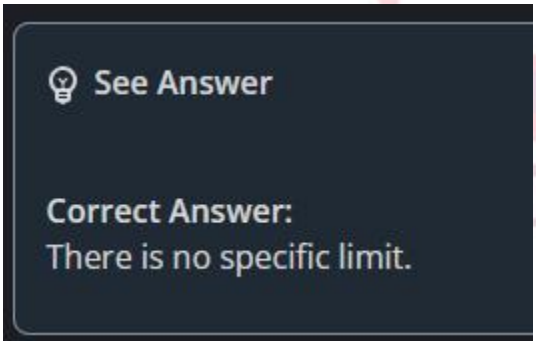
A. Scalar subquery
B. Correlated subquery
C. Nested subquery
D. Inline subquery



39.Question:

What is the maximum number of subqueries that can be nested in a single SQL statement?

A. 1
B. 5
C. 10
D. There is no specific limit.

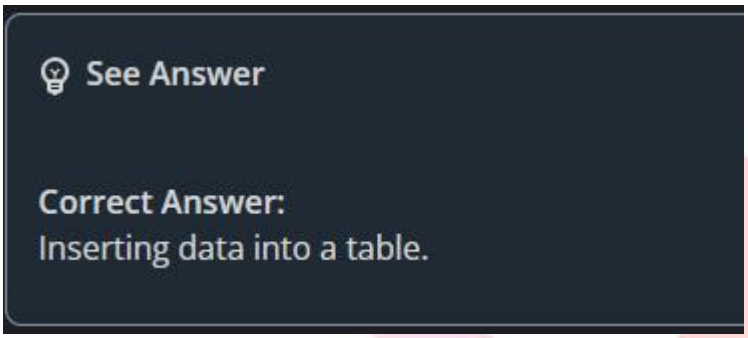


40.Question:

Which of the following is not a valid use case for subqueries?

A. Filtering rows based on aggregate functions.

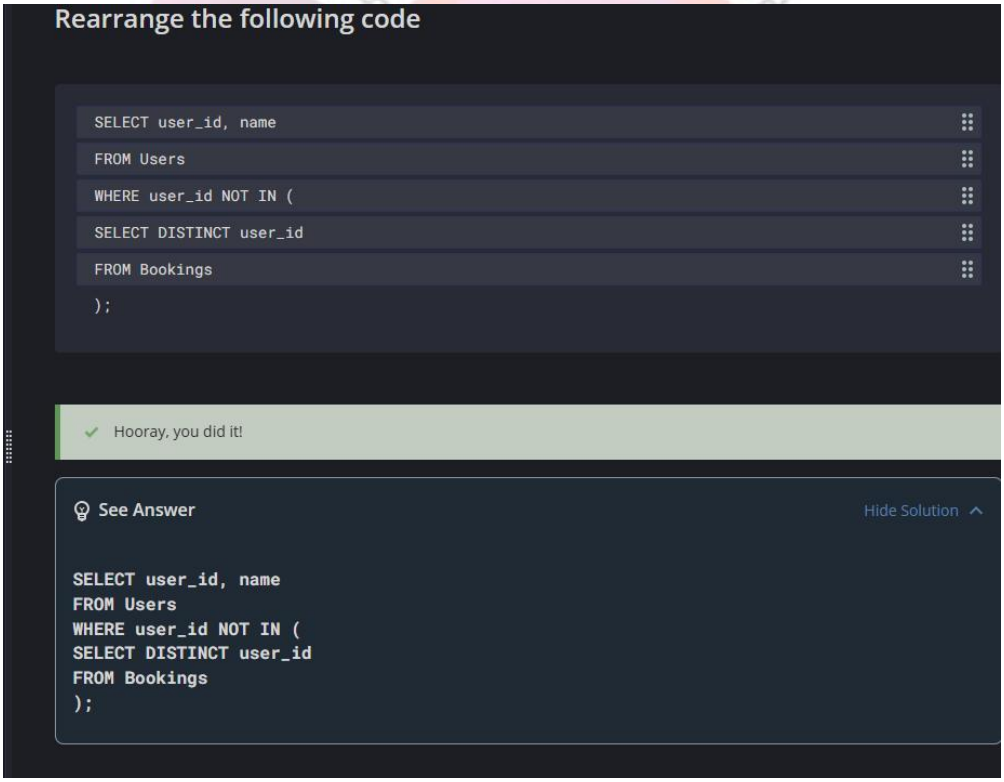
B. Retrieving data from multiple tables.
C. Inserting data into a table.
D. Updating data in a table.



41.Question:

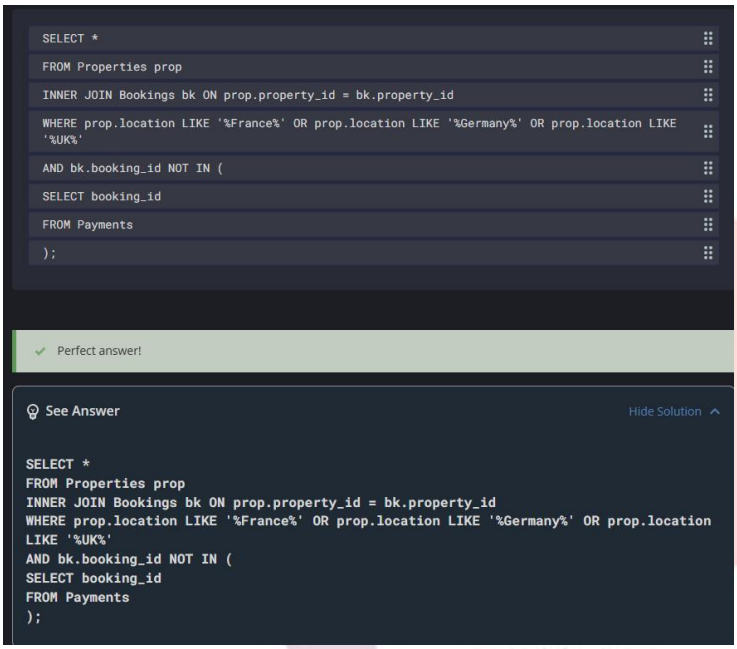
Retrieve the names and user IDs of users who have not made any bookings in the HappyHaven Home Rental App.

Think critically and rearrange the subqueries to unveil the SQL solution!



42.Question:

Retrieve the details of properties located in countries that have names containing "France," "Germany," or "UK" and have pending payments for bookings.



```
SELECT *
FROM Properties prop
INNER JOIN Bookings bk ON prop.property_id = bk.property_id
WHERE prop.location LIKE '%France%' OR prop.location LIKE '%Germany%' OR prop.location LIKE
'%UK%'
AND bk.booking_id NOT IN (
SELECT booking_id
FROM Payments
);
```

Perfect answer!

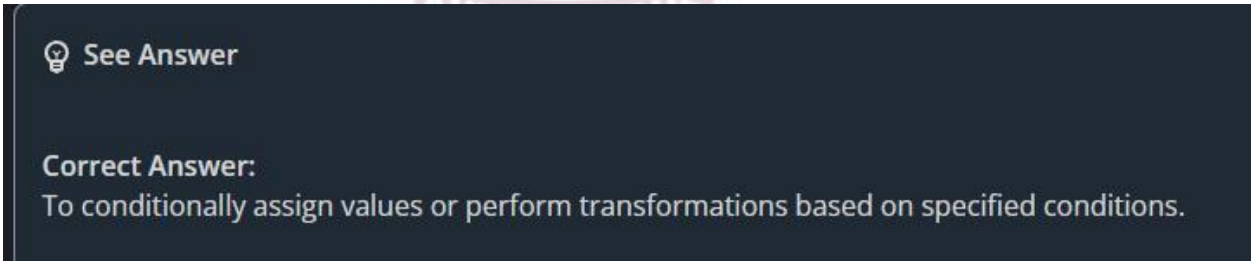
See Answer — Hide Solution

```
SELECT *
FROM Properties prop
INNER JOIN Bookings bk ON prop.property_id = bk.property_id
WHERE prop.location LIKE '%France%' OR prop.location LIKE '%Germany%' OR prop.location
LIKE '%UK%'
AND bk.booking_id NOT IN (
SELECT booking_id
FROM Payments
);
```

43.Question:

What is the purpose of a CASE WHEN expression in SQL?

A. To filter rows based on specific condition
B. To perform mathematical calculations.
C. To update data in a table.
D. To conditionally assign values or perform transformations based on specified conditions.



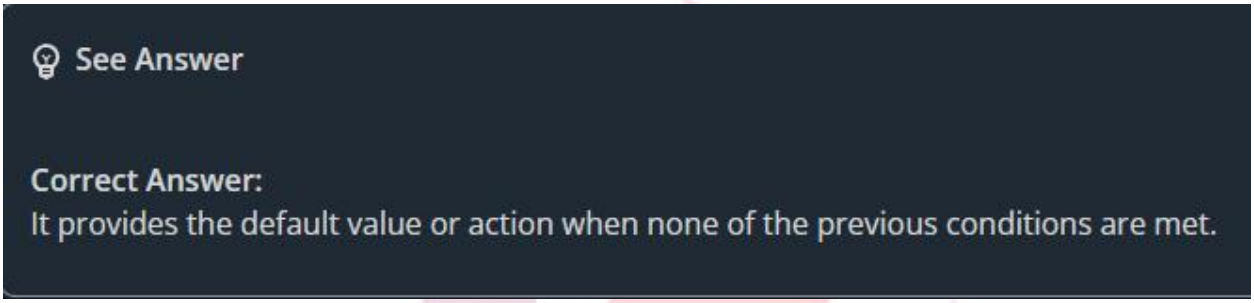
See Answer

Correct Answer:
To conditionally assign values or perform transformations based on specified conditions.

44.Question:

What is the purpose of the ELSE statement in a CASE WHEN expression?

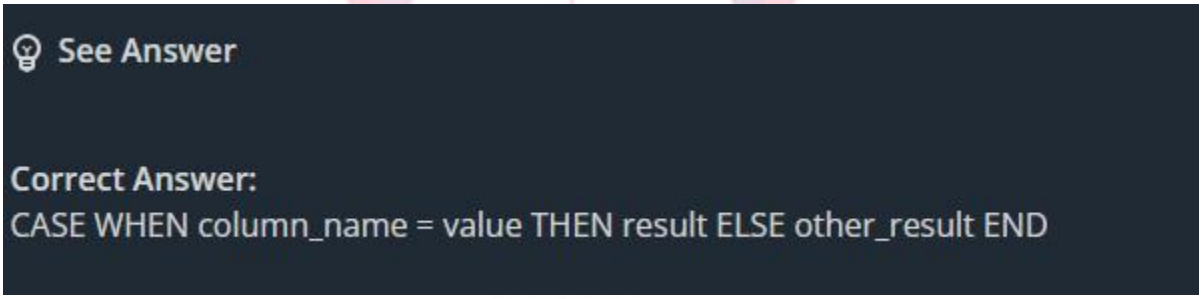
A. It specifies the condition for executing the CASE WHEN expression.
B. It defines the column aliases for the result set.
C. It provides the default value or action when none of the previous conditions are met.
D. It represents the end of the CASE WHEN expression



45.Question:

Which option represents the correct syntax for using CASE WHEN in SQL?

A. CASE column_name WHEN value THEN result END
B. CASE column_name WHEN value THEN result ELSE other_result END
C. CASE WHEN column_name = value THEN result ELSE other_result END
D. CASE WHEN column_name = value THEN result END



46.Question:

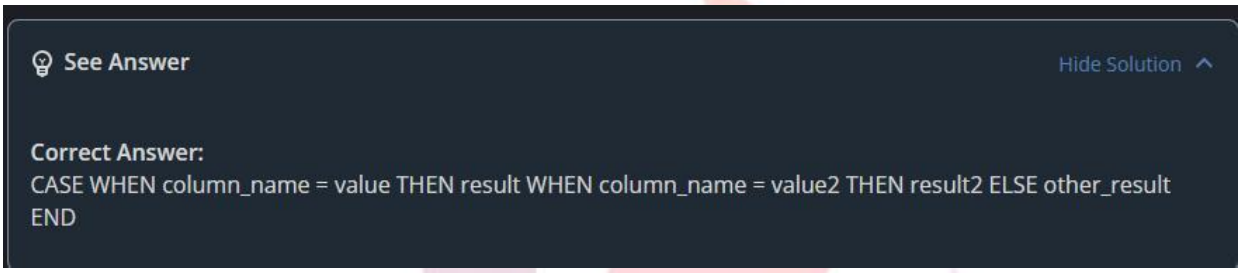
Which option represents the correct syntax for using CASE WHEN with multiple conditions in SQL?

A. CASE column_name WHEN value THEN result WHEN value2 THEN result2 END

B. CASE WHEN column_name = value THEN result WHEN column_name = value2 THEN result2 ELSE other_result END

C. CASE column_name WHEN value THEN result ELSE WHEN value2 THEN result2 END

D. CASE WHEN column_name = value THEN result WHEN column_name = value2 THEN result2 END

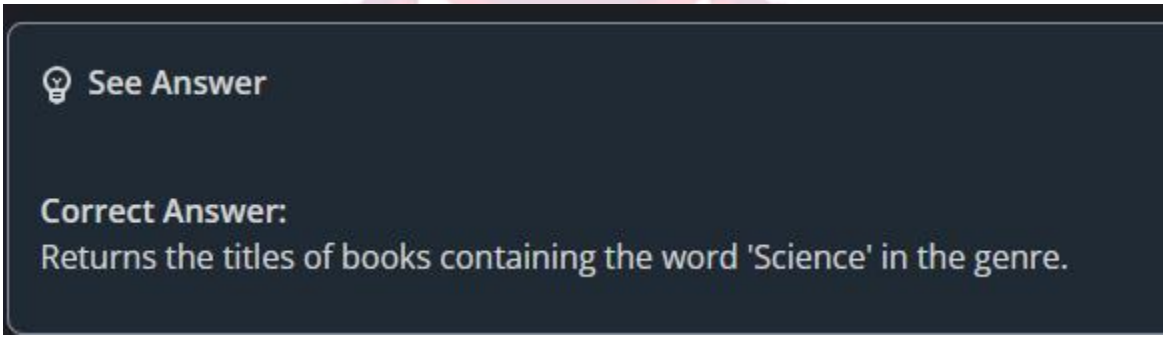


47.Question:

What is the output of the following expression?

SELECT titleFROM BooksWHERE genre LIKE '%Science%';

A. Returns the titles of books containing the word **'Science'** in the genre.
B. Returns the count of books containing the word **'Science'** in the genre.
C. Returns the titles of books not categorized as **'Science Books'**.
D. Returns the count of books not categorized as **'Science Books'**.



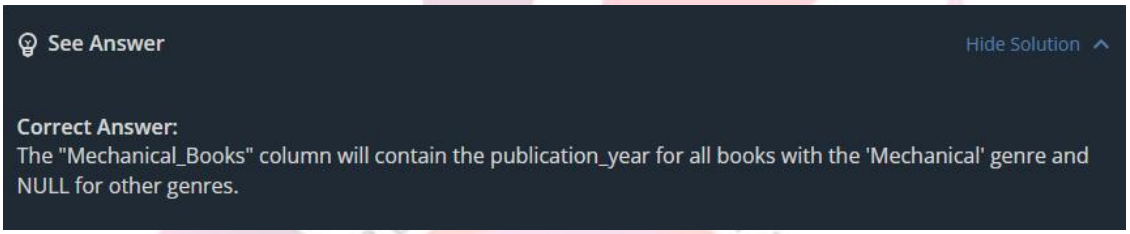
48.Question:
What is the output of the following CASE WHEN expression?

A. The **"Mechanical_Books"** column will contain the publication year for all books with the '**Mechanical**' genre and NULL for other genres.

B. The **"Mechanical_Books"** column will contain the title for all books with the '**Mechanical**' genre and NULL for other genres.

C. The **"Mechanical_Books"** column will contain the genre for all books with the '**Mechanical**' genre and NULL for other genres.

D. The **"Mechanical_Books"** column will contain the title for all books in the dataset, regardless of their genre.



49.Question:

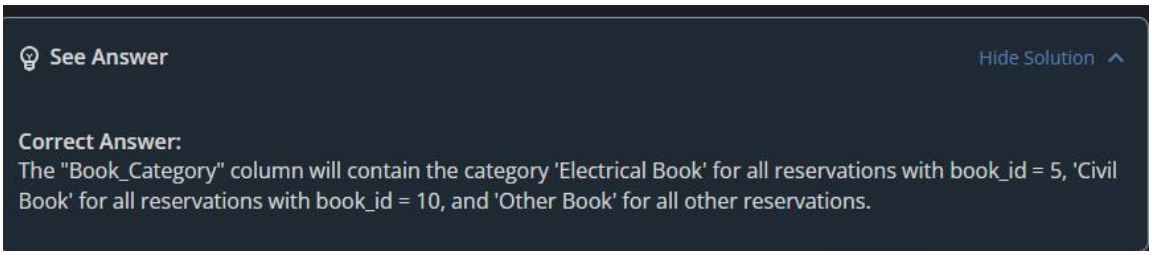
What is the output of the following CASE WHEN expression?

A. The **"Book_Category"** column will contain the reservation_id for all reservations with book_id = 5 and NULL for other reservations.

B. The **"Book_Category"** column will contain the book_id for all reservations with book_id = 10 and NULL for other reservations.

C. The **"Book_Category"** column will contain **'Electrical Book'** for reservations with book_id = 5, **'Civil Book'** for reservations with book_id = 10, and **'Other Book'** for all other reservations.

D. The **"Book_Category"** column will contain the reservation_id for all reservations, regardless of their book_id.



50.Question

The "Book_Category" column will contain the category 'Old Book' for all books published before 2015 and 'Recent Book' for all books published in or after 2015.

· The "Book_Category" column will contain the title for all books published before 2015 and NULL for all other books.

· The "Book_Category" column will contain the publication_year for all books published after 2015 and NULL for all other books.

· The "Book_Category" column will contain the title for all books in the dataset, regardless of their publication_year.

· The "Book_Category" column will contain the category 'Old Book' for all books published before 2015 and 'Recent Book' for all books published in or after 2015**.**



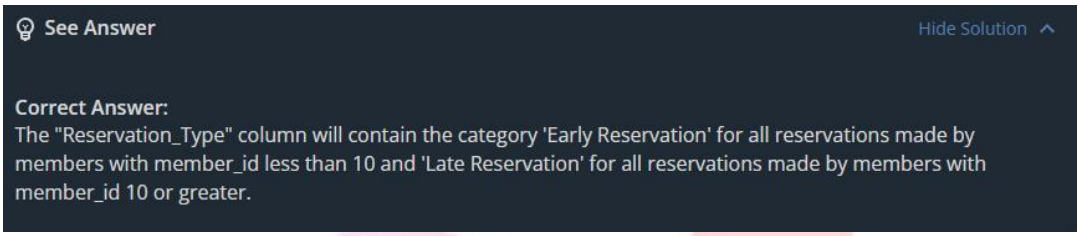
51.Question

The "Reservation_Type" column will contain the category 'Early Reservation' for all reservations made by members with member_id less than 10 and 'Late Reservation' for all reservations made by members with member_id 10 or greater.

A· The "Reservation_Type" column will contain the reservation_id for all reservations made by members with member_id less than 10 and NULL for all other reservations.

B. · The "Reservation_Type" column will contain the member_id for all reservations made by members with member_id 10 or greater and NULL for all other reservations.

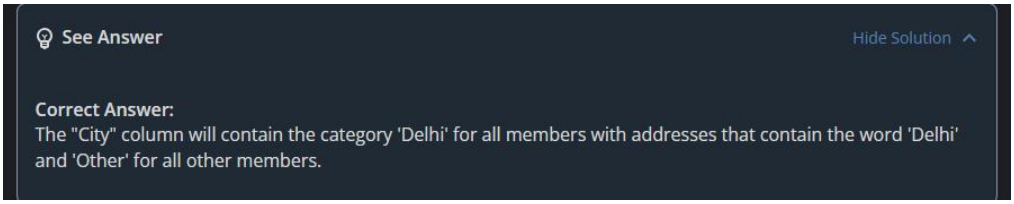
C. · The "Reservation_Type" column will contain the category 'Early Reservation' for all reservations made by members with member_id less than 10 and 'Late Reservation' for all reservations made by members with member_id 10 or greater.
D. · The "Reservation_Type" column will contain the reservation_id for all reservations in the dataset, regardless of their member_id.



52.Question:

What is the output of the following CASE WHEN expression?

- · The "City" column will contain the member_name for all members with addresses that contain the word 'Delhi' and NULL for all other members.
- · The "City" column will contain the category 'Delhi' for all members with addresses that contain the word 'Delhi' and 'Other' for all other members. ✓
- · The "City" column will contain the address for all members with addresses that do not contain the word 'Delhi' and NULL for all other members.
- · The "City" column will contain the member_name for all members in the dataset, regardless of their address.



RETAIL INVENTORY AND STOCK REPLENISHMENT

DBMS Mini Project (SQL and PL/SQL)

Table: RETAIL_INVENTORY | Database: Oracle

1. Problem Statement

Develop an inventory management system to monitor stock levels, calculate inventory value, and identify products requiring replenishment. The system is built on a single table, **RETAIL_INVENTORY**, with SQL queries for analysis and PL/SQL subprograms for processing sales, valuing stock and generating replenishment reports.

2. Dataset

Records: 25 products across 5 categories (Groceries, Electronics, Clothing, Home & Kitchen, Stationery). Prices are in INR.

Source datasets: Kaggle - *Retail Store Inventory Forecasting Dataset* (Product ID, Category, Inventory Level, Units Sold, Price) and the UCI Machine Learning Repository - *Online Retail* dataset (StockCode, Description, Quantity, UnitPrice). The field structure follows these sources; the 25 records are a curated retail sample.

Field mapping and added fields:

Field in RETAIL_INVENTORY	Origin
product_id	Kaggle Product ID / UCI StockCode (re-keyed P101 to P505)
product_name	UCI Description
category	Kaggle Category (mapped to 5 categories)
stock_qty	Kaggle Inventory Level
unit_price	Kaggle Price / UCI UnitPrice
units_sold	Kaggle Units Sold (aggregated)
reorder_level	ADDED FIELD: minimum stock before reordering
supplier_name	ADDED FIELD
last_restock_date	ADDED FIELD

Dataset (25 records)

ID	Product	Category	Stock	Reorder	Price	Sold	Supplier	Last Restock
P101	Basmati Rice 5kg	Groceries	120	50	520.00	340	Agro Fresh Traders	2026-09-10
P102	Sunflower Oil 1L	Groceries	35	60	165.00	410	Agro Fresh Traders	2026-09-02
P103	Whole Wheat Flour 5kg	Groceries	0	40	245.00	295	Golden Mills	2026-08-28
P104	Tea Powder 500g	Groceries	80	40	210.00	260	Nilgiri Suppliers	2026-09-15
P105	Sugar 1kg	Groceries	55	55	48.00	520	Golden Mills	2026-09-12
P201	Bluetooth Earphones	Electronics	25	20	1,499.00	130	TechNova Distributors	2026-09-05
P202	Smartphone Charger 20W	Electronics	18	25	799.00	210	TechNova Distributors	2026-08-30
P203	Power Bank 10000mAh	Electronics	0	15	1,299.00	95	VoltEdge Traders	2026-08-20
P204	USB Type-C Cable	Electronics	140	60	199.00	480	TechNova Distributors	2026-09-18
P205	LED Bulb 9W	Electronics	200	80	99.00	610	BrightLine Electricals	2026-09-20
P301	Men's Cotton T-Shirt	Clothing	60	30	499.00	185	StyleHub Garments	2026-09-08
P302	Women's Kurti	Clothing	22	30	899.00	150	StyleHub Garments	2026-09-01
P303	Denim Jeans	Clothing	45	25	1,599.00	110	DenimWorks	2026-09-14
P304	Cotton Socks (3 Pack)	Clothing	0	40	249.00	230	StyleHub Garments	2026-08-25
P305	School Backpack	Clothing	30	15	1,099.00	70	BagCraft	2026-09-11
P401	Non-stick Frying Pan	Home & Kitchen	28	20	749.00	90	HomeEase	2026-09-09
P402	Steel Water Bottle 1L	Home & Kitchen	75	35	399.00	300	HomeEase	2026-09-16
P403	Mixer Grinder 500W	Home & Kitchen	12	15	3,299.00	45	KitchenPro Appliances	2026-09-03
P404	Dinner Set 24pc	Home & Kitchen	16	10	2,199.00	38	HomeEase	2026-09-06
P405	Pressure Cooker 5L	Home & Kitchen	20	20	1,899.00	62	KitchenPro Appliances	2026-09-13
P501	A4 Notebook 200pg	Stationery	300	100	65.00	750	PaperLine	2026-09-17
P502	Ball Pen Box (10)	Stationery	90	50	120.00	430	PaperLine	2026-09-19
P503	Geometry Box	Stationery	14	20	180.00	165	EduMart	2026-09-04
P504	Marker Set (4)	Stationery	0	25	140.00	120	EduMart	2026-08-27
P505	Stapler Medium	Stationery	48	20	110.00	85	PaperLine	2026-09-07

3. Table Creation

```
CREATE TABLE retail_inventory (
    product_id        VARCHAR2(10)   CONSTRAINT pk_retail_inventory PRIMARY KEY,
    product_name      VARCHAR2(60)   NOT NULL,
    category          VARCHAR2(30)   NOT NULL
                      CONSTRAINT ck_ri_category CHECK (category IN
                      ('Groceries','Electronics','Clothing','Home & Kitchen','Stationery')),
    stock_qty         NUMBER(6)      DEFAULT 0 NOT NULL CONSTRAINT ck_ri_stock CHECK (stock_qty >= 0),
    reorder_level     NUMBER(6)      NOT NULL CONSTRAINT ck_ri_reorder CHECK (reorder_level >= 0),
    unit_price        NUMBER(10,2)   NOT NULL CONSTRAINT ck_ri_price CHECK (unit_price > 0),
    units_sold        NUMBER(8)      DEFAULT 0 NOT NULL CONSTRAINT ck_ri_sold CHECK (units_sold >= 0),
    supplier_name     VARCHAR2(50),
    last_restock_date DATE
);
```

Constraints: primary key on product_id; NOT NULL on key business columns; CHECK constraints prevent negative stock, negative reorder level, non-positive price, negative units sold and invalid categories.

4. Data Insertion

```
INSERT INTO retail_inventory VALUES ('P101','Basmati Rice 5kg','Groceries',120,50,520.00,340,'Agro Fresh Traders',DATE
     '2026-09-10');
INSERT INTO retail_inventory VALUES ('P102','Sunflower Oil 1L','Groceries',35,60,165.00,410,'Agro Fresh Traders',DATE
    '2026-09-02');
INSERT INTO retail_inventory VALUES ('P103','Whole Wheat Flour 5kg','Groceries',0,40,245.00,295,'Golden Mills',DATE
    '2026-08-28');
INSERT INTO retail_inventory VALUES ('P104','Tea Powder 500g','Groceries',80,40,210.00,260,'Nilgiri Suppliers',DATE
    '2026-09-15');
INSERT INTO retail_inventory VALUES ('P105','Sugar 1kg','Groceries',55,55,48.00,520,'Golden Mills',DATE '2026-09-12');
INSERT INTO retail_inventory VALUES ('P201','Bluetooth Earphones','Electronics',25,20,1499.00,130,'TechNova
    Distributors',DATE '2026-09-05');
INSERT INTO retail_inventory VALUES ('P202','Smartphone Charger 20W','Electronics',18,25,799.00,210,'TechNova
    Distributors',DATE '2026-08-30');
INSERT INTO retail_inventory VALUES ('P203','Power Bank 10000mAh','Electronics',0,15,1299.00,95,'VoltEdge
    Traders',DATE '2026-08-20');
INSERT INTO retail_inventory VALUES ('P204','USB Type-C Cable','Electronics',140,60,199.00,480,'TechNova
    Distributors',DATE '2026-09-18');
INSERT INTO retail_inventory VALUES ('P205','LED Bulb 9W','Electronics',200,80,99.00,610,'BrightLine Electricals',DATE
     '2026-09-20');
INSERT INTO retail_inventory VALUES ('P301','Men''s Cotton T-Shirt','Clothing',60,30,499.00,185,'StyleHub
    Garments',DATE '2026-09-08');
INSERT INTO retail_inventory VALUES ('P302','Women''s Kurti','Clothing',22,30,899.00,150,'StyleHub Garments',DATE
    '2026-09-01');
INSERT INTO retail_inventory VALUES ('P303','Denim Jeans','Clothing',45,25,1599.00,110,'DenimWorks',DATE
    '2026-09-14');
INSERT INTO retail_inventory VALUES ('P304','Cotton Socks (3 Pack)','Clothing',0,40,249.00,230,'StyleHub
    Garments',DATE '2026-08-25');
INSERT INTO retail_inventory VALUES ('P305','School Backpack','Clothing',30,15,1099.00,70,'BagCraft',DATE
    '2026-09-11');
INSERT INTO retail_inventory VALUES ('P401','Non-stick Frying Pan','Home & Kitchen',28,20,749.00,90,'HomeEase',DATE
    '2026-09-09');
INSERT INTO retail_inventory VALUES ('P402','Steel Water Bottle 1L','Home & Kitchen',75,35,399.00,300,'HomeEase',DATE
    '2026-09-16');
INSERT INTO retail_inventory VALUES ('P403','Mixer Grinder 500W','Home & Kitchen',12,15,3299.00,45,'KitchenPro
    Appliances',DATE '2026-09-03');
INSERT INTO retail_inventory VALUES ('P404','Dinner Set 24pc','Home & Kitchen',16,10,2199.00,38,'HomeEase',DATE
    '2026-09-06');
INSERT INTO retail_inventory VALUES ('P405','Pressure Cooker 5L','Home & Kitchen',20,20,1899.00,62,'KitchenPro
    Appliances',DATE '2026-09-13');
INSERT INTO retail_inventory VALUES ('P501','A4 Notebook 200pg','Stationery',300,100,65.00,750,'PaperLine',DATE
    '2026-09-17');
INSERT INTO retail_inventory VALUES ('P502','Ball Pen Box (10)','Stationery',90,50,120.00,430,'PaperLine',DATE
    '2026-09-19');
INSERT INTO retail_inventory VALUES ('P503','Geometry Box','Stationery',14,20,180.00,165,'EduMart',DATE '2026-09-04');
INSERT INTO retail_inventory VALUES ('P504','Marker Set (4)','Stationery',0,25,140.00,120,'EduMart',DATE
    '2026-08-27');
INSERT INTO retail_inventory VALUES ('P505','Stapler Medium','Stationery',48,20,110.00,85,'PaperLine',DATE
    '2026-09-07');
COMMIT;
```

5. SQL Queries with Execution Results

Q1. Display all products with category, stock quantity, reorder level and unit price

```
SELECT product_id, product_name, category, stock_qty, reorder_level, unit_price
FROM   retail_inventory
ORDER  BY category, product_id;
```

Result (25 rows):

PRODUCT_ID	PRODUCT_NAME	CATEGORY	STOCK_QTY	REORDER_LEVEL	UNIT_PRICE
P301	Men's Cotton T-Shirt	Clothing	60	30	499.00
P302	Women's Kurti	Clothing	22	30	899.00
P303	Denim Jeans	Clothing	45	25	1,599.00
P304	Cotton Socks (3 Pack)	Clothing	0	40	249.00
P305	School Backpack	Clothing	30	15	1,099.00
P201	Bluetooth Earphones	Electronics	25	20	1,499.00
P202	Smartphone Charger 20W	Electronics	18	25	799.00
P203	Power Bank 10000mAh	Electronics	0	15	1,299.00
P204	USB Type-C Cable	Electronics	140	60	199.00
P205	LED Bulb 9W	Electronics	200	80	99.00
P101	Basmati Rice 5kg	Groceries	120	50	520.00
P102	Sunflower Oil 1L	Groceries	35	60	165.00
P103	Whole Wheat Flour 5kg	Groceries	0	40	245.00
P104	Tea Powder 500g	Groceries	80	40	210.00
P105	Sugar 1kg	Groceries	55	55	48.00
P401	Non-stick Frying Pan	Home & Kitchen	28	20	749.00
P402	Steel Water Bottle 1L	Home & Kitchen	75	35	399.00
P403	Mixer Grinder 500W	Home & Kitchen	12	15	3,299.00
P404	Dinner Set 24pc	Home & Kitchen	16	10	2,199.00
P405	Pressure Cooker 5L	Home & Kitchen	20	20	1,899.00
P501	A4 Notebook 200pg	Stationery	300	100	65.00
P502	Ball Pen Box (10)	Stationery	90	50	120.00
P503	Geometry Box	Stationery	14	20	180.00
P504	Marker Set (4)	Stationery	0	25	140.00
P505	Stapler Medium	Stationery	48	20	110.00

Q2. Products whose available stock is below or equal to the reorder level

```
SELECT product_id, product_name, category, stock_qty, reorder_level,
       (reorder_level - stock_qty) AS shortfall
FROM   retail_inventory
WHERE  stock_qty <= reorder_level
ORDER  BY shortfall DESC, product_id;
```

Result (11 rows):

PRODUCT_ID	PRODUCT_NAME	CATEGORY	STOCK_QTY	REORDER_LEVEL	SHORTFALL
P103	Whole Wheat Flour 5kg	Groceries	0	40	40
P304	Cotton Socks (3 Pack)	Clothing	0	40	40
P102	Sunflower Oil 1L	Groceries	35	60	25
P504	Marker Set (4)	Stationery	0	25	25
P203	Power Bank 10000mAh	Electronics	0	15	15
P302	Women's Kurti	Clothing	22	30	8
P202	Smartphone Charger 20W	Electronics	18	25	7
P503	Geometry Box	Stationery	14	20	6
P403	Mixer Grinder 500W	Home & Kitchen	12	15	3
P105	Sugar 1kg	Groceries	55	55	0
P405	Pressure Cooker 5L	Home & Kitchen	20	20	0

Q3. Total inventory value for each product category

```
SELECT category, COUNT(*) AS products, SUM(stock_qty) AS total_units,
       SUM(stock_qty * unit_price) AS inventory_value
FROM   retail_inventory
GROUP  BY category
ORDER  BY inventory_value DESC;
```

Result (5 rows):

CATEGORY	PRODUCTS	TOTAL_UNITS	INVENTORY_VALUE
Home & Kitchen	5	151	163,649.00
Clothing	5	157	154,643.00
Electronics	5	383	99,517.00
Groceries	5	290	87,615.00
Stationery	5	452	38,100.00

Q4. Five best-selling products based on units sold

```
SELECT product_id, product_name, category, units_sold
FROM   retail_inventory
ORDER  BY units_sold DESC
FETCH FIRST 5 ROWS ONLY;
```

Result (5 rows):

PRODUCT_ID	PRODUCT_NAME	CATEGORY	UNITS_SOLD
P501	A4 Notebook 200pg	Stationery	750
P205	LED Bulb 9W	Electronics	610
P105	Sugar 1kg	Groceries	520
P204	USB Type-C Cable	Electronics	480
P502	Ball Pen Box (10)	Stationery	430

Q5. Out-of-stock products with estimated sales revenue

```
SELECT product_id, product_name, category, stock_qty, units_sold, unit_price,
       (units_sold * unit_price) AS est_sales_revenue
FROM   retail_inventory
WHERE  stock_qty = 0
ORDER  BY est_sales_revenue DESC;
```

Result (4 rows):

PRODUCT_ID	PRODUCT_NAME	CATEGORY	STOCK_QTY	UNITS_SOLD	UNIT_PRICE	EST_SALES_REVENUE
P203	Power Bank 10000mAh	Electronics	0	95	1,299.00	123,405.00
P103	Whole Wheat Flour 5kg	Groceries	0	295	245.00	72,275.00
P304	Cotton Socks (3 Pack)	Clothing	0	230	249.00	57,270.00
P504	Marker Set (4)	Stationery	0	120	140.00	16,800.00

6. PL/SQL Subprograms

```
-- 1. Procedure: process a sale (validates input, checks availability, updates stock)
CREATE OR REPLACE PROCEDURE sp_process_sale (
    p_product_id IN retail_inventory.product_id%TYPE,
    p_qty        IN NUMBER
) AS
    v_name      retail_inventory.product_name%TYPE;
    v_stock     retail_inventory.stock_qty%TYPE;
    v_reorder   retail_inventory.reorder_level%TYPE;
    v_new_stock retail_inventory.stock_qty%TYPE;
    e_invalid_qty        EXCEPTION;
    e_insufficient_stock EXCEPTION;
BEGIN
    IF p_qty IS NULL OR p_qty <= 0 OR p_qty <> TRUNC(p_qty) THEN
        RAISE e_invalid_qty;
    END IF;

    SELECT product_name, stock_qty, reorder_level
    INTO   v_name, v_stock, v_reorder
    FROM   retail_inventory
    WHERE  product_id = p_product_id
    FOR UPDATE;

    IF v_stock < p_qty THEN
        RAISE e_insufficient_stock;
    END IF;

    UPDATE retail_inventory
    SET    stock_qty  = stock_qty - p_qty,
           units_sold = units_sold + p_qty
    WHERE  product_id = p_product_id;

    COMMIT;
    v_new_stock := v_stock - p_qty;
    DBMS_OUTPUT.PUT_LINE('SALE OK: ' || v_name || ' | sold ' || p_qty ||
                         ' | stock ' || v_stock || ' -> ' || v_new_stock);
    IF v_new_stock <= v_reorder THEN
        DBMS_OUTPUT.PUT_LINE('ALERT: ' || v_name || ' is at/below reorder level (' || v_reorder || ').');
    END IF;
EXCEPTION
    WHEN e_invalid_qty THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: Invalid quantity. Enter a positive whole number.');
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: Product ' || p_product_id || ' not found.');
    WHEN e_insufficient_stock THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('ERROR: Insufficient stock for ' || v_name ||
                             '. Available: ' || v_stock || ', requested: ' || p_qty);
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('UNEXPECTED ERROR: ' || SQLERRM);
END sp_process_sale;
/

-- 2. Function: inventory value of a product (stock_qty * unit_price)
CREATE OR REPLACE FUNCTION fn_inventory_value (
    p_product_id IN retail_inventory.product_id%TYPE
) RETURN NUMBER AS
    v_value NUMBER(14,2);
BEGIN
    SELECT stock_qty * unit_price
    INTO   v_value
    FROM   retail_inventory
    WHERE  product_id = p_product_id;
    RETURN v_value;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20001, 'Product ' || p_product_id || ' not found.');
END fn_inventory_value;
/

-- 3. Procedure: stock replenishment report (suggested order = 2 x reorder level - stock)
CREATE OR REPLACE PROCEDURE sp_replenishment_report AS
    CURSOR c_low IS
        SELECT product_id, product_name, stock_qty, reorder_level, supplier_name,
               (2 * reorder_level - stock_qty) AS suggested_qty
        FROM   retail_inventory
        WHERE  stock_qty <= reorder_level
        ORDER  BY stock_qty - reorder_level, product_id;
    v_count NUMBER := 0;
BEGIN
    DBMS_OUTPUT.PUT_LINE('===== STOCK REPLENISHMENT REPORT =====');
    FOR r IN c_low LOOP
        v_count := v_count + 1;
```

```
        DBMS_OUTPUT.PUT_LINE(RPAD(r.product_id,6) || RPAD(r.product_name,26) ||
            'Stock:' || LPAD(r.stock_qty,4) || '  Reorder:' || LPAD(r.reorder_level,4) ||
            '  Order Qty:' || LPAD(r.suggested_qty,4) || '  ' || r.supplier_name ||
            CASE WHEN r.stock_qty = 0 THEN '  [OUT OF STOCK]' END);
    END LOOP;
    IF v_count = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No products require replenishment.');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Total products to reorder: ' || v_count);
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('UNEXPECTED ERROR: ' || SQLERRM);
END sp_replenishment_report;
/
```

Execution examples

```
SET SERVEROUTPUT ON
EXEC sp_process_sale('P101', 10);
SELECT fn_inventory_value('P201') FROM dual;
EXEC sp_replenishment_report;
```

7. Exception Handling and Test Cases

Exceptions handled: invalid quantity (user-defined e_invalid_qty), insufficient stock (user-defined e_insufficient_stock), missing product (NO_DATA_FOUND) and any other error (WHEN OTHERS with rollback). The function raises ORA-20001 for a missing product.

Procedure tests: sp_process_sale

Run in this order on a fresh load of the data.

T1 Valid sale (P101, 10)

```
SQL> EXEC sp_process_sale('P101', 10);
SALE OK: Basmati Rice 5kg | sold 10 | stock 110 -> 100
```

T2 Sale reaching reorder level (P405, 3)

```
SQL> EXEC sp_process_sale('P405', 3);
SALE OK: Pressure Cooker 5L | sold 3 | stock 17 -> 14
ALERT: Pressure Cooker 5L is at/below reorder level (20).
```

T3 Insufficient stock (P102, 50)

```
SQL> EXEC sp_process_sale('P102', 50);
ERROR: Insufficient stock for Sunflower Oil 1L. Available: 35, requested: 50
```

T4 Out-of-stock product (P103, 1)

```
SQL> EXEC sp_process_sale('P103', 1);
ERROR: Insufficient stock for Whole Wheat Flour 5kg. Available: 0, requested: 1
```

T5 Missing product (P999, 5)

```
SQL> EXEC sp_process_sale('P999', 5);
ERROR: Product P999 not found.
```

T6a Negative quantity (P101, -5)

```
SQL> EXEC sp_process_sale('P101', -5);
ERROR: Invalid quantity. Enter a positive whole number.
```

T6b Zero quantity (P101, 0)

```
SQL> EXEC sp_process_sale('P101', 0);
ERROR: Invalid quantity. Enter a positive whole number.
```

T6c NULL quantity (P101, NULL)

```
SQL> EXEC sp_process_sale('P101', NULL);
ERROR: Invalid quantity. Enter a positive whole number.
```

Function tests: fn_inventory_value

Call	Expected result
fn_inventory_value('P201')	37,475.00 (25 x 1499.00)
fn_inventory_value('P203')	0.00 (out of stock)
fn_inventory_value('P999')	ORA-20001: Product P999 not found.

Replenishment report test

EXEC sp_replenishment_report; (run after T1 and T2)

```
===== STOCK REPLENISHMENT REPORT =====
P103  Whole Wheat Flour 5kg     Stock:   0  Reorder:  40  Order Qty:  80  Golden Mills  [OUT OF STOCK]
P304  Cotton Socks (3 Pack)     Stock:   0  Reorder:  40  Order Qty:  80  StyleHub Garments  [OUT OF STOCK]
P102  Sunflower Oil 1L          Stock:  35  Reorder:  60  Order Qty:  85  Agro Fresh Traders
P504  Marker Set (4)            Stock:   0  Reorder:  25  Order Qty:  50  EduMart  [OUT OF STOCK]
P203  Power Bank 10000mAh       Stock:   0  Reorder:  15  Order Qty:  30  VoltEdge Traders  [OUT OF STOCK]
P302  Women's Kurti             Stock:  22  Reorder:  30  Order Qty:  38  StyleHub Garments
P202  Smartphone Charger 20W    Stock:  18  Reorder:  25  Order Qty:  32  TechNova Distributors
P503  Geometry Box              Stock:  14  Reorder:  20  Order Qty:  26  EduMart
P403  Mixer Grinder 500W        Stock:  12  Reorder:  15  Order Qty:  18  KitchenPro Appliances
P405  Pressure Cooker 5L        Stock:  17  Reorder:  20  Order Qty:  23  KitchenPro Appliances
P105  Sugar 1kg                 Stock:  55  Reorder:  55  Order Qty:  55  Golden Mills
Total products to reorder: 11
```

8. Conclusion

The system tracks stock levels, values inventory by category, ranks best-selling products, flags out-of-stock and low-stock items, and safely processes sales with input validation and exception handling. Future extensions include a SALES transaction table, a trigger to log stock changes, and automatic purchase orders.